

中国科学技术大学 USTC

智能决策博弈与数字创新经济安徽省重点实验室

Anhui Province Key Laboratory of Intelligent Decision Games and Digital Economic Advancements

IDEA 重点实验室内部论文阅读集

数据智能讨论班

2026 年 11 月

目录

1. 菜单定价：Menu Pricing of Large Language Models (CFDP, 2026) 3

2. 随机车辆路径：Data-Driven Stochastic Vehicle Routing Problems with Deadlines Under Decision-Dependent Travel Time (MSOM, 2026) 4

3. 序贯决策：Improving Human Sequential Decision Making with Reinforcement Learning (MS, 2026) 5

4. 护士配置：Nurse Staffing Under Absenteeism: A Distributionally Robust Optimization Approach (MSOM, 2025) 6

5. 无人机基站选址：3-D Dynamic UAV Base Station Location Problem (IJOC, 2021) 7

1. 菜单定价：Menu Pricing of Large Language Models (CFDP, 2026)

来源：Cowles Foundation Discussion Paper No. 2502, 2026.

https://arxiv.org/abs/2502.07736v2

作者：Dirk Bergemann, Alessandro Bonatti, Alex Smolin

关键词：大语言模型；最优定价；菜单定价；微调

推荐人：李昊洋

推荐理由：本文建立大语言模型定价与产品设计框架，研究服务商如何向任务估值不同的用户销售 token 预算菜单。在齐次生产技术下，用户的高维类型可由一个标量指标概括，从而把定价问题化为一维筛选。最优机制表现为承诺支出合约：用户购买预算，再按边际成本在不同 token 类别之间分配。研究进一步讨论多种差异化模型，以及专有模型与开源竞争者并存的情形，说明竞争如何改变算力供给的规模和覆盖范围，并将模型结果与预算套餐、多模型分层和线性 API 定价联系起来。

arXiv:2502.07736v2 [econ.TH] 9 Mar 2026

Menu Pricing of Large Language Models*

Dirk Bergemann Alessandro Bonatti Alex Smolin

March 10, 2026

Abstract

We develop a framework for the optimal pricing and product design of LLMs in which a provider sells menus of token budgets to users who differ in their valuations across a continuum of tasks. Under a homogeneous production technology, we show that users' high-dimensional type profiles are summarized by a scalar index, reducing the seller's problem to one-dimensional screening. The optimal mechanism takes the form of committed-spend contracts: buyers pay for a budget that they allocate across token classes priced at marginal cost. We extend the analysis to environments with multiple differentiated models and to competition between a proprietary leader and an open-source fringe, showing that competitive pressure reshapes both the intensive and extensive margins of compute provision. Each element of our theory (token-budget menus, maximum- and minimum-spend plans, multi-model versioning, and linear API pricing) has a direct counterpart in the observed pricing practices of providers such as Anthropic, OpenAI, and GitHub.

Keywords: Large Language Models, Optimal Pricing, Menu Pricing, Fine-Tuning.
JEL Codes: D47, D82, D83.

*Bergemann: Department of Economics, Yale University, dirk.bergemann@yale.edu. Bonatti: MIT Sloan, bonatti@mit.edu. Smolin: Toulouse School of Economics, alexey.v.smolin@gmail.com. We thank Mark Armstrong, Mert Demirer, Scott Kominers, Antonio Russo, Ron Siegel, and Frank Yang for valuable comments and discussions, as well as audiences at the Triangle Conference, USC, Caltech, Virginia, the Chicago workshop, the Sciences Po workshop, the NBER Market Design workshop, VSET, Brown, LSE, the TSE Digital Economics Conference, the IESE Economics of AI Conference, and Cambridge. An extended abstract of an earlier version of this paper appeared as "The Economics of Large Language Models: Token Allocation, Fine-Tuning, and Optimal Pricing" in the proceedings of EC'25. Dirk Bergemann gratefully acknowledges financial support from NSF SES 2049754 and ONR MURI. Alessandro Bonatti gratefully acknowledges financial support from NSF SES 2519401. Alex Smolin gratefully acknowledges funding from the French National Research Agency (ANR) under the Investments for the Future program (grant ANR-17-EURE-0010) and through the AI Interdisciplinary Institute ANITI (grant ANR-23-IACL-0002).

1 Introduction

Access to large language models is fast becoming a major input to economic activity. Businesses use LLMs for coding, customer support, legal research, and content generation; individual users rely on them for writing, analysis, and decision-making. The leading providers, Anthropic, OpenAI, and Google, collectively serve hundreds of millions of users and generate billions of dollars in annualized revenue. Yet the pricing of these services remains strikingly ad hoc: subscription tiers, token-based metering, credit systems, and volume commitments coexist with no clear theoretical foundation. Understanding how a profit-maximizing provider should price LLM access, and quantifying the distortions that arise when it does, is the goal of this paper.[1]

Pricing access to LLMs is fundamentally a multidimensional screening problem. An LLM user faces a continuum of tasks, each with a different value. The provider offers token budgets—bundles of input, output, and fine-tuning tokens—that the user allocates across tasks at her discretion. The provider can meter total token usage but cannot observe or contract on the user's task-by-task allocation, creating a combined adverse-selection and moral-hazard problem. User types are infinite-dimensional (a function from tasks to values), the allocation space is high-dimensional (tokens of multiple classes across many tasks), and the user's hidden action (token allocation across tasks) further compounds the difficulty. A priori, this problem seems intractable.

Our central result is that it is not. Under a homogeneous production technology—meaning that the gain function is homogeneous, so the optimal composition of inference tokens is scale-invariant—each user's entire type profile collapses to a scalar index, the *aggregate type*. Users with the same aggregate type make the same total token demands, receive the same total surplus, and choose the same menu item, regardless of the fine structure of their task valuations. This aggregation property reduces the provider's problem to one-dimensional screening à la Mussa and Rosen (1978), despite the underlying complexity of the environment. Building on this reduction, we develop the analysis in four steps.

First, we characterize the efficient allocation (Section 3). Efficiency requires that all tasks employ inference token classes in common proportions; only the scale of token usage varies with a task's marginal value. When the provider faces capacity constraints in each token class, the efficient allocation can be implemented through linear prices equal to inflated shadow costs (Corollary 1), a result that rationalizes the linear, pay-per-token pricing universally observed in developer-facing API markets.

[1] For the practical challenges associated with LLM pricing, see https://www.wsj.com/articles/no-one-knows-how-to-price-ai-tools-f346ea8a.

Second, we characterize the optimal mechanism for a single-model monopolist (Section 4). By the aggregation result, the buyer's indirect utility from a token-budget bundle takes a multiplicative form: aggregate type times aggregate quality. The optimal menu therefore excludes low types and distorts quality downward for (almost) all others, exactly as in the standard one-dimensional framework. Crucially, the optimal mechanism admits intuitive indirect implementations (Proposition 4): it can be realized as a maximum-spend mechanism (a budget of credits allocated across tokens priced at marginal cost), as a minimum-spend mechanism (volume commitments that unlock lower per-token prices), or as a menu of two-part tariffs. These implementations are not merely theoretical curiosities; they correspond precisely to the pricing structures observed at leading platforms.

Third, we extend the framework to multiple differentiated models (Section 5). When models share a common returns-to-scale parameter in inference but differ in returns to fine-tuning, the buyer's payoff has a constant elasticity of substitution over model qualities. Cost minimization implies that each type uses a single model for all tasks (Proposition 6). This generates a natural versioning structure: higher-tier plans grant access to more capable models, not merely larger usage allocations. Anthropic's pricing illustrates the single-model benchmark of Section 4: all paid tiers access the same models, with differentiation occurring through compute budgets. OpenAI, by contrast, screens jointly on usage and model access, reserving its most compute-intensive reasoning model for the highest tier, consistent with the multi-model menu of Section 5.

Fourth, we study competition between a proprietary leader and an open-source fringe that sells tokens at marginal cost (Section 5). The leader designs an optimal menu subject to the buyer's ability to supplement usage from the fringe. Three regions emerge: low types purchase exclusively from the fringe; intermediate types buy from the leader at quantities that exactly deter fringe top-up; and high types are served as under pure monopoly, with standard downward distortions (Proposition 7). Competition thus reshapes both the intensive margin (how much compute is sold to each buyer) and the extensive margin (which buyers adopt the proprietary model at all).

We conclude by connecting each theoretical construct to observed pricing (Section 6). Consumer subscriptions at Anthropic and OpenAI implement the nonlinear menus of Sections 4 and 5. Model aggregators (e.g., Quora's Poe and GitHub Copilot) implement the committed-spend mechanisms of Section 4.2, differing in whether they enforce a hard budget cap (maximum spend) or allow overages at linear prices (minimum spend). API pricing is linear in tokens with no volume discounts, consistent with providers prioritizing adoption over rent extraction. Thus, pricing practices observed in the industry suggest that these mechanisms capture fundamental economic forces rather than incidental design choices.

Related Literature Our paper contributes to the rapidly growing literature on the economics of AI. Existing work has examined the impact of LLMs on price competition (Fish, Gonczarowski, and Shorrer, 2024), token auctions (Duetting, Mirrokni, Paes Leme, Xu, and Zuo, 2024), and sponsored search (Bergemann, Bojko, Dütting, Paes Leme, Xu, and Zuo, 2024), and Demirer, Fradkin, Tadelis, and Peng (2025) document pricing and market-share patterns. By contrast, our focus is on the provider's own design problem, how to price and version LLM access, which has received surprisingly little theoretical attention. For example, Mahmood (2024) studies competition among producers of generative AI who can specialize in different tasks, but restricts attention to linear token pricing. We go further and characterize the fully optimal nonlinear mechanism, leveraging the sufficient-statistic reduction generated by the homogeneity of the gain function, which renders the joint screening-and-moral-hazard problem solvable and yields sharp, implementable predictions.

At their core, LLMs are an information technology, connecting our work to the literature on selling information (Babaioff, Kleinberg, and Paes Leme, 2012; Bergemann, Bonatti, and Smolin, 2018; Yang, 2022). However, LLMs possess distinctive features: they are general-purpose technologies deployed across many tasks, usage is metered in tokens, and precision is improved by combining inference and fine-tuning, which give rise to a structurally different design problem.

Methodologically, our aggregation result relates to the literature on "1.5-dimensional" mechanism design (Fiat, Goldner, Karlin, and Koutsoupias, 2016; Devanur, Goldner, Saxena, Schvartzman, and Weinberg, 2020). Unlike most of that literature, however, we must address potential incentive clashes across the continuum of subproblems that arise from the buyer's hidden allocation of tokens, requiring an additional argument based on the production technology. Even in simpler settings, such as one-dimensional screening with moral hazard (Castro-Pires, Chade, and Swinkels, 2024) or multidimensional screening and bundling without moral hazard (Armstrong, 1996; Rochet and Stole, 2003; Daskalakis, Deckelbaum, and Tzamos, 2017), tractability issues arise. Our environment combines both, and our solution exhibits a constrained efficiency property as in Laffont and Tirole (1990) and Doligalski, Dworczak, Krysta, and Tokarski (2025).

2 Model

2.1 Task Environment

A buyer of LLM services faces a unit measure of tasks indexed by $i \in [0, 1]$. In our baseline setting, an LLM provider offers one type of model only. The buyer can decide on (i) how

many *inference* tokens to use for each task i, $x_i \in \mathbb{R}^J_+$ and (ii) how many *fine-tuning* tokens to use to improve the model's performance (e.g., precision) on all tasks, $z \in \mathbb{R}^K_+$. In language models, a token is a unit of text, typically a word.

The model's performance on task i is given by the *gain function* $g(x_i, z)$. The function g is common across tasks and takes the following form:

$$g(x_i, z) = \Psi(x_i)\,\Phi(z). \tag{1}$$

Thus, the gain function is multiplicatively separable in inference and fine-tuning tokens.

Diminishing returns to inference tokens on a single task are a fundamental feature of LLM performance, consistent with empirically observed scaling laws. Thus the function Ψ is positive, increasing, strictly concave, differentiable, Inada at 0,[2] and homogeneous of degree $\sigma \in (0,1)$, i.e., $\Psi(rx_i) = r^\sigma \Psi(x_i)$ for all $r > 0$. This function captures the returns to inference tokens. A canonical example of such a function is a CES function, $\Psi(x_i) = \left(\sum_{j=1}^J \alpha_j x_{ij}^\rho\right)^{\sigma/\rho}$, where $\alpha_j > 0, \sum_j \alpha_j = 1, \rho < 1$. The function $\Phi(z)$ is positive, increasing, such that g is strictly concave (e.g., $\Phi(z)^{1/(1-\sigma)}$ is strictly concave) and $(1-\sigma)$-Inada at 0.[3] This function captures the returns to fine-tuning.

These properties are similar in shape to the scaling laws for training LLMs. Concavity captures the fundamental assumption that time and computing resources spent on a single task exhibit diminishing marginal returns. Homogeneity is the key tractability assumption. It ensures that the optimal mix of token classes is the same for every task; only the scale of token usage varies. This property drives the aggregation result in Section 3, which reduces the seller's infinite-dimensional screening problem to a one-dimensional problem. The parameter σ captures the returns to scale.

The buyer's marginal value of performance on each task is captured by $w = (w_i)_{i\in[0,1]}$,

$$w : [0,1] \to [0,1], \tag{2}$$

which we refer to as the buyer's *type*. Using a z-fine-tuned model with a profile of $(x_i)_{i\in[0,1]}$ inference tokens delivers the following total payoff for buyer type w:

$$\int_0^1 w_i g(x_i, z) di. \tag{3}$$

The buyer's type is distributed according to a commonly known distribution F_w. The buyer

[2] That is, for all j and $x_{i,-j} \in \mathbb{R}^{J-1}_+$ such that $\Psi(0, x_{i,-j}) > 0$, $\lim_{x_{ij}\downarrow 0} \Psi_j(x_{ij}, x_{i,-j}) = +\infty$.

[3] That is, there exists $z_0 \geq 0$, $z_0 \neq 0$, such that $\lim_{r\downarrow 0} \Phi(rz_0)/r^{1-\sigma} = +\infty$. Sufficient conditions for this property are (i) $\Phi(0) > 0$ or (ii) Φ is homogeneous of degree $\hat{\sigma} \in (0,1)$ with $\sigma + \hat{\sigma} < 1$.

knows his type, while the provider does not.

The provider bears the cost of processing tokens. We assume that the marginal processing costs are constant but can vary across different token classes. The cost of inference tokens is $c_j > 0$, $j \in [J]$, and the cost of fine-tuning tokens is $\hat{c}_k > 0$, $k \in [K]$. We let $c \in \mathbb{R}^J_{++}$ and $\hat{c} \in \mathbb{R}^K_{++}$ denote the corresponding cost profiles.

Notation Throughout the text, we use the following notation. "$\triangleq$" indicates a definition. $[K] \triangleq \{1, \ldots, K\}$. $\mathbb{R}^K_+ \triangleq \{x \in \mathbb{R}^K : x_k \geq 0,\ k \in [K]\}$ and $\mathbb{R}^K_{++} \triangleq \{x \in \mathbb{R}^K : x_k > 0,\ k \in [K]\}$. All stand-alone qualifiers such as "positive," "increasing," and "concave" are understood in the weak sense, that is, as "non-negative," "non-decreasing," and "weakly concave," respectively. For constants, subscripts refer to variables or tasks; for functions, they indicate partial derivatives. All proofs are in Appendix A.

2.2 Mechanism Design

The provider contracts on the total number of tokens of each class used by the buyer. In other words, the provider sells budgets of various kinds of tokens. A mechanism consists of an arbitrary menu of the form

$$\{(X_1(w), \ldots, X_J(w), Z_1(w), \ldots, Z_K(w), t(w))\}_w, \tag{4}$$

where X_j is the total number of inference tokens of class j. Upon purchase, the buyer can freely distribute those tokens across tasks. (We relax this restriction in Appendix B, where we allow the provider to contract on the usage of all tokens by the buyer.) Z_k is the number of fine-tuning tokens of class k, which is used for fine-tuning and cannot be distributed.

The seller's problem is an optimal mechanism design problem with infinite-dimensional private information and moral hazard. However, we show that the homogeneity of the gain function renders the problem tractable.

2.3 Mapping the Setting to Practice

Before turning to the analysis, we describe how the primitives of our setting relate to the design and operation of contemporary large language models (LLMs).

Fine-tuning tokens, $z \in \mathbb{R}^K_+$ The case $z = 0$ corresponds to a baseline model that has not been fine-tuned.[4] In this case, the model can be interpreted as a *foundation model*. Its

[4]For tractability, we will often study the case $g(x, 0) = 0$. This can be viewed as a normalization that doesn't affect the economics of the problem.

quality depends on its size (i.e., the number of parameters)[5], its architecture, the quantity and quality of training data, and the details of the training procedure. Scaling up model size typically improves capability and accuracy, but state-of-the-art performance generally requires commensurate scaling of training data as well (Kaplan et al., 2020). In our setting, we take pretraining as given. Accordingly, we focus on inference-time compute rather than training-time compute.

When $z > 0$, this variable captures the number of tokens used to fine-tune the foundation model. Fine-tuning resembles pretraining in that it updates the model's parameters, without changing its size or architecture, but it is much smaller in scale. It is performed on a dataset directly relevant to a class of tasks (e.g., labeled X-ray scans, code examples, or conversational transcripts) and can be interpreted as injecting task-specific knowledge and behavior into the model.

Inference tokens, $\boldsymbol{x_i} \in \mathbb{R}_+^J$ These are the tokens used directly to process a given task. The two standard classes of inference tokens are input and output tokens. Input tokens are those provided by the user. Increasing the number of input tokens can improve predictive quality for two reasons. First, richer prompts provide more context, enabling more tailored and appropriate responses. Second, additional input may supply the data on which the model is meant to operate. For example, retrieval-augmented generation (RAG) uses the prompt to retrieve relevant passages from an external corpus and appends them to the prompt.

Output tokens are those generated by the model. A larger number of output tokens can also improve quality. First, it allows the model to provide more detailed and nuanced answers. Second, it provides opportunities for multi-step reasoning, often described as chain-of-thought (CoT) computation. This additional reasoning channel can enable the model to solve more complex tasks, at the cost of generating many intermediate tokens, often hidden from the user. This inference-time reasoning is qualitatively distinct from pretraining, remains only partially understood, and is an active frontier for improving LLM performance.[6]

Gain function, $\boldsymbol{g}$ Tokens of different classes enter the large language model as distinct inputs. They are neither perfect substitutes nor perfect complements, but each can contribute to higher predictive quality. This motivates our gain-function formulation in (1), which is also in line with the empirically observed scaling laws for inference-time computation (e.g., Wu, Sun, Li, Welleck, and Yang, 2025).

[5]The parameter counts of leading models are estimated to be in the several-billion range; see https://codingscape.com/blog/most-powerful-llms-large-language-models.

[6]Recent progress has leveraged this channel; see, for instance, https://arcprize.org/blog/oai-o3-pub-breakthrough.

Furthermore, one can envision other contractible token categories, either by refining the distinctions above (e.g., separating reasoning tokens from response tokens) or by introducing new modalities and resources (e.g., media files, external databases, etc.). Accordingly, our model accommodates an arbitrary number of inference and fine-tuning token classes.

Costs, $c_j, \hat{c}_k$ Processing tokens is costly because it requires executing the neural network and, for fine-tuning, computing parameter updates, which in turn requires energy and specialized hardware. Because tokens within a given class are processed symmetrically from a computational perspective, we assume that the marginal cost per token within a class is constant across tasks.[7] However, because different token classes correspond to different computational routines, we allow marginal costs to differ across classes.

3 Efficient Solution

In this section, we begin by analyzing the socially efficient allocation of tokens to buyers and tasks. This solution provides a useful benchmark and also coincides with the optimal monopoly solution when the buyer's type is known. We then turn to the problem of a social planner who faces capacity constraints for each token class, and we use the constrained-efficient solution to characterize the buyer's utility from purchasing a fixed token budget.

Unconstrained problem Given a type $w = (w_i)_{i\in[0,1]}$, the efficient allocation solves the following problem:

$$\max_{(x_i)_{i\in[0,1]}, z\geq 0} \int_0^1 w_i \Psi(x_i)\Phi(z)di - \sum_{j=1}^{J} c_j \int_0^1 x_{ij} di - \sum_{k=1}^{K} \hat{c}_k z_k. \tag{5}$$

Consider the allocation of inference tokens first. A key implication of the homogeneity of the production function Ψ is that inference tokens are optimally employed in the same proportions: the efficient allocation of tokens across tasks differs solely by the scale at which these inputs are employed. To see this, consider the system of J first-order conditions for each $x_i = (x_{i1}, ..., x_{iJ})$:

$$w_i \nabla\Psi(x_i)\Phi(z) = c. \tag{6}$$

By equation (6), for all i, $\nabla\Psi(x_i)$ belongs to a ray with a direction c. Because Ψ is homogeneous and strictly concave, the ray in the space of gradients corresponds to a ray in the

[7] Note that fine-tuning typically does not change the model's architecture or size and therefore should not directly affect marginal inference costs.

space of tokens. Thus, for each i, any optimal x_i can be written as:

$$x_i = r_i d,$$

where $d \in \mathbb{R}^J_+$ is the unique vector that solves $\nabla\Psi(d) = c$, i.e., the cost-minimizing input shares. Furthermore, because Ψ is homogeneous of degree σ, Ψ_j is homogeneous of degree $\sigma - 1$, so that the optimal scale is given by

$$r_i = w_i^{\frac{1}{1-\sigma}} \Phi(z)^{\frac{1}{1-\sigma}}.$$

Substituting back in the objective function, both the total surplus and the total consumption of each token class depend on the buyer's type w only through $\int_0^1 w_i^{1/(1-\sigma)} di$. It is then convenient to define the aggregate type $\theta(w)$ as

$$\theta(w) \triangleq \left(\int_0^1 w_i^{\frac{1}{1-\sigma}} di \right)^{1-\sigma}. \tag{7}$$

Our first result establishes that the total surplus and the total amount of inference tokens depend only on the aggregate type and not on the finer details of the type profile w.

Proposition 1 (Efficient Allocation). *Under the efficient allocation, all buyer types w with the same aggregate type $\theta(w)$ consume the same number of fine-tuning tokens, consume the same total number of inference tokens in each class, and obtain the same total payoff. The number of inference tokens allocated to task i is proportional to $w_i^{\frac{1}{1-\sigma}}$.*

Proposition 1 has a key implication: two buyers with very different task profiles—one who values a few tasks intensely and many tasks little, and another who values all tasks moderately—consume the same total resources and obtain the same total surplus, provided they share the same aggregate type. The seller therefore cannot distinguish between them on the basis of total token consumption, nor would it want to. This indistinguishability is not an assumption but a consequence of the technology: homogeneity of Ψ ensures that the efficient mix of token classes is task-independent, so only the scale of usage varies, and the aggregator (7) is the natural summary of how much total scale a buyer demands.

We now show that a similar logic can be used to characterize the socially efficient solution under capacity constraints in each class of tokens (training, computation, input, output).

Constrained problem We now introduce capacity constraints on each token class. This detour is not merely a generalization: the special case in which all constraints bind is precisely the problem faced by a buyer who has purchased a fixed bundle of token budgets. Thus,

the constrained-efficient allocation provides the foundation for the seller's mechanism design problem in Section 4.

Fix a type w and capacity constraints $X_j > 0$, $Z_k > 0$ for all j, k. The capacity-constrained planner's problem is

$$\max_{(x_i)_{i\in[0,1]}, z\geq 0} \int_0^1 w_i \Psi(x_i)\Phi(z)\, di - \sum_{j=1}^{J} c_j \int_0^1 x_{ij} di - \sum_{k=1}^{K} \hat{c}_k z_k,$$
$$\text{s.t.} \int_0^1 x_{ij} di \leq X_j,\ z_k \leq Z_k \text{ for } j \in [J], k \in [K].$$

Our next result establishes that the capacity-constrained efficient allocation coincides with the (unconstrained) efficient allocation for marginal costs inflated by the shadow costs of the capacity constraints, $(c', \hat{c}') \geq (c, \hat{c})$. This observation yields an immediate implementation of the efficient solution.

Corollary 1 (Constrained Efficient Allocation).

1. *In the capacity-constrained efficient allocation, all buyer types w with the same aggregate type $\theta(w)$ consume the same total number of fine-tuning tokens and inference tokens in each class, and they obtain the same total payoff.*

2. *The capacity-constrained efficient allocation can be implemented via linear prices equal to inflated marginal costs.*

Thus, capacity constraints do not change the qualitative properties of the solution, but lead to an inefficiency in the relative allocation of token types, and in the split between those buyer types who choose to fine-tune ($z > 0$) and those who do not ($z = 0$).

The special case where all capacity constraints bind is of particular interest, because it is instrumental in the characterization of the buyer's demand for token budgets in Section 4. When all constraints bind, the total production cost is pinned down. The planner's and the buyer's solutions then coincide, i.e., the constrained efficient allocation solves the problem of a buyer who has access to inference and fine-tuning token budgets (X, Z).

The optimal allocation of a fixed budget (X, Z) solves the following problem:

$$\max_{x_{ij}\geq 0} \int_0^1 w_i \Psi(x_i)\Phi(Z)\, di, \quad \text{s.t.} \int_0^1 x_{ij}\, di = X_j \text{ for } j \in [J]. \tag{8}$$

Applying the homogeneity argument again, $\Phi(Z)$ factors out, and the buyer optimally sets $x_i = r_i\, d$ with $r_i \propto w_i^{1/(1-\sigma)}$. Since the budget constraint for each token class j requires $d_j \int_0^1 r_i\, di = X_j$, the common direction is $d = X/\int_0^1 r_i\, di$, which yields a simple expression for the buyer's optimal payoff.

The following result is the key step in our analysis. It shows that the buyer's indirect utility from any token bundle is multiplicatively separable in the aggregate type and an aggregate quality index, placing the seller's problem squarely in the Mussa-Rosen framework.

Proposition 2 (Buyer Indirect Utility). *For any inference and fine-tuning token budgets* $X = (X_1, \ldots, X_J) \geq 0$ *and* $Z = (Z_1, \ldots, Z_K) \geq 0$*, the indirect utility of buyer type* w *is*

$$U(w, X, Z) = \theta(w)\, \Psi(X)\Phi(Z),$$

where $\theta(w)$ *is the aggregate type defined in* (7)*.*

We therefore obtain a tractable expression for the buyer's payoff by means of a representative task, whose value is given by the aggregate type θ, to which the buyer assigns the entire token budget. Heterogeneity across tasks washes out, and only the aggregate matters. A notable implication is that the buyer's unobserved allocation of tokens across tasks generates no additional incentive constraints beyond those of the standard screening problem. Under homogeneity, the cost-minimizing input mix is independent of scale, so the buyer's within-bundle allocation is pinned down regardless of type. This product form is the key input to the seller's problem, which we turn to next.

4 Menus of Token Budgets

The main result of this section is that the seller's optimal mechanism takes the form of a menu of committed-spend contracts: each buyer pays an upfront fee for a spending budget that she allocates freely across token classes priced at the provider's marginal cost. Higher types purchase larger budgets at higher fees, with quantity discounts that are consistent with observed industry pricing. The formal analysis proceeds in two steps: we first characterize the optimal direct mechanism (Section 4.1) and then show that it admits intuitive indirect implementations (Section 4.2).

We now characterize the provider's profit-maximizing menu of token budgets. By Proposition 2, the buyer's indirect utility from a bundle (X, Z) is $\theta(w)\, \Psi(X)\Phi(Z)$. Two features of this expression are worth emphasizing. First, private information enters only through the scalar θ: the seller's multidimensional screening problem reduces to a one-dimensional problem. Second, the payoff is multiplicatively separable in type and allocation, placing us in the framework of Mussa and Rosen (1978).

4.1 Optimal Budget Mechanism

Define the effective type as θ and the effective allocation as the *aggregate quality* given by

$$Q(X, Z) \triangleq \Psi(X)\Phi(Z). \tag{9}$$

The effective production costs of producing the aggregate quality Q can be found via a cost-minimization problem:

$$C(Q) \triangleq \min_{X,Z\geq 0} \sum_{j=1}^{J} c_j X_j + \sum_{k=1}^{K} \hat{c}_k Z_k, \quad \text{s.t. } \Psi(X)\Phi(Z) = Q. \tag{10}$$

The solution to the cost-minimization problem has the following properties.

Lemma 1 (Cost Function). *The cost function $C(Q)$ is strictly increasing and strictly convex and satisfies $C'_+(0) = 0$.*[8]

Convexity follows from the strict concavity of the gain function g: producing higher aggregate quality requires disproportionately more tokens. The property $C'_+(0) = 0$ follows from the Inada conditions. Economically, $C'_+(0) = 0$ means that the first unit of aggregate quality is arbitrarily cheap to produce. Combined with convexity, this implies that it is never efficient to exclude any buyer type entirely. Thus, in the monopoly problem, exclusion is driven by information rents rather than production costs.

By Lemma 1, the analysis of Mussa and Rosen (1978) applies. Denote the prior distribution of θ by F, which is derived from F_w. The seller chooses an aggregate quality schedule $Q(\theta)$ and a transfer schedule $t(\theta)$ to solve

$$\max_{Q(\cdot),\, t(\cdot)} \ \mathbb{E}_F\big[t(\theta) - C(Q(\theta))\big],$$

subject to incentive compatibility and individual rationality.

As usual, let

$$\varphi(\theta) \triangleq \theta - \frac{1 - F(\theta)}{f(\theta)}, \tag{11}$$

denote the virtual (aggregate) type. If the virtual value φ is not increasing, let $\bar{\varphi}$ denote its Myerson-ironed version. Then, all θ with $\bar{\varphi}(\theta) \leq 0$ are excluded. For all other θ, the optimal allocation is uniquely pinned down by:

$$\bar{\varphi}(\theta) = C'(Q(\theta)). \tag{12}$$

[8] In Section 5, we show that if $\Phi(Z)$ is also a homogeneous function, then $C(Q)$ is simply a power function.

Denoting the corresponding optimal transfers by

$$t(\theta) = \theta\, Q(\theta) - \int_0^{\theta} Q(r)dr, \tag{13}$$

we can then fully characterize the optimal mechanism.

Proposition 3 (Optimal Menu). *The optimal menu of token budgets is* $\{(X(\theta), Z(\theta), t(\theta))\}_\theta$, *where* $(X(\theta), Z(\theta))$ *are the cost-minimizing token budgets that deliver quality* $Q = Q(\theta)$ *at prices* $t(\theta)$ *as defined in* (10)-(13).

The optimal menu offers a continuum of plans. All types w with the same aggregate type $\theta(w)$ choose the same item. Low-type buyers are excluded. All served buyers receive distorted-downward quality (fewer tokens than the efficient allocation would prescribe) except at the very top. Buyers with higher aggregate types purchase strictly larger token budgets and pay strictly higher transfers, but enjoy quantity discounts: the average price per unit of quality falls with θ.

4.2 Indirect Mechanisms

The optimal direct mechanism specifies token quantities for each type. In practice, providers do not announce menus of token quantities; they post prices and spending limits. We now show that three natural pricing formats implement the optimum.

We first consider committed-spend mechanisms, where the terms of the contract with a buyer are related to a maximum or minimum total expenditure.

Definition 1 (Maximum-Spend Mechanism). A maximum-spend mechanism is a collection of token prices p and a menu of monetary budgets and transfers $\{(B_n, T_n)\}_n$ such that, upon selecting item n, the buyer pays T_n for access to budget B_n, which he can freely spend on tokens priced at p.

While the optimal budget (direct) mechanism is a cap on quantities, the maximum-spend mechanism limits quality by capping expenditures at fixed prices. Those expenditures are virtual in that they do not contribute to payment beyond T_n, but they govern token allocation. As we will see in Section 6, the maximum-spend mechanism is used in practice, for example, by Quora's Poe (see Section 6), and it is closely related to the Cost-Based tariffs (where $p = c$) introduced in Armstrong (1996). Indeed, Proposition 8 in Appendix B derives analogous conditions to those in Armstrong (1996) under which a maximum-spend mechanism is optimal across all mechanisms, including those that specify a task-by-task token allocation.

An alternative implementation does not impose a cap on spending, but exposes the buyer to variable marginal prices, much like committed-spend mechanisms in cloud computing.

Definition 2 (Minimum-Spend Mechanism). A minimum-spend mechanism is a menu of monetary budgets and transfers $\{(p_n, T_n)\}_n$ such that, upon selecting item n, the buyer commits to spending at least T_n on tokens priced at p_n.

A minimum-spend mechanism allows buyers to commit to larger budgets (i.e., total expenditures) to unlock variable-price discounts. In contrast to a maximum-spend mechanism, payment comes from actual token consumption rather than from an upfront payment.

Definition 3 (Two-Part-Tariff Mechanism). A two-part-tariff mechanism is a menu of prices and transfers $\{(p_n, T_n)\}_n$ such that, upon selecting item n, the buyer pays T_n and can buy any number of tokens priced at p_n.

A two-part tariff is a classical mechanism that combines upfront and consumption payments but does not impose any token consumption limits.

Define a type-dependent *markup* $m(\theta)$ as:

$$m(\theta) \triangleq \frac{\theta}{\bar{\varphi}(\theta)}. \tag{14}$$

The markup $m(\theta)$ is the ratio of the true type to the virtual type; it captures the information-rent wedge that the seller imposes. Higher markups on lower types reflect greater information rents extracted from higher types in the menu.

Proposition 4 (Indirect Implementation). *An optimal menu of token budgets can be implemented via a maximum-spend mechanism. If $m(\theta)$ is decreasing (e.g., if F has an increasing hazard rate), then an optimal menu of token budgets can be implemented via a minimum-spend mechanism and a two-part tariff mechanism.*

Implementability by a maximum-spend mechanism is straightforward and relies on the fact that the optimal token allocation is constrained-efficient, as in Doligalski et al. (2025): it suffices to set prices equal to marginal costs and to choose token expenditures and transfers so as to mimic those under the direct mechanism.

Implementability by minimum-spend and two-part tariff mechanisms requires more care. Because Ψ is homogeneous, the cost-minimizing input mix is the same for every quality level Q; only the scale changes. A two-part tariff that preserves this input mix must therefore set token prices proportional to marginal costs; any other price vector would distort the buyer's allocation away from cost minimization. The proportionality factor is the markup $m(\theta) =$

$\theta/\bar{\varphi}(\theta)$, which inflates all marginal costs equally. The condition that $m(\theta)$ is decreasing ensures that higher types, who select larger budgets, face lower per-token prices, so the menu is incentive-compatible. In the minimum-spend implementation, the markup equals the ratio between total revenue and total cost for type θ. Section 6 shows that each of these three implementations corresponds to an observed pricing format at leading platforms.

5 Multiple Models and Competition

In practice, every major LLM provider offers multiple models that differ in capability and cost (e.g., Anthropic's Haiku, Sonnet, and Opus, or OpenAI's GPT-4o-mini and o1). A buyer can assign different tasks to different models, and the provider can screen on both usage quantity and model access. This section extends our framework to this richer environment. Two new questions arise: How does a buyer optimally allocate tasks across models? And how does a profit-maximizing provider design menus that screen on both dimensions?

Specifically, we extend our setting and allow there to be L models, each with a model-specific gain function:

$$g_l(x_i, z) = \Psi_l(x_i)\Phi_l(z), \tag{15}$$

where Ψ_l and Φ_l are assumed to have the same properties as in the baseline model, with Ψ_l being homogeneous of degree σ_l. We order the models so that $\sigma_1 \leq \sigma_2 \leq \cdots \leq \sigma_L$. The token costs are model-specific $(c_l, \hat{c}_l)$.

The buyer can process different tasks with different models; however, he cannot use two models for the same task.[9] If the buyer of type w processes tasks $i \in I_l$ with model l, his total payoff ignoring the payment is:

$$\sum_{l=1}^{L} \int_{I_l} w_i g_l(x_{li}, z_l) di. \tag{16}$$

In this setting, a bundle specifies token budgets for all available models $(X_l, Z_l)_{l=1}^L = (X_{l1}, \ldots, X_{lJ}, Z_{l1}, \ldots, Z_{lK})_{l=1}^L$.

As a preliminary step for the subsequent analysis, we show that the buyer's value for a bundle $(X_l, Z_l)_{l=1}^L$ depends only on the aggregate quality of each model,

$$Q_l \triangleq g_l(X_l, Z_l), \tag{17}$$

[9]Equivalently, we could assume that the buyer can combine tokens from two models on any task, with a performance given by the sum of individual gain functions.

and admits a tractable closed-form expression. To this end, we fix w and order the tasks so that w_i is increasing. We assume that $Q_l > 0$ for all l (if $Q_l = 0$ then the l-th model can be ignored). If the buyer allocates tasks in the set I_l to model l, then his payoff from model l is, by the arguments of Proposition 2,

$$U_l(I_l, Q_l) = \theta_l(I_l) Q_l, \tag{18}$$

where $\theta_l(I_l) \triangleq (\int_{I_l} w_i^{1/(1-\sigma_l)} di)^{1-\sigma_l}$. The total payoff from the task-model allocation $(I_l)_{l=1}^L$ is

$$\sum_{l=1}^{L} U_l(I_l, Q_l) = \sum_{l=1}^{L} \theta_l(I_l) Q_l. \tag{19}$$

The optimal payoff $U^*(Q_1, \ldots, Q_L) = \sum_{l=1}^L U_l(I_l^*, Q_l)$ is evaluated at the optimal task-model allocation $(I_l^*)_{l=1}^L$.

Lemma 2 (Buyer-Optimal Payoff). *If $\sigma_l = \sigma$ for all $l \in [L]$, then*

$$U^*(Q_1, \ldots, Q_L) = \theta \left(\sum_{l=1}^{L} Q_l^{1/\sigma} \right)^{\sigma}. \tag{20}$$

By Lemma 2, despite the combinatorial complexity of assigning a continuum of tasks to multiple models, the buyer's payoff from a bundle of tokens across different models admits a simple product decomposition into an aggregate type and a CES-style aggregate quality over models. The elasticity of substitution across models is pinned down by the common returns-to-scale parameter σ, and the single-model tractability of Sections 3–4 extends to the multi-model environment.

5.1 Efficient Solution

We first compute the efficient allocation in this setting. Since the buyer's value takes a simple form (20) as a function of aggregate qualities of each model, the efficient allocation delivers the efficient amount of these qualities in a cost-efficient way.

Assumption 1 (Homogeneous Fine-Tuning). For each $l = 1, ..., L$:

1. Ψ_l is homogeneous of degree $\sigma \in (0, 1)$.
2. Φ_l is homogeneous of degree $\hat{\sigma}_l < 1 - \sigma$.

All subsequent analysis holds under Assumption 1, so we omit it from the statements. Assumption 1 allows us to obtain closed-form expressions because of the following result:

Lemma 3 (Model-Specific Cost Function). *The cost function of delivering aggregate quality Q_l through model l is $C_l(Q_l) = \kappa_l Q_l^{1/(\sigma+\hat{\sigma}_l)}$ for some constant $\kappa_l > 0$.*

By Lemma 3, the minimal cost of obtaining quality Q from L models is

$$C(Q) = \min_{Q_1,\ldots,Q_L \geq 0} \sum_{l=1}^{L} \kappa_l Q_l^{1/(\sigma+\hat{\sigma}_l)}, \quad \text{s.t.} \sum_{l=1}^{L} Q_l^{1/\sigma} = Q^{1/\sigma}.$$

The optimal solution utilizes a single model, resulting in

$$C(Q) = \min_{l \in [L]} \kappa_l Q^{1/(\sigma+\hat{\sigma}_l)}. \tag{21}$$

Thus, the rich functional model heterogeneity (15) reduces to heterogeneity in two parameters $(\kappa_l, \hat{\sigma}_l)$ that roughly correspond to "cost-effectiveness" and "propensity to fine-tune." Heterogeneity in κ_l is vertical: higher κ_l correspond to overall costlier models. Heterogeneity in $\hat{\sigma}_l$ is horizontal: models with high $\hat{\sigma}_l$ are better at generating high Q whereas models with low $\hat{\sigma}_l$ are better at generating low Q. As a result, higher values of Q are optimally produced by models with higher $\hat{\sigma}_l$. In other words, models with higher returns to fine-tuning have flatter cost curves and are cheaper for producing high qualities, introducing a natural form of horizontal differentiation among models.

Notably, the cost function $C(Q)$ is not convex but piecewise convex, with kinks at the model switches, which translates into unusual properties of the efficient and profit-maximizing allocations. Specifically, the efficient quality allocation for type θ when using model l solves:

$$\max_{Q \geq 0} \ \theta Q - \kappa_l Q^{1/(\sigma+\hat{\sigma}_l)},$$

leading to the following characterization:

Proposition 5 (Efficient Allocation). *In the efficient allocation, each type θ processes all tasks with a single model:*

$$l^* \in \arg\max_l \ (1-\sigma-\hat{\sigma}_l) \left(\frac{(\sigma+\hat{\sigma}_l)}{\kappa_l} \right)^{(\sigma+\hat{\sigma}_l)/(1-\sigma-\hat{\sigma}_l)} \theta^{1/(1-\sigma-\hat{\sigma}_l)}, \tag{22}$$

with the aggregate quality:

$$Q^*(\theta) = \left(\frac{\theta(\sigma+\hat{\sigma}_{l^*})}{\kappa_{l^*}} \right)^{(\sigma+\hat{\sigma}_{l^*})/(1-\sigma-\hat{\sigma}_{l^*})}. \tag{23}$$

The efficient token allocation is given by Lemma 3 evaluated at Q^* from (23).

Every type uses a single model. Higher aggregate types θ consume higher aggregate qualities by using models with higher fine-tuning capabilities $\hat{\sigma}_l$, i.e., $Q^*(\theta)$ is increasing.

Because $C(Q)$ is not continuously differentiable, the allocation is discontinuous at the model switch. Indeed, it follows from (23) and (22) that if the efficient allocation at type θ is indifferent between Q_l of model l and $Q_{l'}$ of model l' such that $\hat{\sigma}_{l'} > \hat{\sigma}_l$, then

$$\frac{Q_{l'}}{Q_l} = \frac{1-\sigma-\hat{\sigma}_l}{1-\sigma-\hat{\sigma}_{l'}} > 1.$$

5.2 Multi-Model Monopolist

We now assume that all L models are sold by a monopolist who offers a menu. Each item specifies a transfer and a collection of token budgets for different models $(X_l, Z_l)_{l=1}^L$.

By Lemma 2, the seller's problem is equivalent to the optimal screening problem in Mussa and Rosen (1978), in which the buyer's type is $\theta \in \mathbb{R}$, the seller chooses a menu of (Q, t), the buyer's payoff is $U(\theta, Q) = \theta Q$, and the cost of obtaining a quality Q is equal to $C(Q)$ as in (21). Although $C(Q)$ is not convex, the Envelope theorem applies and this problem is equivalent to a maximization of virtual surplus:

$$\max_{Q(\cdot)\text{: increasing}} \int_0^1 \Big(\varphi(\theta)Q(\theta) - C(Q(\theta))\Big) dF(\theta), \tag{24}$$

where $\varphi(\theta) = \theta - (1-F(\theta))/f(\theta)$.

Assuming $\varphi(\theta)$ is increasing, the solution can be obtained pointwise to equal an efficient allocation with respect to the virtual type: $Q^{\mathrm{m}}(\theta) = Q^*(\varphi(\theta))$. By Proposition 5, if $\varphi(\theta) < 0$, then the buyer is excluded, $Q^{\mathrm{m}} = 0$; if $\varphi(\theta) \geq 0$,

$$l^{\mathrm{m}}(\theta) \in \arg\max_l \ (1-\sigma-\hat{\sigma}_l) \left(\frac{(\sigma+\hat{\sigma}_l)}{\kappa_l}\right)^{(\sigma+\hat{\sigma}_l)/(1-\sigma-\hat{\sigma}_l)} \varphi(\theta)^{1/(1-\sigma-\hat{\sigma}_l)}, \tag{25}$$

$$Q^{\mathrm{m}}(\theta) = \left(\frac{\varphi(\theta)(\sigma+\hat{\sigma}_{l^{\mathrm{m}}})}{\kappa_{l^{\mathrm{m}}}}\right)^{(\sigma+\hat{\sigma}_{l^{\mathrm{m}}})/(1-\sigma-\hat{\sigma}_{l^{\mathrm{m}}})}. \tag{26}$$

Since Q^* is increasing, the pointwise solution solves the problem. Optimal transfers are

$$t^{\mathrm{m}}(\theta) = \theta Q^{\mathrm{m}}(\theta) - \int_0^\theta Q^{\mathrm{m}}(r)dr. \tag{27}$$

As under the efficient allocation, because $C(Q)$ is not continuously differentiable, the quality and transfer schedules are discontinuous.

Proposition 6 (Multi-Model Monopoly). *If $\varphi(\theta)$ is increasing, then the optimal token-*

budget menu is given by $\{((X_l(\theta), Z_l(\theta))_{l=1}^L, t(\theta))\}_\theta$*, where* $(X_l(\theta), Z_l(\theta))_{l=1}^L$ *are efficient token budgets that deliver quality* $Q^{\mathrm{m}}(\theta)$ *at prices* $t(\theta)$ *as defined in (26) and (27).*

The monopolist assigns each buyer type to a single model, with higher types using more capable models. All types w with the same aggregate type $\theta(w)$ choose the same item and use one model, $l^{\mathrm{m}}(\theta)$ as defined in (25), for all tasks. The quality schedule exhibits discrete jumps at model-switching thresholds: a buyer at the margin between two models jumps to a strictly higher quality when upgrading. OpenAI's tier structure, which reserves its most compute-intensive reasoning model for the highest-paying subscribers, provides a direct empirical counterpart; see the discussion in Section 6.

5.3 Leader-Fringe Competition

We now study competition between a proprietary leader and an open-source competitive fringe. The key economic forces are: (i) the fringe provides an outside option whose value depends on the buyer's type; (ii) for intermediate types, the leader must provide enough tokens to deter fringe top-up; (iii) for high types, the leader acts as an unconstrained monopolist. The resulting allocation has three distinct regions, generating a richer pattern of distortions than either the single-model monopoly or the multi-model monopoly.

Specifically, we assume that there is a single leader, corresponding to the highest-capability model ($l = L$) in the previous section, and a continuum of firms in a competitive fringe that sell their tokens at fixed per-token prices equal to marginal costs. We assume that the leader possesses a proprietary model characterized by the aggregate cost parameter $c_L > 0$, returns to intensity $\sigma_L = \sigma > 0$, and returns to fine-tuning $\hat{\sigma}_L > 0$, such that $\sigma + \hat{\sigma}_L < 1$. The fringe possesses an open-source model characterized by the lower aggregate cost parameter $c_F \in [0, c_L)$, the same returns to intensity $\sigma_F = \sigma$, and lower returns to fine-tuning $\hat{\sigma}_F \in [0, \hat{\sigma}_L)$.

The buyer has a private type w. By Lemma 2, only the aggregate type $\theta(w)$ matters, so we take θ as a primitive and assume that θ is distributed according to F with strictly positive density f everywhere on $[0, 1]$. The buyer can multi-home and can buy at most one item from the leader.

We want to solve the leader's problem, which can be viewed as a monopolistic design of an optimal menu (q, t) subject to the competitive pressure from the fringe. For notational simplicity, define the *quantities*:

$$q_L \triangleq Q_L^{1/(\sigma+\hat{\sigma}_L)} = g_L(X_{L1}, \ldots, X_{LJ}, Z_{L1}, \ldots, Z_{LK})^{1/(\sigma+\hat{\sigma}_L)},$$
$$q_F \triangleq Q_F^{1/(\sigma+\hat{\sigma}_F)} = g_F(X_{F1}, \ldots, X_{FJ}, Z_{F1}, \ldots, Z_{FK})^{1/(\sigma+\hat{\sigma}_F)}.$$

The payoff of type θ from having purchased quantity q_L from the leader is

$$\max_{q_F \geq 0} \ \theta(q_L^{(\sigma+\hat{\sigma}_L)/\sigma} + q_F^{(\sigma+\hat{\sigma}_F)/\sigma})^{\sigma} - c_F q_F, \tag{28}$$

with a (type-dependent) outside option corresponding to $q_L = 0$. The first-order condition for the buyer's problem of purchasing fringe quantity is

$$\theta(\sigma + \hat{\sigma}_F)(q_L^{(\sigma+\hat{\sigma}_L)/\sigma} + q_F^{(\sigma+\hat{\sigma}_F)/\sigma})^{\sigma-1} q_F^{\hat{\sigma}_F/\sigma} = c_F. \tag{29}$$

In general, (29) does not admit a closed-form solution. It does so in the case of $\hat{\sigma}_F = 0$. Specifically, define

$$\hat{q}(\theta) \triangleq \left(\frac{\theta\sigma}{c_F}\right)^{\sigma/((1-\sigma)(\sigma+\hat{\sigma}_L))}, \quad \psi(\theta) \triangleq \theta^{\frac{1}{1-\sigma}}(1-\sigma)\left(\frac{\sigma}{c_F}\right)^{\sigma/(1-\sigma)}. \tag{30}$$

If the quantity purchased from the leader is $q_L < \hat{q}(\theta)$, the buyer will purchase fringe tokens to achieve the optimal total quantity $\hat{q}(\theta)$; if $q_L > \hat{q}(\theta)$, the buyer will single-home with the leader. The buyer's outside option utility corresponding to $q = 0$ is $\psi(\theta)$. We then obtain the following characterization of the buyer's demand for the leader's quantity.

Lemma 4 (Buyer's Payoff). *The leader's problem is equivalent to one in which the buyer has a zero outside option and the following payoff:*

$$u(\theta, q) = \begin{cases} c_F q^{(\sigma+\hat{\sigma}_L)/\sigma}, & \text{if } q < \hat{q}(\theta), \\ \theta q^{\sigma+\hat{\sigma}_L} - \psi(\theta), & \text{if } q \geq \hat{q}(\theta). \end{cases} \tag{31}$$

The payoff $u(\theta, q)$ in Lemma 4 is continuous, differentiable, and increasing in both parameters. In the region $q < \hat{q}(\theta)$, it is convex in q and independent of θ. In the region $q > \hat{q}(\theta)$, it is concave in q and supermodular in q and θ.

The two-regime structure has a clear economic interpretation. In the first regime ($q < \hat{q}(\theta)$), the buyer supplements leader tokens with fringe tokens to achieve the optimal total quantity, so her marginal value of additional leader tokens is pinned down by the fringe price c_F; the leader faces a perfectly elastic residual demand. In the second regime ($q \geq \hat{q}(\theta)$), the buyer single-homes with the leader, and her value is the standard concave payoff minus the outside option $\psi(\theta)$ she forgoes by not using the fringe. The transition between regimes is where the leader's competitive constraint binds.

The leader's problem is an instance of screening with a type-dependent outside option (Jullien, 2000); the reformulation (31) absorbs this outside option into the buyer's payoff.

An optimal menu can then be characterized by standard methods.[10] Indeed, $u(\theta, q)$ satisfies the (weak) single-crossing constraints: $u_{\theta q}(\theta, q) = 0$ if $q < \hat{q}(\theta)$ and $u_{\theta q}(\theta, q) > 0$ if $q \geq \hat{q}(\theta)$. Allocation $q(\theta)$ is implementable only if it is increasing whenever $q \geq \hat{q}(\theta)$, and the profit associated with allocation q is:

$$\Pi(q) = \int_0^1 \left(u(\theta, q(\theta)) - c_L q(\theta) - \frac{1 - F(\theta)}{f(\theta)} u_\theta(\theta, q(\theta)) \right) f(\theta) d\theta. \tag{32}$$

In regular environments, this profit can be maximized pointwise. To this end, denote by $q^{\mathrm{m}}(\theta)$ the optimal monopoly quantity in the absence of the competitive fringe

$$q^{\mathrm{m}}(\theta) = \left(\frac{(\sigma + \hat{\sigma}_L)\varphi(\theta)}{c_L} \right)^{1/(1-\sigma-\hat{\sigma}_L)}. \tag{33}$$

Proposition 7 (Leader-Fringe Competition). *Let F satisfy a monotone hazard rate and $\hat{\sigma}_F = 0$. Then, there exist $\underline{\theta}$ and $\overline{\theta}$, $\underline{\theta} \leq \overline{\theta}$, such that (i) for $\theta \leq \underline{\theta}$, $q(\theta) = 0$, (ii) for $\theta \in (\underline{\theta}, \overline{\theta})$, $q(\theta) = \hat{q}(\theta)$ given by (30), and (iii) for $\theta \geq \overline{\theta}$, $q(\theta) = q^{\mathrm{m}}(\theta)$ given by (33).*

Under the optimal mechanism, low types purchase exclusively from the fringe. All higher types purchase exclusively from the leader. Out of those, the midline types are on the margin of whether to buy from the fringe, whereas the highline types strictly prefer to purchase from the leader. Depending on parameters, the midline region may not exist, $\underline{\theta} = \overline{\theta}$, or the highline region may not exist, $\overline{\theta} > 1$.

The three-region structure reflects three distinct economic regimes. In the *fringe-only* region ($\theta \leq \underline{\theta}$), the buyer's willingness to pay for the leader's superior fine-tuning capability is too low to justify adoption. In the *deterrence* region ($\underline{\theta} < \theta < \overline{\theta}$), the leader offers exactly enough tokens to make the buyer indifferent between single-homing with the leader and supplementing with the fringe; this pins the leader's quantity to $\hat{q}(\theta)$, which rises with type. In the *monopoly* region ($\theta \geq \overline{\theta}$), the buyer's value is high enough that fringe competition no longer constrains the leader, who reverts to the standard downward-distorted monopoly quantity $q^{\mathrm{m}}(\theta)$. The novel region is the deterrence region, which has no analog in either the single-model monopoly or the multi-model monopoly.

[10] See Calzolari and Denicolò (2013) for a related analysis of nonlinear competition between a dominant firm and a competitive fringe under private information about demand.

Comparison with Efficient Allocation It is instructive to compare the allocation under leader-fringe competition with the efficient benchmark. By Proposition 5, defining

$$u_l^*(\theta) = (1-\sigma_l-\hat{\sigma}_l)\theta^{1/(1-\sigma_l-\hat{\sigma}_l)}\left(\frac{\sigma_l+\hat{\sigma}_l}{c_l}\right)^{(\sigma_l+\hat{\sigma}_l)/(1-\sigma_l-\hat{\sigma}_l)},$$

type θ uses the fringe model if and only if $u_F^*(\theta) > u_L^*(\theta)$. The indifference type at which $u_F^*(\theta) = u_L^*(\theta)$ is (recall that $\sigma_L = \sigma_F = \sigma$ and $\hat{\sigma}_F = 0$)

$$\hat{\theta} = \left(\frac{1-\sigma}{1-\sigma-\hat{\sigma}_L}\right)^{(1-\sigma-\hat{\sigma}_L)(1-\sigma)/\hat{\sigma}_L}\left(\frac{\sigma}{c_F}\right)^{\sigma(1-\sigma-\hat{\sigma}_L)/\hat{\sigma}_L}\left(\frac{c_L}{\sigma+\hat{\sigma}_L}\right)^{(\sigma+\hat{\sigma}_L)(1-\sigma)/\hat{\sigma}_L}. \tag{34}$$

If $\theta < \hat{\theta}$, then the type uses the fringe model: $q_F(\theta) = (\theta\sigma/c_F)^{1/(1-\sigma)}$ and $q_L(\theta) = 0$. If $\theta \geq \hat{\theta}$, then the type uses the leader model: $q_F(\theta) = 0$ and $q_L(\theta) = (\theta(\sigma+\hat{\sigma}_L)/c_L)^{1/(1-\sigma-\hat{\sigma}_L)}$.

Comparing this with the fringe-competition allocation characterized in Proposition 7, we see two main differences: First, we see that the "highline" leader quantity $q^{\mathrm{m}}(\theta)$ is distorted downward from efficiency because $\varphi(\theta)$ is smaller than θ. This is natural, since in that range the leader is unconstrained by the fringe and acts as a monopolist. Second, there is an additional *extensive margin distortion*, because $\hat{\theta} < \underline{\theta}$.[11]

Comparison with Multi-Model Monopoly Another natural benchmark is the case of a monopolist possessing the leader and fringe models. By Proposition 6, the monopolist effectively faces a Mussa-Rosen problem with the buyer's payoff:

$$u(\theta, q_L, q_F) = \theta(q_F + q_L^{(\sigma+\hat{\sigma}_L)/\sigma})^{\sigma}.$$

and will be supplying an efficient allocation with respect to the virtual type. If $\theta < \theta^{\mathrm{m}}$, where $\varphi(\theta^{\mathrm{m}}) \triangleq \hat{\theta}$ and $\hat{\theta}$ is defined at (34), then the type uses the fringe model and $q_F(\theta) = (\varphi(\theta)\sigma/c_F)^{1/(1-\sigma)}$ and $q_L(\theta) = 0$. If $\theta \geq \theta^{\mathrm{m}}$, then the type uses the leader model with $q_F(\theta) = 0$ and $q_L(\theta) = (\varphi(\theta)(\sigma+\hat{\sigma}_L)/c_L)^{1/(1-\sigma-\hat{\sigma}_L)}$.

Relative to the efficiency benchmark, the allocation is distorted downward within each model, except at the very top. In addition, fewer types consume the leader model.

Relative to the leader-fringe environment of Proposition 7, the integrated multi-model monopolist internalizes the fringe technology and therefore does not face a competitive outside option. This has two implications. First, the fringe allocation is no longer pinned down by marginal-cost pricing: types that use the fringe model receive $q_F(\theta) = (\varphi(\theta)\sigma/c_F)^{1/(1-\sigma)}$

[11] Indeed, $\frac{\hat{\theta}}{\underline{\theta}} = \left[\left(\frac{1-\sigma}{1-\sigma-\hat{\sigma}_L}\right)^{1-\sigma-\hat{\sigma}_L}\left(\frac{\sigma}{\sigma+\hat{\sigma}_L}\right)^{\sigma+\hat{\sigma}_L}\right]^{(1-\sigma)/\hat{\sigma}_L}$, and thus, by the strict concavity of the logarithm and Jensen's inequality, $\ln\left(\frac{\hat{\theta}}{\underline{\theta}}\right) = \frac{1-\sigma}{\hat{\sigma}_L}\left[(1-\sigma-\hat{\sigma}_L)\ln\left(\frac{1-\sigma}{1-\sigma-\hat{\sigma}_L}\right) + (\sigma+\hat{\sigma}_L)\ln\left(\frac{\sigma}{\sigma+\hat{\sigma}_L}\right)\right] < 0$.

(and are excluded whenever $\varphi(\theta) < 0$), whereas under leader-fringe competition the corresponding types purchase the efficient quantity $(\theta\sigma/c_F)^{1/(1-\sigma)}$ from the fringe. Second, the "midline" region $\theta \in (\underline{\theta}, \overline{\theta})$ in which the leader supplies $q = \hat{q}(\theta)$ to deter top-up purchases from the fringe disappears. Because the monopolist controls both models, she uses the fringe model directly as the low-quality product in the Mussa-Rosen menu and assigns each type to a single model with a single cutoff θ^{m}. Finally, once the fringe constraint is slack, the intensive-margin provision coincides and equals the monopoly quantity $q^{\mathrm{m}}(\theta)$; differences between the two environments are therefore concentrated among low and intermediate types.[12]

Figure 1 illustrates the optimal allocation and distortions in a uniform example. All calculations for this example are in Appendix A.

The top panel plots the leader quantity $q(\theta)$, and the bottom panel plots the fringe quantity $q_F(\theta)$. Four cutoffs, ordered $\hat{\theta} < \underline{\theta} < \theta^{\mathrm{m}} < \overline{\theta}$, partition the type space. The efficient allocation (solid) features a single switch at $\hat{\theta}$: types below $\hat{\theta}$ use only the fringe, types above use only the leader. The leader–fringe allocation (dotted) shifts this switch rightward to $\underline{\theta}$, reflecting the extensive-margin distortion. Between $\underline{\theta}$ and $\overline{\theta}$, the leader supplies the deterrence quantity $\hat{q}(\theta)$, which lies strictly below the efficient leader quantity. Above $\overline{\theta}$, the fringe constraint is slack and the leader reverts to the monopoly quantity $q^{\mathrm{m}}(\theta)$, which coincides with the integrated monopolist's allocation (dashed). In the bottom panel, the efficient and leader–fringe curves for q_F coincide over their respective ranges, since the fringe prices at marginal cost in both cases; the difference is that fringe usage persists up to $\underline{\theta}$ rather than $\hat{\theta}$. The integrated monopoly distorts fringe provision downward and switches to the leader at the later cutoff θ^{m}.

6 LLM Pricing in Practice

A striking feature of current LLM pricing is that different market segments implement each of the optimal mechanisms in our theoretical framework: consumer subscriptions implement the nonlinear menus of Sections 4 and 5; model aggregators implement the committed-spend mechanisms of Section 4.2; and developer APIs implement the constrained-efficient linear pricing of Corollary 1.

Our main findings are as follows. The two largest proprietary providers, Anthropic and OpenAI, offer remarkably similar price points ($20 and $200) but implement fundamentally different screening mechanisms. Anthropic screens through quantity alone, holding model ac-

[12] Calzolari and Denicolò (2015) show that a dominant firm with a competitive advantage can profitably impose exclusive dealing on privately informed buyers; the mechanism here is analogous, though exclusivity arises from the optimal nonlinear tariff rather than from contractual restrictions.

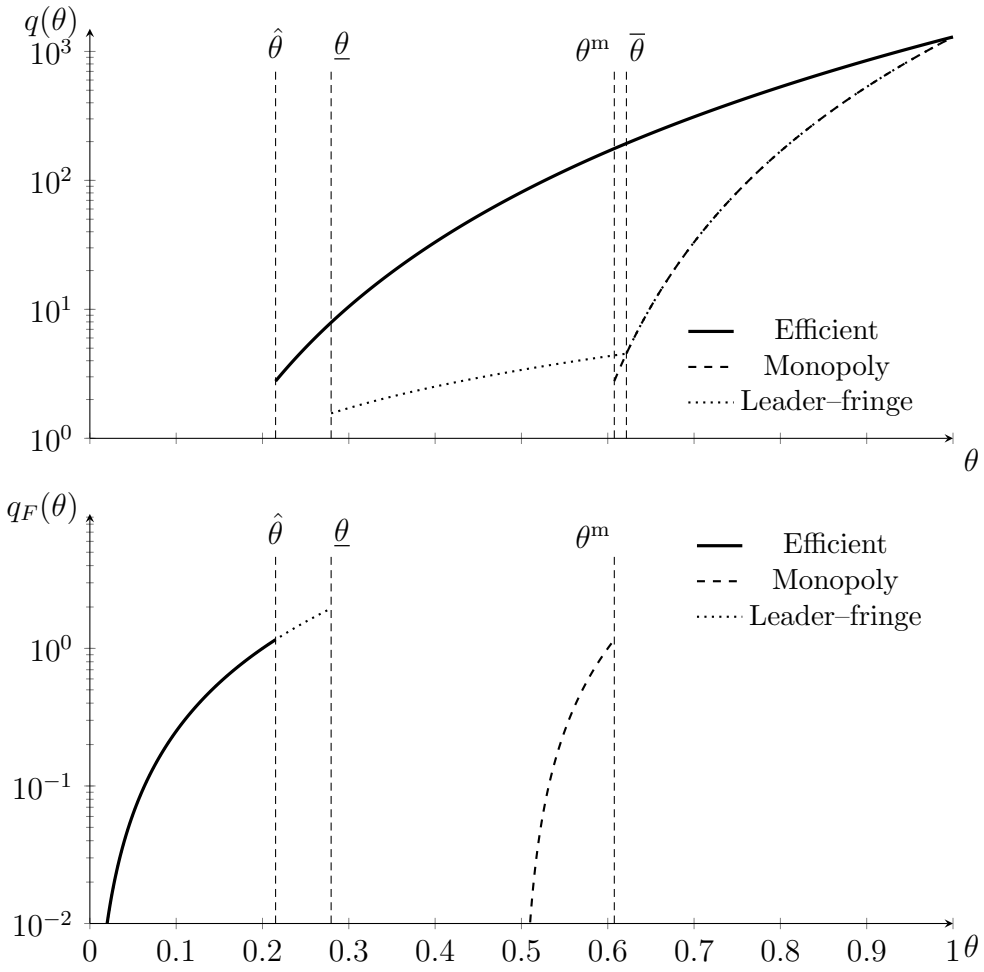


Figure 1: Leader-Fringe allocations across different regimes. Example with $\theta \sim U[0,1]$, $\sigma = 1/2$, $\hat{\sigma}_L = 1/4$, $c_F = 1/10$, $c_L = 1/8$.

cess constant across paid tiers, matching the token-budget mechanism of Proposition 3. OpenAI screens through both quantity and model access, reserving its most compute-intensive reasoning model for the highest tier, matching the multi-model menu of Proposition 6. This contrast illustrates the distinction between the single-model screening of Section 4 and the multi-model versioning of Section 5. Model aggregators, which resell access to upstream providers through a single interface, implement the committed-spend mechanisms of Section 4.2: Quora's Poe enforces a hard budget cap (maximum spend), while GitHub Copilot allows overage purchasing at linear prices (minimum spend). Finally, API pricing across all major providers is linear in tokens with no volume discounts, resembling the constrained-efficient benchmark of Corollary 1 rather than any profit-maximizing mechanism, consistent with providers prioritizing market share over rent extraction in the developer segment.

6.1 Anthropic

Anthropic's consumer pricing for Claude best illustrates the token-budget mechanism of Section 4. Anthropic offers the same model family across all paid tiers, with differentiation occurring through usage allocations. As in the optimal menu characterized in Proposition 3, users with higher aggregate types θ select items with larger token budgets while consuming the same underlying technology.

Figure 2 summarizes the structure of paid tiers as of January 2026. All paid tiers, namely Pro ($20/month), Max 5x ($100/month), and Max 20x ($200/month), grant access to Haiku 4.5, Sonnet 4.5, and Opus 4.5. The key distinction is the quantity of compute: Anthropic measures usage limits in computational intensity. For example, Opus queries consume resources approximately 5× faster than Sonnet. This aligns with Proposition 3, where the optimal mechanism defines users' budgets in total tokens consumed across classes. Although Anthropic offers multiple models, all paid tiers grant access to the same set, so the relevant screening dimension is quantity rather than model access. The different depletion rates across models (e.g., Opus consuming resources 5× faster than Sonnet) function as heterogeneous per-unit costs within a single compute-budget mechanism, analogous to the token-class costs c_j in Proposition 3.

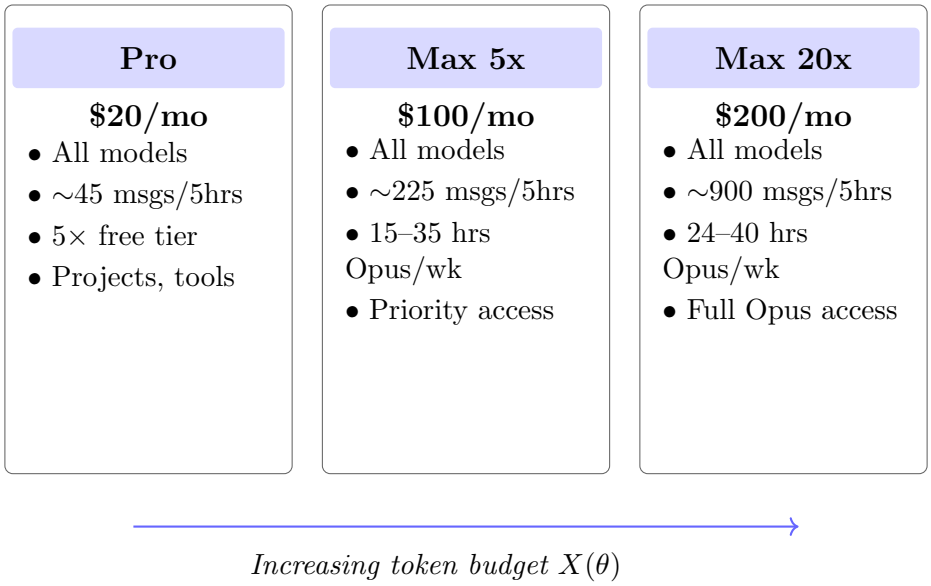


Figure 2: Anthropic subscription tiers (January 2026). Model access is constant across paid tiers; differentiation occurs through usage allocations.

6.2 OpenAI

Unlike Anthropic's tier structure, which differentiates mostly through constraints on usage, OpenAI's ChatGPT pricing differentiates through both quantity and exclusive model access: higher tiers grant access to more capable models and relax usage constraints. This joint screening aligns with the multi-model monopolist of Section 5.2.

Figure 3 summarizes OpenAI's tier structure as of January 2026. The Free tier provides limited GPT-4o access with automatic downgrade to GPT-4o-mini when capacity is constrained. Plus ($20/month) expands to approximately 80 GPT-4o messages per 3-hour window and adds reasoning models (o3, o4-mini) with a separate weekly limit of roughly 100 queries. Pro ($200/month) removes most constraints and provides exclusive access to o1-pro, OpenAI's most compute-intensive reasoning model. Thus, both Anthropic and OpenAI converge on similar price points ($20 standard, $200 premium), but implement very different screening strategies.

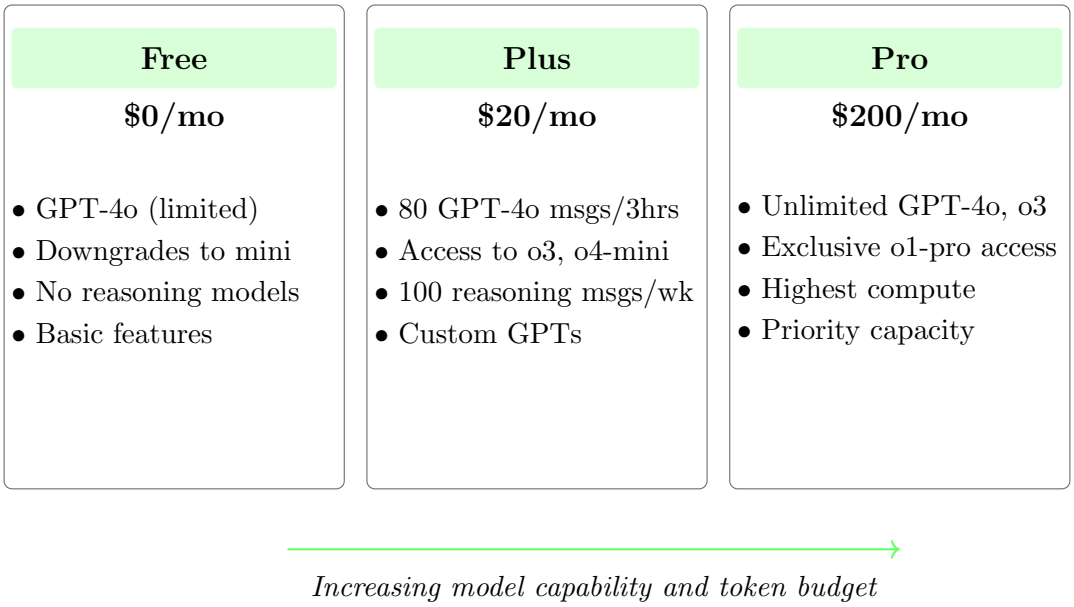


Figure 3: OpenAI ChatGPT subscription tiers (January 2026). Higher tiers grant access to more capable models and larger usage allocations.

In Proposition 6, we showed that when models differ in cost curvature, the optimal mechanism assigns higher-capability models to higher buyer types. OpenAI's decision to reserve its o1-pro model for the highest tier is consistent with this prediction. In consumer subscriptions, the relevant cost differences across models may reflect inference-time compute intensity rather than user fine-tuning, but the qualitative implication of a monotone assignment with discrete jumps at upgrade thresholds is the same.

6.3 Model Aggregators: GitHub and Quora

A number of AI platforms, including GitHub Copilot and Quora's Poe, do not develop their own models. Instead, they aggregate models produced by others (OpenAI, Anthropic, Google, Mistral, and others) and sell access to users through a single interface. A common contractual arrangement is one whereby subscribers pay a monthly fee, receive a budget of platform-specific credits, and allocate those credits across the available models. Selecting a more capable model typically depletes the budget faster. We now describe Quora's and GitHub's platforms in detail and then relate their pricing structures to the indirect implementation of the optimal mechanism discussed in Section 4.2.

Quora-Poe Poe offers subscribers access to over 100 AI models through a unified chat interface. A subscriber pays a fixed monthly fee and receives an allocation of "compute points," which serve as the platform's internal currency. Each model is priced in points per message at a rate that reflects its inference cost.[13]

Table 1 summarizes Poe's tier structure. The entry-level Basic plan provides 300,000 points per month for $5; the top-tier Premium plan provides 12,500,000 points for $250. Above the Basic tier, the effective price per million points is constant at $20, so that higher tiers simply offer a proportionally larger budget at a fixed unit rate. Table 2 reports point costs for selected models, illustrating the wide dispersion across model classes.

Tier	Monthly Price	Points/Month	Eff. $/M Points
Basic	$5	300,000	$16.67
Standard	$20	1,000,000	$20.00
Pro	$50	2,500,000	$20.00
Advanced	$100	5,000,000	$20.00
Premium	$250	12,500,000	$20.00

Table 1: Poe subscription tiers (2025).

The connection to the maximum-spend framework of Section 4.2 is immediate. The monthly subscription fee corresponds to the transfer T_n; the point allocation corresponds to the budget B_n; and the per-model point costs correspond to marginal-cost pricing of different token classes. The subscriber chooses how to allocate the budget taking the marginal prices into account. Finally, once the budget is exhausted, access to all models is suspended until the next billing cycle, and points do not roll over. Thus, Poe's menu is best captured by a maximum-spend mechanism.

[13] For instance, a message to GPT-4o-mini costs 9 points, while a message to Claude Opus 4 costs 4,105 points, a ratio of roughly 450×.

Class	Model	Points/Message
Budget	GPT-4o-mini	9
	Gemini-2.0-Flash	9
Mid-tier	GPT-4o	224
	Claude-3.5-Sonnet	276
Frontier	Claude-3-Opus	1,697
	Claude-Opus-4	4,105

Table 2: Poe per-model point costs (2025).

GitHub Copilot GitHub Copilot provides AI-assisted coding within a developer's integrated environment. Like Poe, Copilot offers multiple pricing tiers, each bundling a monthly fee with a budget of "premium requests" that the developer allocates across model calls. Table 3 summarizes the plan structure.

Plan	Monthly Fee	Premium Requests	Notes
Free	\$0	50	Limited quota, basic completions
Pro	\$10	300	Unlimited completions, chat, IDE support
Pro+	\$39	1,500	Larger quota, advanced models
Business	\$19/user	300/user	Org controls, policy management
Enterprise	\$39/user	1,000/user	Enterprise features

Table 3: GitHub Copilot pricing tiers and premium request budgets (2026).

The mechanism by which Copilot meters usage differs slightly from Poe's point system but serves the same function. Rather than assigning each model a point cost per message, Copilot assigns each model a multiplier that determines how many premium requests a single interaction consumes. A set of baseline models, including GPT-5 mini, GPT-4.1, and GPT-4o, carry a multiplier of 0 on paid plans: interactions with these models are included at no additional budget cost. Efficient reasoning models such as Claude Haiku 4.5 and o4-mini carry multipliers of 0.25–0.33. Standard models such as Claude Sonnet 4.x and GPT-5.x carry a multiplier of 1. At the frontier, Claude Opus 4.5 carries a multiplier of 3, and GPT-4.5 carries a multiplier of 50.

This mechanism matches the framework of Section 4.2 quite closely. The plan fee corresponds to T_n; the monthly premium-request allocation corresponds to B_n; and the model multipliers play the role of heterogeneous marginal costs. A distinctive feature of Copilot is that it extends this budget mechanism with an overage option. When a developer (i.e., a buyer) exhausts her monthly allocation, Copilot allows continued use at a fixed charge

of $0.04 per premium request. The minimum-spend framework of Section 4.2 matches this feature rather well: each user commits to a certain amount of monthly spending, there are no refunds, but the developer can increase her consumption ex post at linear prices.[14]

Thus, both Poe and Copilot implement the core logic of the committed-spend mechanisms: a non-bankable budget, priced up front via a subscription fee, that the user freely allocates across inputs with heterogeneous per-unit costs. The key distinction lies in how each platform handles the overages. Poe enforces a hard cap: once points are exhausted, access is suspended until the next billing cycle. This corresponds exactly to the maximum-spend mechanism. Copilot, by contrast, allows the cap to be relaxed at a known marginal price (which varies across models through the multiplier system), effectively implementing a minimum-spend mechanism.

6.4 API Pricing

Alongside consumer subscriptions, every major LLM provider operates a developer-facing API through which applications can programmatically submit prompts and receive completions. A developer who calls the API pays per token, with separate rates for input tokens (the prompt) and output tokens (the model's response). There is no subscription fee, no bundled allocation, and no minimum commitment: the developer pays only for what she uses. Table 4 reports prices for selected models across three providers as of January 2025.

Provider	Model	Input ($/M)	Output ($/M)
OpenAI	GPT-4o-mini	0.15	0.60
OpenAI	GPT-4o	2.50	10.00
OpenAI	o1 (reasoning)	15.00	60.00
Anthropic	Claude 3.5 Haiku	0.80	4.00
Anthropic	Claude 3.5 Sonnet	3.00	15.00
Anthropic	Claude 3 Opus	15.00	75.00
Google	Gemini 1.5 Flash	0.075	0.30
Google	Gemini 1.5 Pro	1.25	5.00

Table 4: API pricing across major providers (January 2025). Prices per million tokens.

The pricing structure is notably uniform. Output tokens are priced at 3–5× input tokens across all providers, reflecting that output generation is sequential and autoregressive (each token requires a forward pass conditioned on all preceding tokens), whereas input encod-

[14] As in the theory, the overage charge interacts with the multiplier system: a query to a 1× model costs $0.04 in the overage region, while a query to a 3× model (Claude Opus 4.5) costs $0.12.

ing is parallelizable and therefore substantially cheaper per token. Pricing is strictly linear: there are no quantity discounts, volume commitments, or tiered rates in standard API access. Although outside our model, all providers offer batch discounts (typically 50%) for asynchronous processing and prompt-caching discounts (up to 90%) for repeated inputs.

This linear, pay-per-token structure stands in sharp contrast to the nonlinear menus that characterize consumer subscriptions. It corresponds instead to the constrained-efficient allocation of Corollary 1: when a provider faces capacity constraints but wishes to maximize total surplus rather than profit (for instance, to lock in consumers in anticipation of future monetization), the constrained-efficient allocation can be implemented via linear prices equal to marginal costs inflated by shadow costs. The absence of nonlinear screening in API pricing is consistent with providers currently prioritizing adoption over rent extraction.

The uniformity of the pricing format across providers is itself evidence of competitive pressure: if any single provider were to introduce nonlinear screening in the API segment, it would risk losing developers to competitors who maintain simpler, linear pricing.

Evidence from pricing dynamics is consistent with this interpretation. First, API prices have declined rapidly: GPT-4-class capability fell from \$30/\$60 per million tokens at launch (March 2023) to \$2.50/\$10 by August 2024, a 90% reduction in 16 months. Second, providers appear to cut prices in response to competitive launches rather than cost improvements; OpenAI's August 2024 reductions followed Claude 3.5 Sonnet and Llama 3.1 releases. Third, gross margins of 50–75% imply prices remain above marginal cost but are being compressed by competition. The entry of DeepSeek in January 2025 with reasoning-model pricing at \$0.55/\$2.19, versus OpenAI's \$15/\$60 for comparable capability, suggested that further compression is viable. See Demirer et al. (2025) for a comprehensive treatment of market share dynamics in the API segment.

7 Conclusion

A priori, the problem of pricing LLM access appears intractable: users have high-dimensional private information, the allocation space is also high-dimensional, and the user's hidden allocation of tokens across tasks introduces moral hazard. In this paper, we have shown that homogeneity of the gain function generates a sufficient-statistic reduction that makes the problem solvable. Users' high-dimensional type profiles collapse to a scalar aggregate type; the seller's problem reduces to one-dimensional screening; and the optimal mechanism admits simple implementations (committed-spend contracts and two-part tariffs) that correspond precisely to the pricing structures observed at leading providers.

While we have phrased the analysis in terms of large language models, the framework applies more broadly to any setting in which a provider sells access to a general-purpose technology with multiple input classes and heterogeneous users. Cloud computing, where providers offer a multiplicity of services with both horizontal and vertical differentiation, is a natural example. In both settings, the core pricing problem is how to allocate and monetize costly computing resources, particularly when operating under capacity constraints.

Several extensions would enrich the analysis. First, we assumed that all tasks are homogeneous in their use of input, output, and fine-tuning tokens, and differ only vertically in how valuable a given task is to the buyer. Allowing the gain function parameters to vary across tasks would break the aggregation result. Understanding how the optimal mechanism changes when the scalar sufficient statistic no longer exists is a challenging but important open question, especially in the context of competing specialized models.

Second, we assumed that all buyer types have fine-tuning data readily available. In practice, buyers face constraints on data availability or may be unwilling to share data with the provider because of privacy, compliance, or intellectual-property concerns. A data-poor buyer may then substitute toward inference-time tokens, while a data-rich buyer substitutes toward fine-tuning tokens. When data availability is the user's private information, the provider's problem involves screening on two dimensions (aggregate type and data availability), which may require new techniques.

Third, LLM platforms exhibit network effects and data externalities that our static framework abstracts from. A provider that attracts more users may improve its model through fine-tuning, reinforcement learning from human feedback, or simply from the revenue that funds further training. These dynamic complementarities between adoption and model quality could substantially reshape optimal pricing, potentially justifying the aggressive below-average-cost pricing observed in the API segment not just as a means to capture market share but as investment in model improvement.

Fourth, our analysis treats each provider as either a monopolist or a leader facing a competitive fringe. A full oligopoly analysis with multiple differentiated proprietary models would capture the strategic interactions among Anthropic, OpenAI, Google, and others that increasingly shape the market. The tractability of our framework through the reduction to one-dimensional types suggests that such an extension may be feasible and would yield further predictions about equilibrium pricing and product differentiation.

The LLM industry is at an inflection point between growth-oriented pricing and profit-maximizing pricing. Our framework provides a theoretical foundation for understanding the mechanisms through which this transition will unfold, and for evaluating whether the resulting market structure efficiently allocates this critical economic input.

A Proofs and Derivations

Proof of Proposition 1 For any z and i, the optimal allocation of inference tokens on task i solves

$$\max_{x_{ij}\geq 0} \; w_i\Psi(x_i)\Phi(z) - \sum_{j=1}^{J} c_j x_{ij}.$$

If $w_i = 0$ or $\Phi(z) = 0$, then $x_i = 0$. Otherwise, by the Inada assumption, the problem admits an interior solution, which satisfies the system of J first-order conditions:

$$w_i\nabla\Psi(x_i)\Phi(z) = c. \tag{35}$$

By (35), for all i, $\nabla\Psi(x_i)$ belongs to a ray with a direction c. Because Ψ is homogeneous and strictly concave, the ray in the space of gradients corresponds to a ray in the space of tokens.[15] Thus, for each i, any optimal x_i can be written as:

$$x_i = dy_i,$$

where $d \in \mathbb{R}^J_+$ is the unique vector that solves $\nabla\Psi(d) = c$.

Therefore, by the homogeneity of Ψ, we have $\nabla\Psi(x_i) = y_i^{\sigma-1}\nabla\Psi(d)$, and (35) can be rewritten as:

$$w_i y_i^{\sigma-1}\nabla\Psi(d)\Phi(z) = \nabla\Psi(d),$$

which pins down the optimal scale as:

$$y_i = w_i^{\frac{1}{1-\sigma}}\Phi(z)^{\frac{1}{1-\sigma}}.$$

The resulting total surplus ignoring fine-tuning costs is:

$$\begin{aligned}\int_0^1 w_i\Psi(x_i)\Phi(z)di - \sum_{j=1}^{J} c_j\int_0^1 x_{ij}di &= \int_0^1 w_i y_i^{\sigma}\Psi(d)\Phi(z) - y_i(d\cdot c)\,di \\ = \int_0^1 w_i^{\frac{1}{1-\sigma}}\Phi(z)^{\frac{1}{1-\sigma}}(\Psi(d) - d\cdot c)\,di &= \theta(w)^{\frac{1}{1-\sigma}}\Phi(z)^{\frac{1}{1-\sigma}}(1-\sigma)\Psi(d),\end{aligned} \tag{36}$$

[15] By homogeneity, for all $x \in \mathbb{R}^J_+$ and $r > 0$, $\nabla\Psi(rx) = r^{\sigma-1}\nabla\Psi(x)$, so any token ray corresponds to a gradient ray. By strict concavity, if $x_1 \neq x_2$, then $\nabla\Psi(x_1) \neq \nabla\Psi(x_2)$, so distinct token rays map to distinct gradient rays.

where $\theta(w)$ is the *aggregate type* defined as

$$\theta(w) \triangleq \left(\int_0^1 w_i^{\frac{1}{1-\sigma}} di \right)^{1-\sigma}, \tag{37}$$

and the last equality in (36) follows from the definition of d and Euler's theorem for homogeneous functions: $d \cdot \nabla \Psi(d) = \sigma \Psi(d)$.

Similarly, the total amount of inference tokens of class j is:

$$\int_0^1 x_{ij} di = \int_0^1 d_j y_i di = \int_0^1 d_j w_i^{\frac{1}{1-\sigma}} \Phi(z)^{\frac{1}{1-\sigma}} di = \theta(w)^{\frac{1}{1-\sigma}} d_j \Phi(z)^{\frac{1}{1-\sigma}}.$$

This completes the proof. □

Proof of Corollary 1 The Lagrangian approach applies. Thus, associating Lagrange multipliers $\lambda_j \geq 0$ and $\hat{\lambda}_k \geq 0$ with the capacity constraints for the different token classes, the optimal solution solves

$$\max_{(x_i)_{i \in [0,1]}, z \geq 0} \int_0^1 w_i g(x_i, z) di - \sum_{j=1}^{J} (c_j + \lambda_j) \int_0^1 x_{ij} di - \sum_{k=1}^{K} (\hat{c}_k + \hat{\lambda}_k) z_k.$$

As such, the solution is an efficient allocation given the adjusted costs $c'_j \triangleq c_j + \lambda_j$ and $\hat{c}'_k \triangleq \hat{c}_k + \hat{\lambda}_k$. The result follows. □

Proof of Proposition 2 Consider the buyer-optimal inference token allocation across tasks for any given token budgets (X, Z) such that $\Phi(Z) > 0$:

$$\max_{x_{ij} \geq 0} \int_0^1 w_i \Psi(x_i) \Phi(Z) di, \quad \text{s.t.} \int_0^1 x_{ij} di = X_j \text{ for } j \in [J]. \tag{38}$$

Because $\Phi(Z)$ is a strictly positive constant, it can be ignored. Assume that for each j, $X_j > 0$; otherwise, $x_{ij} = 0$ for all i. Lagrangian approach applies. We associate Lagrange multipliers $\lambda = (\lambda_1, \ldots, \lambda_J)$ with the budget constraints of (38); the resulting first-order conditions are:

$$w_i \nabla \Psi(x_i) = \lambda.$$

By the same logic as in the efficiency analysis, these conditions imply that the relative ratios of input tokens are the same across all tasks:

$$x_i = y_i d,$$

for some $d \in \mathbb{R}^J_+$. The budget constraints imply that $(\int_0^1 y_i di)d = X$ and hence d is proportional to X and can be normalized to satisfy $\sum_{j=1}^J d_j = 1$, leading to

$$d = \frac{1}{\sum_{j=1}^J X_j} X. \tag{39}$$

The optimal scales y_i solve:

$$\max_{y_i \geq 0} \int_0^1 w_i y_i^\sigma \Psi(d) di, \quad \text{s.t.} \int_0^1 y_i di = \sum_{j=1}^J X_j.$$

At the optimum, the marginal gains $w_i \sigma y_i^{\sigma-1}$ are equalized across tasks, so:

$$y_i = w_i^{\frac{1}{1-\sigma}} \frac{\sum_{j=1}^J X_j}{\int_0^1 w_r^{\frac{1}{1-\sigma}} dr}.$$

The optimal buyer's payoff is:

$$\begin{aligned} U &= \int_0^1 w_i \left(w_i^{\frac{1}{1-\sigma}} \frac{\sum_{j=1}^J X_j}{\int_0^1 w_r^{\frac{1}{1-\sigma}} dr} \right)^\sigma \Psi(d)\Phi(Z) di \\ &= \left(\int_0^1 w_i^{\frac{1}{1-\sigma}} di \right)^{1-\sigma} \left(\sum_{j=1}^J X_j \right)^\sigma \Psi(d)\Phi(Z) = \theta(w)\Psi(X)\Phi(Z), \end{aligned}$$

where in the last equality we used the homogeneity of Ψ and (39). □

Proof of Lemma 1 The fact that $C(Q)$ is strictly increasing and strictly convex is immediate because $g(X, Z) = \Psi(X)\Phi(Z)$ is strictly increasing and strictly concave in its arguments whenever $g(X, Z) > 0$.

To show $C'_+(0) = 0$, observe that since $C(Q)$ is convex, $C'_+(0) = \inf_{Q>0} C(Q)/Q$. Pick any X_0 such that $\Psi(X_0) > 0$ and Z_0 as in Footnote 3. For $r > 0$, consider $(X_r, Z_r) = r(X_0, Z_0)$ and set $Q_r = g(X_r, Z_r)$. We have,

$$\frac{C(Q_r)}{Q_r} \leq \frac{c \cdot X_r + \hat{c} \cdot Z_r}{Q_r} = \frac{r(c \cdot X_0 + \hat{c} \cdot Z_0)}{r^\sigma \Psi(X_0)\Phi(rZ_0)} = \frac{c \cdot X_0 + \hat{c} \cdot Z_0}{\Psi(X_0)} \frac{1}{\Phi(rZ_0)/r^{1-\sigma}}.$$

By the choice of Z_0, $\Phi(rZ_0)/r^{1-\sigma} \to +\infty$ as $r \downarrow 0$, hence the right-hand side goes to 0. Since $Q_r \downarrow 0$, it follows that $\inf_{Q>0} C(Q)/Q = 0$. □

Proof of Proposition 3 The result is an application of Mussa and Rosen (1978) to the buyer's payoff θQ from Proposition 2 and the cost function $C(Q)$ from Lemma 1. The optimal quality schedule (12) and transfers (13) follow from standard arguments. □

Proof of Proposition 4 Fix an optimal direct mechanism $(X^*(\theta), Z^*(\theta), T^*(\theta))$ with the associated aggregate quality $Q^*(\theta) = \Psi(X^*(\theta))\Phi(Z^*(\theta))$.

Maximum-spend mechanism. Consider a maximum-spend mechanism with $p = (c, \hat{c})$, $T(\theta) = T^*(\theta)$, and $B(\theta) = C(Q^*(\theta))$. Under this mechanism, since tokens are priced at marginal cost, each type w purchasing budget $B(\theta')$ would optimally allocate tokens in a constrained-efficient way, thus deriving payoff $\theta(w)Q$ where $C(Q) = B(\theta')$, i.e., $Q = Q^*(\theta')$. Hence, the choice across items in this mechanism is equivalent to the choice in the direct mechanism, and this mechanism implements the allocation of the direct mechanism.

Two-part-tariff mechanism. When type w faces any two-part tariff, his optimal token allocation problem is equivalent to the efficient allocation problem (5) with prices taking the roles of costs. Therefore, all types w with the same $\theta(w)$ have the same payoff from any possible two-part tariff and can be treated as a single type.

Faced with a given item, the problem of type θ can be written as value-maximization-payment-minimization:

$$\max_{Q \geq 0} \ \theta Q - P(Q),$$

where

$$P(Q) \triangleq \min_{X_1,\ldots,X_J,Z_1,\ldots,Z_K \geq 0} \sum_{j=1}^{J} p_j X_j + \sum_{k=1}^{K} \hat{p}_k Z_k, \quad \text{s.t. } \Psi(X)\Phi(Z) = Q.$$

Consider the following two-part-tariff mechanism:

$$\begin{aligned} &p_j(\theta) = m(\theta)c_j, \ \hat{p}_k(\theta) = m(\theta)\hat{c}_k, \ j \in [J], \ k \in [K], \\ &p_0(\theta) = t(\theta) - m(\theta)C(Q(\theta)), \end{aligned}$$

where $C(Q)$ is as defined in (10), $Q(\theta)$ is as defined in (12), and $t(\theta)$ is as defined in (13). Under this mechanism, $\sum_{j=1}^{J} p_j X_j + \sum_{k=1}^{K} \hat{p}_k Z_k = m(\theta)(\sum_{j=1}^{J} c_j X_j + \sum_{k=1}^{K} \hat{c}_k Z_k)$. Thus, the buyer-optimal allocation is efficient, and $P(Q) = m(\theta)C(Q)$.

The rest of the argument is standard (e.g., Tirole (1988, pp. 154-157)). A buyer-optimal $Q(\theta)$ is determined by the first-order condition:

$$\theta = P'(Q(\theta)) = m(\theta)C'(Q(\theta)),$$

and thus the buyer-optimal level of quality satisfies the optimality condition:

$$\bar{\varphi}(\theta) = C'(Q(\theta)).$$

If $m(\theta)$ is decreasing, then the implied payment schedule in units of $C(Q)$ is concave. Thus, a menu of two-part tariffs with markups $m(\theta)$, constant across inputs, implements the desired quality schedule $Q(\theta)$, with each type θ consuming the optimal amount of tokens and paying the optimal total transfer.

Minimum-spend mechanism. Consider a minimum-spend mechanism that consists of $p(\theta) = r(\theta)(c, \hat{c})$ and $B(\theta) = T^*(\theta)$ for all types such that $Q^*(\theta) > 0$, where $r(\theta) = \frac{T^*(\theta)}{C(Q^*(\theta))}$. By construction, since prices are proportional to costs, type w, when consuming aggregate quality Q, would optimally derive value $\theta(w)Q$. Thus, the buyer's problem of optimal spending can be formulated in terms of Q. The restriction of minimum spend when reporting θ' can be written as $r(\theta')C(Q) \geq T^*(\theta')$, which simplifies to $Q \geq Q^*(\theta')$. Thus, the optimal payoff of type θ when reporting type θ' is

$$U(\theta, \theta') = \max_{Q \geq Q^*(\theta')} \theta Q - r(\theta')C(Q).$$

This mechanism implements the allocation of the direct mechanism if each type prefers to report truthfully and choose $Q = Q^*(\theta)$. Since $B(\theta)$ is increasing, for this to happen, $r(\theta)$ must be decreasing.

In fact, $r(\theta)$ being decreasing is not only necessary but also sufficient for implementability. To see this, note that $Q^*(\theta)$ is continuously increasing and $r(\theta) \geq 1$. Thus, the optimal deviation is always weakly below the maximum quality offered in the direct menu:

$$\arg \max_{Q \geq Q^*(\theta')} \theta Q - r(\theta')C(Q) \leq \arg \max_{Q \geq Q^*(\theta')} \overline{\theta} Q - C(Q) = Q^*(\overline{\theta}),$$

where $\overline{\theta}$ is the maximum point in $\operatorname{supp} F$. Moreover, if type θ deviates to type θ' and consumes quality $Q \leq Q^*(\overline{\theta})$, then there exists a type $\theta'' \geq \theta'$ such that $Q = Q^*(\theta'')$. Since r is decreasing, it is more profitable for type θ to deviate to type θ'' and consume $Q^*(\theta'')$. But such a deviation is equivalent to the deviation to θ'' in the direct menu and is therefore not profitable.

It remains to show that decreasing m implies decreasing r. Assume that m is decreasing. Since m and r are continuous, it suffices to show that their derivatives are negative at all

differentiable points. At those points, $m'(\theta) \leq 0$, which implies $\bar{\varphi}(\theta) - \bar{\varphi}'(\theta)\theta \leq 0$, and

$$r'(\theta) = \frac{T^{*\prime}(\theta)C(Q^*(\theta)) - T^*(\theta)C'(Q^*(\theta))Q^{*\prime}(\theta)}{C(Q^*(\theta))^2} = \frac{Q'(\theta)}{C(Q^*(\theta))^2}(\theta C(Q^*(\theta)) - T^*(\theta)\bar{\varphi}(\theta)),$$

where we used the fact that $T^{*\prime}(\theta) = \theta Q^{*\prime}(\theta)$ and $C'(Q^*(\theta)) = \bar{\varphi}(\theta)$. Since Q is increasing, it suffices to show that $\frac{\theta}{\bar{\varphi}(\theta)} \leq \frac{T^*(\theta)}{C(Q^*(\theta))}$, i.e., the markup in the two-part tariff implementation is lower than the markup in the minimum-spend implementation. To show this, observe that at the maximal excluded type θ_0, $\theta C(Q^*(\theta)) - T^*(\theta)\bar{\varphi}(\theta) = 0$, and for all types $\theta > \theta_0$,

$$\left(\frac{\theta C(Q^*(\theta)) - T^*(\theta)\bar{\varphi}(\theta)}{\theta}\right)' = \frac{T(\theta)}{\theta^2}(\bar{\varphi}(\theta) - \bar{\varphi}'(\theta)\theta) \leq 0. \tag{40}$$

Therefore, $\theta C(Q^*(\theta)) - T^*(\theta)\bar{\varphi}(\theta) \leq 0$, and r is decreasing. □

Proof of Lemma 2 For general, not necessarily identical, σ_l, the marginal gain from assigning an infinitesimal task with value w_i to model l is, by (18):

$$\delta_l(w_i) = \frac{1-\sigma_l}{(\int_{I_l} w_r^{1/(1-\sigma_l)} dr)^{\sigma_l}} Q_l w_i^{1/(1-\sigma_l)}. \tag{41}$$

Under an optimal task split, if a task with value w_i is assigned to a model l, then it must be that for all m, $\delta_l(w_i) \geq \delta_m(w_i)$. Because $\delta_m(w_i)/\delta_l(w_i)$ is increasing in w_i for all $m > l$, it follows that an optimal task-model allocation exists that forms a monotone partition, with models of higher l (and thus higher σ_l) being allocated the more valuable tasks.

Denoting by i_l the delimiting points, with $i_0 = 0$ and $i_L = 1$, the optimal partition is determined by equalizing the marginal gains, $\delta_l(w_{i_l}) = \delta_{l+1}(w_{i_l})$ for $l \in [L-1]$:

$$\frac{(1-\sigma_l)Q_l w_{i_l}^{1/(1-\sigma_l)}}{(1-\sigma_{l+1})Q_{l+1} w_{i_l}^{1/(1-\sigma_{l+1})}} = \frac{(\int_{I_l} w_i^{1/(1-\sigma_l)} di)^{\sigma_l}}{(\int_{I_{l+1}} w_i^{1/(1-\sigma_{l+1})} di)^{\sigma_{l+1}}}. \tag{42}$$

When $\sigma_l \equiv \sigma$, the optimality condition (42) simplifies to the following expression:[16]

$$\frac{Q_l}{Q_{l+1}} = \frac{(\int_{I_l^*} w_i^{1/(1-\sigma)} di)^{\sigma}}{(\int_{I_{l+1}^*} w_i^{1/(1-\sigma)} di)^{\sigma}}.$$

[16] In the general case of heterogeneous σ_l, the optimal task split and the resulting buyer's payoff do not admit tractable closed-form solutions. In particular, the fine details of the profile w could matter.

Thus,

$$\int_{I_l^*} w_i^{1/(1-\sigma)} di = \frac{Q_l^{1/\sigma}}{\sum_m Q_m^{1/\sigma}} \int_0^1 w_i^{1/(1-\sigma)} di,$$

or, equivalently,

$$\theta_l(I_l^*) = \left(\frac{Q_l^{1/\sigma}}{\sum_m Q_m^{1/\sigma}} \right)^{1-\sigma} \theta.$$

The collection $(\theta_l(I_l^*))_{l=1}^L$ determines an optimal (monotone) partition $(I_l^*)_{l=1}^L$.[17] All models with $Q_l > 0$ are employed at some tasks. The resulting optimal payoff is then (20). □

Proof of Lemma 3 Drop the l index. First, consider the case $\hat{\sigma} > 0$. Define the unit-cost indices

$$e \triangleq \min_{x \geq 0} \{c \cdot x : \Psi(x) \geq 1\}, \quad \hat{e} \triangleq \min_{z \geq 0} \{\hat{c} \cdot z : \Phi(z) \geq 1\}.$$

For strictly concave, increasing, homogeneous Ψ, Φ these are finite and attained, and the respective minimizers $d, \hat{d}$ are unique.

By homogeneity and the definition of e, the minimal cost to reach $\Psi(x) = \Psi_0$ is

$$\min_{x \geq 0} \{c \cdot x : \Psi(x) \geq \Psi_0\} = \Psi_0^{1/\sigma} e,$$

attained at $x = \Psi_0^{1/\sigma} d$. Similarly, $\min_{z \geq 0} \{\hat{c} \cdot z : \Phi(z) \geq \Phi_0\} = \Phi_0^{1/\hat{\sigma}} \hat{e}$, attained at $z = \Phi_0^{1/\hat{\sigma}} \hat{d}$.

The cost minimization reduces to

$$\min_{\Psi_0, \Phi_0 \geq 0} \Psi_0^{1/\sigma} e + \Phi_0^{1/\hat{\sigma}} \hat{e} \quad \text{s.t.} \quad \Psi_0 \Phi_0 = Q.$$

Straightforward calculation gives:

$$\Psi_0 = Q^{\frac{\sigma}{\sigma+\hat{\sigma}}} \left(\frac{\hat{e}\,\sigma}{e\,\hat{\sigma}} \right)^{\frac{\sigma\hat{\sigma}}{\sigma+\hat{\sigma}}}, \quad \Phi_0 = Q^{\frac{\hat{\sigma}}{\sigma+\hat{\sigma}}} \left(\frac{\hat{e}\,\sigma}{e\,\hat{\sigma}} \right)^{-\frac{\sigma\hat{\sigma}}{\sigma+\hat{\sigma}}},$$

which corresponds to the optimal token allocation:

$$x^*(Q) = Q^{\frac{1}{\sigma+\hat{\sigma}}} \left(\frac{\hat{e}\,\sigma}{e\,\hat{\sigma}} \right)^{\frac{\hat{\sigma}}{\sigma+\hat{\sigma}}} d, \quad z^*(Q) = Q^{\frac{1}{\sigma+\hat{\sigma}}} \left(\frac{\hat{e}\,\sigma}{e\,\hat{\sigma}} \right)^{-\frac{\sigma}{\sigma+\hat{\sigma}}} \hat{d}.$$

The resulting cost function is:

$$C(Q) = e\Psi_0^{1/\sigma} + \hat{e}\Phi_0^{1/\hat{\sigma}} = e^{\frac{\sigma}{\sigma+\hat{\sigma}}} \hat{e}^{\frac{\hat{\sigma}}{\sigma+\hat{\sigma}}} \left[\left(\frac{\sigma}{\hat{\sigma}} \right)^{\frac{\hat{\sigma}}{\sigma+\hat{\sigma}}} + \left(\frac{\hat{\sigma}}{\sigma} \right)^{\frac{\sigma}{\sigma+\hat{\sigma}}} \right] Q^{\frac{1}{\sigma+\hat{\sigma}}} = C_0 Q^{\frac{1}{\sigma+\hat{\sigma}}}.$$

[17] Non-monotone partitions may also be optimal, but $\theta_l(I_l^*)$ are uniquely determined.

The case $\hat{\sigma} = 0$ is analogous and simpler because in that case $\Phi(z) \equiv \Phi_0 > 0$ and fine-tuning can be ignored. The resulting cost function is

$$C(Q) = e\left(\frac{Q}{\Phi_0}\right)^{\frac{1}{\sigma}} = C_0 Q^{\frac{1}{\sigma}}.$$

This completes the proof. □

Proof of Proposition 5 By Lemma 3, each model's surplus $\theta Q - \kappa_l Q^{1/(\sigma+\hat{\sigma}_l)}$ is concave in Q and can be maximized in closed form. The efficient allocation selects the model that yields the highest surplus, yielding (22)–(23). Single-model usage follows because the cost function (21) is concave in $Q^{1/\sigma}$. □

Proof of Proposition 6 The result follows by pointwise maximization of virtual surplus with $\varphi(\theta)$ in place of θ in Proposition 5. Monotonicity of φ ensures that the pointwise solution $Q^{\mathrm{m}}(\theta) = Q^*(\varphi(\theta))$ is increasing and therefore implementable. □

Proof of Lemma 4 When $\hat{\sigma}_F = 0$, (29) simplifies into:

$$q_L^{(\sigma+\hat{\sigma}_L)/\sigma} + q_F = \left(\frac{\theta\sigma}{c_F}\right)^{1/(1-\sigma)}. \tag{43}$$

The outside option corresponding to $q_L = 0$ is

$$\psi(\theta) = \theta^{\frac{1}{1-\sigma}}(1-\sigma)\left(\frac{\sigma}{c_F}\right)^{\sigma/(1-\sigma)}. \tag{44}$$

The formulation (31) follows. □

Proof of Proposition 7 Denote the expression in the brackets of (32) by $\Pi(\theta, q)$ and observe that

$$\Pi(\theta, q(\theta)) = \begin{cases} c_F q(\theta)^{(\sigma+\hat{\sigma}_L)/\sigma} - c_L q(\theta), & \text{if } q(\theta) < \hat{q}(\theta), \\ \varphi(\theta) q(\theta)^{\sigma+\hat{\sigma}_L} - c_L q(\theta) - \psi(\theta) + \frac{1-F(\theta)}{f(\theta)}\psi'(\theta), & \text{if } q(\theta) > \hat{q}(\theta). \end{cases} \tag{45}$$

Consider the pointwise maximization of $\Pi(\theta, q(\theta))$. In the region $q(\theta) \in [0, \hat{q}(\theta)]$, $\Pi(\theta, q(\theta))$ is convex in $q(\theta)$ and therefore attains its maximum at a corner, $q(\theta) = 0$ or $q(\theta) = \hat{q}(\theta)$.

Direct calculation shows that $\Pi(\theta, 0) > \Pi(\theta, \hat{q}(\theta))$ if and only if $\theta < \theta_1$, where

$$\theta_1 \triangleq \frac{c_F}{\sigma} \left(\frac{c_L}{c_F} \right)^{(1-\sigma)(\sigma+\hat{\sigma}_L)/\hat{\sigma}_L}. \tag{46}$$

In the region $q(\theta) \geq \hat{q}(\theta)$, if $\varphi(\theta) \leq 0$, then optimally $q(\theta) = \hat{q}(\theta)$. If $\varphi(\theta) > 0$, then $\Pi(\theta, q(\theta))$ is concave in $q(\theta)$ and therefore the maximum is either at the corner, $q(\theta) = \hat{q}(\theta)$, or in the interior, in which case it equals the optimal monopoly quantity

$$q^{\mathrm{m}}(\theta) = \left(\frac{(\sigma + \hat{\sigma}_L)\varphi(\theta)}{c_L} \right)^{1/(1-\sigma-\hat{\sigma}_L)}. \tag{47}$$

Thus, the solution over the region $q(\theta) \geq \hat{q}(\theta)$ is interior if and only if $\Pi_q(\theta, \hat{q}(\theta)) > 0$, or

$$\varphi(\theta) > \frac{c_L}{\sigma + \hat{\sigma}_L} \left(\frac{\theta\sigma}{c_F} \right)^{\frac{\sigma(1-\sigma-\hat{\sigma}_L)}{(1-\sigma)(\sigma+\hat{\sigma}_L)}}. \tag{48}$$

Condition (48) can hold on disjoint intervals of θ even if $\varphi(\theta)$ is increasing, because the right-hand side is increasing in θ. However, the power of θ on the right-hand side is strictly less than 1. Thus, if F satisfies monotone hazard rate, then $\varphi'(\theta) \geq 1$, and condition (48) holds, if at all, for all $\theta > \theta_2$ where θ_2 solves:

$$\varphi(\theta_2) = \frac{c_L}{\sigma + \hat{\sigma}_L} \left(\frac{\theta_2\sigma}{c_F} \right)^{\frac{\sigma(1-\sigma-\hat{\sigma}_L)}{(1-\sigma)(\sigma+\hat{\sigma}_L)}}. \tag{49}$$

If (49) doesn't admit a solution, then the solution is never interior and thus for all $\theta < \theta_1$, $q(\theta) = 0$ and for all $\theta > \theta_1$, $q(\theta) = \hat{q}(\theta)$. If (49) admits a solution at $\theta_2 > \theta_1$, then: for all $\theta < \theta_1$, $q(\theta) = 0$; for all $\theta \in (\theta_1, \theta_2)$, $q(\theta) = \hat{q}(\theta)$; for all $\theta \geq \theta_2$, $q(\theta) = q^{\mathrm{m}}(\theta)$. Finally, if (49) admits a solution at $\theta_2 < \theta_1$, then $\theta_3 \in [\theta_2, \theta_1]$ exists such that for all $\theta < \theta_3$, $q(\theta) = 0$ and for all $\theta \geq \theta_3$, $q(\theta) = q^{\mathrm{m}}(\theta)$. (The threshold $\theta_3 > \theta_2$ is determined by the condition $\Pi(\theta_3, q^{\mathrm{m}}(\theta_3)) = 0$.)

Since all these (relaxed) solutions are implementable, the result follows. □

Leader-Fringe Competition with Uniform Types

Competitive Mechanism— With $\hat{\sigma}_F = 0$ the buyer's first-order condition implies

$$q_L^{(\sigma+\hat{\sigma}_L)/\sigma} + q_F = \left(\tfrac{\theta\sigma}{c_F} \right)^{1/(1-\sigma)}.$$

On the *fringe-indifference boundary* ($q_F = 0$) the leader's quantity solves

$$\hat{q}(\theta) = \left(\frac{\theta\,\sigma}{c_F}\right)^{\frac{\sigma}{(1-\sigma)(\sigma+\hat{\sigma}_L)}}.$$

When the buyer single-homes on the leader and the fringe constraint is slack,

$$q^{\mathrm{m}}(\theta) = \left(\frac{(\sigma+\hat{\sigma}_L)\,\varphi(\theta)}{c_L}\right)^{\frac{1}{1-\sigma-\hat{\sigma}_L}} = \left(\frac{(\sigma+\hat{\sigma}_L)\,(2\theta-1)}{c_L}\right)^{\frac{1}{1-\sigma-\hat{\sigma}_L}}.$$

We want the optimal menu to be characterized by $0 < \theta_1 < \theta_2 < 1$ such that

$$q^{LF}(\theta) = \begin{cases} 0, & \theta \le \theta_1, \\ \hat{q}(\theta), & \theta_1 < \theta < \theta_2, \\ q^{\mathrm{m}}(\theta), & \theta \ge \theta_2, \end{cases} \qquad q_F^{LF}(\theta) = \begin{cases} \left(\dfrac{\theta\,\sigma}{c_F}\right)^{\frac{1}{1-\sigma}}, & \theta \le \theta_1, \\ 0, & \theta > \theta_1. \end{cases}$$

The entry cutoff is

$$\theta_1 = \frac{c_F}{\sigma}\left(\frac{c_L}{c_F}\right)^{\frac{(1-\sigma)(\sigma+\hat{\sigma}_L)}{\hat{\sigma}_L}}.$$

The boundary-interior switch θ_2 is the unique solution to

$$\varphi(\theta) = \frac{c_L}{\sigma+\hat{\sigma}_L}\left(\frac{\theta\,\sigma}{c_F}\right)^{\frac{\sigma(1-\sigma-\hat{\sigma}_L)}{(1-\sigma)(\sigma+\hat{\sigma}_L)}},$$

which exists in $(1/2, 1)$ if and only if $\frac{c_L}{\sigma+\hat{\sigma}_L}\left(\frac{\sigma}{c_F}\right)^{\frac{\sigma(1-\sigma-\hat{\sigma}_L)}{(1-\sigma)(\sigma+\hat{\sigma}_L)}} < 1$. Furthermore, $\theta_2 > \theta_1$ if and only if $\theta_1 < (\sigma+\hat{\sigma}_L)/(\sigma+2\hat{\sigma}_L)$. Therefore, for $0 < \theta_1 < \theta_2 < 1$ to take place, the necessary and sufficient conditions are:

$$\frac{c_L}{\sigma+\hat{\sigma}_L}\left(\frac{\sigma}{c_F}\right)^{\frac{\sigma(1-\sigma-\hat{\sigma}_L)}{(1-\sigma)(\sigma+\hat{\sigma}_L)}} < 1, \quad \frac{c_F}{\sigma}\left(\frac{c_L}{c_F}\right)^{\frac{(1-\sigma)(\sigma+\hat{\sigma}_L)}{\hat{\sigma}_L}} < \frac{\sigma+\hat{\sigma}_L}{\sigma+2\hat{\sigma}_L}.$$

Efficient Allocation— Efficiency features single-homing with a model-switch cutoff $\hat{\theta}$:

$$(q_L^*(\theta), q_F^*(\theta)) = \begin{cases} \left(0,\ \left(\dfrac{\theta\,\sigma}{c_F}\right)^{\frac{1}{1-\sigma}}\right), & \theta < \hat{\theta}, \\ \left(\left(\dfrac{\theta(\sigma+\hat{\sigma}_L)}{c_L}\right)^{\frac{1}{1-\sigma-\hat{\sigma}_L}},\ 0\right), & \theta \ge \hat{\theta}, \end{cases}$$

with

$$\hat{\theta} = \left(\frac{1-\sigma}{1-\sigma-\hat{\sigma}_L}\right)^{\frac{(1-\sigma-\hat{\sigma}_L)(1-\sigma)}{\hat{\sigma}_L}}\left(\frac{\sigma}{c_F}\right)^{\frac{\sigma(1-\sigma-\hat{\sigma}_L)}{\hat{\sigma}_L}}\left(\frac{c_L}{\sigma+\hat{\sigma}_L}\right)^{\frac{(\sigma+\hat{\sigma}_L)(1-\sigma)}{\hat{\sigma}_L}}.$$

Multi-Model Monopoly— Virtual type is $\varphi(\theta) = 2\theta - 1$. Types $\theta < 1/2$ are excluded. Among

served types the monopolist assigns a single model, switching at $\theta^{\mathrm{m}} = (1+\hat{\theta})/2$. Optimal quantities are

$$(q_L^{\mathrm{m}}(\theta), q_F^{\mathrm{m}}(\theta)) = \begin{cases} (0,0), & \theta < \frac{1}{2}, \\ \left(0,\ \left(\dfrac{\sigma\,\varphi(\theta)}{c_F}\right)^{\frac{1}{1-\sigma}}\right), & \frac{1}{2} \le \theta < \theta^{\mathrm{m}}, \\ \left(\left(\dfrac{(\sigma+\hat{\sigma}_L)\,\varphi(\theta)}{c_L}\right)^{\frac{1}{1-\sigma-\hat{\sigma}_L}},\ 0\right), & \theta \ge \theta^{\mathrm{m}}. \end{cases}$$

B Contractible Tasks

Throughout the manuscript, we focus on the more realistic case in which the LLM provider cannot contract on the allocation of tokens across the buyer's tasks. However, the full-contracting setting provides a natural benchmark and we study it in this section. Thus, the seller designs a direct menu that specifies a token allocation across different tasks:

$$\{((x_{i1}(w),\dots,x_{iJ}(w))_{i\in[0,1]}, z_1(w),\dots,z_K(w), t(w))\}_w. \tag{50}$$

The allocation being contractible means the buyer has no freedom to reallocate tokens across tasks. This naturally increases the scope for screening. This also makes the problem intractable in full generality, because the reduction to a one-dimensional aggregate type and quality as in Proposition 2 does not apply. Nevertheless, in this section we obtain complete characterizations in two special settings: the case of separable type distributions (Section B.1) and the case of two types (Section B.2). In the former case, the optimal solution can be obtained via a menu of token budgets. In the latter case, the optimal solution can be obtained by identifying the structure of binding incentive constraints.

B.1 Separable Type Distribution and Cost-Based Pricing

In this section, we provide a sufficient condition on type distribution under which contracting on tasks is not profitable. In fact, under that condition an even less contractually restrictive class of mechanisms, cost-based tariffs, is optimal:

Definition 4 (Cost-Based Tariff). A cost-based tariff is a menu of monetary budgets and transfers $\{(B_n, T_n)\}_n$ such that, upon purchasing item n, the buyer pays T_n for access to budget B_n, which he can freely spend on tokens priced at their marginal costs.

We build on the analysis of Armstrong (1996) and introduce the conditions that are jointly sufficient for the optimality of cost-based tariffs: demand separability and type separability. We show that the former is always satisfied in our setting, whereas the latter is equivalent to the aggregate type not being informative about the relative task weights.

To this end, consider a problem of optimal token allocation by type w given a monetary budget B and token prices equal to marginal costs:

$$V(w,B) = \max_{\{(x_i)_{i\in[0,1]}, z\geq 0\}} \int_0^1 w_i g(x_i, z)di, \quad \text{s.t.} \int_0^1 \sum_{j=1}^{J} c_j x_{ij} di + \sum_{k=1}^{K} \hat{c}_k z_k = B,$$

Definition 5 (Demand Separability). Demand is separable if there exist functions $V_1(w)$ and $V_2(B)$ such that

$$V(w, B) = V_1(w)V_2(B). \tag{51}$$

If demand is separable, then faced with a cost-based tariff, all types w with the same $V_1(w)$ purchase the same item.

Demand separability always holds in our setting. Indeed, by Proposition 2, if type w purchases a total amount of inference tokens $X = (X_1, \ldots, X_J)$ and fine-tuning tokens $Z = (Z_1, \ldots, Z_K)$, then his optimal payoff is $\theta(w)g(X, Z)$. Therefore, (51) holds with $V_1(w) = \theta(w)$ and

$$V_2(B) = \max_{X \geq 0, Z \geq 0} \; g(X, Z), \quad \text{s.t.} \; \sum_{j=1}^{J} c_j X_j + \sum_{k=1}^{K} \hat{c}_k Z_k = B.$$

Definition 6 (Type Separability). Type distribution is separable if f_1, f_2 exist such that $f(w) = f_1(\theta(w))f_2(w)$ for all w and f_2 is homogeneous of degree zero.

A separable type distribution means that knowing $\theta(w)$ provides no information about which ray from the origin w lies on, that is, about the relative weights the buyer assigns to different tasks. Equivalently, denoting by $\|\cdot\|_p$ a standard L_p norm, $\theta(w) = \|w\|_{1/(1-\sigma)}$ and the type separability is equivalent to $\|w\|_{1/(1-\sigma)}$ and $w/\|w\|_{1/(1-\sigma)}$ to be independent. Thus, any type distribution generated by a draw of a "total size" $\|w\|_{1/(1-\sigma)}$ according to an arbitrary distribution together with an independent draw of "relative weights" $w/\|w\|_{1/(1-\sigma)}$ according to any other distribution, is separable.

Proposition 8 (Cost-Based Optimality). *If the type distribution is separable, then a cost-based tariff is optimal.*

Proof. Since in our setting demand is always separable, the result follows from the arguments presented by Armstrong (1996). Specifically, we can relax the IC constraints between types located on different rays from the origin. Then, applying the Envelope Theorem ray by ray, we can find a solution to the relaxed problem as follows:

$$(x^*, z^*) \in \arg \max_{\{(x_i)_{i\in[0,1]}, z \geq 0\}} \Big(1 - \frac{\int_1^\infty r^{n-1} f(rw) dr}{f(w)}\Big) u(w, x, z) - c(x, z).$$

Given the demand separability, this condition can be rewritten as:

$$(x^*, z^*) \in \arg \max_{\{(x_i)_{i\in[0,1]}, z \geq 0\}} u(w, x, z), \quad \text{s.t.} \; c(x, z) \leq B^*(w),$$

and

$$B^*(w) \in \arg\max_{B \geq 0} \Big(1 - \frac{\int_1^\infty r^{n-1} f(rw) dr}{f(w)}\Big) \theta(w) V_2(B) - B.$$

At the same time, the optimal cost-based tariff implements an allocation

$$(x^*, z^*) \in \arg \max_{\{(x_i)_{i \in [0,1]}, z \geq 0\}} u(w, x, z), \quad \text{s.t. } c(x, z) \leq B^*(w),$$

where

$$B^*(w) \in \arg\max_{B} \Big(\theta(w) - \frac{1 - F(\theta(w))}{f(\theta(w))}\Big) V_2(B) - B.$$

If the type distribution is separable, then

$$\Big(1 - \frac{\int_1^\infty r^{n-1} f(rw) dr}{f(w)}\Big) \theta(w) = \theta(w) - \frac{1 - F(\theta(w))}{f(\theta(w))},$$

and the cost-based tariff implements the solution to the relaxed problem. Therefore, it is (indirectly) optimal. □

B.2 Binary Types

Alternatively, let there be only two types, w_1 and w_2, which occur with prior strictly positive probabilities f_1 and f_2, respectively. In this case, the optimal mechanism depends on what happens if the seller attempts to extract the first-best level of surplus, that is, if she offers a menu containing the efficient amounts of tokens for each type with prices equal to their respective added values. If this menu is incentive compatible, then it is clearly optimal, and we call the type with the higher payment "high" and associate the label H with it. If this menu is not incentive compatible, then we call "high" the type whose incentive constraint is violated. We call the other type "low" and associate the label L with it.

To design an optimal menu, it is important to determine which type out of w_1 and w_2 is high and which one is low. If w_2 dominates w_1 for every task, then it is clearly high; alternatively, the types are in some sense horizontally differentiated and the distinction is less clear. However, it turns out that the ranking can be derived from the aggregate types, and the optimal menu admits a simple characterization.

Proposition 9 (Binary Types). *The high type is the one with the higher aggregate type, i.e., $\theta(w_H) \geq \theta(w_L)$. In the optimal menu, the token allocation of w_H is always efficient. If*

$$\int_0^1 (w_{Hi} - w_{Li}) w_{Li}^{\frac{\sigma}{1-\sigma}} di \leq 0, \tag{52}$$

then the token allocation of w_L is also efficient and the seller extracts full surplus. Otherwise, the token allocation of w_L is efficient with respect to a virtual type $w_L - (w_H - w_L) f_H / f_L$.

Proof. With a small abuse of notation relative to previous sections, denote by $q = (q_i)_{i \in [0,1]}$ the *profile* of qualities delivered to each task, $q_i \triangleq g(x_i, z)$, and denote by $C(q)$ the minimal total cost of generating a given profile q:

$$C(q) \triangleq \min_{x_i, z \geq 0} \int_0^1 \sum_{j=1}^{J} c_j x_{ij} di + \sum_{k=1}^{K} \hat{c}_k z_k, \quad \text{s.t. } g(x_i, z) = q_i, \forall\, i \in [0,1].$$

Because the set of feasible profiles q is convex, it follows from the analysis of Haghpanah and Siegel (2024) on general screening problems with two buyer types that in our setting either (i) the seller extracts full surplus; or (ii) the incentive constraint of type w_H and the individual rationality constraint of type w_L bind.

It follows that if the seller cannot extract full surplus, then the seller's problem can be written as

$$\begin{aligned} &\max_{q_L, q_H, t_L, t_H} f_L(t_L - C(q_L)) + f_H(t_H - C(q_H)) \\ &\qquad \text{s.t.} \int_0^1 w_{Hi} q_{Hi} di - t_H = \int_0^1 w_{Hi} q_{Li} di - t_L, \ \int_0^1 w_{Li} q_{Li} di - t_L = 0. \end{aligned}$$

Solving for transfers from the constraints, the problem can be restated as:

$$\max_{q_L, q_H} f_L \left(\int_0^1 \left(w_{Li} - \frac{f_H}{f_L} (w_{Hi} - w_{Li}) \right) q_{Li} di - C(q_L) \right) + f_H \left(\int_0^1 w_{Hi} q_{Hi} di - C(q_H) \right),$$

which is solved by the allocation efficient relative to the virtual types.

It is left to determine which type is high and provide conditions for full surplus extraction. Consider a mechanism that attempts full surplus extraction. By the efficiency analysis behind Proposition 1, under this mechanism type w, when reporting type $\tilde{w}$, obtains added value

$$u(w, \tilde{w}) = \int_0^1 w_i \Psi(x_i(\tilde{w})) \Phi(z(\tilde{w})) di = \int_0^1 w_i \tilde{w}_i^{\frac{\sigma}{1-\sigma}} \Psi(d) \Phi(z(\tilde{w}))^{\frac{1}{1-\sigma}} di.$$

Because under truth-telling each type obtains zero rents, the corresponding payment is

$t(w) = u(w, w)$, and the incentive constraint $u(w, w) - t(w) \geq u(w, \tilde{w}) - t(\tilde{w})$ is violated if and only if:

$$\Psi(d)\Phi(z(\tilde{w}))^{\frac{1}{1-\sigma}} \int_0^1 (w_i - \tilde{w}_i)\tilde{w}_i^{\frac{\sigma}{1-\sigma}} di > 0. \tag{53}$$

By Jensen's inequality and the concavity of the logarithm:

$$w_i \tilde{w}_i^{\frac{\sigma}{1-\sigma}} = w_i^{\frac{1-\sigma}{1-\sigma}} \tilde{w}_i^{\frac{\sigma}{1-\sigma}} \leq (1-\sigma) w_i^{\frac{1}{1-\sigma}} + \sigma \tilde{w}_i^{\frac{1}{1-\sigma}}.$$

Therefore, inequality (53) implies

$$\int_0^1 (w_i^{\frac{1}{1-\sigma}} - \tilde{w}_i^{\frac{1}{1-\sigma}}) di > 0,$$

and the incentive violation is possible only if $\theta(w) > \theta(\tilde{w})$, i.e., from high to low aggregate type. Therefore, if full surplus extraction is not incentive compatible, then the high type is the one with the higher aggregate type. If full surplus extraction is incentive compatible, then the high type is the one with the higher aggregate type directly by the analysis behind Proposition 1. The result follows. □

Note that even though the virtual types in Proposition 9 follow the standard formula of the single-dimensional case, each type there is infinite-dimensional.

One might wonder whether the correspondence between the incentive order and aggregate types extends beyond the binary-type case. This is not the case. In Section B.3, we study a special case of our model in which the buyer's type can be effectively parameterized by two variables: the number of ex ante homogeneous tasks to which the buyer attaches positive value, and the value he attaches to each of those tasks. We show that the incentive constraints that bind in the optimal mechanism do not admit a one-dimensional structure. Instead, they form an infinite collection of one-dimensional segments. In that setting the optimal mechanisms also admit a natural implementation via a menu of two-part tariffs. However, in contrast to the indirect implementations described in Section 4.2, each item in the menu must be accompanied by *task caps*, that is, restrictions on the number of tasks the buyer can process.

B.3 Value-Scale Heterogeneity

In this section, we characterize an optimal menu with contractible token allocations across tasks in the case of value-scale heterogeneity. In this case, each multidimensional type w is

characterized (with a small abuse of notation) by two parameters (w, s) such that

$$w_i = \begin{cases} w, \text{ if } i \leq s, \\ 0, \text{ if } i > s, \end{cases} \tag{54}$$

in which w and s are independently distributed according to CDFs F_w and F_s, with F_w featuring increasing virtual values. We will argue that in this case, under additional assumptions, the binding incentive constraints are those within each scale, and the seller is able to generate the same profit as if the scale were observable.

For this section, we drop the product structure and the homogeneity requirements of (1) and instead require that $g(x_i, z)$ is (i) positive and continuous on $\mathbb{R}_+^{J+K}$, (ii) strictly monotone, twice continuously differentiable, strictly concave with negative definite Hessian, and with all cross-partial derivatives strictly positive on $\mathbb{R}_{++}^{J+K}$, (iii) $g(0) = 0$, and (iv) Inada at zero, i.e., $\lim_{y_m \downarrow 0} g_{y_m}(y_m, y_{-m}) = +\infty$ for all $y_{-m} \in \mathbb{R}_+^{J+K-1}$ such that $g(0, y_{-m}) > 0$.[18]

Consider the problem in which the scale is commonly known to be $s > 0$. The buyer's payoff from any given item on the menu (50) is

$$w \int_{i=0}^{s} g(x_i, z) di - t.$$

For any reported w the seller should optimize token allocation to deliver a promised level of (total) quality q,

$$wq - t,$$

with the minimal cost function $C(q, s)$ of delivering a given quality being:

$$C(q, s) = \min_{(x_i)_{i \in [0,1]}, z \geq 0} \int_{i=0}^{s} \sum_{j=1}^{J} c_j x_{ij} di + \sum_{k=1}^{K} \hat{c}_k z_k, \quad \text{s.t.} \int_{i=0}^{s} g(x_i, z) di = q. \tag{55}$$

Since g is strictly concave, the solution to this problem is achieved by allocating the inference tokens uniformly across the s tasks. The problem can be equivalently stated as:

$$C(q, s) = \min_{x, z \geq 0} \; s \sum_{j=1}^{J} c_j x_j + \sum_{k=1}^{K} \hat{c}_k z_k, \quad \text{s.t. } sg(x, z) = q. \tag{56}$$

The resulting cost function $C(q, s)$ satisfies, over the domain of admissible (q, s), the following properties:

[18] The Inada condition is imposed only for simplicity of arguments in Lemma 5.

Lemma 5 (Cost Function).

1. *$C(q,s)$ is strictly increasing and strictly convex in q with $C_q(0,s)=0$.*
2. *$C(q,s)$ is strictly decreasing in s for $q>0$.*
3. *$C(q,s)$ is submodular, i.e., $C_q(q,s)$ is decreasing in s for all q.*

Proof. The first two statements follow directly from our assumptions on g. To establish the third property, posit the Lagrangian for the cost minimization problem:

$$L = s\sum_{j=1}^{J} c_j x_j + \sum_{k=1}^{K} \hat{c}_k z_k + \lambda(q - sg(x,z)).$$

By the Envelope Theorem, $C_q(q,s) = \lambda(q,s)$. Thus, it suffices to show that $\lambda(q,s)$ is decreasing in s for a fixed q.

To this end, for any $q>0$, by the Inada condition, the solution must be interior, $(x,z) \gg 0$. The constraint binds, so $\lambda>0$. Denoting (x,z) by y, the first-order conditions are:

$$g_{x_j}(y) = c_j/\lambda, \quad g_{z_k}(y) = \hat{c}_k/(\lambda s). \tag{57}$$

Define the Hessian function $H(y) \triangleq \nabla^2 g(y)$. Denote by $A, B \in \mathbb{R}^{J+K}$ vectors such that $A_r = c_r/\lambda^2$ if $r \le J$ and $= \hat{c}_{r-J}/(\lambda^2 s)$ if $r > J$, $B_r = 0$ if $r \le J$ and $= \hat{c}_{r-J}/(\lambda s^2)$ if $r > J$. Then, differentiating (57) with respect to s, we obtain:

$$H\frac{dy}{ds} = -A\frac{d\lambda}{ds} - B. \tag{58}$$

At the same time, differentiating the constraint $sg(y(s)) = q$ with respect to s, we obtain:

$$g(y) + s\nabla g(y)^\top \frac{dy}{ds} = 0. \tag{59}$$

Solving for dy/ds in (58) and substituting it into (59), we obtain, omitting the dependence on y:

$$\frac{d\lambda}{ds} s\nabla g^\top(-H^{-1}A) = -g - s\nabla g^\top(-H^{-1}B). \tag{60}$$

Now, observe that for all $y \in \mathbb{R}^{J+K}_{++}$, by assumption on g, $H(y)$ is symmetric negative definite; moreover, by the positivity of cross-derivative, all off-diagonal entries of $H(y)$ are strictly positive. Then, $-H(y)$ is a Stieltjes matrix and, consequently, all elements of $H(y)^{-1}$ are negative. It immediately follows from (60) that whenever $q>0$ and $s>0$, $d\lambda/ds < 0$. □

As such, the seller's problem for any given s is analogous to Mussa and Rosen (1978). Since w and s are independently distributed, we can drop the dependence on s and define the virtual value as

$$\varphi(w) \triangleq w - \frac{1 - F_w(w)}{f_w(w)}. \tag{61}$$

Since $\varphi(w)$ is increasing, all w with $\varphi(w) \leq 0$ are excluded and all other w receive the quality level $q(w, s)$ that solves:

$$\varphi(w) = C_q(q(w, s), s). \tag{62}$$

The corresponding optimal transfers are

$$t(w, s) = w\, q(w, s) - \int_0^w q(r, s)dr. \tag{63}$$

Lemma 5 shows that it is cheaper to generate an extra unit of (total) quality when you have more tasks. This property is intuitive given that the returns on each task are diminishing. Thus, the optimal quality, and hence the buyer's rent, increase in scale for any given value w.

Assumption 2 (Bounded Rent Increase). For all w, s, the function $q(w, s)$ defined in (62) satisfies $\int_0^w sq_s(r, s)dr \leq wq(w, s)$.

Assumption 2 requires that the buyer's rent does not grow too quickly and, specifically, that the marginal increase of buyer rent from having an additional task is smaller than the average equilibrium value generated by LLM across existing tasks.

Proposition 10 (Optimal Menu of Token Allocations). *Under Assumption 2, an optimal menu is*

$$((x_i(w, s))_{i\in[0,1]}, z(w, s), t(w, s))_{(w,s)},$$

where for each (w, s), $((x_i(w, s))_{i\in[0,1]}, z(w, s))$ are cost-minimizing tokens from (55) that deliver quality $q(w, s)$ as defined in (62), and $t(w, s)$ is as defined in (63).

Proof. If each type reports truthfully, then the menu attains the profits of the observable-scale benchmark and is thus optimal.

If type (w, s) deviates to $(w, \tilde{s})$ with $\tilde{s} \leq s$, then, under the proposed menu, he obtains exactly the same payoff as type $(w, \tilde{s})$, because he processes the same number of tasks with the same willingness to pay for quality. By Lemma 5, $C_q(q, s)$ is decreasing in s for all q, and thus $q(w, s)$ is increasing in s for all w. Therefore, the rents accrued by type (w, s) under truth-telling,

$$U(w, s) = \int_0^w q(r, s)dr,$$

are increasing in s for all w. Therefore, (w,s) does not want to deviate to $(w,\tilde{s})$ with $\tilde{s} \leq s$. Furthermore, by incentive compatibility within a given $\tilde{s}$, $(w,\tilde{s})$ does not want to deviate to $(\tilde{w},\tilde{s})$. Therefore, (w,s) does not want to deviate to any $(\tilde{w},\tilde{s})$ with $\tilde{s} \leq s$.

If type (w,s) deviates to $(\tilde{w},\tilde{s})$ with $\tilde{s} > s$, then he obtains gross payoff $wq(\tilde{w},\tilde{s})\, s/\tilde{s}$ and pays the transfer $t(\tilde{w},\tilde{s})$. Therefore, the optimal double deviation strategy for a misreporting type solves

$$\max_{\tilde{w}\geq 0}\left[wq(\tilde{w},\tilde{s})\frac{s}{\tilde{s}} - \tilde{w}q(\tilde{w},\tilde{s}) + \int_0^{\tilde{w}} q(r,\tilde{s})dr\right].$$

Because the mechanism incentivizes truthful reporting by any $\tilde{s}$-truthtelling type, including the type $(ws/\tilde{s},\tilde{s})$, it follows that

$$\tilde{w}^* = \frac{ws}{\tilde{s}}, \quad U(w;s,\tilde{s}) = \int_0^{\frac{ws}{\tilde{s}}} q(r,\tilde{s})dr.$$

The condition that discourages local deviations to $\tilde{s} > s$ is:

$$s\int_0^w q_s(r,s)dr \leq wq(w,s),$$

for all (w,s), which is precisely Assumption 2. Furthermore, observe that for $\tilde{s} > s$:

$$\begin{aligned} U_{\tilde{s}}(w;s,\tilde{s}) &= \int_0^{\frac{ws}{\tilde{s}}} q_s(r,\tilde{s})dr - \frac{ws}{\tilde{s}^2}q(\frac{ws}{\tilde{s}},\tilde{s}) \\ &= \frac{1}{\tilde{s}}\left(\tilde{s}\int_0^{\frac{ws}{\tilde{s}}} q_s(r,\tilde{s})dr - \frac{ws}{\tilde{s}}q(\frac{ws}{\tilde{s}},\tilde{s})\right) \leq 0, \end{aligned}$$

where the inequality holds by Assumption 2 with (w,s) replaced by $(sw/\tilde{s},\tilde{s})$. Therefore, the upward global deviations in $\tilde{s}$ are suboptimal and the result follows. □

References

Armstrong, M. (1996): "Multiproduct Nonlinear Pricing," *Econometrica*, 64, 51–76.

Babaioff, M., R. Kleinberg, and R. Paes Leme (2012): "Optimal Mechanisms for Selling Information," in *Proceedings of the 13th ACM Conference on Electronic Commerce*, 92–109.

Bergemann, D., M. Bojko, P. Dütting, R. Paes Leme, H. Xu, and S. Zuo (2024): "Data-Driven Mechanism Design: Jointly Eliciting Preferences and Information," *arXiv preprint arXiv:2412.16132*.

Bergemann, D., A. Bonatti, and A. Smolin (2018): "The Design and Price of Information," *American Economic Review*, 108, 1–48.

Calzolari, G. and V. Denicolò (2013): "Competition with Exclusive Contracts and Market-Share Discounts," *American Economic Review*, 103, 2384–2411.

——— (2015): "Exclusive Contracts and Market Dominance," *American Economic Review*, 105, 3321–3351.

Castro-Pires, H., H. Chade, and J. Swinkels (2024): "Disentangling Moral Hazard and Adverse Selection," *American Economic Review*, 114, 1–37.

Daskalakis, C., A. Deckelbaum, and C. Tzamos (2017): "Strong Duality for a Multiple-Good Monopolist," *Econometrica*, 85, 735–767.

Demirer, M., A. Fradkin, N. Tadelis, and S. Peng (2025): "The Emerging Market for Intelligence: Pricing, Supply, and Demand for LLMs," Tech. rep., National Bureau of Economic Research.

Devanur, N. R., K. Goldner, R. R. Saxena, A. Schvartzman, and S. M. Weinberg (2020): "Optimal Mechanism Design for Single-Minded Agents," in *Proceedings of the 21st ACM Conference on Economics and Computation*, 193–256.

Doligalski, P., P. Dworczak, J. Krysta, and F. Tokarski (2025): "Incentive separability," *Journal of Political Economy Microeconomics*, 3, 539–567.

Duetting, P., V. Mirrokni, R. Paes Leme, H. Xu, and S. Zuo (2024): "Mechanism design for large language models," in *Proceedings of the ACM on Web Conference 2024*, 144–155.

Fiat, A., K. Goldner, A. R. Karlin, and E. Koutsoupias (2016): "The Fedex Problem," in *Proceedings of the 2016 ACM Conference on Economics and Computation*, 21–22.

Fish, S., Y. A. Gonczarowski, and R. I. Shorrer (2024): "Algorithmic Collusion by Large Language Models," *arXiv preprint arXiv:2404.00806*.

Haghpanah, N. and R. Siegel (2024): "Screening Two Types," Tech. rep., Penn State.

Jullien, B. (2000): "Participation Constraints in Adverse Selection Models," *Journal of Economic Theory*, 93, 1–47.

Kaplan, J., S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei (2020): "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*.

Laffont, J.-J. and J. Tirole (1990): "The regulation of multiproduct firms: Part I: Theory," *Journal of Public Economics*, 43, 1–36.

Mahmood, R. (2024): "Pricing and Competition for Generative AI," *arXiv preprint arXiv:2411.02661*.

Mussa, M. and S. Rosen (1978): "Monopoly and Product Quality," *Journal of Economic Theory*, 18, 301–317.

Rochet, J.-C. and L. A. Stole (2003): "The Economics of Multidimensional Screening," *Econometric Society Monographs*, 35, 150–197.

Tirole, J. (1988): *The Theory of Industrial Organization*, Cambridge: MIT Press.

Wu, Y., Z. Sun, S. Li, S. Welleck, and Y. Yang (2025): "Inference Scaling Laws: An Empirical analysis of Compute-Optimal Inference for LLM Problem-Solving," in *The Thirteenth International Conference on Learning Representations*.

Yang, K. H. (2022): "Selling Consumer Data for Profit: Optimal Market-Segmentation Design and Its Consequences," *American Economic Review*, 112, 1364–1393.

2. 随机车辆路径：**Data-Driven Stochastic Vehicle Routing Problems with Deadlines Under Decision-Dependent Travel Time (MSOM, 2026)**

来源：Manufacturing & Service Operations Management, 2026.
https://doi.org/10.1287/msom.2024.0899

作者：Shanshan Wang, Erick Delage, Leandro C. Coelho

关键词：车辆路径；截止时间；内生不确定性；非参数估计；Benders分解

推荐人：徐光辉

推荐理由：针对外卖配送中行程时间随路径决策变化的问题，本文研究带截止时间和内生不确定性的数据驱动随机车辆路径模型。作者采用近邻和核密度估计刻画与决策相关的行程时间分布，并设计增强的逻辑型Benders分解，包括子路径延误割、热启动、局部搜索和机器学习下界启发式。真实配送数据上的实验表明，将路径决策纳入非参数模型能够降低预期延误，优于所比较的参数模型、稳健分配方案和实际行驶路径。

https://pubsonline.informs.org/journal/msom

MANUFACTURING & SERVICE OPERATIONS MANAGEMENT
Articles in Advance, pp. 1–21
ISSN 1523-4614 (print), ISSN 1526-5498 (online)

Data-Driven Stochastic Vehicle Routing Problems with Deadlines Under Decision-Dependent Travel Time

Shanshan Wang,[a] Erick Delage,[b,c,*] Leandro C. Coelho[b,d]

[a] School of Economics and Management, Beihang University, Beijing 100191, China; [b] GERAD, Montréal, Quebec H3T 1N8, Canada; [c] Department of Decision Sciences, HEC Montréal, Montréal, Quebec H3T 2A7, Canada; [d] Department of Operations and Decision Systems, Université Laval, Quebec City, Quebec G1V 0A6, Canada

*Corresponding author

Contact: shanshanwang@buaa.edu.cn, https://orcid.org/0000-0002-5122-4501 (SW); erick.delage@hec.ca, https://orcid.org/0000-0002-6740-3600 (ED); leandro.coelho@fsa.ulaval.ca, https://orcid.org/0000-0002-9797-1019 (LCC)

Received: March 7, 2024
Revised: June 6, 2025; January 21, 2026
Accepted: February 12, 2026
Published Online in Articles in Advance: June 11, 2026

https://doi.org/10.1287/msom.2024.0899

Copyright: © 2026 INFORMS

Abstract. ***Problem definition*:** Vehicle routing problems (VRPs) with deadlines have received significant attention around the world. Motivated by a real-world food delivery problem, we assume that the travel time depends on the routing decisions, and we study a data-driven stochastic VRP with deadlines and endogenous uncertainty. ***Methodology/results*:** We use the nonparametric approaches, including *k*-nearest neighbor (*k*NN) and kernel density estimation (KDE), to estimate the decision-dependent probability distribution of travel time. To solve the resulting problem efficiently, we employ a logic-based Benders decomposition (LBBD) algorithm with several algorithmic enhancements. In particular, we propose a novel family of optimality cuts that includes the expected delay for all the subroutes. Moreover, we solve a total travel cost minimization problem to warm start the algorithm. We also use a local search procedure to improve the current routing decision and propose a machine learning–based lower bound heuristic to efficiently solve problems of realistic size. A practical case study for a food delivery routing problem using real-world data is conducted to show the efficiency of the proposed techniques and the advantage of the data-driven stochastic VRP in reducing the expected delay. ***Managerial implications*:** In our case study, we show that incorporating routing decisions into a nonparametric model outperforms a state-of-the-art data-driven parametric model by 23% on average in terms of the expected delay and the order-assignment decisions obtained from a robust model with travel-time predictors by 26% on average. Moreover, compared with the drivers' actual routes and arc-based VRP models that ignore the endogenous uncertainty, our suggested routes can significantly improve the on-time performance of delivery services. We also quantify the value of the proposed routes with different service deadlines.

Funding: S. Wang was partially supported by the Natural Sciences and Engineering Research Council of Canada [Grant RGPIN-2016-05208], IVADO, and a joint project between the Fonds de Recherche du Québec - Société et Culture (FRQSC) and the National Natural Science Foundation of China (NSFC) [Grant 295837]. She is also most recently supported by the National Natural Science Foundation of China [Grants 72501014, 72371022, and 72272014].

Supplemental Material: The online appendix is available at https://doi.org/10.1287/msom.2024.0899.

Keywords: data-driven stochastic vehicle routing • machine learning • logic-based Benders decomposition • prescriptive analytics • deadline • endogenous uncertainty

1. Introduction

Vehicle routing problems (VRPs) define a highly studied class of combinatorial optimization problems in which products are delivered from a single depot to fulfill customers' demands using a fleet of capacitated vehicles (Toth and Vigo 2014). Deliveries typically incur penalties for violated deadlines, and the goal is to minimize the total penalty cost while determining optimal vehicle routes to fulfill the demand. As a fundamental problem in the field of operations research and management science, VRPs have received significant attention and have been widely applied in transportation and logistics operations management, for example, food delivery, school bus routing and scheduling, traveling salesman problems, waste collection, and ride-sharing services.

The classical VRPs assume that all the model parameters are known in advance. However, this assumption usually does not hold in real-world VRP applications. For example, in a food delivery problem, the driver's travel time is highly uncertain because it largely depends on many factors (e.g., driver

characteristics). These might lead to optimal decisions for deterministic VRPs to fail to be executed by the drivers in practice in order to deliver their targeted on-time service. To overcome these difficulties, one can employ stochastic programming (e.g., Birge and Louveaux 2011) and robust optimization (RO) (e.g., Bertsimas et al. 2011) paradigms to model and address the uncertainties involved in stochastic VRPs. Depending on the method, it is assumed that either the distribution of uncertain parameters is estimated from historical observations or that it is unknown but resides in a predefined uncertainty set. Methodologically, most of the existing studies address the stochastic VRPs using the above two approaches (Carlsson and Delage 2013, Jaillet et al. 2016, Liu and Luo 2023).

1.1. The Motivation for Endogenous Travel Time Uncertainty

Following the relevant literature (Adulyasak and Jaillet 2016, Zhang et al. 2021), we also assume that the travel time is uncertain and its distribution can be estimated using historical observations. However, unlike these papers, we assume that the distribution of uncertain travel time depends on the routes. That is to say, travel uncertainty is endogenous, and the assignment and routing decisions made in the stochastic VRPs could affect the travel time on the arc. This type of decision (or route)-dependent travel time uncertainty is motivated by real-world applications.

In food delivery services, the provider prepares the food at its central kitchen, which is referred to as the depot hereafter, and delivers it to customers within a certain radius of the depot. Customers place orders before each cutoff time and are promised delivery by a deadline. Orders with the same cutoff time are dispatched together and share the same delivery deadline. The planners determine order assignments and routes for each driver (Liu et al. 2021). If the last customer cannot be served before the deadline, a delay cost will be incurred per unit of time.

In our case study, a hypothesis test is conducted to validate whether the travel time on an arc of the network depends on the total route. We observed that on a moderate percentage of the traveled arcs, the null hypothesis that travel time is independent of the route could safely be rejected in favor of a dependence relation. There are several possible explanations. First, delivery drivers may adjust their speed to meet the deadline based on their experience and real-time road conditions (Cui et al. 2023). For instance, if a completion time for the last order estimated by a driver is close to or exceeds the deadline, the driver may speed up, leading to a shorter travel time. In last-mile delivery, drivers have more freedom to decide their speed because they can use electric bikes on the bike lanes. Second, the time spent at a node may be dependent on the route, for example, if the node is near a traffic light where it is faster to perform a right turn (Jenelius and Koutsopoulos 2013, Rahmani et al. 2015). Therefore, it is reasonable and essential for decision makers to construct a route-dependent travel time distribution to better capture the drivers' behaviors and inform the decisions on assigning orders to drivers and routing the deliveries. As indicated by the case study, compared with the routes from drivers' decisions and arc-based VRP models, our suggested routes significantly reduce the expected delays and/or improve stability of out-of-sample performance.

For endogenous travel times, in traffic assignment or traffic flow problems, travel times are endogenized with traffic arc flow decisions (e.g., Beckmann et al. 1956, Nie et al. 2004, Jafari et al. 2017). It has been shown that the arc travel times on a road network are functions of the arc traffic flow decisions, and the traffic assignment problems could be determined by solving convex mathematical programs if the arc travel time functions are separable/integrable, convex, and monotonically increasing. Unlike traffic assignment problems, we assume that travel time on an arc of the network depends on the total routing decisions, and we use nonparametric approaches to estimate the decision-dependent probability distribution of travel time instead of the assumption on the functional form of arc travel times.

Recently, optimization problems under the endogenous uncertainty have been studied in the context of stochastic programming (e.g., Goel and Grossmann 2006) and distributionally robust optimization (DRO) (e.g., Yu and Shen 2022), which assume that decisions can directly affect the distribution of uncertain parameters. Given the endogenous nature of uncertainty, they are notoriously hard both to model and solve, and not surprisingly, the relevant literature is quite sparse. To address the modeling challenge, with the help of the emerging field of prescriptive analytics (e.g., Bertsimas and Kallus 2020, Jiang et al. 2024, Sadana et al. 2025), we employ prescriptive machine learning (ML) tools to estimate a route-dependent distribution of travel time directly from historical data. To address the computational challenge, we develop an efficient logic-based Benders decomposition (LBBD) algorithm with acceleration strategies to solve real-world VRP applications of realistic size.

As the most relevant literature to our problem, Liu et al. (2021) also assumed that the travel time is dependent on the order-assignment decisions and proposed a data-driven decision-making framework that integrates mixed-integer linear program (MILP) representable travel-time predictors with an order-assignment problem. Our work differs in two ways (at least). First, it extends the model beyond MILP representable travel time. Second, Liu et al. (2021) only assign orders to

drivers without detailed route planning considering that significant deviations may exist between the drivers' routes and the planned ones. In this paper, we also employ predictors that account for a routing decision, that is, the order in which the deliveries should be made. Given that our model learns about the characteristics of the real routes to design routes, these routes could be more acceptable to drivers. As indicated by a real-world food delivery application presented in Section 5, we improve the on-time performance of delivery services by making joint decisions regarding order assignment and routing when compared with Liu et al. (2021). Moreover, by learning the deviation from the suggested and actual delivery routes, one could further capture drivers' preferences and improve the likelihood of drivers accepting the suggested routes (see, e.g., Fu et al. 2023). Therefore, from the perspective of future implementation, designing routes plays an important role in increasing drivers' trust in the platform. The goal of this paper is to contribute a prescriptive ML-based decision support methodology that can use covariate data for making both the assignment and routing decisions of stochastic VRP in a data-driven context.

1.2. Contributions of This Paper

Whereas the emerging field of prescriptive analytics has received much attention in recent years, to the best of our knowledge, this paper presents the first comprehensive study of a prescriptive ML approach for a general data-driven stochastic VRP with deadlines and endogenous travel time uncertainty. We provide novel model formulations, efficient solution algorithms, and an extensive real-world case study. Our contributions are described as follows.

- We propose a novel prescriptive ML approach for a data-driven stochastic VRP with deadlines and uncertain travel time. Motivated by real-world food delivery problems, we consider that the uncertain travel time depends on the routes and thus incorporate endogenous travel time uncertainty in our VRP model. We then use nonparametric approaches, including k-nearest neighbor (kNN) and kernel density estimation (KDE), to estimate the route-dependent probability distribution of travel time. Moreover, we extend the data-driven stochastic VRP to a DRO framework under an ambiguity set based on the Wasserstein distance of order $q \in [1, +\infty)$ and obtain closed-form expressions for the inner worst-case expectation problem.
- We develop an efficient LBBD method with acceleration strategies to solve the data-driven stochastic VRP, including general optimality cuts, warm start, and local search. However, given the difficulty involved with the route-dependent travel time uncertainty, we further propose an ML-based lower bound heuristic to efficiently solve instances of realistic size. Specifically, we introduce a distribution that is first-order stochastically dominated by the distribution of travel time. Motivated by the conformal prediction, we then use a finite-scenario method to model the distribution and add the corresponding linear cuts to the VRP model. To the best of our knowledge, this is the first time that an ML-based lower bound cut generation strategy is used when implementing the LBBD method in the literature.
- We conduct a practical case study for a food delivery routing problem using real-world data. We find that for the data-driven stochastic VRP with the kNN method, the acceleration strategies and lower bound improvement heuristic implemented in the LBBD decrease the average absolute gap by about 93% when compared with the default LBBD and 90% when compared with a genetic algorithm based on Vidal et al. (2012). For KDE, the improvement is about 98% on average when compared with the default LBBD and 91% when compared with the genetic algorithm. For the out-of-sample simulated performance, the results show that our nonparametric model with routing decisions reduces the median of the out-of-sample expected delay by about 23.1% on average when compared with an adjusted version of the assignment parametric models proposed by Liu et al. (2021) and about 25.8% when compared with the order-assignment decisions obtained from the DOA-DRO (data-driven order assignment with DRO) model used in Liu et al. (2021). Moreover, our suggested routes significantly outperform arc-based VRP models that ignore the endogenous uncertainty and the implemented route decisions observed in the real data set. We also quantify the value of the proposed routes with different service deadlines.

The remainder of this paper is organized as follows. Section 2 reviews the literature on stochastic VRPs with deadlines, ML-based VRPs, and the LBBD method for VRPs. Section 3 presents our data-driven stochastic VRP. We describe our LBBD solution scheme to solve the problem in Section 4. We further propose several acceleration strategies and develop a lower bound heuristic. We finally conduct an extensive case study using real-world food delivery data in terms of model training, computational efficiency of the proposed LBBD solution scheme, and out-of-sample performance of our data-driven solutions in Section 5. Section 6 concludes this paper with a summary of the important findings. The e-companion contains the appendices of this paper, which include all the proofs and supporting material. For instances, Appendix EC.8 discusses the data-driven DRO-VRP formulation under type q-Wasserstein ambiguity set. The appendix also presents closed-form expressions for the inner worst-case expectation problem, a solution scheme for data-driven DRO-VRP, and the out-of-sample performance for the data-driven DRO-VRP model.

2. Literature Review

We identify three streams of relevant literature. In Section 2.1, we briefly review the studies on the stochastic VRPs with deadlines. In Section 2.2, we review the studies using ML methods to handle VRPs and also briefly connect the general problem with the framework of integrated ML and optimization under uncertainty. Finally, Section 2.3 reviews the studies on the LBBD method for VRPs and the ML-based cut generation strategies for the decomposition method.

2.1. Stochastic VRPs with Deadlines

VRPs are well-studied problems in the field of transportation science, which have received great attention. There are several reviews and surveys for VRPs in the literature. For instance, Laporte (1992) reviewed the main exact and approximate algorithms developed for VRPs in the early days. Coelho et al. (2014) gave a comprehensive review of the literature on VRPs integrated with inventory management.

Most studies assume that all the parameters are deterministic (e.g., Laporte 1992). However, some parameters (e.g., customer demand, travel time) can be uncertain in practice. Thus, a number of studies on stochastic VRPs have proposed mathematical optimization frameworks for solving VRPs under different sources of uncertainties. Generally speaking, there are mainly two types of solution schemes for handling uncertainties in the literature: stochastic programming and RO. The stochastic programming approach typically assumes that the distribution of random uncertain parameters is known, which can be estimated by historical samples. In contrast, RO assumes that the distribution of random uncertain parameters is unknown and is assumed to belong to a predefined uncertainty set.

Because our work is related to stochastic VRPs with deadlines, in the following, we mainly focus on the literature on stochastic VRPs with deadlines and different sources of uncertainties. Extensive studies have explored the use of a stochastic programming approach for this type of problem. Generally, existing studies fit broadly into different categories in terms of the sources of uncertainty: stochastic demand (e.g., Hooshmand Khaligh and MirHassani 2016, Ghosal et al. 2023), uncertain travel time (e.g., Adulyasak and Jaillet 2016), and stochastic availability of vehicles (Torres et al. 2022). In particular, Hooshmand Khaligh and MirHassani (2016) proposed a multistage stochastic programming model for a single-vehicle routing problem under endogenous demand uncertainty from a dynamic viewpoint when the expected total routing cost is minimized. However, the problem size that can be addressed by their method is limited.

Our work differs from the above studies because (1) we propose a novel prescriptive ML approach for a data-driven stochastic VRP with deadlines and uncertain travel time; (2) motivated by real-world food delivery problem, we capture the feature of VRP where the uncertain travel time depends on the routes, and thus incorporate the endogenous uncertainty in the VRP; and (3) we propose an efficient LBBD algorithm for solving the resulting model with instances of realistic size.

2.2. Machine Learning for VRPs

With advances in data science techniques and the rapid availability of data in real-life applications, there has been rapidly increasing interest in the applications of ML tools to address the challenges (e.g., uncertainty and dynamics) that are involved in real-world VRP applications (Soeffker et al. 2022). These ML tools have been integrated with analytical/optimization approaches to strengthen problem formulations or improve algorithmic performance under different problem settings.

One of the main streams of research is to use reinforcement learning (RL) to address the dynamics of real-time vehicle routing and relocation problems. Its idea is to employ RL techniques to improve the VRP algorithms and construct solutions (e.g., Nazari et al. 2018). It trains a state-based decision policy by interacting with the environment and observing the rewards. For example, Nazari et al. (2018) proposed an end-to-end framework for solving VRPs using RL, where they trained a single policy model that finds near-optimal solutions only by observing the reward signals and following feasibility rules. We refer interested readers to surveys by Hildebrandt et al. (2023) for more recent advances in using RL for dynamic VRPs in logistics and supply chain management. Although RL-based methods provide an efficient way to handle the dynamics of real-world VRPs, they require a tremendous amount of data to explore high-dimensional state-action spaces and simplify the problem by ignoring uncertainties and related covariates to ensure efficient training.

Recently, the method of prescriptive analytics has been rapidly developed for data-driven decision-making problems. The main idea is to first predict or estimate the uncertainty behavior using data and then find optimal decisions by solving an optimization model that depends on the output of prediction or estimation (e.g., Nguyen et al. 2020, Sadana et al. 2025). The method of prescriptive analytics has also been widely used in various applications (Mišić and Perakis 2020), for example, portfolio selection, inventory management, and last-mile delivery.

Our work is more closely related to this stream of research, which mainly relies on this prescriptive ML method to address a general data-driven stochastic VRP. We employ ML techniques to handle the travel time uncertainty and estimate the distribution of travel

time, which is dependent on the routing decisions. To the best of our knowledge, our work is the first to propose a prescriptive ML approach to address a general data-driven stochastic VRP with deadlines and endogenous travel time in the literature. Because related studies for applying the discussed prescriptive ML method to address a general stochastic VRP are rare, here, we also include studies that are close to VRPs and their variants, for example, meal delivery with fixed routes (Hildebrandt and Ulmer 2022), order assignment (Liu et al. 2021), and deterministic last-mile delivery problem (Serkan Özarık et al. 2024). Since the initial release of our work, Serrano et al. (2024) studied a contextual VRP with time windows, where side information is used to predict the joint distribution of exogenous arc travel times. They introduced three approximations of the problem, each taking a different perspective in dealing with uncertain travel times and features. Besides, they combined side information with node and route characteristics, such as positions and time windows of customers in the routes, and adopted a supervised learning setting to approximate the expected penalty of a solution. Using the instances from Solomon (1987), they showed that the feature-dependent sample average approximation performs best among the proposed perspective models. We note that although our work focuses on a *noncontextual* version of the VRP with *route-dependent* travel time distributions, the approach we propose could be modified to account for contextual information. Our work, however, focuses on how to handle the dependence of travel time on routing decision with the help of nonparametric ML approaches and on characterizing the gains that can be achieved by accounting for it in a real-world setting.

Perhaps the most closest related study consists of Liu et al. (2021), who proposed a data-driven decision-making framework that integrates MILP representable travel-time predictors with an order-assignment problem. Leveraging recent advances in stochastic and DRO, they discussed tractable predictors and linear prediction models and derived the reformulations of integrated models, where they assumed that the distribution of service time is unknown but resides in a moment-based ambiguity set. The resulting model can be efficiently solved by the proposed branch-and-price algorithm. Using data for a real-world food delivery problem, they demonstrated the superior performance of their proposed order-assignment models over the sample average approximation (SAA) approach. Our work differs from theirs with respect to the following aspects: (1) instead of only deciding the order dispatch decision, we further incorporate the routing decisions in the data-driven decision-making model and address a general VRP model with deadlines; (2) our model accounts for arbitrary ML travel-time predictors, whereas Liu et al. (2021) needed MILP representable ones; (3) we consider travel time uncertainty and use nonparametric ML approaches to estimate its distribution, which is dependent on the routing decisions; in contrast, Liu et al. (2021) provided a point estimate on the average total travel time; (4) we also propose an efficient LBBD algorithm with acceleration strategies to solve the resulting VRP model of realistic size; and (5) using the same set of real-world data, we show that our proposed model outperforms their model in terms of the expected delay.

2.3. Logic-Based Benders Decomposition for VRPs

Another stream of related research consists of using LBBD for solving VRPs and ML for improving the decomposition schemes. The LBBD method is a generalized Benders decomposition that allows subproblems to be mixed-integer programs (as opposed to linear programs in classical Benders decomposition). The LBBD method has been proven to efficiently solve a large number of combinatorial optimization problems, such as bin-packing problems, VRPs, home healthcare delivery, and location-transportation problems. We refer interested readers to a survey by Rahmaniani et al. (2017) for more recent advances in LBBD.

Specifically, several studies employ the LBBD method to solve VRP variants in the literature (e.g., Li et al. 2022). Most of these studies implement LBBD with enhancements, including optimization cut, branch-and-check, and subproblem relaxation.

In this paper, we also propose an efficient LBBD method with acceleration strategies to solve stochastic VRP, including warm start, local search, and optimality cuts. However, given the difficulty of our model involving route-dependent travel time uncertainty, we propose an ML-based lower bound heuristic to solve the problems of realistic size efficiently. To the best of our knowledge, we are the first to propose an ML-based lower bound cut generation strategy in the implementation of the LBBD method in the literature.

In this regard, our work is also related to the literature on the use of ML tools to generate or learn the cuts for improving the algorithmic efficiency of decomposition approaches when solving stochastic programs. However, the related literature is very limited. More recently, Larsen et al. (2022) proposed a method to quickly predict expected tactical descriptions of operational solutions in the context of two-stage stochastic programming when the second stage is demanding computationally. Larsen et al. (2024) studied mixed-integer linear two-stage stochastic programs where the second stage is computationally demanding. Their idea is to gain large reductions in online solution time while incurring small reductions in first-stage solution accuracy by substituting the exact second-stage solutions of Benders decomposition with fast and accurate

supervised ML predictions. Unlike the aforementioned studies, we use an ML-based method to model a distribution, which is approximately first-order stochastically dominated by the distribution of travel time, and add the corresponding linear constraints to the master problem of LBBD method.

3. Problem Description and Formulation

In Section 3.1, we introduce the proposed routing set (see, e.g., Adulyasak and Jaillet 2016, Zhang et al. 2021) used in our model. Then, we present a data-driven stochastic model formulation for VRP with deadlines and endogenous travel time uncertainty in Section 3.2.

3.1. Description of the Routing Set

The routing set is to design at most L a priori routes to service all customers. It requires that each customer location be serviced exactly once and by only one vehicle; each vehicle, if used, starts from the depot and ends at the last visited customer location without exceeding its capacity. The problem is defined on a complete and directed graph $\hat{\mathcal{G}} = (\hat{\mathcal{N}}, \hat{\mathcal{A}})$, where $\hat{\mathcal{N}} = \{0\} \cup \mathcal{N}_c$ and $\hat{\mathcal{A}} = \{(i,j) | i,j \in \hat{\mathcal{N}}, i \neq j\}$ represent a set of nodes and arcs, respectively. Node 0 is the depot, which is equipped with L homogeneous vehicles of maximum capacity Q, and nodes in set $\mathcal{N}_c = \{1, \ldots, I\}$ represent a set of I customer locations. Each node $i \in \mathcal{N}_c$ has a demand d_i. The latest time for starting the service (i.e., deadline) is denoted by τ, and all customer locations have the same τ. For each arc $(i,j) \in \hat{\mathcal{A}}$, the random travel time is denoted by $\tilde{t}_{ij}$. Without loss of generality, we assume that for $(i,j) \in \hat{\mathcal{A}}$, $\tilde{t}_{ij}$ includes the service time at node $i \in \mathcal{N}_c$.

We add L dummy nodes for vehicles: $\mathcal{N}_d = \{I+1, \ldots, I+L\}$ to denote the destinations (see, e.g., Adulyasak and Jaillet 2016). We then transform the original graph into an extended one $\mathcal{G} = \{\mathcal{N}, \mathcal{A}\}$, where $\mathcal{N} = \{0\} \cup \mathcal{N}_c \cup \mathcal{N}_d$, $\mathcal{A} = \mathcal{A}_o \cup \mathcal{A}_c \cup \mathcal{A}_d$ with $\mathcal{A}_o = \{(0,j) | j \in \mathcal{N}_c\}$, $\mathcal{A}_c = \{(i,j) | i,j \in \mathcal{N}_c, i \neq j\}$, and $\mathcal{A}_d = \{(i,j) | i \in \mathcal{N}_c, j \in \mathcal{N}_d\}$. In the extended graph, for the node $i \in \mathcal{N}_d$, the demand $d_i = 0$, and the travel time of the arcs in $\mathcal{A}_d$ is set to zero. Given any subset $\mathcal{H} \subseteq \mathcal{N}$ of nodes, the sets of their outgoing and incoming arcs are defined as

$$E^+(\mathcal{H}) = \{(i,j) \in \mathcal{A} \mid i \in \mathcal{H}, j \in \mathcal{N} \backslash \mathcal{H}\},$$
$$E^-(\mathcal{H}) = \{(j,i) \in \mathcal{A} \mid i \in \mathcal{H}, j \in \mathcal{N} \backslash \mathcal{H}\}.$$

For simplicity, we use (i,j) and a interchangeably to represent an arc in $\mathcal{A}$, and $E^+(i)$ and $E^+(\{i\})$ interchangeably for $i \in \mathcal{N}$.

The following decision variables are used in the routing set. The binary variable $x_a = 1$ if a is part of some vehicle's route, and zero otherwise. The binary variable s_a^ℓ is equal to one if arc a is part of the route to node $\ell \in \mathcal{N}_d$. We denote $\boldsymbol{s} = (\boldsymbol{s}^\ell)_{\ell \in \mathcal{N}_d}$ and $\boldsymbol{s}^\ell \in \{0,1\}^{|\mathcal{A}|}$, where $|\cdot|$ is the cardinality of a set. A feasible routing set $\mathcal{S}$ is defined as

$$\mathcal{S} := \{\boldsymbol{x} \in \{0,1\}^{|\mathcal{A}|}, \boldsymbol{s} \in \{0,1\}^{|\mathcal{A}| \times |\mathcal{N}_d|} | (1a) - (1g)\},$$

namely, that there exist an $\boldsymbol{x} \in \{0,1\}^{|\mathcal{A}|}$ such that

$$\sum_{a \in E^+(i)} x_a = \sum_{a \in E^-(i)} x_a = 1, \qquad \forall i \in \mathcal{N}_c, \tag{1a}$$

$$\sum_{a \in E^+(0)} x_a = \sum_{\ell \in \mathcal{N}_d} \sum_{a \in E^-(\ell)} x_a, \tag{1b}$$

$$\sum_{a \in E^-(\ell)} x_a \leq 1, \qquad \forall \ell \in \mathcal{N}_d, \tag{1c}$$

$$\sum_{a \in E^+(0)} s_a^\ell = \sum_{a \in E^-(\ell)} s_a^\ell = \sum_{a \in E^-(\ell)} x_a, \quad \forall \ell \in \mathcal{N}_d, \tag{1d}$$

$$\sum_{a \in E^-(i)} s_a^\ell - \sum_{a \in E^+(i)} s_a^\ell = 0, \qquad \forall \ell \in \mathcal{N}_d, i \in \mathcal{N}_c, \tag{1e}$$

$$s_a^\ell \leq x_a, \qquad \forall \ell \in \mathcal{N}_d, a \in \mathcal{A}, \tag{1f}$$

$$\sum_{a \in E^+(\mathcal{H})} x_a \geq \left\lceil \frac{\sum_{i \in \mathcal{H}} d_i}{Q} \right\rceil, \qquad \forall \mathcal{H} \subseteq \mathcal{N}_c, |\mathcal{H}| \geq 2. \tag{1g}$$

Constraints (1a) state that all customers must be visited. Constraints (1b) ensure that the numbers of vehicles leaving the depot and arriving at the destination equal each other. Constraints (1c) denote that one can choose whether to visit the nodes in set $\mathcal{N}_d$. Constraints (1d) are used to guarantee that the route to node ℓ should start at the depot and arrive at node ℓ. Constraints (1e) denote that the flow balance should hold at the intermediate nodes. Constraints (1f) link the arc variables x_a and the route variables s_a^ℓ. Constraints (1g) are subtour elimination and vehicle capacity constraints, which are also referred to as rounded capacity inequalities (Lysgaard et al. 2004). One can also add the following set of symmetry-breaking constraints:

$$\sum_{a \in \mathcal{A}} s_a^\ell \leq \sum_{a \in \mathcal{A}} s_a^{\ell-1}, \quad \forall \ell \in \mathcal{N}_d \backslash \{I+1\}. \tag{2}$$

Constraints (2) impose that the number of customers served by vehicle ℓ is larger than the number served by $\ell - 1$, which implicitly ensure that if vehicle ℓ is used to serve the customers, vehicle $\ell - 1$ must be used.

For our multiple capacitated vehicles problem, as stated by Adulyasak and Jaillet (2016), one could use the original graph and incorporate the index $\ell \in \{1, \ldots, L\}$ into variables $\boldsymbol{x}$ and $\boldsymbol{s}$. Then, one could impose vehicle capacity constraints on variable $\boldsymbol{x}$ for each vehicle separately. However, the problem size would be significantly increased. Therefore, in this paper, we use the extended graph to formulate the feasible routing set.

3.2. Data-Driven Stochastic VRP Formulation

Motivated by a real-world food delivery problem, we construct a route-dependent travel time distribution in

order to better capture the drivers' behaviors and make the routing decisions for the drivers. We consider the VRP with deadlines and endogenous uncertainty, where the distribution of the random travel time for routes depends on the decision $\boldsymbol{s}$. The objective is to minimize the total expected delay for each route, which is widely used in the literature about VRP with deadlines (e.g., Adulyasak and Jaillet 2016). More specifically, the VRP model is given as follows:

$$\underset{(\boldsymbol{x},\boldsymbol{s})\in\mathcal{S}}{\text{minimize}} \sum_{\ell\in\mathcal{N}_d} \mathbb{E}_{\mathbb{P}_{(\boldsymbol{s}^\ell)}}(\tilde{\xi}-\tau)^+, \tag{3}$$

where $\tilde{\xi}$ is the random travel time for route $\boldsymbol{s}^\ell$ with the distribution $\mathbb{P}_{(\boldsymbol{s}^\ell)}$. If the travel time uncertainty $\tilde{\boldsymbol{t}}$ for each arc follows a known probability distribution $\mathbb{P}$ independent of $\boldsymbol{s}^\ell$, then $\tilde{\xi}=\tilde{\boldsymbol{t}}^\top\boldsymbol{s}^\ell$, and the distribution $\mathbb{P}_{(\boldsymbol{s}^\ell)}$ in Problem (3) can be replaced by the distribution $\mathbb{P}$.

In a realistic situation, the decision maker can collect data on the travel time, thus providing a more accurate description of the uncertain parameters that can be used to make the present decision. Given a set of historical data $\{\hat{\xi}^1,\dots,\hat{\xi}^N\}$, where N is the number of scenarios, $\hat{\xi}^i\in\mathbb{R}_+$ represents the total travel time about the route $i=\{0,n^i_{(1)},\dots,n^i_{(v_i)}\}$ for $i=1,\dots,N$, $n^i_{(j)}\in\mathcal{N}_c$ is the jth customer location in the route i, and v_i is the number of customer locations serviced by the route i, we exploit the real data to capture the drivers' behavior and describe the uncertainty with

$$\mathbb{P}_{(\boldsymbol{s}^\ell)}=\sum_{i=1}^{N(\boldsymbol{s}^\ell)} p_i(\boldsymbol{s}^\ell)\delta_{\xi^i(\boldsymbol{s}^\ell)}, \tag{4}$$

where $N(\boldsymbol{s}^\ell)$, $p_i(\boldsymbol{s}^\ell)$, and $\xi^i(\boldsymbol{s}^\ell)\in\mathbb{R}^+$ are the number of scenarios, the probability (or weight), and the total travel time for scenario $i\in\{1,\dots,N(\boldsymbol{s}^\ell)\}$, and are all represented as functions of $\boldsymbol{s}^\ell$. The $\delta_{\xi^i(\boldsymbol{s}^\ell)}$ is a Dirac distribution concentrating its unit mass at $\xi^i(\boldsymbol{s}^\ell)$. Then, the stochastic data-driven VRP (3) can be rewritten as follows:

$$\underset{(\boldsymbol{x},\boldsymbol{s})\in\mathcal{S}}{\text{minimize}} \sum_{\ell\in\mathcal{N}_d}\sum_{i=1}^{N(\boldsymbol{s}^\ell)} p_i(\boldsymbol{s}^\ell)(\xi^i(\boldsymbol{s}^\ell)-\tau)^+. \tag{5}$$

The Objective Function (5) can be seen as a weighted average of historical delay observations.

There are several ways for ML algorithms to construct such probability functions. In this paper, we focus on kNN and KDE methods, which are popular nonparametric ML methods with great prediction power (Davis et al. 2011, Biau and Devroye 2015). For historical route $i\in\{1,\dots,N\}$, let $\hat{\boldsymbol{\gamma}}^i\in\mathbb{R}_+$ be the feature of this route and $\bar{\boldsymbol{\gamma}}^\ell$ be the feature of the route $\boldsymbol{s}^\ell$.[1] We describe the following probability functions:

Definition 1. The kNN for constructing a probability function is defined as follows:

$$p_i(\boldsymbol{s}^\ell)=\begin{cases}1/k, & \text{if } \hat{\boldsymbol{\gamma}}^i \text{ is a } k\text{-nearest neighbor of } \bar{\boldsymbol{\gamma}}^\ell,\\ 0, & \text{otherwise.}\end{cases}$$

$\hat{\boldsymbol{\gamma}}^i$ is a k-nearest neighbor of $\bar{\boldsymbol{\gamma}}^\ell$ if $|\{j\in\{1,\dots,n\}\backslash i: \|\hat{\boldsymbol{\gamma}}^j-\bar{\boldsymbol{\gamma}}^\ell\|<\|\hat{\boldsymbol{\gamma}}^i-\bar{\boldsymbol{\gamma}}^\ell\|\}|<k$, where it is assumed for simplicity that there are no ties.

Definition 2. The KDE for constructing a probability function is defined as follows:

$$p_i(\boldsymbol{s}^\ell)=\frac{K_H(\bar{\boldsymbol{\gamma}}^\ell-\hat{\boldsymbol{\gamma}}^i)}{\sum_{j=1}^n K_H(\bar{\boldsymbol{\gamma}}^\ell-\hat{\boldsymbol{\gamma}}^j))},$$

where $K(\cdot)$ is a kernel function, $K_{\boldsymbol{H}}(\boldsymbol{x})=|\boldsymbol{H}|^{-1/2}K(\boldsymbol{H}^{-1/2}\boldsymbol{x})$, and $\boldsymbol{H}$ is bandwidth diagonal matrix. In this paper, we consider the Gaussian kernel function given by

$$K(\boldsymbol{x})=(2\pi)^{-d/2}\exp\left(-\frac{1}{2}\boldsymbol{x}^\top\boldsymbol{x}\right).$$

Nonparametric methods (also called distribution-free methods) are statistical methods that do not rely on the assumption that the data are drawn from a specific parametric family of probability distributions (Wasserman 2006). For the kNN and KDE methods, extremely large values of the parameter k and elements in the bandwidth matrix $\boldsymbol{H}$ imply that the VRP Model (5) reduces to a problem with an equal probability over all observations for all routes. Conversely, by using small values, most of the probability mass is assigned to the samples closest to route $\boldsymbol{s}^\ell$. For more details on the above methods, see García-Portugués (2023).

For more complex conditional generative models with complicated nonlinear structures, such as the conditional Gaussian mixture model (CGMM) (see, e.g., Gilardi et al. 2002), they are also compatible with our VRP model. It is well-known that conditional generative models can easily generate numerical characteristics of the underlying distributions or other performance estimations through Monte Carlo methods (Huang et al. 2022). Therefore, compared with the kNN and KDE methods, generative models are able to generate new data, which have no obvious connection to probability distributions over samples of input variables and could be essential for evaluating the out-of-sample performance of routes obtained from Problem (5) and demonstrating the computational efficiency of our solution scheme that is proposed in Section 4.

In terms of the feature selection, we use the features proposed in the literature, which are based on the number of locations, visiting area, distance, dispersion, and their interactive terms (Liu et al. 2021). Çavdar and Sokol (2015) used $\sqrt{I(cstedv_a cstedv_b)}$ to address the dispersion, where $cstedv_a$ and $cstedv_b$ are the standard deviations of the latitudinal and longitudinal differences between the depot and customer locations,

respectively. Chien (1992) and Kwon et al. (1995) suggested $\sqrt{SI}$, $I\sqrt{S}$, $\sqrt{S'(I+1)}$, $\sqrt{S'(I+1)^3}$, and $\sqrt{\frac{S'}{I+1}}$ to account for the visiting area, where S and S' are the area of the smallest rectangle covering the customer locations visited by a driver and covering both the depot and the customer locations visited by a driver, respectively. As a result, the above features, as well as the number of customer locations I and the length of the routes, are included in the feature vector. Because of the availability of data, we focus our attention on the characteristics of the route itself in the empirical application. Although driver characteristics are not considered in this paper, these and other features (e.g., day of the week, weather) can be easily incorporated into our model.

4. A Logic-Based Benders Decomposition Solution Scheme

In this section, we first propose an LBBD solution procedure for the data-driven stochastic VRP with deadlines in Section 4.1. Section 4.2 presents our proposed acceleration strategies. In Section 4.3, we develop an ML-based lower bound heuristic.

4.1. A Logic-Based Benders Decomposition Algorithm

Although some ML models can be reformulated as convex representations under some assumptions, it is hard to extend them to more general problems, and this poses a major challenge in solving larger and more realistic instances. For example, Liu et al. (2021) used tractable feature vectors and linear ML models, which are compatible with their order-assignment optimization, to predict travel time. Grimstad and Andersson (2019) and Anderson et al. (2020) assumed that the nonlinear operator in deep neural networks is a rectified linear unit. They formulated the deep neural networks as a zero-one mixed integer linear program with the big-M method. When it comes to larger and more realistic deep neural networks, the computing time can become too large.

To overcome these issues, in this section, we use an LBBD to solve our proposed VRP with the features proposed in Section 3.2. One advantage of the LBBD algorithm is that it allows us to solve the data-driven VRP without any assumptions about the problem structure. Moreover, as indicated by our numerical study, using the strategies proposed later in this paper can greatly improve the performance of the LBBD algorithm. In particular, we decompose the problem as a master problem and a set of subproblems. In the master problem, we determine the routes without considering the expected delay, that is, the master problem is a relaxation of the data-driven VRP where the expected delay is ignored. Hence, the master problem is a combinatorial optimization problem. For each route obtained from the master problem, we compute the probabilities based on the nonparametric ML methods proposed in Section 3.2 and the expected delay in the subproblems.

Because the subproblems are always feasible, when we obtain a feasible integer solution from the master problem, we only generate optimality cuts for each subproblem and add them to the master problem. Unlike the traditional LBBD, which solves the master problem and a set of subproblems iteratively and introduces optimality cuts in the master problem, we add the optimality cuts in a branch-and-cut framework, so the master problem is solved only once.

The master problem is defined as follows:

$$\text{(MP)} \quad \underset{(\boldsymbol{x},\boldsymbol{s})\in\mathcal{S},\,\boldsymbol{\eta}\geq\mathbf{0}}{\text{minimize}} \sum_{\ell\in\mathcal{N}_d}\eta_\ell. \tag{6}$$

Let $(\bar{\boldsymbol{x}},\bar{\boldsymbol{s}})$ be a feasible solution of (MP). LBBD relies on adding valid optimality cuts for each η_ℓ that will be added to the MP. Whereas one could simply employ the simple optimality cut

$$\eta^\ell \geq obj^\ell - obj^\ell \sum_{a:\bar{s}_a^\ell=1}(1-s_a^\ell), \quad \forall \ell\in\mathcal{N}_d, \tag{7}$$

where $obj^\ell := \sum_{i=1}^{N(\bar{\boldsymbol{s}}^\ell)} p_i(\bar{\boldsymbol{s}}^\ell)(\xi^i(\bar{\boldsymbol{s}}^\ell)-\tau)^+$, one can obtain richer optimality cuts by assuming that the travel time distribution of a route is independent of which vehicle is servicing it. To formalize this assumption, we first define for each vehicle $\ell\in\mathcal{N}_d$, the set of arcs of the route that it follows; that is:

$$\mathcal{R}(\bar{\boldsymbol{s}}^\ell) := \{(0,n^\ell_{(1)}),\ldots,(n^\ell_{(v_\ell-1)},n^\ell_{(v_\ell)})\},$$

where $n^\ell_{(i)}\in\mathcal{N}_c$ is the ith customer location in the route, and v_ℓ is the number of customer locations serviced by the route.

Assumption 1 (Vehicle-Insensitive Travel Time). *For all $\ell_1,\ell_2\in\mathcal{N}_d$ and all $(\boldsymbol{x}_1,\boldsymbol{s}_1),(\boldsymbol{x}_2,\boldsymbol{s}_2)\in\mathcal{S}$:*

$$\mathcal{R}(\bar{\boldsymbol{s}}_1^{\ell_1}) = \mathcal{R}(\bar{\boldsymbol{s}}_2^{\ell_2}) \Rightarrow \mathbb{P}_{(\bar{\boldsymbol{s}}_1^{\ell_1})} = \mathbb{P}_{(\bar{\boldsymbol{s}}_2^{\ell_2})}.$$

Assumption 1 is a natural one to employ when the fleet of vehicles is homogeneous and historical data cannot be used to infer the route-dependent travel time distribution of each specific driver.

Assumption 1 allows to replace the optimality cuts described in (7) with

$$\eta^j \geq obj^\ell - obj^\ell \sum_{a\in\mathcal{A}^j(\bar{\boldsymbol{s}}^\ell)}(1-s_a^j), \quad \forall j,\ell\in\mathcal{N}_d, \tag{8}$$

where

$$\mathcal{A}^j(\bar{\boldsymbol{s}}^\ell) = \{(0,n^\ell_{(1)}),\ldots,(n^\ell_{(v_\ell)},j)\}.$$

Optimality Cuts (8) indicate that if $s_a^j=1$ for all $a\in\mathcal{A}^j(\bar{\boldsymbol{s}}^\ell)$, then the expected delay is at least obj^ℓ, whereas it is otherwise at least zero. This cut is valid because the expected delay is always nonnegative and, according to Assumption 1, is equal to obj^ℓ when its route is the same as the route of $\bar{\boldsymbol{s}}^\ell$.

Algorithm 1 in Appendix EC.1.1 in the e-companion describes the detailed procedure of the LBBD algorithm. It is well-known that the LBBD algorithm terminates in finitely many iterations, as it is based on branching on a finite number of binary variables. In our implementation, a new feasible solution obtained from the LP relaxation of (MP) always has zero objective value; hence, the LB is equal to zero except that the algorithm could enumerate all the possible feasible routes or obtain a very small upper bound (UB) value. Therefore, the LBBD algorithm generally yields a large gap, and the computational cost might be prohibitive if the historical travel times are significantly larger than the service deadline τ.

4.2. Acceleration Strategies

In Section 4.2.1, we provide a class of general optimality cuts extended from the Traditional Optimality Cuts (8) for the LBBD algorithm. We present a warm-starting strategy in Section 4.2.2 and local search procedures in Section 4.2.3.

4.2.1. A General Optimality Cut. Optimality Cuts (8) can be weak, which are useful only when $s_a^j = 1$ for all $a \in \mathcal{A}^j(\bar{\boldsymbol{s}}^\ell)$ and $j \in \mathcal{N}_d$; that is, it only includes the expected delay for one route. Moreover, in the worst case, it needs to enumerate all possible feasible routes to get an optimal solution, thus requiring long computing time, especially for large-scale instances. Herein, we extend Cuts (8) to new general optimality cuts, which contain Cuts (8) and the expected delay for all the subroutes. Thus, within a tight time limit, (MP) could include more information about the routes by adding the general cuts when compared with Cuts (8). In particular, for the general optimality cuts, we start with the first customer location visited by a vehicle, and then we sequentially add other locations to the cuts such that any subroute has the value of expected delay.

Given a feasible route $\bar{\boldsymbol{s}}^\ell$ obtained from (MP), for $i = 1, \ldots, v_\ell$, let the expected delay for the partial route $\{(0, n^\ell_{(1)}), \ldots, (n^\ell_{(i-1)}, n^\ell_{(i)})\}$ be denoted by $obj^\ell_{(i)}$, where $n^\ell_{(0)}$ is node 0. Then, the general optimality cuts take the following form:

$$\begin{aligned}\eta^j \geq\ & obj^\ell_{(1)} s^j_{(0,n_{(1)})} - M \sum_{i \in \mathcal{N}_c \setminus \{n_{(1)}\}} s^j_{(0,i)} \\ & + (obj^\ell_{(2)} - obj^\ell_{(1)}) s^j_{(n_{(1)},n_{(2)})} - M \sum_{i \in \mathcal{N}_c \setminus \{n_{(2)}\}} s^j_{(n_{(1)},i)} + \\ & \cdots \\ & + (obj^\ell_{(v_\ell)} - obj^\ell_{(v_\ell-1)}) s^j_{(n_{(v_\ell-1)},n_{(v_\ell)})} \\ & - M \sum_{i \in \mathcal{N}_c \setminus \{n_{(v_\ell)}\}} s^j_{(n_{(v_\ell-1)},i)} - M \sum_{i \in \mathcal{N}_c} s^j_{(v_\ell,i)}, \ \forall j \in \mathcal{N}_d, \end{aligned} \tag{9}$$

where M is a positive constant guaranteeing that the general cuts are inactive when the vehicle does not visit these subroutes. We can simply set M equal to the sum of positive coefficients in the cut. The following proposition gives the validity of the General Optimality Cuts (9).

Proposition 1. *Given Assumption 1, the General Optimality Cuts (9) are valid for the data-driven VRP (5).*

4.2.2. Warm Start. LBBD algorithms are well-known to suffer from ineffective initial iterations because of low-quality solutions generated at the beginning of the algorithm. In this section, we use a warm-starting procedure to generate a good initial solution for (MP). The underlying idea of the warm-starting procedure is to use the hybrid genetic search proposed by Vidal et al. (2012) to solve a total travel cost minimization problem, and then a feasible solution of the problem with the smallest expected delay is used to warm start the LBBD algorithm. Specifically, the total travel cost minimization problem is

$$\underset{(\boldsymbol{x},\boldsymbol{s}) \in \mathcal{S}}{\text{minimize}} \sum_{a \in \mathcal{A}} c_a x_a, \tag{10}$$

where c_a is transportation cost of arc $a \in \mathcal{A}$. In this paper, we take c_a as the distance associated with arc a. To equally use all the vehicles, one can limit the number of customer locations visited by one vehicle to no more than $\lceil I/L \rceil + 1$.

The hybrid genetic search proposed by Vidal et al. (2012) can be used to solve Problem (10). Vidal et al. (2012) proposed an algorithm framework to address the capacitated VRP with constrained route duration, and the goal is to minimize the total travel cost. The general scheme of the heuristic includes a population of individuals with feasible and infeasible subpopulations and then selects two parent individuals to produce a new individual, which is enhanced by using local search procedures. It inserts the new enhanced individual into the appropriate subpopulation based on its feasibility. In order to use the hybrid genetic search method, we change the constrained route duration to address the number of customer location limit and obtain a population of individuals with feasible subpopulation $\{\boldsymbol{x}^m, \boldsymbol{s}^m\}_{m=1,\ldots,M'}$. Then, we compute the expected delay obj^m of $(\boldsymbol{x}^m, \boldsymbol{s}^m)$ for $m = 1, \ldots, M'$ and let $m^* = \arg\min_{m=1,\ldots,M'} obj^m$. Finally, we reorder the vehicle indices for $(\boldsymbol{x}^{m^*}, \boldsymbol{s}^{m^*})$ to satisfy the Symmetry-Breaking Constraints (2) and let $\hat{\eta} = obj^{m^*}$. Thus, $(\boldsymbol{x}^{m^*}, \boldsymbol{s}^{m^*}, \hat{\eta})$ is a feasible solution of Problem (5) and can be used as a warm start for the LBBD algorithm. Algorithm 2 in Appendix EC.1.2 outlines the warm-starting procedure. As stated in the case study, solving the hybrid genetic search algorithm with the objective of minimizing the travel cost is computed within seconds. Thus, it doesn't increase the computational burden.

4.2.3. Local Search. To speed up the computation, particularly for large-scale instances, we introduce local

search procedures into our LBBD algorithm. The local search procedures have been used in many heuristic frameworks, such as the hybrid genetic search (Vidal et al. 2012) and the variable neighborhood search. In Section 5, we numerically show that the LBBD algorithm can be significantly improved by incorporating the local search procedures and outperforms a heuristic based on the hybrid genetic search with the local search procedures.

The local search procedures include relocations, which insert one or two consecutive customers to another position; swaps, which exchange one or two consecutive customers with another position; 2-opt, an intraroute move where a selected sequence of customers is reversed; and 2-opt*, where the end parts of two different routes are interchanged, or the start and end parts of two different routes are connected, respectively. The local search phase stops when all possible procedures have been successively tried and failed to produce any improvement. For further details about the local search procedures, interested readers are referred to Vidal et al. (2012) and Vidal (2022).

To incorporate the local search procedures, we start from an integer solution $(\bar{\boldsymbol{x}}, \bar{\boldsymbol{s}})$ obtained from (MP) and calculate the corresponding expected delay $\bar{obj}$. $\bar{\boldsymbol{s}}$ is then improved by the local search procedures. If a new solution $\bar{\boldsymbol{s}}'$ is better than the incumbent $\bar{\boldsymbol{s}}$ in terms of the expected delay, then we construct $\bar{\boldsymbol{x}}'$ based on $\bar{\boldsymbol{s}}'$ after reordering the vehicle indices to satisfy the symmetry-breaking constraints, and we set the new solution $(\bar{\boldsymbol{x}}', \bar{\boldsymbol{s}}')$ as the incumbent solution and update the UB; otherwise, it continues with the next iteration. The incorporation of local search procedures is formalized as Algorithm 3 in Appendix EC.1.3 in the e-companion.

4.3. Lower Bound Heuristic

It is well-known that the capacitated VRP is NP-hard. As a generalization of capacitated VRP, our Data-Driven Stochastic VRP (5) is also NP-hard. Although the numerical results show that the proposed acceleration strategies could effectively reduce the absolute gap, large-sized instances remain difficult to solve. Moreover, the lower bound obtained from the LBBD algorithm is usually equal to zero within a tight time limit. Thus, we introduce a novel lower bound heuristic to reduce the computational cost in proving optimality.

We first show how to use an $\boldsymbol{s}$ dependent distribution $\bar{\mathbb{P}}_{(\boldsymbol{s})}$, which is first-order stochastically dominated by the distribution under $\mathbb{P}_{(\boldsymbol{s})}$, for all $\boldsymbol{s}$, to obtain an equivalent formulation of problem (3) in Lemma 1.

Lemma 1. *For all $(\boldsymbol{x}, \boldsymbol{s}) \in \mathcal{S}$, let the distribution of $\tilde{\xi}$ under some $\bar{\mathbb{P}}_{(\cdot)}$ be uniformly first-order stochastically dominated by its distribution under $\mathbb{P}_{(\cdot)}$, that is,*

$$\bar{\mathbb{P}}_{(\boldsymbol{s}^\ell)}(\tilde{\xi} \le z) \ge \mathbb{P}_{(\boldsymbol{s}^\ell)}(\tilde{\xi} \le z), \ \forall z \in \mathbb{R}, \ \forall \ell \in \mathcal{N}_d, \ \forall (\boldsymbol{x}, \boldsymbol{s}) \in \mathcal{S},$$

and then Problem (3) is equivalent to the following problem:

$$\underset{(\boldsymbol{x},\boldsymbol{s})\in\mathcal{S},\boldsymbol{\eta}}{\text{minimize}} \ \sum_{\ell\in\mathcal{N}_d} \eta_\ell \tag{11a}$$

$$\text{subject to} \ \ \eta_\ell \ge \mathbb{E}_{\mathbb{P}_{(\boldsymbol{s}^\ell)}}[(\tilde{\xi}-\tau)^+], \quad \forall \ell \in \mathcal{N}_d, \tag{11b}$$

$$\eta_\ell \ge \mathbb{E}_{\bar{\mathbb{P}}_{(\boldsymbol{s}^\ell)}}[(\tilde{\xi}-\tau)^+], \quad \forall \ell \in \mathcal{N}_d. \tag{11c}$$

To model $\bar{\mathbb{P}}_{(\cdot)}$, we can use a discrete distribution model $\bar{\mathbb{P}}_{(\boldsymbol{s}^\ell)}(\tilde{\xi} \le y) := (1/(T+1))\mathbf{1}\{0 \le y\} + (1/(T+1)) \sum_{t=1}^{T} \mathbf{1}\{\sum_{a\in\mathcal{A}} \theta_a^t s_a^\ell + \bar{\theta}^t \le y\}$ for some $\theta_a^t, \bar{\theta}^t \in \mathbb{R}$ and $T \in \mathbb{Z}_+$. Here, $\mathbf{1}\{\cdot\}$ is an indicator function, which returns one if the expression in $\{\cdot\}$ is true. Then, Constraints (11c) can be expressed as follows:

$$\eta_\ell \ge (1/(T+1)) \sum_{t=1}^{T} \left(\sum_{a\in\mathcal{A}} \theta_a^t s_a^\ell + \bar{\theta}^t - \tau \right)^+, \quad \forall \ell \in \mathcal{N}_d. \tag{12}$$

Motivated by conformal prediction, which uses past experience to determine precise levels of confidence in new predictions (Shafer and Vovk 2008), we assemble a data set $\{(\hat{\boldsymbol{s}}^j, \boldsymbol{q}_j)\}_{j=1}^{J}$, where each $\boldsymbol{q}_j \in \mathbb{R}^T$ such that $q_j^t := Q^{t/(T+1)}_{\mathbb{P}_{(\hat{\boldsymbol{s}}_j)}}(\tilde{\xi})$, and $Q^{\alpha}_{\mathbb{P}}(Y)$ is the α quantile of Y under the distribution $\mathbb{P}$. One then solves the following linear program:

$$\underset{\boldsymbol{\theta}\ge 0}{\text{maximize}} \ \sum_{t=1}^{T} \sum_{j=1}^{J} \left(\sum_{a\in\mathcal{A}} \theta_a^t \hat{s}_a^j + \bar{\theta}^t \right) \tag{13a}$$

$$\text{subject to} \ \ q_j^t \ge \sum_{a\in\mathcal{A}} \theta_a^t \hat{s}_a^j + \bar{\theta}^t, \quad \forall t = 1,\dots,T, \ j = 1,\dots,J, \tag{13b}$$

$$\boldsymbol{\theta}^{t+1} \ge \boldsymbol{\theta}^t, \quad \forall t = 1,\dots,T-1. \tag{13c}$$

To evaluate $\boldsymbol{\theta}^t$ and $\bar{\theta}^t$ obtained from Problem (13) for $t = 1,\dots,T$, we randomly generate a set of routes $\{\hat{\boldsymbol{s}}'^j\}_{j=1,\dots,J'}$ and compute the following probability that the expected delay obtained by the finite scenario model is smaller than the one from the distribution $\mathbb{P}_{(\hat{\boldsymbol{s}}'^j)}$:

$$\frac{\sum_{j=1}^{J'} \mathbf{1}\{(1/(T+1)) \sum_{t=1}^{T} (\sum_{a\in\mathcal{A}} \theta_a^t \hat{s}_a'^j + \bar{\theta}^t - \tau)^+ \le \mathbb{E}_{\mathbb{P}_{(\hat{\boldsymbol{s}}'^j)}}[(\tilde{\xi}-\tau)^+]\}}{J'}.$$

The probability might be small; to correct the value of $\boldsymbol{\theta}^t$ and $\bar{\theta}^t$, we can replace Constraints (12) with the following constraints by using a hyperparameter $\rho \in \mathbb{R}_+$:

$$\eta_\ell \ge \rho(1/(T+1)) \sum_{t=1}^{T} \left(\sum_{a\in\mathcal{A}} \theta_a^t s_a^\ell + \bar{\theta}^t - \tau \right)^+, \quad \forall \ell \in \mathcal{N}_d. \tag{14}$$

Here, ρ is used to reduce the likelihood of introducing Incorrect Cuts (12). This could occur when the expected

delay obtained from $\boldsymbol{\theta}$ is larger than the exact one with a high probability.

Given a threshold ϵ', if the probability is less than ϵ', we decrease ρ; otherwise, we increase it. We can use a bisection method to find the proper ρ such that the probability meets the threshold. More specifically, the bisection method is presented as Algorithm 4 in Appendix EC.1.4 in the e-companion.

In order to use the LBBD algorithm with the acceleration strategies to solve Problem (11), we first add Constraints (14) to the Master Problem (6), leading to a new master problem of the following form:

$$\underset{(\boldsymbol{x},\boldsymbol{s})\in\mathcal{S},\,\boldsymbol{\eta}}{\text{minimize}} \quad \sum_{\ell\in\mathcal{N}_d}\eta_\ell \tag{15a}$$

$$\text{subject to} \quad (14). \tag{15b}$$

As indicated by our numerical study, when τ is small, the lower bound of the LBBD algorithm can be positive. The warm-starting procedure proposed in Section 4.2.2 can also provide an initial solution to the Master Problem (15). If a solution obtained from Problem (15) is integer, we apply the local search procedure to improve the solution and set the new solution as the incumbent solution if it has a smaller expected delay. Then, we add the general optimality cuts based on the incumbent solution to the Master Problem (15). Otherwise, we continue branching. The algorithm stops when the node list is empty or the lower bound is larger than or close to the upper bound. Algorithm 5 in Appendix EC.1.5 in the e-companion describes the procedure of our implementation of the LBBD algorithm, including the acceleration strategies and the lower bound heuristic.

5. A Real-World Food Delivery Case Study

In Section 5.1, we describe the real-world food delivery data set. Section 5.2 presents a Hoeffding's D independence test. In Section 5.3, we show how to estimate the ground-truth distribution of the delivery travel time for the routes, which is done by designing a hypothesis test. Section 5.4 describes the procedures for training the ML models used in this paper. In Section 5.5, we present the computational performance of our LBBD algorithms with enhancements. Finally, we provide the out-of-sample performance of our data-driven VRP solutions with benchmark models and policies in Section 5.6.

5.1. Data Description

We use real-world data from a food delivery service provider in Shanghai, China, to show the advantages of the data-driven VRP model and the performance of the proposed algorithm. The data set has 839 customer locations and 4,076 routes for two months in 2015. Specifically, the data set records the longitude and latitude, the number of items ordered for the customer locations, the actual travel time, the travel sequences from the depot to the last visited customer location, and a batch identifier for the routes, where all orders with the same service deadline are referred to as one batch. Thus, a batch represents an instance, where the optimal vehicle routes are determined to fulfill the demands of customer locations with the same service deadline. We split the real data as a training set and a test set, of which 3,268 routes in the training set and the remaining 808 routes serve as the test set (i.e., 80% training data and 20% test data). See Liu et al. (2021) for more details about the data set.

We note that, as indicated in Bertsimas and Kallus (2020), in order to employ this data set to learn the nature of the endogenous effect of the route on travel time, we make the following assumption about the way that it was produced.

Assumption 2 (Ignorability). *Let $\mathcal{R}$ be the set of all possible routes, and let $\tilde{\xi}(r)$ denote the random travel time given that route $r := \{0, n_{(1)}, \ldots, n_{(v)}\} \in \mathcal{R}$ is selected. The data set $\{(\hat{r}^i, \hat{\xi}^i)\}_{i=1}^N$ describing the observed pairs of random routes $\tilde{r}$ and travel times $\tilde{\xi}(\tilde{r})$ is such that for all $r \in \mathcal{R}$, $\tilde{\xi}(r)$ is independent of $\tilde{r}$.*

In other words, given any route r and any situation where a historical observation was made, the travel time that would have been experienced by implementing route r instead of $\tilde{r}$ is independent of the route $\tilde{r}$ that was selected by the manager. This assumption commonly needs to be made when relying on causal inference Rosenbaum and Rubin (1983) and was implicitly made by Liu et al. (2021) given that they performed causal inference using this data set. In the case that the manager exploited some features $\tilde{\zeta}$ that are correlated with $\tilde{\xi}(\cdot)$ (e.g., weather, time of day, etc.) to produce $\tilde{r}$, our method would need to also condition the travel time predictions on $\tilde{\zeta}$ in order to ensure that the hypothesis tests results and prescribed routes are not biased by this influence.

5.2. Hypothesis Test to Validate Endogenous Arc Travel Time

In order to validate that arcs' travel times are dependent on the choice of route, we apply a Hoeffding's D independence test (see Hoeffding 1994) between arc travel time and length of the route on arcs with more than 10 visits (i.e., part of more than 10 batches) in our data set. More specifically, for each arc a among these arcs, we gather the data set $\{t_i^a, \bar{d}_i\}_{i\in\mathcal{N}(a)}$, where t_i^a is the travel time of arc a in the route i, $\bar{d}_i$ is the length of the route i, and $\mathcal{N}(a)$ is the set of routes containing the arc a. The Hoeffding's D independence test compares the following hypotheses:

- *Null hypothesis:* "The travel time on the arc is independent of the length of the route."
- *Alternative hypothesis:* "The travel time on the arc is dependent on the length of the route."

We calculate p-values based on the nonparametric bootstrapping method. Specifically, for each arc, the bootstrapping method samples new data sets (of the same size) with (arc travel time, route length) pairs, where the arc travel time and route length are respectively and independently drawn from the empirical distributions observed in the original data set. Using the empirical distribution of the test statistics in the bootstrapping, we calculate the p-value of the test statistics measured on the original sample. We set the significance threshold of the test to 5% and observe that among the 173 arcs with more than 10 visits, the null hypothesis is rejected for 26 of them. Focusing on the arcs that have more than 60 visits, we observed a rejection rate of 33% (five out of 15 arcs). We conclude from this evidence that the route length has some influence on the arc travel time. We believe that this influence could be further characterized by using more data or looking at the impact of other route characteristics, such as the number of stops or the presence of stops in heavy traffic regions. Interested readers may also consult Appendix EC.3 in the e-companion for an investigation of whether the endogeneity of arc travel time can be explained by the way that nodes are traversed, for example, if the node is at an intersection where it is easy to do a right turn.

5.3. Ground-Truth Distribution Estimation

To assess the out-of-sample performance of implementing the solutions obtained from the data-driven VRP, we need to estimate the ground-truth distribution of the delivery travel time for the routes proposed by the different approaches. Then, the delivery travel time is sampled from the estimated ground-truth distribution to evaluate the out-of-sample delivery delays. In particular, we use CGMM as our ground-truth distribution, which could capture any distribution as long as the number of Gaussians in the mixture is large enough (Gilardi et al. 2002).

To verify the statistical quality of an estimated distribution of the delivery travel time for the routes, we design a hypothesis test that verifies the null hypothesis. Specifically, let $\{x_i, y_i\}_{i=1}^N$ be the feature and total travel time for each observed route, which are sampled from the ground-truth distribution $\mathbb{P}$ on the outcome space $(\mathcal{X} \times \mathbb{R})$. The role of a conditional distribution estimator $\hat{F}_{Y|X}$ is such that $\hat{F}_{Y|X}(y;x) \approx \mathbb{P}(Y \le y | X = x)$ for all $y \in \mathbb{R}$ almost surely when $x \sim \mathbb{P}$. Then, the null hypothesis and alternative hypothesis are defined as

- *Null hypothesis:* "$\mathbb{P}(Y \le y | X = x) = \hat{F}_{Y|X}(y;x)$"
- *Alternative hypothesis:* "$\mathbb{P}(Y \le y | X = x) \ne \hat{F}_{Y|X}(y;x)$"

To test the null hypothesis, we first augment the data set with $Z := 1 + \sum_{k=1}^K \mathbf{1}\{Y > \hat{Y}_k(X)\}$, where each $\hat{Y}_k(X)$ is drawn independent and identically distributed (i.i.d.) from $\hat{F}_{Y|X}(y;X)$ for some $K > 1$. The test statistic, parameterized by $K > 1$, takes the form of

$$\chi^2_K := \sum_{k=1}^{K+1} \frac{(\sum_{i=1}^N \mathbf{1}\{z_i = k\})^2}{N/(K+1)} - N,$$

where $z_i := 1 + \sum_{k=1}^K \mathbf{1}\{y_i > \hat{Y}_k(x_i)\}$ for all $i = 1, \ldots, N$.

Lemma 2. *Assuming that $\hat{F}_{Y|X}(y;x)$ is continuous for all x, then, under the null hypothesis, we have that χ^2_K converges to a chi-square distribution with K degrees of freedom.*

Lemma 2 gives rise to the following statistical testing procedure:

- For all $i = 1, \ldots, N$, let $z_i := 1 + \sum_{k=1}^K \mathbf{1}\{y_i > \hat{Y}_k(x_i)\}$.
- Calculate the sample statistic $\chi^2_K := \sum_{k=1}^{K+1} (\sum_{i=1}^N \mathbf{1}\{z_i = k\})^2 / (N(K+1)) - N$.
- Compute the p-value of the test statistic using an inverse χ^2 distribution with K degrees of freedom.
- If p-value is below the desired confidence level then reject the null hypothesis.

The p-value can be used to compare the goodness-of-fit of the different proposals for $\hat{F}_{Y|X}$.

CGMM is trained using the training data and evaluated in the test set. To train CGMM, we first divide our data set into training (3,268 points), validation (3,268 points), and test sets (808 points). Here, the training and validation sets use the same data set. We set the number of Gaussians in the mixture from five to 20 and the maximum number of iterations to 2,000. The mixture distribution parameters are output by a neural network and trained to maximize overall log likelihood. The whole training procedure is implemented in PyTorch. In addition, during the training phase, for each number of mixtures and each iteration, we train the model using the training set. Then, we generate $K = 1{,}633$ samples for each data in the validation set from the current model such that $N_{\text{valid}}/(K+1) = 2$ and compute the overall p-value, where N_{valid} is the number of points in the validation set. At the end of the training procedure, we use the one with a minimum p-value as our generative model. To evaluate the generative model, we generate $K = 1{,}633$ samples for each point in the test set and compute the p-value, which is 0.26. So the hyperparameters are chosen from the training and validation sets and evaluated using the test set. If we choose a significance level of 5%, then we cannot reject the null hypothesis for CGMM; that is, we can accept the hypothesis that CGMM is the same as the ground-truth distribution. Therefore, in the remainder of this section, we use the CGMM as the ground-truth model

to demonstrate the computational efficiency and evaluate the out-of-sample performance.

5.4. Model Training

All the ML models in this paper are also trained using the training data. We use tenfold cross-validation to select the best hyperparameters, that is, the k for kNN and bandwidth for KDE, and the shrinkage parameters in the least absolute shrinkage and selection operator regression (LASSO) used in Liu et al. (2021). The training and validation procedures are implemented in Python for kNN and KDE and in R for LASSO. Specifically, we randomly split the training data set into 10 equal-sized groups. One of the groups is used as the validation set for testing the model, and the remaining groups are used as the training set. The cross-validation process is repeated 10 times; at each time, the model is trained on the training set and scored on the validation set by computing the mean squared error (MSE). At the end of the process, the hyperparameter with the minimum average cross-validation MSE over 10 times is used. The average cross-validation root MSE for the kNN and KDE methods are equal to 9.4 and 9.6, respectively. Compared with the KDE method, the kNN method achieves a lower error. Therefore, in the remainder of this section, the kNN method will be among the models used by the optimization model. It is also important to note that when the size of data set is small, there may exist routes that are far away from the training set. In this situation, the DRO framework is an alternative to the stochastic programming paradigm, where the solution of DRO can be robust to estimation error (Esfahani and Kuhn 2018). This motivates us to develop a DRO-VRP model under type q-Wasserstein ambiguity set centered at the route-dependent travel time distributions in Appendix EC.8 in the e-companion.

5.5. Computational Performance

We study the computational performance of our proposed algorithm. We start in Section 5.5.1 by presenting some details regarding the implementation of the LBBD algorithm, followed by a description of how the lower bound heuristic was calibrated in Section 5.5.2. Section 5.5.3 presents the computational results under the endogenous travel time model with the kNN method identified in Section 5.4, whereas the computational performance for the lower bound heuristic and KDE method is presented in Appendices EC.4 and EC.5 in the e-companion, respectively.

5.5.1. Implementation Details. To evaluate the numerical performance of using the LBBD algorithm with the proposed acceleration strategies and the lower bound heuristic to solve the data-driven VRP, in our test examples, we vary the service deadline τ from 25 minutes to 65 minutes, which could adjust different food preparation times. To set a reasonable number of vehicles and fairly compare the performance of different variants, following Liu et al. (2021), we let the maximum capacity $Q := 30$, and the number of customer locations visited by one vehicle is no more than eight. The number of drivers L used for each batch is set to be $\min\{I_c * 1.5, I_c + 4\}$, where I_c is the minimum number of drivers needed according to the capacity restriction and the number of customer locations limit. Based on the training data from the real data, 10 new training data sets were generated by using CGMM for the VRP to obtain statistics about the performance of the different algorithms. The kNN and KDE methods are trained using the training data. Then, the decisions from the data-driven VRP are derived and evaluated for each batch in the test set. For the test data, only the batches that contain more than 10 customer locations are used, so there are 44 batches in total, where the number of customer locations varies from 10 to 58.

All experiments are conducted on the Cedar cluster of the Digital Alliance of Canada with a single CPU core and 32 GB memory. The algorithm was implemented with the C++ programming language using IBM's CPLEX solver, version 12.10. For all computations, we set a maximum runtime of 20 minutes for all algorithms, which corresponds to 20 minutes for practical food preparation. The CPLEX default settings are used for all algorithms, and we apply the LBBD method with optimality cuts added as lazy constraints, and we set the number of threads to one. The warm start and the $\boldsymbol{\theta}$ in the lower bound heuristic are generally computed within seconds, so we did not present these runtimes in the tables.

5.5.2. Computational Results on the Lower Bound Heuristic. In this section, we provide details on training $\boldsymbol{\theta}$ and choosing ρ for the lower bound heuristic with kNN and KDE methods, which are defined in Section 4.3. Given a set of customers $\mathcal{N}_c = \{1, \ldots, I\}$, considering the computational cost, we let $J = I \times I$ and the set of routes for training $\boldsymbol{\theta}$ be $\{\{0, i\}, \{0, i, j\}\}_{i,j \in \mathcal{N}_c, i \neq j}$. We let $T = 9$ and calculate the 10th, 20th, …, and 90th quantiles based on the discrete distribution constructed by the kNN and KDE methods. We then solve Problem (13) to obtain an optimal solution. To choose the value of ρ, we set the threshold $\varepsilon' = 0.85$ and randomly generate 8,000 routes with different numbers of customer locations. Then, we use a bisection algorithm (see Algorithm 3 in the e-companion) to choose ρ. Table 1 presents the lower bound probability and the average absolute gap (in minutes) when the lower bound value is larger than the expected delay for the kNN method based on a newly generated set of routes with different numbers of locations.

Table 1. The Lower Bound Probability and the Average Absolute Gap Are Reported

τ	25	35	45	55	65
LBProb	0.85	0.85	0.85	0.85	0.85
Avg_AGap	0.3	0.22	0.16	0.11	0.06

Note. LBProb, lower bound probability; Avg_AGap, average absolute gap (in minutes).

From Table 1, we can see that the out-of-sample lower bound probability is around the threshold ε', and the average absolute gap is small. Although parameter $\boldsymbol{\theta}$ is trained with at most two customer locations, we use different numbers of locations to choose ρ and evaluate the heuristic. The results in Table 1 indicate that the lower bound value could meet the threshold value. When the lower bound value is larger than the expected delay, this gap is less than one minute on average. Therefore, for the data-driven VRP with the kNN method, we conclude that the lower bound heuristic could provide a valid lower bound value. Similar observations can be found for the KDE method (see Appendix EC.5.1 in the e-companion).

5.5.3. Computational Results Under the *k*NN Model. We now present our results on the acceleration strategies and lower bound improvement heuristic implemented in the LBBD Algorithm 5 in the e-companion for the data-driven VRP with the kNN method. In this section, we consider the service deadline $\tau \in \{25, 35, 45, 55, 65\}$. We illustrate the performance of the following six different variants:

- BDWLG_LH refers to using the default LBBD algorithm with the warm starting, local search procedures, the general optimality cuts, and the lower bound improvement heuristic on the Data-Driven VRP Model (5).
- BDWLG refers to using the default LBBD algorithm with the warm-starting and local search procedures and the general optimality cuts.
- BDWL refers to using the default LBBD algorithm with the warm-starting and local search procedures.
- BDW refers to using the default LBBD algorithm only with the warm-starting procedure.
- BD refers to using the default LBBD method with neither acceleration strategies nor the lower bound improvement heuristic.
- Genetic refers to using a genetic heuristic, where the objective function of the total travel cost minimization in the hybrid genetic search heuristic (Vidal et al. 2012) is adjusted to the Data-Driven VRP Model (5).

Table 2 displays the average upper bound and estimated lower bound, the average absolute gap, the solution time for the instances that can be solved to optimality, the average time spent for the subproblem and local search procedures, the average number of cuts and nodes, and the optimal ratio for the kNN method, where the optimal ratio is computed as the number of solved instances over the total 440 instances. The results for the KDE method are presented in Appendix EC.5.2 in the e-companion.

From Table 2, we can observe that BDWLG_LH achieves the smallest average absolute gap and largest optimal ratio among all six methods for smaller service deadlines, whereas BDWLG has a better average upper bound. Only when the service deadline $\tau = 65$, BDWLG_LH gives a comparable performance to BDWLG and BDWL in terms of the average absolute gap and optimal ratio. In particular, when $\tau = 55$, compared with BDWLG, BDWLG_LH reduces the average absolute gap by more than 38% and improves the optimal ratio by about 46%. However, the average upper bound obtained by BDWLG_LH is modestly increased. This is mainly because BDWLG_LH has a larger problem size than BDWLG; BDWLG_LH could explore a larger number of nodes and return fewer integer solutions. This translates to a larger average upper bound. Moreover, when $\tau = 25$, a significantly smaller average absolute gap is obtained from BDWLG_LH, although it is still larger than 10. In such cases, the average estimated lower bound of the expected delay is larger than 35 minutes, possibly not meeting the decision makers' objectives. Thus, one needs to increase the number of vehicles to achieve the desired goal. In addition, BDWL yields slightly poorer performance compared with BDWLG. By using the general optimality cuts, either the average absolute gap or the solution time for the solved instances is reduced. Furthermore, BDWL has a significantly better performance than BDW, indicating that the local search procedures could provide improved integer solutions of reasonably good quality, although it does require some time to implement the local search procedures. This increase in the local search time can be related to the reduction in the number of optimality cuts and nodes explored in the algorithm. Moreover, BDW significantly outperforms BD. This confirms that the acceleration strategies and the lower bound heuristic proposed in Section 4 are crucial to improving the efficiency of the LBBD algorithm. We also notice that when compared with the genetic heuristic, the LBBD algorithm with acceleration strategies delivers significant improvement in terms of the average absolute gap and the optimal ratio. Only BD performs worse than the genetic heuristic. These findings support the following main insight across different service deadline levels for the problem instance studied in our case study.

Insight 1. *BDWLG has the best integer solutions for the data-driven VRP with the kNN method. In the meantime, BDWLG_LH could decrease the average absolute gap and improve the optimal ratio.*

Table 2. The Computational Results Under the *k*NN Model Are Reported

τ	Method	UB	Est_LB	Gap	AvT	AvT_SP	Number of cuts	Number of nodes	Ratio
25	BDWLG_LH	48.67	35.58	13.09	129.8	147.7	318	70,664	0.17
	BDWLG	47.67	0.00	47.67	N/A	675.5	1,716	2,667	0.00
	BDWL	47.71	0.00	47.71	N/A	695.1	1,727	2,270	0.00
	BDW	67.76	0.00	67.76	N/A	10.3	24,972	19,496	0.00
	BD	101.29	0.00	101.29	N/A	10.3	24,163	18,185	0.00
	Genetic	84.78	0.00	84.78	N/A	N/A	N/A	N/A	0.00
35	BDWLG_LH	18.70	12.58	6.12	156.3	122.1	277	74,460	0.21
	BDWLG	17.84	0.00	17.84	N/A	648.3	1,904	3,348	0.00
	BDWL	17.96	0.00	17.96	N/A	680.8	1,977	2,968	0.00
	BDW	32.60	0.00	32.60	N/A	9.4	22,598	20,848	0.00
	BD	61.74	0.00	61.74	N/A	9.8	23,096	19,278	0.00
	Genetic	46.00	0.00	46.00	N/A	N/A	N/A	N/A	0.00
45	BDWLG_LH	5.35	3.11	2.24	183.5	122.1	395	78,170	0.20
	BDWLG	5.05	0.00	5.05	2.7	595.5	2,012	3,900	0.02
	BDWL	5.05	0.00	5.05	5.9	636.1	2,138	3,616	0.02
	BDW	12.53	0.00	12.53	3.5	9.3	22,470	20,292	0.005
	BD	33.29	0.00	33.29	8.2	9.9	22,962	19,346	0.005
	Genetic	22.42	0.00	22.42	N/A	N/A	N/A	N/A	0.01
55	BDWLG_LH	1.00	0.44	0.57	152.1	97.7	430	48,045	0.22
	BDWLG	0.91	0.00	0.91	86.5	475.9	1,819	3,895	0.12
	BDWL	0.91	0.00	0.91	95.4	505.8	1,963	3,802	0.12
	BDW	3.68	0.00	3.68	43.3	8.1	19,379	20,553	0.03
	BD	15.97	0.00	15.97	3.2	8.7	20,373	18,565	0.03
	Genetic	8.55	0.00	8.55	N/A	N/A	N/A	N/A	0.05
65	BDWLG_LH	0.10	0.01	0.09	170.2	59.7	293	14,553	0.45
	BDWLG	0.09	0.00	0.09	111.6	221.3	974	2,516	0.49
	BDWL	0.09	0.00	0.09	107.5	238.2	1,050	2,603	0.48
	BDW	0.87	0.00	0.87	15.2	5.8	12,695	16,685	0.11
	BD	6.68	0.00	6.68	25.2	6.7	15,088	15,153	0.03
	Genetic	2.63	0.00	2.6	N/A	N/A	N/A	N/A	0.22

Note. UB, average upper bound; Est_LB, average estimated lower bound; Gap, average absolute gap; AvT, the average solution time (in seconds) for the instances solved optimally; AvT_SP, average time (in seconds) spent for subproblem and local search procedures; number of cuts, average number of cuts; number of nodes, average number of nodes; ratio, optimal ratio.

5.6. Out-of-Sample Performance of the Data-Driven VRP Solutions

To evaluate the out-of-sample performance of the routes obtained from the data-driven VRP with the *k*NN and KDE methods, in this section, we use the real data as the training data set and generate $\{\bar{\xi}_\ell^1, \ldots, \bar{\xi}_\ell^{N'}\}_{N'=10,000}$ samples from the CGMM for the optimal route $\boldsymbol{s}^{*\ell}$. We then calculate the following out-of-sample expected delay for each batch:

$$\text{out-of-sample expected delay} := \frac{1}{N'} \sum_{i=1}^{N'} \sum_{\ell \in \mathcal{N}_d} (\bar{\xi}_\ell^i - \tau)^+.$$

We first present in Section 5.6.1 the performance of the data-driven VRP that employs *k*NN and KDE methods and compare it to an adjusted version of the assignment model proposed by Liu et al. (2021). In Section 5.6.2, we then compare the *k*NN method to data-driven stochastic and deterministic arc-based VRP models that both ignore the travel time endogeneity. Finally, Sections 5.6.3 and 5.6.4, respectively, compare the performance of order-assignment decisions obtained from the original DOA-DRO model used in Liu et al. (2021) and the implemented route decisions observed in the real data set.

5.6.1. Comparisons with Benchmark Models. In this section, we show the out-of-sample performance of the data-driven VRP with the *k*NN and KDE methods using the feature vector proposed in Section 3.2 and the following three methods:

- ALiu_{opt} refers to adjusting the assignment model proposed by Liu et al. (2021) to our problem by letting the travel time include the service time, where the routing decisions are optimal (provided by an "oracle") given the assignment decisions. Please refer to the detailed formulations and solution schemes of ALiu_{opt} in Appendix EC.6 in the e-companion.
- ALiu_{kNN} refers to the use of ALiu_{opt} but using the *k*NN method to optimize the routing decisions. See Appendix EC.6 in the e-companion for more details.

• *k*NNT refers to the data-driven VRP with the *k*NN method using the tractable feature vectors used in Liu et al. (2021).

The experiment setting is analogous to the setting used in Section 5.5.1 with the service deadline $\tau \in \{45, 55, 65\}$. For ALiu_{opt}, it can be seen as the best version on the out-of-sample performance achieved by the assignment model based on Liu et al. (2021). Figure 1 presents the statistics of the out-of-sample expected delay for the *k*NN and KDE methods and the assignment models based on Liu et al. (2021) for three different service deadlines.

Figure 1 shows that the *k*NN method with the features used in this paper performs better than other methods for the service deadline $\tau = 55$ and 65. The *k*NN method significantly outperforms the KDE method. Meanwhile, when compared with Aliu_{kNN}, the median of the out-of-sample expected delay obtained by Aliu_{opt} is reduced by an average of about 25.7%. Moreover, compared with Aliu_{opt}, the *k*NN method using the tractable features in Liu et al. (2021) improves the median of the out-of-sample expected delay by about 20.9% on average; the *k*NN method with the features used in this paper further reduces the median by 23.1%. This indicates that both the *k*NN method and the features used in this paper could yield improvements in the out-of-sample performance over the assignment model based on Liu et al. (2021) for $\tau = 55$ and 65. For smaller service deadline ($\tau = 45$), we notice that *k*NN, *k*NNT, and Aliu_{opt} have comparable out-of-sample performance and better performance when compared with Aliu_{kNN} and KDE. These results suggest the following insight across different service deadline levels in our case study when assuming that the ground-truth distribution estimated in Section 5.3 is accurate.

Insight 2. *Using the tractable features proposed by Liu et al. (2021), our nonparametric model with routing decisions significantly outperforms the parametric assignment model* Aliu_{opt}. *This performance is further improved by using the features in this paper. Therefore, the kNN method with the features used in this paper appears to be the preferred method for such a data-driven problem.*

5.6.2. Comparisons with Data-Driven Arc-Based VRP Models. We compared the data-driven VRP model with the *k*NN method to the following stochastic and deterministic arc-based VRP models that ignore endogenous uncertainty:

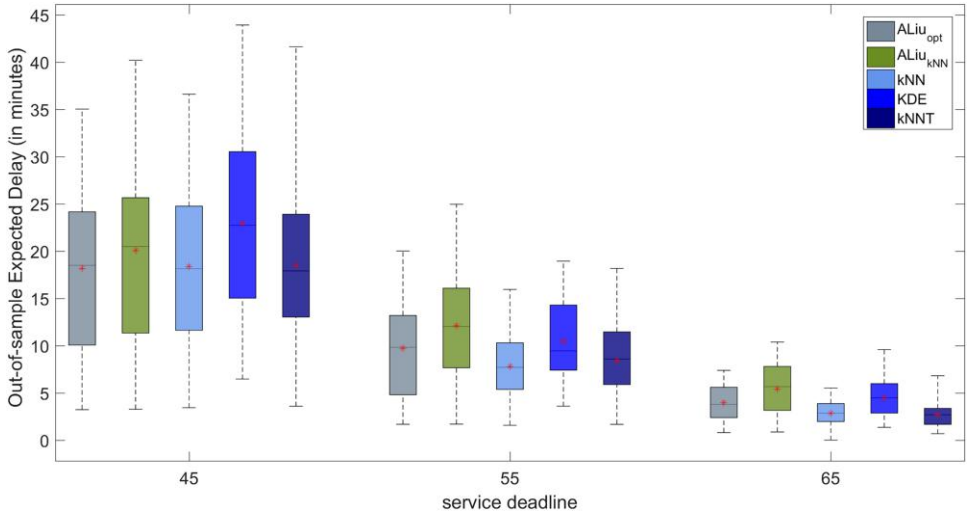
• kNN_{SArc} refers to the stochastic arc-based VRP model with the arc travel time distributions, where the *k*NN method with the features proposed in Section 3 is used to estimate the distributions of travel time for each arc.

• $k\text{NN}_{\text{DArc}}$ refers to the deterministic arc-based VRP model with the average arc travel times, where the average arc travel times are calculated from the arc travel time distributions learned for $k\text{NN}_{\text{SArc}}$.

We refer the reader to Appendix EC.7 in the e-companion for details about the formulation and solution scheme of $k\text{NN}_{\text{SArc}}$ and $k\text{NN}_{\text{DArc}}$.

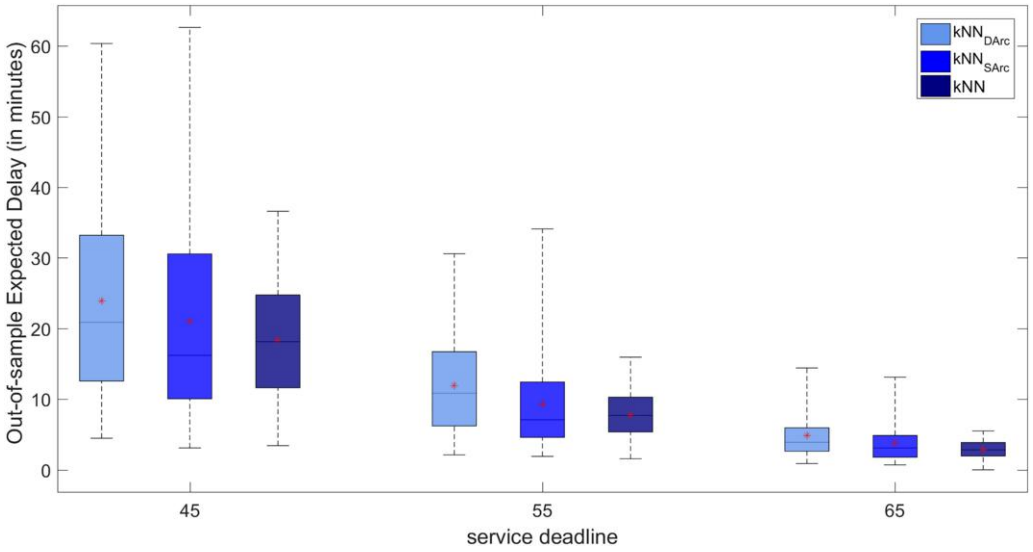
The experiment setting is again analogous to the setting used in Section 5.5.1 with the service deadline

Figure 1. (Color online) Statistics of the Out-of-Sample Expected Delay for ALiu_{opt}, Aliu_{kNN}, *k*NN, KDE, and *k*NNT



Notes. The star identifies the mean of the out-of-sample expected delay. ALiu_{opt}, assignment model based on Liu et al. (2021) with optimal routing decisions; ALiu_{kNN}, ALiu_{opt} but using the *k*NN method to optimize the routing decisions; *k*NN, data-driven VRP with the *k*NN method; KDE, data-driven VRP with the KDE method; *k*NNT, *k*NN but using the tractable features in Liu et al. (2021).

Figure 2. (Color online) Statistics of the Out-of-Sample Expected Delay for $k\text{NN}_{\text{DArc}}$, $k\text{NN}_{\text{SArc}}$ and kNN



Notes. The star identifies the mean of the out-of-sample expected delay. $k\text{NN}_{\text{DArc}}$, deterministic arc-based VRP models with the kNN method; kNN_{SArc}, stochastic arc-based VRP models with the kNN method; kNN, data-driven VRP with the kNN method.

$\tau \in \{45, 55, 65\}$. Figure 2 presents the statistics of the out-of-sample expected delay for our data-driven VRP with the kNN method (kNN), the data-driven stochastic ($k\text{NN}_{\text{SArc}}$), and deterministic ($k\text{NN}_{\text{DArc}}$) arc-based VRP models with the kNN method for three different service deadlines.

From Figure 2, we find that our data-driven VRP with the (endogenous) kNN model has a more stable performance than the arc-based VRP models. The data-driven VRP reduces the maximal out-of-sample expected delay by at least 50% when compared with either $k\text{NN}_{\text{DArc}}$ or $k\text{NN}_{\text{SArc}}$ for all three magnitudes of service deadline. A similar observation can be made for the 75th percentile, where the improvement is around 33% for $k\text{NN}_{\text{DArc}}$ compared with 19% for $k\text{NN}_{\text{SArc}}$. In terms of the mean of the out-of-sample expected delay, an improvement is obtained by about 33% for $k\text{NN}_{\text{DArc}}$ and 8% for $k\text{NN}_{\text{SArc}}$.

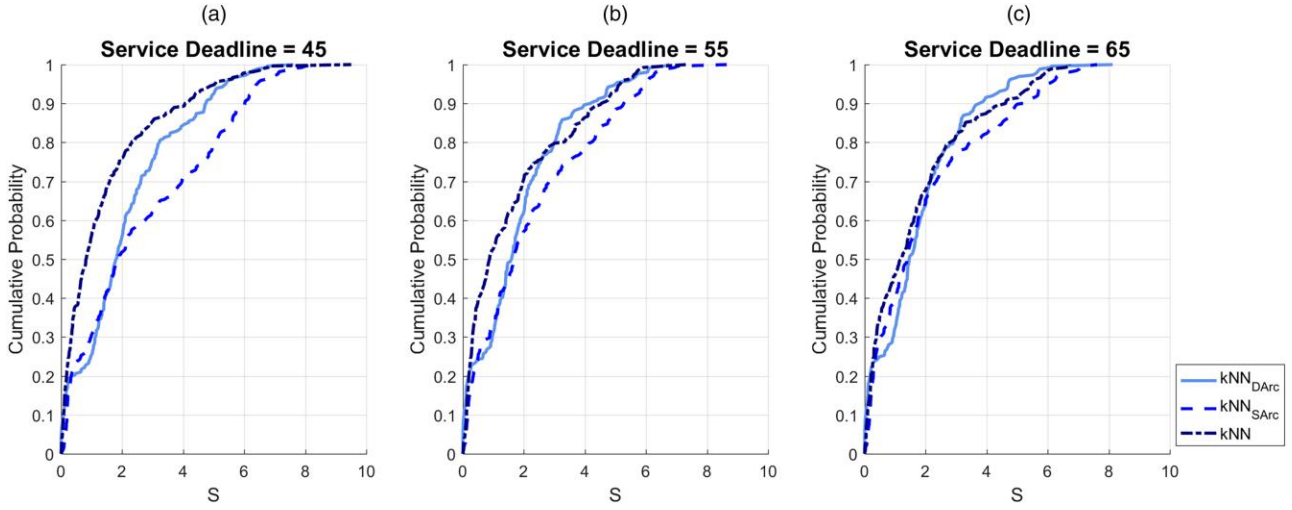
Motivated by an ablation that identified S, that is, the smallest rectangle covering the customer locations visited by a driver, as the feature of routes that contributes most to the performance of kNN, Figure 3 presents cumulative probabilities of S for the three methods as the service deadline varies. Figure 3(a) shows that the distribution for kNN stochastically dominates the distribution obtained from the two other methods when the service deadline is $\tau = 45$. Specifically, the kNN method has more chances of producing a route with lower S than any target level. As the service deadline increases to 55 and 65, this dominance declines, although it remains partially recognizable. One can also confirm that the average S level remains lower for kNN compared with $k\text{NN}_{\text{DArc}}$ and $k\text{NN}_{\text{SArc}}$ at all service levels, with expected values at 1.61, 1.83, 2.22, and 1.67, 1.74, 1.97, respectively. This seems to suggest that the gain in performance of kNN might be partially attributed to the use of more routes that span a smaller area. We refer the reader to Appendix EC.9 in the e-companion for an example of this spatial phenomenon.

These results suggest the following insight across different service deadline levels for our case study when assuming that the ground-truth distribution estimated in Section 5.3 is accurate.

Insight 3. *Compared with data-driven arc-based VRP models, accounting for stochasticity significantly improves performance in expectation, whereas additionally accounting for endogeneity (e.g., the area covered by the planned routes) improves the stability of the out-of-sample performance.*

5.6.3. Comparisons with the Assignment Decisions from Liu et al. (2021). We compare the data-driven VRP with the kNN method using the features in this paper to the order-assignment decisions obtained from the DOA-DRO model proposed by Liu et al. (2021). For a fair comparison, the parameter settings are set to be the same as those used in Liu et al. (2021), and the kNN model is trained on the same training data set as in Liu et al. (2021) because Liu et al. (2021) used a clustering algorithm for the customer locations in each instance based on geographical proximity such that the customer locations with slight differences in latitude and longitude are considered as one location. Given the assignment decisions from Liu et al. (2021), to obtain the routing decisions, we separate the clusters into the original customer locations. We let the number of

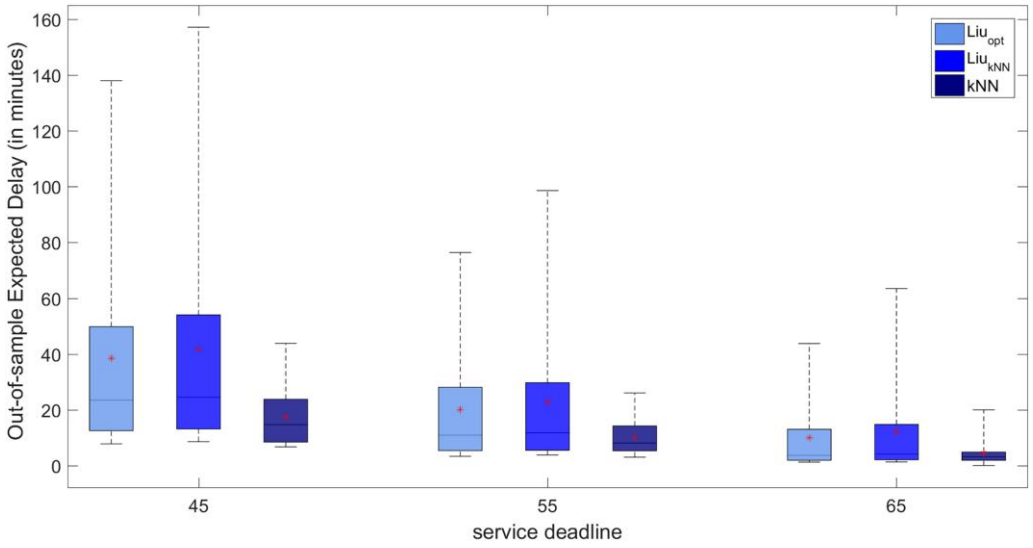
Figure 3. (Color online) Cumulative Probability of S for $k\text{NN}_{\text{DArc}}$, $k\text{NN}_{\text{SArc}}$, and kNN



vehicles equal the one used in Liu et al. (2021). Figure 4 presents the statistics of the out-of-sample expected delay for the kNN method and for the assignment decisions obtained from Liu et al. (2021) with optimal routing decisions (Liu_{opt}) and using the kNN method to optimize the routing decisions (Liu_{kNN}) for three different service deadlines.

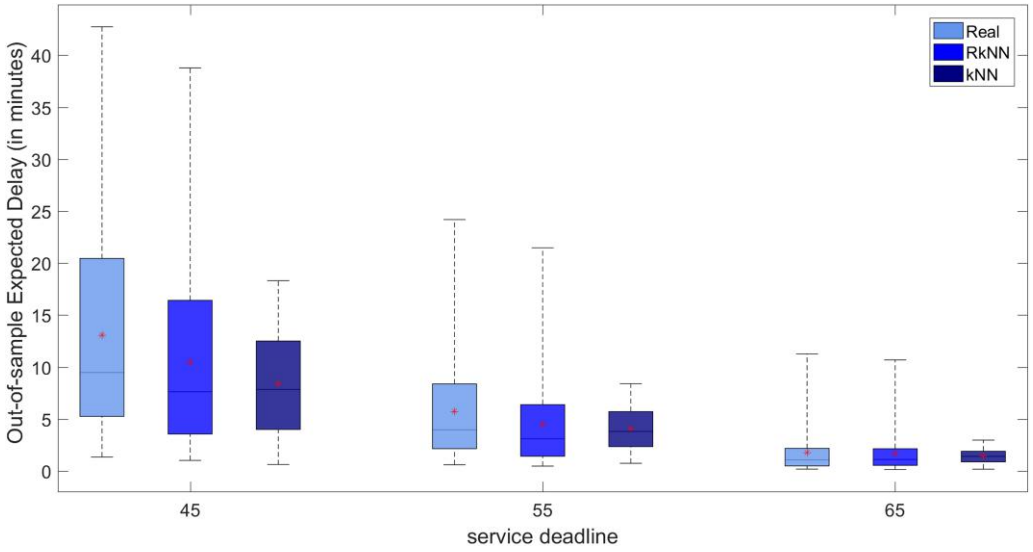
We observe from Figure 4 that the data-driven VRP with the kNN method achieves better performance than the assignment decisions obtained from Liu et al. (2021). Specifically, when compared with Liu_{opt}, kNN reduces the median of out-of-sample expected delay by about 25.8% on average. For the maximal out-of-sample expected delay, a significant improvement is obtained by about 62.8% on average. From Figures 1 and 4, we see that the performance gap between kNN and ALiu_{opt} is smaller than the gap between kNN and Liu_{opt}. This might be because a certain number of customer locations are considered as one customer location by using the clustering algorithm in Liu et al. (2021), then one driver might serve a large number of customer locations (we found that one driver could serve up to 16 customer locations). This might lead to a larger performance gap between kNN and Liu_{opt}. We

Figure 4. (Color online) Statistics of the Out-of-Sample Expected Delay for Liu_{opt}, Liu_{kNN}, and kNN



Notes. The star identifies the mean of the out-of-sample expected delay. Liu_{opt}, assignment decisions obtained from Liu et al. (2021) with optimal routing decisions; Liu_{kNN}, Liu_{opt} but using the kNN method to optimize the routing decisions; kNN, data-driven VRP with the kNN method.

Figure 5. (Color online) Statistics of the Out-of-Sample Expected Delay for Real, R*k*NN, and *k*NN



Notes. The star identifies the mean of the out-of-sample expected delay. Real, implemented routes; R*k*NN, implemented assignment decision with the routes from the *k*NN method; *k*NN, data-driven VRP with the *k*NN method.

conclude with the following insight across different service deadline levels in our case study if the ground-truth distribution estimated in Section 5.3 is accurate.

Insight 4. *Compared with the order-assignment decisions obtained from DOA-DRO model proposed by Liu et al. (2021), our method leads to significant improvement in out-of-sample expected delay.*

5.6.4. Comparison with the Implemented Routing Decisions. We compare our models to the routing decisions observed in the test set (i.e., the implemented routing decisions). We set the number of drivers to be the same as the actual number of dispatched drivers. Figure 5 presents the statistics of the out-of-sample expected delay for the implemented routes (Real), the implemented assignment decisions with the routes from the *k*NN method (R*k*NN), and the data-driven VRP with the *k*NN method (*k*NN) for three service deadlines. Because the implemented service deadline is 55 minutes, we vary the service deadline around the implemented one by letting $\tau \in \{45, 55, 65\}$.

From Figure 5, we can observe that compared with the actual routes chosen by drivers, R*k*NN improves the maximum value of the out-of-sample expected delay from 11.3 to 10.7 when $\tau = 65$, from 24.2 to 21.5 when $\tau = 55$, and from 42.7 to 38.8 when $\tau = 45$. A similar observation can be found for other statistics. Moreover, by using the data-driven VRP with the *k*NN method, the maximum value of the out-of-sample expected delay is significantly improved by about 62% on average and about 16% for the mean of the out-of-sample expected delay when compared with other methods, whereas for the median, 25th, and minimum values of the average delay, the *k*NN method has a comparable performance to the other methods. We draw the following insight in our case study when assuming the ground-truth distribution estimated in Section 5.3.

Insight 5. *Given the implemented assignment decisions, the routes from the kNN method have a better performance than the ones chosen by drivers. A more stable out-of-sample performance is further achieved by our model. This indicates that integrating the assignment and routing decision processes plays an important role in reducing the expected delay.*

6. Concluding Remarks

In this paper, we propose a novel prescriptive ML approach for a data-driven stochastic VRP with deadlines and endogenous travel time. We then use nonparametric approaches, including *k*NN and KDE, to estimate the route-dependent travel time distribution. An LBBD scheme with some acceleration strategies and an ML-based lower bound heuristic was proposed to solve the problem. We conduct a practical case study with real-world data for a food delivery routing problem.

The computational results show that the proposed algorithm can significantly decrease the average absolute gap when compared with the default LBBD and a genetic heuristic based on Vidal et al. (2012). In terms of the out-of-sample performance, our data-driven stochastic VRP with the *k*NN method successfully reduces the out-of-sample expected delay when compared with

an adjusted version of the assignment parametric models proposed by Liu et al. (2021), the stochastic and deterministic arc-based VRP models, the order-assignment decisions obtained from the DOA-DRO model used in Liu et al. (2021), and the implemented route decisions observed in the real data set. We also quantify the value of the proposed routes with different service deadlines. This indicates that integrating the assignment and routing decision processes within a nonparametric framework plays an important role in reducing the expected delay.

We believe that the models and real-world case study for the VRP should motivate its use in other types of applications, for example, network design and facility location. Future research could also benefit from validating the insights gained from the real-world case study in different problem contexts. Moreover, it would also be interesting to extend our proposed VRP to a multiperiod VRP in the context of contextual optimization.

Acknowledgments

The authors are thankful to an anonymous AE and two referees for their valuable comments. The authors also thank Sheng Liu, Long He, and Zuo-Jun Max Shen for providing them with the data used in our case study. Finally, this research was enabled in part by support provided by the Digital Research Alliance of Canada (https://www.alliancecan.ca/).

Endnote

[1] Note that a contextual version of our VRP could be obtained by accounting for side information in the construction of this feature vector.

References

Adulyasak Y, Jaillet P (2016) Models and algorithms for stochastic and robust vehicle routing with deadlines. *Transportation Sci.* 50(2):608–626.

Anderson R, Huchette J, Ma W, Tjandraatmadja C, Vielma JP (2020) Strong mixed-integer programming formulations for trained neural networks. *Math. Programming* 183(1):3–39.

Beckmann M, McGuire CB, Winsten CB (1956) *Studies in the Economics of Transportation* (Yale University Press, New Haven, CT).

Bertsimas D, Kallus N (2020) From predictive to prescriptive analytics. *Management Sci.* 66(3):1025–1044.

Bertsimas D, Brown DB, Caramanis C (2011) Theory and applications of robust optimization. *SIAM Rev.* 53(3):464–501.

Biau G, Devroye L (2015) *Lectures on the Nearest Neighbor Method*, Lectures on the Nearest Neighbor Method (Springer, Cham, Switzerland).

Birge JR, Louveaux F (2011) *Introduction to Stochastic Programming*, Introduction to Stochastic Programming (Springer, New York).

Carlsson JG, Delage E (2013) Robust partitioning for stochastic multivehicle routing. *Oper. Res.* 61(3):727–744.

Çavdar B, Sokol J (2015) A distribution-free TSP tour length estimation model for random graphs. *Eur. J. Oper. Res.* 243(2):588–598.

Chien TW (1992) Operational estimators for the length of a traveling salesman tour. *Comput. Oper. Res.* 19(6):469–478.

Coelho LC, Cordeau JF, Laporte G (2014) Thirty years of inventory routing. *Transportation Sci.* 48(1):1–19.

Cui R, Lu Z, Sun T, Golden JM (2023) Sooner or later? Promising delivery speed in online retail. *Manufacturing Service Oper. Management* 26(1):233–251.

Davis RA, Lii KS, Politis DN (2011) Remarks on some nonparametric estimates of a density function. Davis R, Lii KS, Politis D, eds. *Selected Works of Murray Rosenblatt*, Selected Works in Probability and Statistics (Springer, New York), 95–100.

Esfahani PM, Kuhn D (2018) Data-driven distributionally robust optimization using the Wasserstein metric: Performance guarantees and tractable reformulations. *Math Programming* 171:115–166.

Fu G, Zhang P, Lei D, Qi W, Shen ZJM (2023) Learning for guiding: A framework for unlocking trust and improving performance in last-mile delivery. Preprint, submitted November 30, https://doi.org/10.2139/ssrn.4639706.

García-Portugués E (2023) Notes for nonparametric statistics. Accessed April 20, 2026, https://bookdown.org/egarpor/NP-UC3M/.

Ghosal S, Ho CP, Wiesemann W (2023) A unifying framework for the capacitated vehicle routing problem under risk and ambiguity. *Oper. Res.* 72(2):425–443.

Gilardi N, Bengio S, Kanevski M (2002) Conditional Gaussian mixture models for environmental risk mapping. *Proc. 12th IEEE Workshop Neural Networks Signal Processing* (IEEE, Piscataway, NJ), 777–786.

Goel V, Grossmann IE (2006) A class of stochastic programs with decision dependent uncertainty. *Math. Programming* 108(2):355–394.

Grimstad B, Andersson H (2019) ReLU networks as surrogate models in mixed-integer linear programs. *Comput. Chemical Engrg.* 131:106580.

Hildebrandt FD, Ulmer MW (2022) Supervised learning for arrival time estimations in restaurant meal delivery. *Transportation Sci.* 56(4):1058–1084.

Hildebrandt FD, Thomas BW, Ulmer MW (2023) Opportunities for reinforcement learning in stochastic dynamic vehicle routing. *Comput. Oper. Res.* 150:106071.

Hoeffding W (1994) A non-parametric test of independence. Fisher NI, Sen PK, eds. *The Collected Works of Wassily Hoeffding*, Springer Series in Statistics (Springer, New York), 214–226.

Hooshmand Khaligh F, MirHassani S (2016) A mathematical model for vehicle routing problem under endogenous uncertainty. *Internat. J. Production Res.* 54(2):579–590.

Huang Z, Lam H, Zhang H (2022) Evaluating aleatoric uncertainty via conditional generative models. Preprint, submitted June 9, https://arxiv.org/abs/2206.04287.

Jafari E, Pandey V, Boyles SD (2017) A decomposition approach to the static traffic assignment problem. *Transportation Res. Part B: Methodological* 105:270–296.

Jaillet P, Qi J, Sim M (2016) Routing optimization under uncertainty. *Oper. Res.* 64(1):186–200.

Jenelius E, Koutsopoulos HN (2013) Travel time estimation for urban road networks using low frequency probe vehicle data. *Transportation Res. Part B: Methodological* 53:64–81.

Jiang H, Cao J, Shen ZJM (2024) Intertemporal pricing via nonparametric estimation: Integrating reference effects and consumer heterogeneity. *Manufacturing Service Oper. Management* 26(1):28–46.

Kwon O, Golden B, Wasil E (1995) Estimating the length of the optimal TSP tour: An empirical study using regression and neural networks. *Comput. Oper. Res.* 22(10):1039–1046.

Laporte G (1992) The vehicle routing problem: An overview of exact and approximate algorithms. *Eur. J. Oper. Res.* 59(3):345–358.

Larsen E, Frejinger E, Gendron B, Lodi A (2024) Fast Continuous and Integer L-Shaped Heuristics Through Supervised Learning. *INFORMS Journal on Computing*. 36(1):203–223.

Larsen E, Lachapelle S, Bengio Y, Frejinger E, Lacoste-Julien S, Lodi A (2022) Predicting tactical solutions to operational planning

problems under imperfect information. *INFORMS J. Comput.* 34(1):227–242.

Li Y, Côté JF, Coelho LC, Wu P (2022) Novel formulations and logic-based Benders decomposition for the integrated parallel machine scheduling and location problem. *INFORMS J. Comput.* 34(2):1048–1069.

Liu S, Luo Z (2023) On-demand delivery from stores: Dynamic dispatching and routing with random demand. *Manufacturing Service Oper. Management* 25(2):595–612.

Liu S, He L, Max Shen ZJ (2021) On-time last-mile delivery: Order assignment with travel-time predictors. *Management Sci.* 67(7): 4095–4119.

Lysgaard J, Letchford AN, Eglese RW (2004) A new branch-and-cut algorithm for the capacitated vehicle routing problem. *Math. Programming* 100(2):423–445.

Mišić VV, Perakis G (2020) Data analytics in operations management: A review. *Manufacturing Service Oper. Management* 22(1):158–169.

Nazari M, Oroojlooy A, Snyder L, Takác M (2018) Reinforcement learning for solving the vehicle routing problem. Bengio S, Wallach HM, Larochelle H, Grauman K, Cesa-Bianchi N, eds. *NIPS'18: Proc. 32nd Internat. Conf. Neural Inform. Processing Systems* (Curran Associates Inc., Red Hook, NY), 9861–9871.

Nguyen VA, Zhang F, Blanchet J, Delage E, Ye Y (2020) Distributionally robust local non-parametric conditional estimation. Larochelle H, Ranzato M, Hadsell R, Balcan MF, Lin H, eds. *NIPS'20: Proc. 34th Internat. Conf. Neural Inform. Processing Systems* (Curran Associates Inc., Red Hook, NY), 15232–15242.

Nie Y, Zhang H, Lee DH (2004) Models and algorithms for the traffic assignment problem with link capacity constraints. *Transportation Res. Part B: Methodological* 38(4):285–312.

Özarık SS, Da Costa P, Florio AM (2024) Machine learning for data-driven last-mile delivery optimization. *Transportation Sci.* 58(1): 27–44.

Rahmani M, Jenelius E, Koutsopoulos HN (2015) Non-parametric estimation of route travel time distributions from low-frequency floating car data. *Transportation Res. Part C: Emerging Tech.* 58: 343–362.

Rahmaniani R, Crainic TG, Gendreau M, Rei W (2017) The Benders decomposition algorithm: A literature review. *Eur. J. Oper. Res.* 259(3):801–817.

Rosenbaum PR, Rubin DB (1983) The central role of the propensity score in observational studies for causal effects. *Biometrika* 70(1):41–55.

Sadana U, Chenreddy A, Delage E, Forel A, Frejinger E, Vidal T (2025) A survey of contextual optimization methods for decision making under uncertainty. *Eur. J. Oper. Res.* 320(2):271–289.

Serrano B, Florio AM, Minner S, Schiffer M, Vidal T (2024) Contextual stochastic vehicle routing with time windows. Preprint, submitted February 10, https://arxiv.org/abs/2402.06968.

Shafer G, Vovk V (2008) A tutorial on conformal prediction. *J. Machine Learn. Res.* 9(3):371–421.

Soeffker N, Ulmer MW, Mattfeld DC (2022) Stochastic dynamic vehicle routing in the light of prescriptive analytics: A review. *Eur. J. Oper. Res.* 298(3):801–820.

Solomon MM (1987) Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper. Res.* 35(2):254–265.

Torres F, Gendreau M, Rei W (2022) Vehicle routing with stochastic supply of crowd vehicles and time windows. *Transportation Sci.* 56(3):631–653.

Toth P, Vigo D (2014) *Vehicle Routing: Problems, Methods, and Applications* (Society for Industrial and Applied Mathematics, Philadelphia).

Vidal T (2022) Hybrid genetic search for the CVRP: Open-source implementation and SWAP neighborhood. *Comput. Oper. Res.* 140:105643.

Vidal T, Crainic TG, Gendreau M, Lahrichi N, Rei W (2012) A hybrid genetic algorithm for multidepot and periodic vehicle routing problems. *Oper. Res.* 60(3):611–624.

Wasserman L (2006) *All of Nonparametric Statistics*, Springer Texts in Statistics (Springer, New York).

Yu X, Shen S (2022) Multistage distributionally robust mixed-integer programming with decision-dependent moment-based ambiguity sets. *Math. Programming* 196(1–2):1025–1064.

Zhang Y, Zhang Z, Lim A, Sim M (2021) Robust data-driven vehicle routing with time windows. *Oper. Res.* 69(2):469–485.

Copyright of Manufacturing & Service Operations Management (M&SOM) (INFORMS) is the property of INFORMS: Institute for Operations Research & the Management Sciences and its content may not be copied or emailed to multiple sites without the copyright holder's express written permission. Additionally, content may not be used with any artificial intelligence tools or machine learning technologies. However, users may print, download, or email articles for individual use.

3. 序贯决策：Improving Human Sequential Decision Making with Reinforcement Learning (MS, 2026)

来源：Management Science. Vol. 72, No. 1, 2026, pp. 733–755.
https://doi.org/10.1287/mnsc.2022.02455

作者：Hamsa Bastani, Osbert Bastani, Wichinpong Park Sinchaisri

关键词：行为运营；可解释强化学习；序贯决策；人机交互

推荐人：李鉴岐

推荐理由：序贯决策的效果常常在较长时间后才显现，因此人们难以从经验中识别有效策略。本文提出一种机器学习方法，从行为轨迹中提取简洁、可解释的操作提示，重点缩小人类行为与最优策略在关键动作上的差距。作者通过虚拟厨房任务中的随机对照实验发现，算法生成的提示能够显著改善参与者表现。实验还表明，参与者会把提示与自身经验结合，并发现进一步改善绩效的策略，为人机协作界面的设计提供依据。

https://pubsonline.informs.org/journal/mnsc

MANAGEMENT SCIENCE
Vol. 72, No. 1, January 2026, pp. 733–755
ISSN 0025-1909 (print), ISSN 1526-5501 (online)

Improving Human Sequential Decision Making with Reinforcement Learning

Hamsa Bastani,[a] Osbert Bastani,[b] Wichinpong Park Sinchaisri[c,*]

[a] Operations, Information and Decisions, The Wharton School, University of Pennsylvania, Philadelphia, Pennsylvania 19104; [b] Computer and Information Science, University of Pennsylvania, Philadelphia, Pennsylvania 19104; [c] Haas School of Business, University of California, Berkeley, Berkeley, California 94720

*Corresponding author

Contact: hamsab@wharton.upenn.edu, https://orcid.org/0000-0002-8793-4732 (HB); obastani@seas.upenn.edu, https://orcid.org/0000-0001-9990-7566 (OB); parksinchaisri@berkeley.edu, https://orcid.org/0000-0001-9351-0541 (WPS)

Received: August 11, 2022
Revised: January 1, 2024; November 27, 2024
Accepted: December 10, 2024
Published Online in Articles in Advance: May 22, 2025

https://doi.org/10.1287/mnsc.2022.02455

Copyright: © 2025 INFORMS

Abstract. Workers spend a significant amount of time learning how to make good decisions. Evaluating the efficacy of a given decision, however, can be complicated—for example, decision outcomes are often long-term and relate to the original decision in complex ways. Surprisingly, even though learning good decision-making strategies is difficult, the strategies can often be expressed in simple and concise forms. Focusing on sequential decision making, we design a novel machine learning algorithm that is capable of extracting "best practices" from trace data and conveying its insights to humans in the form of interpretable "tips." Our algorithm selects the tip that best bridges the gap between the actions taken by human workers and those taken by the optimal policy in a way that accounts for which actions are consequential for achieving higher performance. We evaluate our approach through a series of randomized controlled experiments where participants manage a virtual kitchen. Our experiments show that the tips generated by our algorithm can significantly improve human performance relative to intuitive baselines. In addition, we discuss a number of empirical insights that can help inform the design of algorithms intended for human-AI interfaces. For instance, we find evidence that participants do not simply blindly follow our tips; instead, they combine them with their own experience to discover additional strategies for improving performance.

History: Accepted by Elena Katok, Special Issue on the Human-Algorithm Connection.
Funding: This work was supported by the Mack Institute for Innovation Management, the Berkeley Artificial Intelligence Research Open Research Commons, and The Wharton Behavioral Lab.
Supplemental Material: The online appendix and data files are available at https://doi.org/10.1287/mnsc.2022.02455.

Keywords: behavioral operations • interpretable reinforcement learning • sequential decision making • human-AI interface

1. Introduction

Workers spend a significant amount of time on the job learning how to make good decisions that improve their performance (Chui et al. 2012). Yet the impact of a current decision can be long range, affecting future decisions/rewards in complex ways, making it difficult for them to evaluate the quality of a decision. This is exacerbated by the fact that multiple decisions are often made sequentially, making it hard to determine which decisions are responsible for good outcomes even in hindsight. Many jobs require such sequential decision making, for example, doctors ordering tests to optimize patient outcomes (Kleinberg et al. 2015) or workers choosing jobs on gig economy platforms to optimize their daily profits (Marshall 2020, Allon et al. 2023). As a concrete example, physicians seek to learn good strategies for ordering laboratory tests because obtaining test results in a timely fashion is necessary to minimize delays in patient visits; for instance, Song et al. (2017) find that experienced physicians have learned to order these tests early to avoid delays. Despite the simple description of the strategy—"order laboratory and radiology tests as early in the care delivery process as possible"—learning it on the job can be difficult because the connection between when tests are ordered and the overall quality of care are influenced by numerous other decisions made by the physician as well as unrelated changes in the underlying environment (e.g., hospital congestion).

Learning on the job can significantly impact service quality because workers likely make suboptimal decisions during this time. For instance, when surgeons first use new devices, surgery duration increases by roughly a third, which can be costly to both patients and providers (Ramdas et al. 2017). Thus, when possible, workers seek alternative ways to acquire best practices in

decision making. Continuing our example of physician decisions for laboratory testing, Song et al. (2017) find that physicians can learn strategies for reducing service time from their better-performing colleagues. This approach is effective precisely because the strategy is simple and easy to communicate yet time-consuming to discover independently. However, learning from their peers is not always an option; for instance, some workers are comparatively isolated, for example, physicians working in rural hospitals or independent workers in the gig economy. In these cases, workers must wastefully spend time independently rediscovering best practices that are already known to their colleagues.

Thus, a natural question arises: can we *automatically* discover best practices and convey them to workers to help them improve their performance? In particular, over the past two decades, many domains have accumulated large amounts of *trace data* on human decisions. For example, nearly every physician action is logged in electronic medical record data, every movement of a driver is recorded by gig economy platforms, and even retail manager decisions on pricing and inventory management are recorded on a daily basis. These data implicitly encode the collective knowledge acquired by numerous workers about how to effectively perform their jobs. However, trace data are often extremely noisy, granular, and of tremendous volume, rendering them unreadable to humans. At the same time, recent advances in reinforcement learning have enabled machines to achieve human-level or superhuman performance at many challenging sequential decision-making tasks (Mnih et al. 2015, Silver et al. 2016). Thus, we might hope to leverage these techniques to mine high-volume trace data to automatically identify key bottlenecks in current human decision making, as well as promising tips/advice to improve their performance.

In this paper, we perform a large-scale behavioral experiment to study whether reinforcement learning can be used to infer tips that improve human performance in sequential decision-making tasks. There is now a large body of evidence that machine learning predictions can improve human performance in *one-shot* decision-making—where the current decision does not affect future outcomes—for example, bail decisions (Green and Chen 2019), visual question answering (Chandrasekaran et al. 2017, 2018), satellite image analysis (Kneusel and Mozer 2017), and detecting deceptive reviews (Lai and Tan 2019). In these settings, it often suffices to provide the model's prediction to the user, potentially in an interpretable way to improve trust and compliance. However, sequential decision-making settings pose qualitatively different challenges because current decisions can have long-term consequences and affect future observed states. In particular, we must figure out in *which* states we should intervene, which can be informed by examining bottlenecks in the current human policy. To this end, we devise a novel algorithmic framework for inferring simple tips that, if adopted, can improve the performance of the worker. Our algorithm aims to capture the *discrepancy* between the existing human policy (as captured by historical trace data) and the optimal policy, which helps us identify the most performance-improving tips for key bottlenecks in current human decision making.

An additional challenge in sequential decision making is that for these tips to improve performance, the human needs to understand how to operationalize them into their broader workflow. Otherwise, even if they comply with the tip, there is no guarantee that they correctly understand what decisions to make on other time steps to achieve optimal performance. In principle, even if a tip suggests optimal actions for the worker to take and the worker complies with the tip perfectly, the overall performance could degrade because the worker subsequently makes poor decisions. Thus, our search space of candidate tips must focus on interpretable and actionable information that workers can easily operationalize. Whether humans can actually do so is an empirical question; thus, we conduct a large-scale behavioral experiment that studies how humans perceive and improve their own decision making over time (given tips from either our algorithm or via peer feedback or simple descriptive statistics), how they adjust other portions of their workflow to accommodate these changes, and how humans may incorrectly perceive bottlenecks in their own decision making.

To summarize, two criteria are needed to *actually* improve human decision making. First, our algorithm must identify sufficiently useful tips to improve performance (assuming humans comply with and effectively operationalize them). Second, humans must be able to understand and comply with our tip and, furthermore, effectively operationalize it by modifying their broader workflow.

1.1. Algorithm

Our algorithm builds on the idea of model distillation (Buciluǎ et al. 2006, Hinton et al. 2015) for interpretable reinforcement learning (Bastani et al. 2018, Verma et al. 2018), which involves first training a black box decision-making policy using reinforcement learning (Sutton and Barto 2018) and then training an interpretable policy to approximate the black box policy. However, unlike prior work, our goal is to infer an interpretable tip that best minimizes the discrepancy between the existing human policy and the black box policy, rather than to train the best-performing interpretable policy that is agnostic to the current human policy. Thus, the chosen tip is tailored to current bottlenecks in the human decision-making policy, and it accounts for which actions are consequential for achieving higher performance—that is, following the tip is expected to

improve the long-term performance of the human rather than simply mimic the optimal policy. In order to easily convey our insights to humans, we design the search space over tips to consist of if-then-else rules. Despite their simplicity, we find that these tips can capture useful insights that are challenging for humans to learn by themselves in complex sequential decision-making problems.

1.2. Game

To study these issues, we designed and built a sequential decision-making game where human players manage a virtual kitchen, inspired by the popular game *Overcooked*. Our game is based on the discrete-time job shop scheduling problem, where tasks need to be scheduled to virtual workers; each task consists of subtasks with dependencies (e.g., ingredients must be chopped before cooking), and workers have heterogeneous processing times (e.g., a chef is better at cooking; a server is better at plating). Players must assign subtasks to virtual workers in a way that minimizes the time it takes to complete a set of food orders. Our game is deterministic, making it easy for inexperienced players to learn the optimal strategy from a few interactions. Instead, the difficulty in achieving good performance comes from the game's combinatorial state space, encoding worker availability and subtask completion so far. For instance, they must make forward-looking trade-offs, for example, deciding whether to greedily assign a worker to a subtask that they are slow to complete or to leave them idle in anticipation of a more suitable subtask.

Our game captures challenges in a variety of operations problems encountered in the real world. For instance, when assigning tasks to health workers, there can be substitution when patient traffic is high, such as having a nurse practitioner perform tasks usually done by physicians. Another example is delivery workers on a grocery delivery platform choosing which orders to accept, where the worker must account for dependencies (e.g., orders must be picked up before delivery) as well as heterogeneous service times (e.g., bikers have an advantage over drivers in high-traffic locations). More broadly, our game can be viewed as a stylized model of any manager scheduling employees to perform tasks on a daily basis, a gig economy employee scheduling daily workload, or a project manager assigning subtasks to workers to accomplish a longer-term goal. Whereas these examples typically involve more complex challenges such as stochastic demands, we believe our experimental findings on worker learning and compliance can generalize well to these settings.

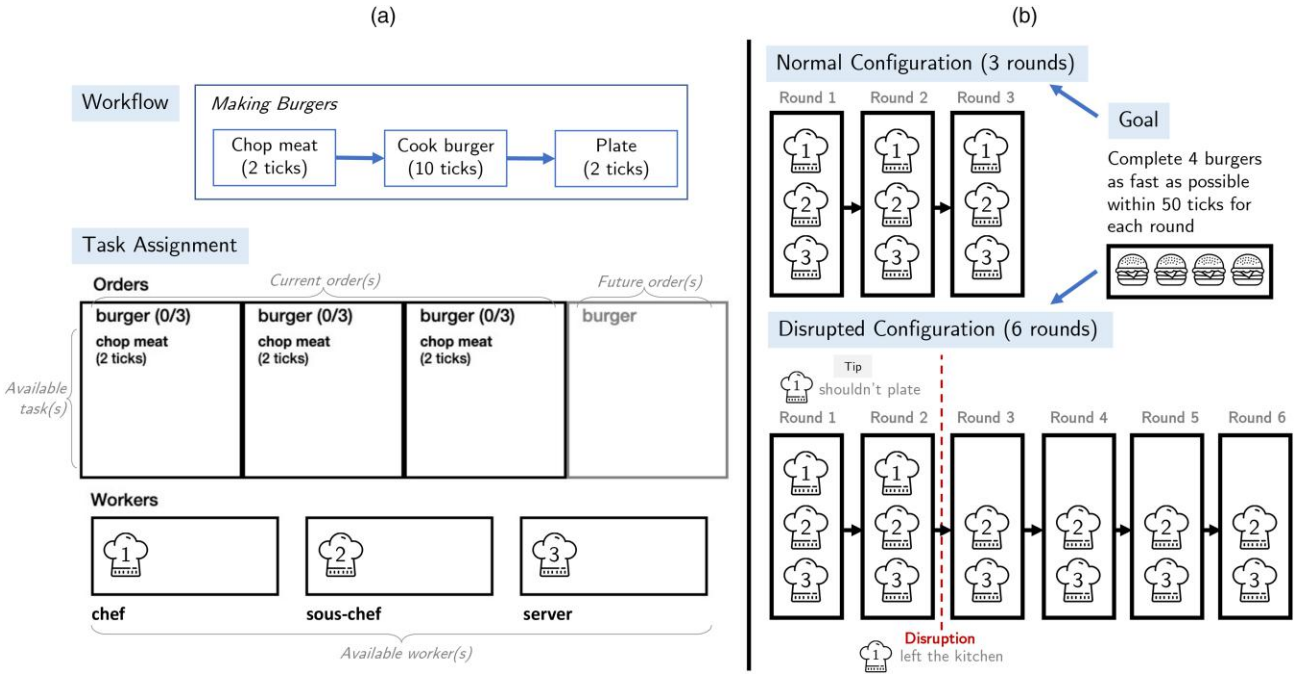
1.3. Experiment

Our primary contribution is a large-scale randomized controlled experiment in the context of this game; Figure 1 illustrates the high-level setup and flow of the game, and Section 3 provides a more detailed description. In particular, we perform a large-scale behavioral study on Amazon Mechanical Turk (AMT) based on two different configurations of our virtual kitchen environment. In the *normal* configuration, the participant plays three identical instantiations of the environment. In the *disrupted* configuration, the first two instantiations of the environment are identical to the ones in the normal configuration, but the remaining four instantiations are modified so that a key worker (namely, the chef) is no longer available. These two configurations are visualized in Figure 1(b). The disrupted configuration is particularly challenging for the human participants because they must unlearn preconceived notions about the optimal strategy acquired during the first two instantiations. For each of these configurations, we leverage our algorithm to learn interpretable tips and then demonstrate how providing this decision-making rule improves the performance of the participants. Our results demonstrate that our algorithm can generate valuable insights that enable human participants to substantially improve their performance compared with counterparts who are not shown the tip or who are shown alternative tips derived from natural baselines. Importantly, we observe that participants do not naively adjust their policy by blindly following the tip. Instead, as they gain experience with the game, they increasingly understand the significance of the tip and improve their performance in ways beyond the surface-level meaning of the tip. Overall, our findings suggest that reinforcement learning can effectively leverage trace data to infer interpretable and useful insights and, furthermore, can successfully convey these insights to humans to improve their decision making.

1.4. Related Literature

1.4.1. Identifying Performance Improvements for Human Workers. Process improvement has long been a focal point in operations management; scholars have especially identified various difficulties associated with sequential decision making and learning. Thus, we study process improvement from the perspective of individual workers through sequential decision making. When workers first experience a new work environment, they may have difficulty adjusting, resulting in various degrees of undesirable performance (Ramdas et al. 2017); for example, unexpected critical medical incidents slow down ambulance activation among paramedics (Bavafa and Jónasson 2021). The situation is exacerbated when inexperienced workers lack guidelines on how to manage their workflow, resulting in suboptimal task prioritization and poor productivity (Ibanez et al. 2018). The complexity of workflows also plays a role. Workers tend to focus on immediate challenges and ignore opportunities for learning (Tucker et al. 2002); furthermore, switching between tasks can

Figure 1. (Color online) Overview of the Kitchen Management Game



Notes. (a) Depiction what participants see: (i) the workflow required to complete a burger order, and (ii) the game screen that allows available tasks to be dragged and dropped to one of three virtual workers. (b) Depicts of the study design: in the normal configuration, participants play the same game for three rounds; in the disrupted configuration, participants play the same game for two rounds, face a disruption in the kitchen (i.e., the chef leaves), and play the disrupted game for four rounds.

significantly hurt productivity (Gurvich et al. 2020). Depending on the features of the sequential decision-making problem, workers may generally follow nonoptimal policies (Kagan et al. 2021).

A common approach to increase reliability and reduce process variation is to standardize processes and offer best practices (Nonaka and Takeuchi 1995, Pfeffer and Sutton 2000, Spear 2005). However, creating standards can be challenging (Szulanski 1996, Argote 2012) and time-consuming (Nonaka and Takeuchi 1995). Workers can learn by trial and error (Dorn and Guzdial 2010), but past experience sometimes makes it challenging to identify best practices (Huckman and Pisano 2006, Kc and Staats 2012). Workers can also learn through soliciting peer feedback (Song et al. 2017, Brattland et al. 2018, Herkenhoff et al. 2018, Jarosch et al. 2021) or working alongside experienced peers (Chan et al. 2014, Tan and Netessine 2019); these mechanisms are especially salient when there are familiarity and collaborative experience between workers (Kim et al. 2020, Akşin et al. 2021). However, these ingredients are often not available. Given well-documented difficulties in learning on the job and identifying best practices, our work proposes an effective approach to automatically extract best practices from logged trace data of historical decisions and outcomes. Whereas recent work has leveraged trace data and machine learning to predict when humans make mistakes in decision making (Fudenberg and Liang 2019, McIlroy-Young et al. 2020, Fudenberg et al. 2022), they do not offer tips to improve human performance.

1.4.2. Using Machine Learning to Improve One-Shot Decision Making. As noted earlier, several recent papers have studied whether machine learning can improve human decision making in the one-shot setting. Key challenges that arise are that humans often erroneously assess their own abilities (Fügener et al. 2022) as well as the predictive model's abilities (Chandrasekaran et al. 2017, 2018; Green and Chen 2019); this, in turn, can result in unwarranted algorithm aversion (Dietvorst et al. 2015) or algorithm appreciation (Logg et al. 2019). This can be overcome by mechanisms such as enabling the predictive model to delegate tasks to humans in a user-aware manner (Fügener et al. 2022), training workers on the success/failures of their specific predictive model (Chandrasekaran et al. 2018), capturing the uncertainty of the model's predictions (Kneusel and Mozer 2017), or accounting for systematic human deviations from the model (Sun et al. 2022). Another important lever is improving the interpretability/explainability of the predictive model (Lu et al. 2019, Stites et al. 2021), which allows workers to gain a deeper understanding of the environment and the potential improvement to be obtained (Sull and Eisenhardt 2015, Gleicher 2016). This can be accomplished by using simple model families

such as decision trees (Breiman et al. 1984, Bertsimas and Dunn 2017) or rule lists (Letham et al. 2015, Wang and Rudin 2015), or by employing posthoc explanation methods such as LIME (Ribeiro et al. 2016).

In contrast to these approaches, we focus on sequential decision making, which is representative of many real-world workflows and poses qualitatively different challenges. For example, adopting a recommended decision on the current time step affects future states/decisions faced by the worker; as a consequence, compliance with a tip may actually *hurt* performance if the worker is unable to appropriately adjust their future workflow. Algorithmically, it is also more challenging to compute interpretable policies because the entire sequence of recommended decisions needs to be interpretable. Thus, we propose a novel framework that adapts interpretable reinforcement learning techniques (Meyer et al. 2014, Puiutta and Veith 2020) to compute interpretable tips that bridge the discrepancy between the human's current policy and the optimal policy. We build on a strategy that first trains a high-performance black box policy and then use imitation learning (Ross et al. 2011) to distill this policy into an interpretable one (Bastani et al. 2018, Verma et al. 2018).

1.5. Contributions

Our work contributes to the literature in two ways. First, we propose a novel algorithm for inferring tips for sequential decision making. Our algorithm leverages techniques from interpretable reinforcement learning to capture the discrepancy between the existing human policy (as captured by trace data) and the optimal policy, thereby identifying the best performance-improving tip targeted toward key bottlenecks in current human decision making.

Second, to the best of our knowledge, we conduct the first large-scale behavioral experiment on Amazon Mechanical Turk to understand how reinforcement learning–based tips can improve human performance in sequential decision-making problems. Unlike one-shot decision making, in order to be effective, humans must understand not only the meaning of a tip but also how to operationalize it into a broader workflow. Our experimental results demonstrate that workers are capable of inferring complex strategies from the limited recommendations provided by our algorithm's tips, but this is not always the case with tips inferred through peer feedback or simple descriptive statistics. We also provide a number of additional insights about how workers comply with tips, as well as how they perceive bottlenecks in their own workflows.

2. Inferring Tips via Interpretable Reinforcement Learning

Consider a human making a sequence of decisions to achieve some desired outcome. We study settings where current decisions affect future outcomes—for instance, if the human decides to consume some resources at the current time step, they can no longer use these resources in the future. These settings are particularly challenging for decision making because of the need to reason about how current actions affect future decisions, making them ideal targets for leveraging tips to improve human performance.

We begin by formalizing the tip inference problem. We model our setting as the human acting to maximize reward in a standard, undiscounted Markov decision process (MDP) $\mathcal{M} = (S, A, R, P)$ over a finite time horizon T. Here, S is the state space, A is the action space, R is the reward function, and P is the transition function. Intuitively, a state $s \in S$ captures the current configuration of the system (e.g., available resources), and an action $a \in A$ is a decision that the human can make (e.g., consume some resources to produce an item). We represent the human as a decision-making policy π_H mapping states to (possibly random) actions. At each time step $t \in \{1, \ldots, T\}$, the human observes the current state s_t and selects an action a_t to take according to the probability distribution $p(a_t \mid s_t) = \pi_H(s_t, a_t)$. Then, they receive reward $r_t = R(s_t, a_t)$, and the system transitions to the next state s_{t+1}, which is a random variable with probability distribution $p(s_{t+1} \mid s_t, a_t) = P(s_t, a_t, s_{t+1})$, after which the process is repeated until $t = T$. A sequence of state-action-reward triples sampled according to this process is called a *rollout*, denoted $\zeta = ((s_1, a_1, r_1), \ldots, (s_T, a_T, r_T))$. We measure the cumulative expected reward of a given policy π as

$$J(\pi) = \mathbb{E}_{\zeta \sim D^{(\pi)}}\left[\sum_{t=1}^{T} r_t\right], \tag{1}$$

where $D^{(\pi)}$ is the distribution of rollouts induced by using policy π. We denote the human policy π_H, which is not directly observed but can be estimated from historical trace data. It will also be useful to define the optimal policy, $\pi^* = \arg\max_\pi J(\pi)$, which maximizes cumulative reward.

2.1. Tips

Now, given the MDP $\mathcal{M}$ and the human policy π_H, our goal is to learn a tip ρ that, conditioned on adoption by the human, most improves the cumulative expected reward. Formally, a tip indicates that in certain states s, the human should use action $\rho(s) \in A$ instead of following their own policy π_H. Thus, we consider tips in the form of a single, interpretable rule:

$$\rho(s) = \text{if } \psi(s), \text{ then take action } a,$$

where $a \in A$ is an action, and $\psi(s) \in \{\text{true}, \text{false}\}$ is a logical predicate over states $s \in S$ (e.g., $\psi(s)$ might be an indicator of whether a sufficient quantity of a certain resource is currently available). In other words, a tip

$\rho = (\psi, a)$ says that if the logical predicate ψ is true, then the human should use the action a prescribed by the tip; otherwise, they should use their own policy π_H.

If the human follows this tip exactly, then the resulting policy they use is $\pi_H \oplus \rho$, where we define the operation

$$(\pi \oplus \rho)(s, a') = \begin{cases} \mathbb{1}(a' = a) & \text{if } \psi(s) \\ \pi(s, a') & \text{otherwise.} \end{cases}$$

Here, $\mathbb{1}$ is the indicator function; that is, the human takes action a with probability one if $\psi(s)$ holds and follows their existing policy otherwise.

Remark 1. In practice, we find that human adoption of tips varies. However, it is difficult to predict the rate of adoption of a tip prior to offering it. Instead, we focus on identifying the best performance-improving tip *conditioned* on adoption. We find that this strategy works sufficiently well to improve performance in our experiments as long as the human can understand both the tip and its rationale. We give a detailed discussion of compliance with tips in Section 5.2.

Our goal is to compute the tip ρ^* that most improves the human's performance; that is,

$$\rho^* = \arg\max_{\rho} J(\pi_H \oplus \rho). \tag{2}$$

This formulation ensures that the chosen tip is *consequential* to improving performance J in Equation (1). There are many other ways to choose tips; for example, one can naively identify state-action pairs that frequently differ between the human and optimal policies. We illustrate the drawbacks to such an approach in our experiments (see Section 5).

2.2. Algorithm

Next, we describe our algorithm for solving Equation (2). Note that we can simply loop through each candidate tip ρ, but we may lack the data to evaluate $J(\pi_H \oplus \rho)$ without additional assumptions. This is because, showing the tip changes the human's behavior, changing the distribution of states $D^{(\pi)}$ they visit to $D^{(\pi \oplus \rho)}$. However, we do not have samples from $D^{(\pi \oplus \rho)}$, which are necessary to estimate Equation (1). One strategy would be to run an experiment with each tip to obtain these samples, but this is prohibitively expensive. Alternatively, one can consider approximating the unobserved distribution $D^{(\pi \oplus \rho)}$ with the observed distribution $D^{(\pi)}$ when evaluating $J(\pi_H \oplus \rho)$, but this has the unfortunate consequence of removing the dependence on the tip ρ entirely from our optimization problem in Equation (2), rendering us unable to identify good tips.

Instead, we describe an approximation that is implementable given observed data and effectively distinguishes between candidate tips; we find that this strategy works well in our experiments. To this end, we leverage the well-studied value- and Q-functions (Watkins and Dayan 1992) (denoted V^* and Q^*, respectively), which can be defined recursively by the Bellman equation

$$V^*(s) = \max_{a \in A} Q^*(s, a),$$
$$Q^*(s, a) = R(s, a) + \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^*(s')].$$

Intuitively, $V^*(s)$ is the cumulative expected reward accrued from state s when using the optimal policy, and $Q^*(s, a)$ is the cumulative expected reward accrued from s by first taking action a and then using the optimal policy. We can compute both V^* and Q^* using Q-learning (Watkins and Dayan 1992). Now, we can rewrite the objective $J(\pi_H \oplus \rho)$ in Equation (2) as follows:

Lemma 1 (Bastani et al. 2018, Lemma 2.2). *For any policy π, we have*

$$J(\pi^*) - J(\pi) = \mathbb{E}_{\zeta \sim D^{(\pi)}}\left[\sum_{t=1}^{T} V_t^*(s_t) - Q_t^*(s_t, \pi(s_t))\right].$$

Applying this lemma to both π_H and $\pi_H \oplus \rho$ and taking the difference, we obtain

$$J(\pi_H \oplus \rho) - J(\pi_H) = \mathbb{E}_{\zeta \sim D^{(\pi_H)}}\left[\sum_{t=1}^{T} V_t^*(s_t) - Q_t^*(s_t, \pi_H(s_t))\right] - \mathbb{E}_{\zeta \sim D^{(\pi_H \oplus \rho)}}\left[\sum_{t=1}^{T} V_t^*(s_t) - Q_t^*(s_t, \pi_H \oplus \rho(s_t))\right].$$

Letting $\overline{D}_t^{(\pi)}$ be the marginal distribution of s_t in the distribution $D^{(\pi)}$ over rollouts, then

$$J(\pi_H \oplus \rho) - J(\pi_H) = \sum_{t=1}^{T} \mathbb{E}_{s_t \sim \overline{D}_t^{(\pi_H)}}[V_t^*(s_t) - Q_t^*(s_t, \pi_H(s_t))] - \mathbb{E}_{s_t \sim \overline{D}_t^{(\pi_H \oplus \rho)}}[V_t^*(s_t) - Q_t^*(s_t, \pi_H \oplus \rho(s_t))].$$

Now, assuming that $\overline{D}_t^{(\pi_H)} \approx \overline{D}_t^{(\pi_H \oplus \rho)}$, we have

$$\begin{aligned} J(\pi_H \oplus \rho) - J(\pi_H) &\approx \sum_{t=1}^{T} \mathbb{E}_{s_t \sim \overline{D}_t^{(\pi_H)}}[V_t^*(s_t) - Q_t^*(s_t, \pi_H(s_t))] - \mathbb{E}_{\zeta \sim \overline{D}_t^{(\pi_H)}}[V_t^*(s_t) - Q_t^*(s_t, \pi_H \oplus \rho(s_t))] \\ &= \mathbb{E}_{\zeta \sim D^{(\pi_H)}}\left[\sum_{t=1}^{T} Q_t^*(s_t, \pi_H \oplus \rho(s_t)) - Q_t^*(s_t, \pi_H(s_t))\right]. \end{aligned} \tag{3}$$

Intuitively, this assumption says that the *indirect* effect on performance because of the shift in the state distribution induced by the tip (i.e., from $\overline{D}_t^{(\pi_H)}$ to $\overline{D}_t^{(\pi_H \oplus \rho)}$) is small; instead, the main effect is because of the *direct* effect on performance because of the change in the

current human action induced by the tip, which is captured by Equation (3). In practice, we do not observe that the state distributions shift substantially, suggesting that this is a good approximation.

Next, we approximate the expectation in our objective using observed rollouts (i.e., historical trace data) $\zeta_1, \ldots, \zeta_k \sim D^{(\pi_H)}$ from the human policy π_H. Thus, our algorithm computes the tip

$$\hat{\rho} = \arg\max_{\rho} \frac{1}{k} \sum_{i=1}^{k} \sum_{t=1}^{T} Q^*(s_{i,t}, (a_{i,t} \oplus \rho)(s_{i,t})). \quad (4)$$

Here, we have dropped the terms $J(\pi_H)$ and $\mathbb{E}_{\zeta \sim D^{(\pi_H)}}[\sum_{t=1}^{T} Q_t^*(s_t, \pi_H(s_t))]$ because they are constant in ρ; for a given tip $\rho = (\psi, a)$ and action a', we have also defined the operation

$$(a' \oplus \rho)(s) = \begin{cases} a & \text{if } \psi(s) = 1 \\ a' & \text{otherwise.} \end{cases}$$

We optimize Equation (4) by enumerating through candidate tips ρ, evaluating the objective, and selecting the tip $\hat{\rho}$ with the highest objective value.

3. Virtual Kitchen Management Game

Our main empirical question is whether human workers can incorporate tips inferred using our algorithm into their broader decision-making policy. Specifically, our tips only provide partial information about the discrepancy between their policy and the optimal policy; thus, workers must not only comply with our tip (which is the usual challenge in improving human performance at one-shot decision-making problems), but they must implicitly infer additional information about the optimal policy in order to effectively operationalize our tip into their broader workflow. To achieve this goal, our environment was designed with two criteria in mind: (i) it should be possible for humans to compute the optimal policy given sufficient thought, but (ii) the optimal policy should not be obvious. We focused on deterministic environments where inexperienced workers could reason about the optimal strategy from very few interactions with the environment. Whereas we believe our insights extend to stochastic environments, they intuitively require more experience/interactions for humans to deduce optimal strategies. Finally, we deliberately designed a problem where we can compute the optimal policy (see Appendix A.2 for a description of this policy), which enables us to evaluate human suboptimality.

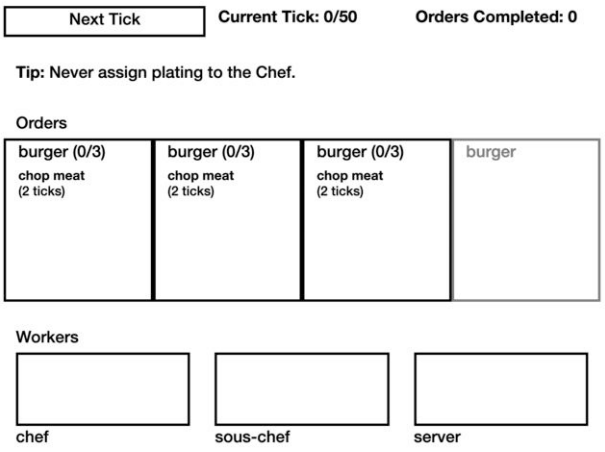
In particular, we build on the job shop scheduling problem, where the goal is to schedule jobs to machines in an optimal way and where there are dependencies between different jobs. To ensure the problem is sufficiently challenging, we introduce additional complexity in the form of heterogeneous machines, where the processing time for different types of jobs varies depending on the machine. To make our problem intuitive to human users, inspired by the popular game *Overcooked*, we represented our decision-making problem as a virtual kitchen management game that can be played by individual human players (see Figure 1). In this game, the player takes the role of a manager of several virtual workers (the "machines")—namely, chef, sous-chef, and server—serving burgers in a virtual kitchen. Each burger consists of a fixed set of subtasks (the "jobs") that must be completed in the order, namely, chopping meat, cooking the burger, and plating the burger. The game consists of discrete time steps; on each time step, the player must decide which (if any) subtask to assign to each idle worker. The worker then completes the subtask across a fixed number of subsequent time steps and then becomes idle again. A burger is completed once all its subtasks are completed, and the player completes the game once four burger orders are completed. The player's goal is to complete the game in as few time steps as possible.

There are two key aspects of the game that make it challenging. First, the subtasks have dependencies; that is, a subtask can only be assigned once previous subtasks of the same order have already been completed. For example, the "plate burger" task can only be assigned once the "cook burger" task is completed. Second, the virtual workers have heterogeneous skills; that is, different workers take different numbers of steps to complete different subtasks. For example, the chef is skilled at chopping/cooking but performs poorly at plating, whereas the server is the opposite, and the sous-chef has average skill on all subtasks; see Table B.1 in Appendix B for details. Ideally, one would match workers to tasks that they are skilled at to reduce completion time. Thus, the player faces the following dilemma. When a worker becomes available but is not skilled at any of the currently available subtasks, then the player must decide between (i) assigning a suboptimal subtask to that worker, potentially creating a bottleneck; or (ii) leaving the worker idle until a more suitable subtask becomes available. For instance, if the server is idle but all available subtasks are "cook burger," then the player must either (i) assign cooking to the unskilled server, thereby slowing down completion of that burger and eliminating the possibility of assigning plating to the server for the near future; or (ii) leave the server idle until a "plate burger" subtask becomes available. Furthermore, players are not shown the number of steps a worker takes to complete a subtask until they assign the subtask to that worker (see Figure 2 and Online Appendix D for example game screenshots); instead, they must experiment to learn this information.

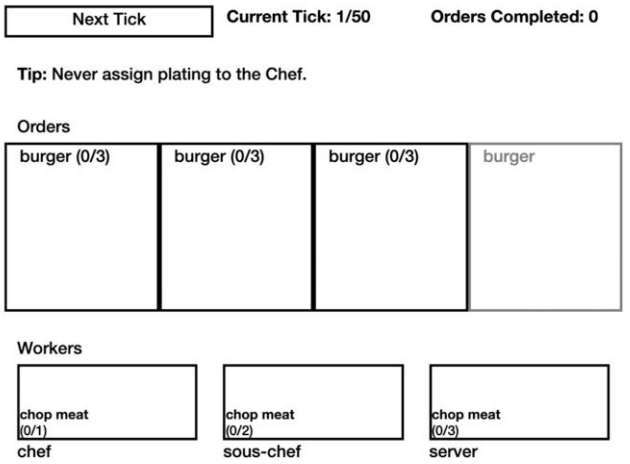
We consider two scenarios of the game, differing only in terms of worker availability. In the first scenario, the kitchen is *fully staffed*, where the human player has access to all three virtual workers (chef, sous-chef, and

Figure 2. Example Screenshots from the Game

(a) The initial state where players observe available subtasks, median times to completion, and three idle virtual workers. The interface also shows the current tick, time limit, current progress, and potential tip



(b) The next state after all three previously available subtasks were assigned to the virtual workers and the true completion times were realized, revealing different levels of virtual workers' skills



server). In the second scenario, the human player faces a disruption, and the kitchen becomes *understaffed*, with only two virtual workers (sous-chef and server). In both scenarios, the goal is to complete four burgers in as few time steps as possible. We describe how this decision-making problem can be formulated as an MDP and the resulting optimal policies in Appendix A. Note that the optimal policy completes four burgers in 20 and 34 time steps for the fully staffed and understaffed scenarios, respectively.

4. Experimental Design

We investigate how humans interpret and follow the tips inferred by our algorithm in the context of our virtual kitchen management game, using preregistered behavioral experiments involving Amazon Mechanical Turk workers.[1] We describe our experimental design in this section.

4.1. Overview

Figure 3 summarizes our experiment, which proceeds in two phases. In phase I, we recruit AMT workers to play our game without showing them any tips, and we collect trace and survey data on their behavior. This phase enables us to collect historical data that would normally already be available for an existing decision-making task, which we use to infer tips.

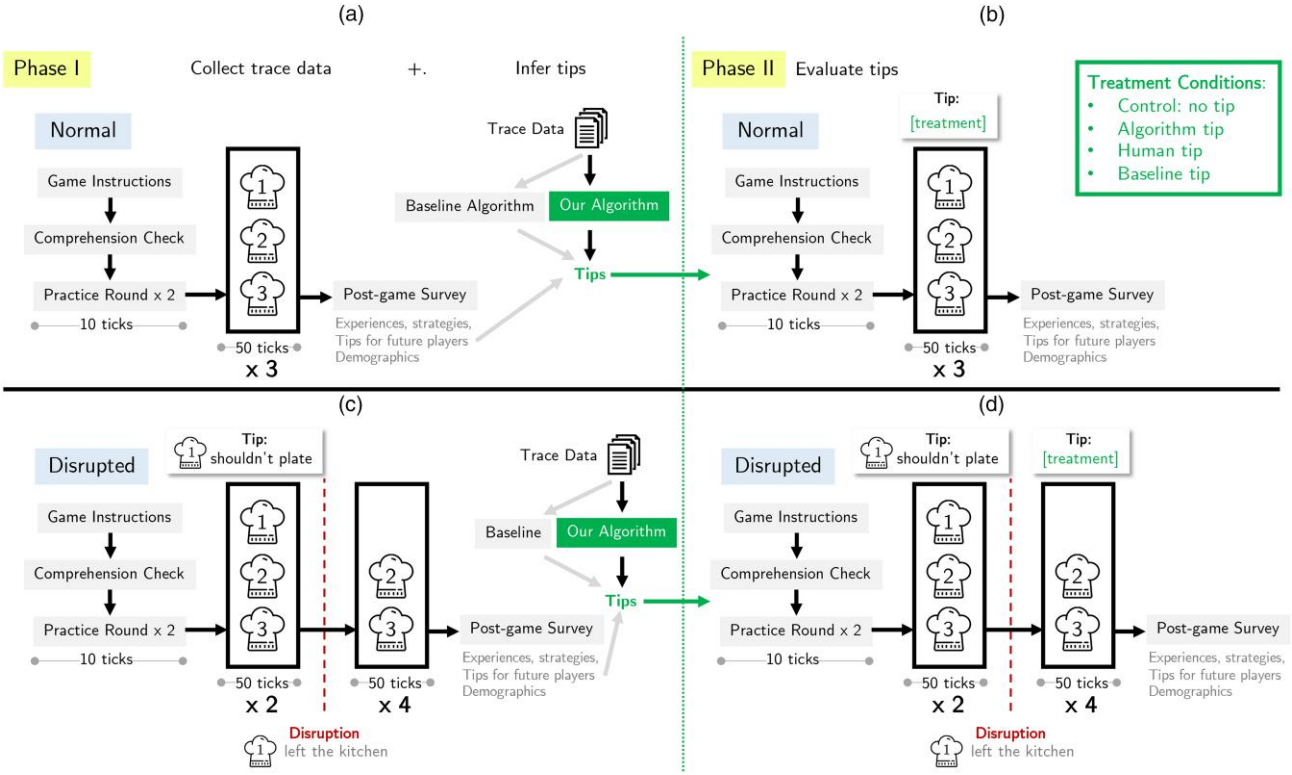
Next, phase II is our actual randomized controlled experiment; in this phase, we again recruit AMT workers to play our game, but this time, we randomize each participant into one of four *advice conditions* and show them a tip that depends on their advice condition (namely, the tip inferred using our algorithm, two alternative tips, and a control group where they are not shown any tip). We measure the performance of the participants, with the goal of determining whether our approach improves over the three alternatives. We describe the four advice conditions below.

In both phases, each participant plays a sequence of three or six *rounds* of our virtual kitchen management game; each round is one instance of our game that is completely independent of the other rounds. The number of rounds is determined by the *game configuration* they are assigned to (normal versus disrupted), which we described below. By having the participant play multiple rounds instead of a single one, we can study both how performance varies with the tip they are shown, as well as how it evolves across games as participants gain experience.

In summary, phase I is purely to gather data for computing tips; in this phase, participants are randomly assigned to one of two conditions (game configuration). Then, phase II is our main experiment, which uses a 2 (game configuration) × 4 (advice condition) between-subjects design; in this phase, participants are assigned randomly to the eight total conditions (two game configuration conditions times four advice conditions). See additional details on the experimental design (e.g., details on inferred tips, performance-based pay) in Appendix B, participant demographics in Online Appendix C, and screenshots of our game in Online Appendix D.

4.1.1. Game Configurations. In both phases of our experiment, participants are randomized into one of two game configurations, each of which determines a sequence of rounds of our game:

Figure 3. (Color online) Overview of Experimental Flow



Notes. (a) and (b) Depiction of phase I (a) and II (b) for the normal configuration, where each participant plays three fully staffed scenarios. (c) and (d) Depiction of phase I (c) and II (d) for the disrupted configuration, where each participant plays two fully staffed and four understaffed scenarios. Phase II participants are randomly assigned to one of four conditions (control, algorithm, human, and baseline). The set of participants across all four configuration-phase pairs is mutually exclusive.

- Normal configuration: Each participant plays three rounds of the fully staffed scenario.
- Disrupted configuration: Each participant plays two rounds of the fully staffed scenario, followed by four rounds of the understaffed scenario (i.e., the chef is no longer available), for a total of six rounds.

Intuitively, the normal configuration studies whether tips can help human participants fine-tune their performance. In contrast, the disrupted configuration is designed to show how tips can help participants adapt to novel situations where the optimal strategy substantially changes. The disrupted scenario is the more interesting one because disruptions often cause workers to struggle to adapt (Ramdas et al. 2017, Bavafa and Jónasson 2021), making tips especially useful.

4.1.2. Advice Conditions. In phase II, participants are randomly assigned not only to a game configuration but also one of four advice conditions:

- "Control group" condition: Participants are not shown any tips.
- "Our algorithm" condition: Participants are shown the tip inferred by our algorithm.
- "Human" condition: Similar to peer feedback, participants are shown the tip most frequently suggested by phase I participants after they have completed all rounds of our game.
- "Baseline algorithm" condition: Participants are shown a tip derived by a baseline algorithm that leverages simple descriptive statistics to identify the state-action pair where human participants and the optimal policy most frequently differ.

These advice conditions, described in more detail in Section 4.2, are chosen to illustrate how our algorithmic approach compares to and complements worker learning in practice.

4.1.3. Phase I Details. In phase I, we have $N = 183$ participants for the normal configuration and $N = 172$ participants for the disrupted configuration.

4.1.4. Phase II Details. In phase II, we have $N = 1{,}317$ participants for the normal configuration and $N = 1{,}011$ participants for the disrupted configuration. In the normal configuration, phase II participants are shown the tip for their advice condition for the fully staffed scenario on all rounds. In the disrupted configuration, they are shown the tip designated by their condition for the understaffed scenario (the last four rounds). In the first two rounds of the disrupted configuration, our goal is to

quickly acclimate participants to the fully staffed scenario in a way that is consistent across conditions. Thus, we show our algorithm tip for the fully-staffed scenario—"chef should never plate"—across all conditions (including control) for the first two rounds; we choose this tip because, as we show in Section 5, it most quickly improves human performance in the fully staffed scenario. After the disruption, we inform participants that the optimal strategy has now changed because of the chef's departure.

4.1.5. Participant Recruitment and Pay. We recruited participants on the Amazon Mechanical Turk platform. Each participant can only participate once across both phases and all conditions—that is, no participant has prior experience with any version of the game. Participants are compensated a flat rate for completing the study, plus a relatively large performance-based bonus determined by how quickly they complete each round of the game (see Appendix B.4 for details).

4.1.6. Hypotheses. Our main outcomes of interest are the average performance in the final round of the game (i.e., the average number of time steps taken by participants to complete all orders in the final round they play), as well as the fraction of participants who ultimately learn the optimal policy. The final round is the fourth round of the normal configuration and the sixth round of the disrupted configuration. Then, our main hypothesis is that for each of the two game configurations, participants in the "our algorithm" advice condition (i.e., shown the tip inferred using our algorithm) outperform participants in the other three advice conditions. In addition to our main hypothesis, we also examine participant behaviors in response to different tips, particularly their compliance, and how they learn to improve their decision making beyond the provided tips.

4.2. Advice Conditions

4.2.1. Control Group. The "control group" condition represents settings where best practices are not readily available, so workers must learn over time based on their own experience; indeed, we observe that performance improves over time without any tips in this condition.

4.2.2. Our Algorithm. The "our algorithm" condition represents our approach. In particular, we use the tip $\hat{\rho}$ inferred using our algorithm (Equation (4)) based on the trace data obtained in phase I. Additional details are provided in Appendix A.

4.2.3. Human. The "human" condition represents settings where one can obtain advice on best practices from more experienced peers (e.g., as in Song et al. 2017). We use phase I to do so. In particular, each participant in phase I is shown a comprehensive list of candidate tips after completing all rounds of our game and is asked to select the tip they believe would most improve the performance of future players. This list is constructed by merging three types of tips:

1. all possible tips of the format described in Appendix A.3 (e.g., "chef should not plate"),
2. a small number of generic player tips that arose frequently in our exploratory pilot studies (e.g., "keep everyone busy at all times"), and
3. a small number of manually constructed tips obtained by studying the optimal policy (e.g., "chef should chop as long as there is no cooking task").

Our algorithm's tip is always contained in this list as part of the first category above. This list contained 13–14 tips (depending on the configuration), which we found to be a reasonable length that did not overwhelm participants in our pilot studies. We take the most frequently chosen tip as the "human tip," capturing the wisdom of the (experienced) crowd. We also considered several variations, such as taking the tip recommended by the best-performing human participants, but these variations all resulted in the same tip; see Online Appendix C.4 for details.

The human tip is designed to demonstrate how our algorithmic approach can exceed the capabilities of humans to offer useful advice, capturing the limitations of relying on peers for advice.

4.2.4. Baseline Algorithm. The "baseline algorithm" condition illustrates a naïve use of descriptive statistics on historical trace data to provide tips—simply looking for frequent differences between the human and optimal policies, rather than leveraging interpretable reinforcement learning to identify the most consequential actions for improving performance. In particular, given rollouts $\zeta_1^*, \ldots, \zeta_h^* \sim D^{(\pi^*)}$ sampled using the optimal policy, we let $C^*(s,a)$ denote the number of times state-action pair (s,a) occurs across these rollouts. Then, given the observed rollouts (i.e., historical trace data from human decision making) $\zeta_1, \ldots, \zeta_k \sim D^{(\pi_H)}$, the baseline algorithm selects the tip

$$\hat{\rho}_{\text{bl}} = \arg\max_{\rho} \frac{1}{k} \sum_{i=1}^{k} \sum_{t=1}^{T} C^*(s_{i,t}, a_{i,t}). \tag{5}$$

In other words, our baseline optimizes the same objective but with Q^* replaced with C^*. Intuitively, this baseline strategy tries to directly imitate the optimal policy, whereas our strategy prioritizes state-action pairs that are more relevant to achieving high rewards. In this condition, we show participants the tip $\hat{\rho}_{\text{bl}}$ inferred by the baseline algorithm (Equation (5)) based on the phase I data.

This baseline algorithm ignores the sequential nature of our decision-making problem. It is designed to highlight the complexity of sequential structure compared with the one-shot decision-making setting studied in prior work and, in particular, the importance of accounting for this sequential structure when inferring tips.

5. Experimental Results

Despite their simplicity and conciseness, we find that our tips significantly improve performance by capturing strategies that are hard for participants to learn on their own; in contrast, alternative tips have empirical shortcomings that limit their effectiveness (Section 5.1). To better understand the underlying mechanisms, we examine how participants comply with different tips. First, we find that compliance increases across rounds, suggesting that participants do not blindly follow our tips but require time to understand and operationalize them (Section 5.2). Moreover, we find evidence that participants combine our tips with their own experience to discover additional strategies beyond the stated tips (Section 5.3). Finally, we find that interventions simply aimed at improving compliance may be insufficient to improve overall performance (Section 5.4). Together, our results suggest that even though our tip only encodes a portion of the optimal strategy, it guides participants to effectively explore and uncover additional insights that help them play optimally.

Figure 4(a) shows the tips inferred in each condition for each configuration using trace and survey data from phase I.

5.1. Performance: Our Tips Substantially Improve Performance

Figure 4 shows performance results across all four conditions and both configurations. Figure 4, (b) and (c) shows participant performance in the final round of our game, Figure 4, (d) and (e) shows how performance improves across rounds, and Figure 4, (f) and (g) shows the fraction of participants achieving optimal performance across rounds. For each configuration, we report performance as the excess ticks (time) taken over the optimal policy, normalized by the optimal policy's ticks; that is,

$$\frac{\text{number of ticks taken} - \text{optimal number of ticks}}{\text{optimal number of ticks}}.$$

Results in terms of the raw number of ticks are shown in Figure C.1 in Online Appendix C.2.

The normal configuration is relatively easy for participants—a substantial fraction (24%) discover the optimal policy by the final round without the aid of tips (control group). As shown in Figure 4(b), participants shown our tip completed the final round in 22.5 steps on average, significantly outperforming participants in the control group ($t(329) = -4.397$, $p < 10^{-4}$), those shown the human-suggested tip ($t(312) = -3.628$, $p = 2 \times 10^{-4}$), and those shown the tip from the baseline algorithm ($t(334) = -4.232$, $p < 10^{-4}$).[2] Our tip speeds up learning by at least one round compared with the other conditions; that is, the performance of participants given our tip on round k was similar to or better than the performance of participants in other conditions on round $k + 1$ (Figure 4(d)). Our tip also helped more participants (35%) achieve optimal performance (20 steps) in the final round, compared with 24%–29% in other conditions.

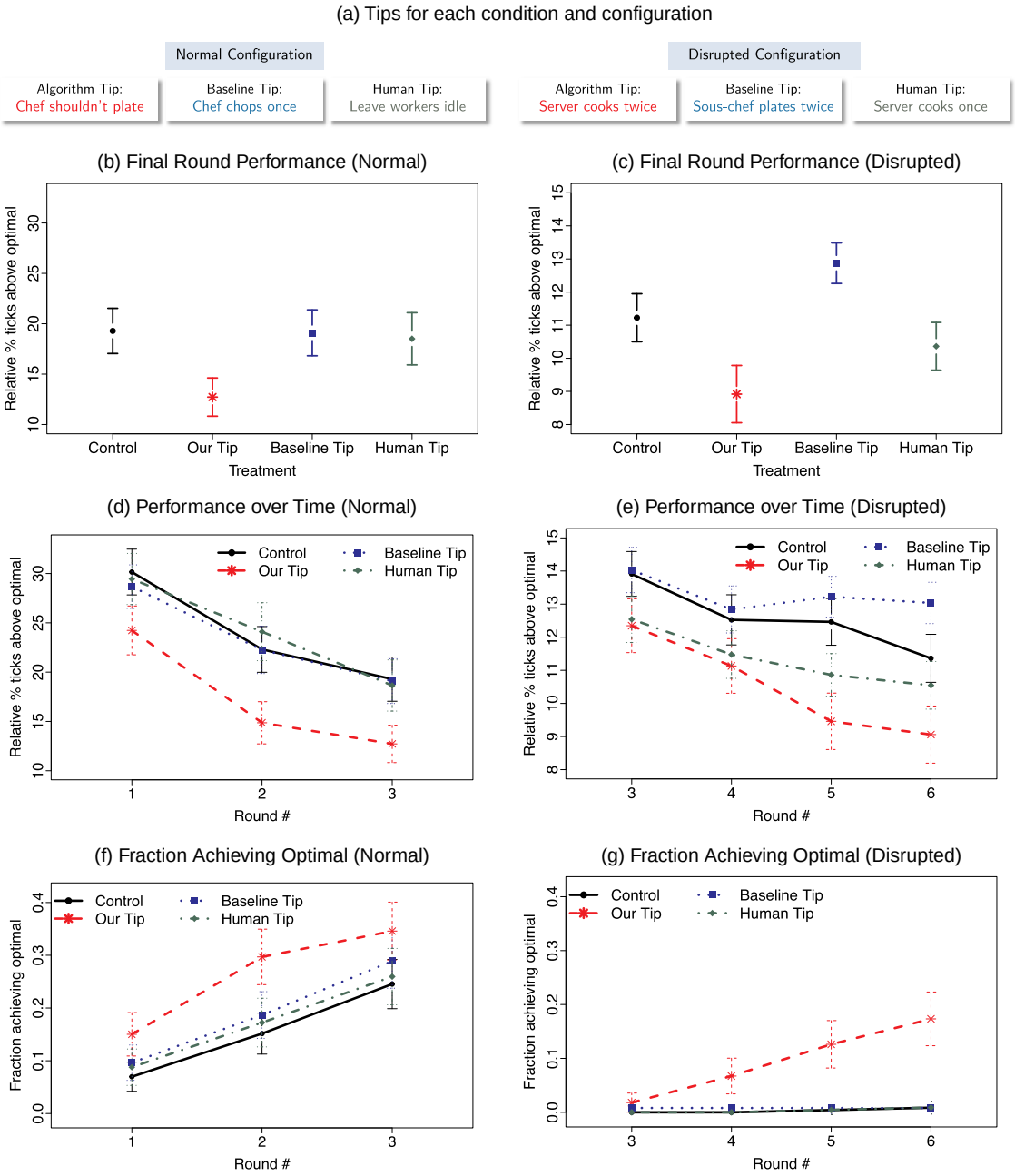
The disrupted configuration is substantially harder because participants must adapt to the more counterintuitive understaffed scenario. Perhaps as a consequence, participants benefit much more from tips: those in the control group took four rounds to achieve the same level of performance as those shown our tip in the first round. Participants shown our tip completed the final round in 37.1 steps, again significantly outperforming participants in the control group ($t(243) = -4.361$, $p < 10^{-4}$), those shown the human-suggested tip ($t(246) = -2.52$, $p = 6 \times 10^{-3}$), and those shown the tip from the baseline algorithm ($t(246) = -7.348$, $p < 10^{-4}$). In the disrupted configuration, the baseline tip actually *reduces* participant performance, likely because participants struggle to operationalize it. More starkly, 19% of participants shown our tip achieved optimal performance (34 steps) in the final round, compared with less than 1% in all other conditions; that is, our tip uniquely helps participants learn to play optimally. Note that there were no significant differences in performance across conditions when playing the two fully staffed rounds in the disrupted configuration. Therefore, the relatively worse performance under other conditions reflects the informativeness of alternative tips.

5.1.1. Shortcomings of Baseline Tips.

As noted earlier, this tip tries to mimic the optimal policy rather than focusing on consequential actions; thus, we expect these tips to be less valuable to participants (for improving performance) than our algorithm's tips. Participants complied with both the baseline algorithm's tips and our algorithm's tips at similar rates (see Section 5.2).

However, the baseline algorithm's tip is still derived from the optimal policy, so it is surprising that it performs *worse* than the control condition in the disrupted configuration. In fact, in Section 5.3, we show that participants who received our algorithm's tips also learned the strategy encoded in the baseline algorithm's tip; however, participants who received the baseline algorithm's tip did not learn the strategy encoded in our algorithm's tip in both configurations. Thus, the problem is not with the *content* of the baseline algorithm's tip but, rather, that participants struggle to *operationalize* the

Figure 4. (Color online) Phase II Participant Performance



Notes. The top row shows the tips derived for each condition and configuration based on phase I data. The remaining rows depict various views of participant performance across conditions in the normal (left) and disrupted (right) configurations. The top row shows performance in the last round of the configuration, the second row shows how participant performance improves over time, and the third row shows the fraction of participants who execute an optimal policy over time.

baseline tip into their workflow (without knowing our algorithm's tip).

In particular, when participants apply a tip, they shift to new unseen portions of the state space and must also learn to act well in those states. By focusing on "high-value" states and critical performance bottlenecks, our algorithm more easily enables participants' off-distribution learning. For example, in the disrupted configuration, the baseline algorithm's tip "sous-chef should plate twice" suggests actions that occur late in the game (hindering participants' ability to explore and alter their strategy) and does not focus on the critical performance bottleneck (cooking). In contrast, our algorithm's tip "server should cook twice" frees the sous-chef to plate later in the game (a strategy, not explicitly conveyed in our tip, that participants automatically learn when given our tip). However, targeting early decisions alone is not sufficient to help participants learn. In the normal configuration, although the baseline

algorithm's tip targets an *earlier* action ("chef should chop once") compared with our algorithm's tip ("chef shouldn't plate"), it fails to help participants learn the entire optimal strategy (see Section 5.3) because it does not address the important bottleneck (keeping the chef from the lengthy task of plating).

5.1.2. Shortcoming of Human Tips. Whereas the human-suggested tips consistently improve performance compared with the control group, they can be overly general or incorrect. In the normal configuration, phase I participants did not translate their strategy into a specific tip; that is, their suggested tip, "strategically leave some workers idle," captures a strategy needed to perform better but fails to convey any necessary details to identify the optimal strategy. Alternatively, in the disrupted configuration, phase I participants provided an *incorrect* tip, suggesting "server should cook once," whereas the optimal policy actually assigns the server to cook twice (as suggested by our tip); that is, participants identified the correct direction of change in response to the understaffing disruption, but at an insufficient magnitude. The tips chosen by participants are remarkably consistent across different participant subgroups, for example, top performers from phase I versus all participants (see Online Appendix C.5), and generally fail to capture counterintuitive properties of the optimal policy. Perhaps because of their more intuitive nature, participants are substantially *more* likely to comply with the human tip than with our algorithm's tip (see Section 5.2). Thus, our results suggest that the worse performance of the human tip is because of the suboptimal quality of the chosen tip.[3]

5.2. Compliance: Participants Comply with Tips More over Time

As discussed earlier, the effectiveness of a tip critically depends on whether humans are able to understand it and implement it effectively. This involves both complying with the tip's suggested actions as well as modifying other portions of their strategy to make full use of the tip. First, we examine compliance with the tips. Note that participants were not informed of the source of the tip (i.e., algorithm or human), so any variation in compliance is because of the content of the tip rather than behavioral reactions to its source (e.g., algorithmic aversion; see Dietvorst et al. 2015).[4]

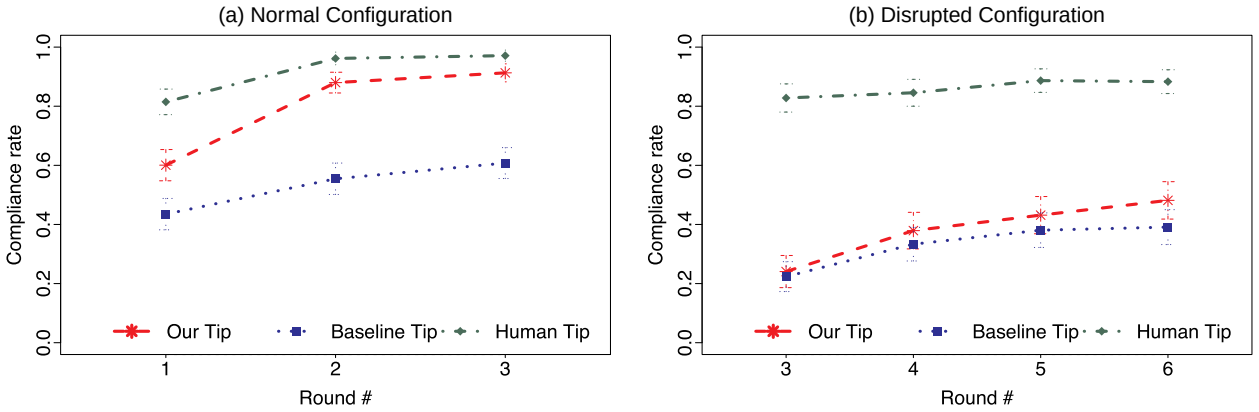
Figure 5 shows the fraction of participants that complied with the tip they were offered in each condition. Specifically, we measure the fraction of participants that act in a way that is consistent with the tip they are shown.[5] We see that participants increasingly comply with the tips shown over time—as they gain experience and better understand the significance of the tip—in all conditions. Compliance with the baseline algorithm's tip was relatively low in both configurations, suggesting that participants did not find it as useful. Alternatively, compliance with the human-suggested tip was higher than compliance with our algorithm's tip, particularly in the disrupted configuration. Based on participants' postgame feedback, we found that this is likely because the human-suggested tip better matches human intuition (because it is devised by humans). The disrupted configuration is illustrative. Although our algorithm's tip is correct (unlike the human-suggested tip), it is highly counterintuitive, hurting adoption. For example, in the disrupted scenario, our tip "server should cook twice" may appear unreasonable because the server is very slow at cooking; in fact, participants *just* learned to never assign the server cooking in the fully staffed scenario prior to the disruption. Yet having the server cook twice is the only way to achieve optimal performance in the understaffed scenario; in contrast, the human-suggested tip is to only have the server cook once, which is a less sharp departure from the previously employed policy. As participants gain experience with the new understaffed scenario, they grow to appreciate the value of our algorithm's tip (i.e., compliance with our algorithm's tip more than doubles over the four rounds). Our results suggest that participants do not blindly follow tips; instead, they only follow them if they believe that the suggested strategy is effective. These hypotheses are supported by a qualitative analysis of participants' perceptions of tips in the postgame survey; that is, they express significantly more positive sentiment toward the human tip than our algorithm's tip (see Online Appendix C.5 for details).

Thus, compliance is a function not just of the interpretability of the tip (which is unchanged across conditions) but also the strategy it encodes. When the optimal strategy is counterintuitive, we observe an intrinsic trade-off between the optimality of the tip and compliance with the tip. Even in the disrupted configuration, our algorithm's tip succeeds despite much lower compliance (relative to the human-suggested tip) because it suggests a highly effective strategy; as seen in Figure 4(g), participants that understand this strategy can achieve optimal performance (whereas essentially none of the participants in the other conditions were able to do so). Interestingly, as we show in the next subsection, participants in the *control group* also comply with the human-suggested tip at a high rate—that is, the human-suggested tip largely captures behaviors that are likely to be adopted even in the absence of tips; in contrast, our algorithm's tip allows participants to learn new strategies that they may not learn otherwise.

5.3. Learning Beyond Tips: Our Tips Help Humans Learn To Perform Optimally

One of the critical challenges in sequential decision making is that the human must learn strategies *beyond* the provided tip to achieve good performance throughout their workflow because the tip only captures a

Figure 5. (Color online) Compliance with Tips



Note. Participant compliance in phase II with the respective tip they were shown in each condition for the normal (a) and disrupted (b) configurations over time.

portion of the optimal policy. To study whether humans learn the optimal policy, we examine what kinds of strategies they learn beyond the specific tips they were shown. More precisely, we study *cross-compliance*, which is the compliance of the participant to tips other than the one they were shown. Naively, there is no reason to expect participants to cross-comply with a tip that we did not show them beyond the cross-compliance exhibited by the control group. Thus, any cross-compliance beyond that of the control group measures how a tip enables participants to learn strategies outside the stated tip. Assuming these strategies are consistent with the optimal policy, cross-compliance serves as a way to measure participants' progress toward operationalizing the tip effectively throughout their broader workflow.

We focus on the disrupted configuration because it is more challenging for participants, leading to more interesting cross-compliance patterns.[6] Figure 6 shows the cross-compliance of participants in each condition with the different tips (algorithm, baseline, human), as well as a new tip ("server chops once") not shown to any participants. This new tip is part of the optimal policy for the understaffed scenario used in the disrupted configuration. Participants in the human and control groups only comply with the human tip. Indeed, the human-suggested tip actually contradicts the optimal policy; thus, despite effectively operationalizing the tip, participants are prevented from learning the other tips that are part of the optimal policy.[7] Participants shown the baseline tip only have high compliance with the baseline tip, indicating that the baseline tip could not help participants uncover the rest of the optimal policy; although the baseline tip is part of the optimal policy, it fails to help participants discover strategies beyond the tip itself because it does not focus on high-value states and critical bottlenecks (see our earlier discussion in Section 5.1). In contrast, participants who received our algorithm's tip have high cross-compliance with *all* parts of the optimal policy (i.e., the baseline tip and the unshown tip); furthermore, our algorithm is the only condition where cross-compliance with the suboptimal human tip decreases over time. That is, our tip uniquely enables participants to combine the tip with their own experience to discover useful strategies (that are consistent with the optimal policy) beyond what is stated in the tip.

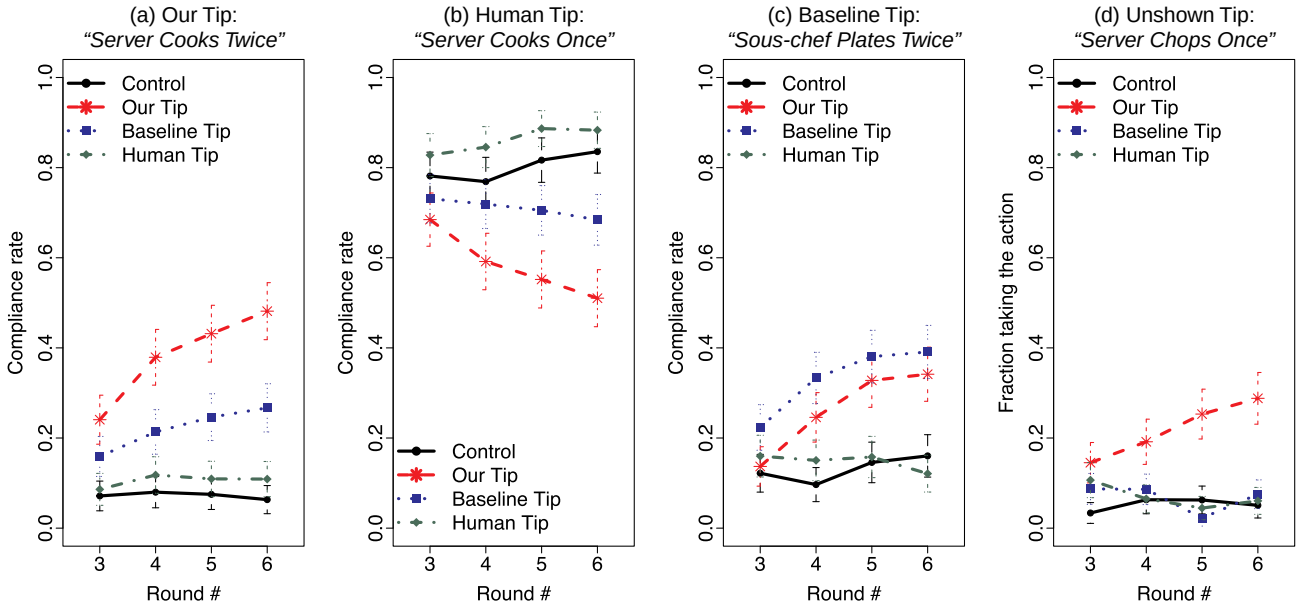
5.4. Compliance Interventions: Improving Compliance May Not Improve Performance

As discussed in Section 5.3, to achieve optimal performance in sequential decision-making tasks, participants must not only comply with the stated tip but also learn other parts of the optimal policy. This suggests that improving compliance alone may not yield performance improvements. To study this, we performed a follow-up user study in the disrupted configuration.[8]

We tested four well-studied interventions aimed at improving compliance with our algorithmic tip: (i) paying users to comply ("Pay"), (ii) suggesting that their high-performing peers complied ("Social"), (iii) a combination of the pay and social interventions ("Pay-Social"), and (iv) using a curriculum to gradually acclimate users to the tip ("Curriculum"). These interventions were only applied in the first two rounds following the disruption (rounds 3 and 4) to avoid distorting performance in the final two rounds (rounds 5 and 6). The control arm ("Tip Only") is identical to the algorithm arm in the original study. We recruited $N = 1{,}496$ participants from the AMT platform and randomized them across these five arms; see Online Appendix C.7 for details.

Figure 7(a) shows compliance rates by condition across all four rounds. As expected, we find that any combination of "Pay" and "Social" interventions improves compliance with our algorithm's tip. Moreover, compliance was "sticky"; that is, participants who complied when

Figure 6. (Color online) Learning Beyond Tips



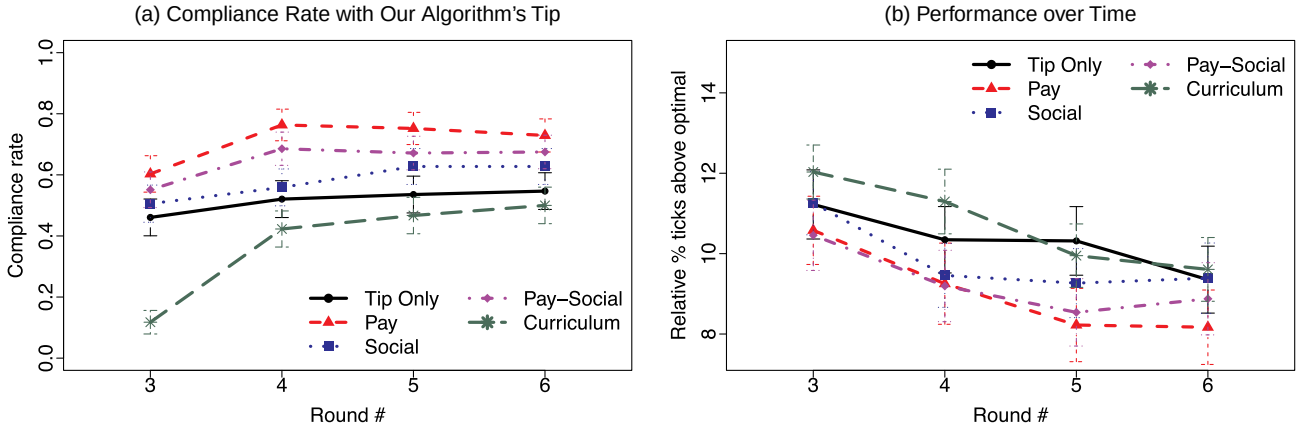
Notes. (a)–(c) The rate at which participants in each condition cross-comply with each offered tip over time in the disrupted configuration. (d) Analogous results for a rule that is part of the optimal policy but was not shown as a tip in any condition.

receiving the intervention continued to comply in the final intervention-free rounds. The "Pay," "Pay-Social," and "Social" interventions significantly improved compliance in round 6 by 18% ($t(527) = 4.434$, $p < 10^{-4}$), 13% ($t(548) = 3.098$, $p = 10^{-3}$) and 8% ($t(528) = 1.886$, $p = 0.03$), respectively, compared with the "Tip Only" condition. The "Curriculum" intervention, which slowly eased people toward our algorithm's tip, did not meaningfully improve compliance by the end of the game.[9]

However, these increases in compliance did not always translate to improvements in overall performance in the final round of the game (see Figure 7(b)). The "Pay," "Pay-Social," and "Social" interventions improved performance by 0.4 steps ($t(527) = -1.873$, $p = 0.03$), 0.01 steps ($t(538) = 0.059$, $p = 0.5$), and -0.2 steps ($t(519) = -0.767$, $p = 0.2$), respectively, compared with the "Tip Only" condition. In other words, even the 13% increase in compliance induced by the "Pay-Social" tip resulted in an essentially null effect on performance; the 18% increase in compliance induced by the "Pay" intervention only increased performance by a mere 0.4 steps.

Thus, our results demonstrate that improving immediate compliance does not necessarily improve longer-term performance; even if there is some positive effect on performance, this effect is smaller or noisier than the

Figure 7. (Color online) Interventions for Compliance



Note. Participant compliance with our algorithm's tip ("server should cook twice") (a) and participant performance (b) in each intervention across the four disrupted rounds.

effect on compliance. These results are consistent with our hypothesis that workers fail to comply in part because they cannot correctly operationalize the tip.

6. Concluding Remarks

We have proposed a novel reinforcement learning algorithm for automatically identifying interpretable tips designed to help improve human sequential decision making. Our large-scale behavioral study demonstrates that the tips inferred by our algorithm can successfully improve human performance at challenging sequential decision-making tasks, speeding up learning by up to three rounds of in-game experience. Furthermore, we find evidence that participants combine our tips with their own experience to discover additional strategies beyond those stated in the tip. In other words, our algorithm is capable of identifying concise insights and communicating them to humans in a way that expands and improves their knowledge. To the best of our knowledge, our work is the first to empirically demonstrate that reinforcement learning–based tips can be used to improve human sequential decision making.

An important ingredient in our framework is incorporating trace data to identify succinct pieces of information that are most likely to help improve the performance of an average worker. Modern-day organizations have benefited from using customer data to inform new product strategies and to provide personalized offerings to their customers, but the data on their own employees are underused. Trace data are often noisy and too granular to be readable and immediately useful to humans. Our machine learning framework provides techniques to leverage the largely untapped potential of readily available trace data in pinpointing areas of performance improvement and identifying new practices. Even when the true optimal strategy is unknown, trace data of experienced or high-performing workers can be used with reinforcement learning to identify good strategies.

Furthermore, we provide a number of insights that can aid the design of human-AI interfaces. First, a significant factor in the performance of a tip is whether humans comply with that tip. Prior work has studied compliance from the perspective of *algorithm aversion* (i.e., whether humans trust other humans more than algorithms) (Eastwood et al. 2012; Dietvorst et al. 2015, 2018) as well as interpretability (i.e., whether the human understands the tip) (Doshi-Velez and Kim 2017, Lage et al. 2018, Rudin 2019). Our results suggest that human compliance additionally depends on whether humans believe (based on their intuition and past experience) that the tip improves performance as well as whether they are able to understand how to operationalize the tip. Second, it takes time for humans to correctly operationalize and adopt the tip—humans need experience to understand why the tip is correct and discover complementary strategies that further improve their performance. Thus, there is an opportunity for human-AI interfaces to help humans *gradually* adapt their behavior to improve performance. Third, as evidenced by the baseline tips, even tips that are part of the optimal policy can hurt participant performance if they focus on actions that are not consequential; avoiding such tips is important because it can cause participants to lose trust in machine learning models. We anticipate that human-AI interfaces will become increasingly prevalent as machine learning algorithms are deployed in real-world settings to help humans make consequential decisions, and a better understanding of how to design trustworthy interfaces will be critical to ensuring that these interfaces ultimately improve human sequential decision making.

Acknowledgments

The authors thank Sinan Aral, Ryan Buell, Paul Leonardi, Bryce McLaughlin, and Jann Spiess as well as conference and seminar participants at Boston University, Harvard Business School, the INFORMS Annual Meeting, Massachusetts Institute of Technology, the MSOM Conference, Stanford University, the University of California, Berkeley, and the University of Pennsylvania for helpful comments. The authors thank the Wharton Behavioral Laboratory, the Wharton Risk Center Ackoff Doctoral Student Fellowship, and the BAIR Open Research Commons for financial support. The authors are grateful for the research assistance of Brandon Chin, Xiteng Lin, Ron Wang, and Yuanxin Zhu.

Appendix A. Tip Inference Algorithm

We first discuss how we formulate the Markov decision process (MDP) for our virtual kitchen management game and the overall structure of the optimal policies for both the fully staffed and understaffed scenarios. Then, we provide detailed information on the design and implementation of our tip inference algorithm.

A.1. MDP Formulation

In our virtual kitchen MDP, the states encode (i) which subtasks have been completed so far across all orders, and (ii) which subtask has been assigned to each virtual worker (if any), as well as how many steps remain to complete this subtask. The actions consist of all possible assignments of available subtasks (i.e., have not yet been assigned) to available virtual workers (i.e., not currently working on any subtask). The reward is -1 at each step until all orders are completed; thus, the total number of steps taken to complete all orders is the negative reward.

A.2. Optimal Policies

We summarize the optimal policy for each scenario. Note that the optimal policy for the understaffed scenario is more counterintuitive than that for the fully staffed scenario.

Fully staffed scenario. Here, the participant has access to all three virtual workers. The optimal number of ticks to

complete this scenario is 20. The key insights to achieving optimality are (i) all three workers should be assigned to chopping in the first time step; (ii) the chef must cook three of the burgers, and the sous-chef must cook one (i.e., the second burger); (iii) the server should never cook and must be kept idle when the third burger becomes available for cooking; they should instead wait to be assigned to plating the first cooked burger; (iv) the chef should never plate; (v) the sous-chef must plate exactly one of the burgers; and (vi) none of the three workers should be left idle except in the previous cases.

Understaffed scenario. Here, the participant has access to only two virtual workers (e.g., the sous-chef and the server). The optimal number of ticks to complete this scenario is 34. The keys insights to achieving optimality are (i) both workers should be assigned to chopping in the first time step; (ii) the sous-chef and the server must cook two burgers each, even though the server is slow at cooking; (iii) the sous-chef must choose chopping over cooking after finishing the first chopping task; (iv) the server's first three tasks must be chopping, cooking, and cooking, in that order; (v) the sous-chef must chop three of the four burgers, and the server must chop one; (vi) both workers must plate two burgers each, even though the sous-chef is slower at plating; (vii) the second cooked burger must not be served until the third and fourth burgers are cooked; and (viii) both workers must be kept busy at all times.

A.3. Search Space of Tips

Each tip is actually composed of a set of rules inferred by our algorithm. Recall that our algorithm considers tips in the form of an if-then-else statement that says to take a certain action in a certain state. One challenge is the combinatorial nature of our action space—there can be as many as $k!/(k-m)!$ actions, where m is the number of workers, and $k = \sum_{j=1}^{n} k_j$ is the total number of subtasks. The large number of actions can make the tips very specific, for example, simultaneously assigning three distinct subtasks to three of the virtual workers. Instead, we decompose the action space and consider assigning a single subtask to a single virtual worker. More precisely, we include three features in the predicate ϕ: (i) the subtask being considered, (ii) the order to which the subtask belongs, and (iii) the virtual worker in consideration. Then, our algorithm considers tips of the form

$$\begin{aligned}&\text{if (order} = o \wedge \text{subtask} = s \wedge \text{virtual worker} = w)\\ &\quad \text{then (assign } (o,s) \text{ to } w),\end{aligned}$$

where o is an order, s is a subtask, and w is a virtual worker.

Even with this action decomposition, we found that these tips are still too complicated for human users to internalize. Thus, we postprocess the tips inferred by our algorithm by aggregating over tuples (o,s,w) that have the same s and w.[10] In particular, consider a tip $\rho = (\psi, a)$ with state predicate ψ and action a, where $a = (o,s,w)$ is a tuple consisting of a subtask s of an order o that is to be assigned to worker w. Our algorithm first aggregates all tips of the form $\rho = (\psi, (o,s,w))$ with the same subtask-worker pair (s,w) to obtain a list $R_{s,w} = \{\rho_1, \ldots, \rho_k\}$ for each (s,w) pair. This (s,w) pair is converted into a tip by counting the number of distinct orders o that occur across $\rho \in R_{s,w}$; if j different orders o occur, then the tip becomes

$$\text{assign } s \text{ to } w, \ j \text{ times}.$$

For example, instead of considering two separate tips

$$\begin{aligned}&\text{if (order} = \text{burger}_1 \wedge \text{subtask} = \text{cooking} \wedge \text{virtual worker} = \text{chef})\\ &\quad \text{then (assign (burger}_1, \text{cooking) to chef)}\\ &\text{if (order} = \text{burger}_2 \wedge \text{subtask} = \text{cooking} \wedge \text{virtual worker} = \text{chef})\\ &\quad \text{then (assign (burger}_2, \text{cooking) to chef)},\end{aligned}$$

we merge them into a tip

$$\text{assign cooking to chef 2 times}.$$

Next, the score our algorithm assigns to the aggregated tip $R_{s,w}$ is $J(R_{s,w}) = \sum_{\rho \in R_{s,w}} J(\rho)$. Finally, our algorithm chooses the tip $R_{s,w}$ with the highest score.

A.4. Tip Inference Procedure

Next, we describe how our algorithm computes optimal tips for our MDP. Whereas our state space is finite, it is still too large for dynamic programming to be tractable. Instead, we use the policy gradient algorithm to (heuristically) learn an expert policy π_* (Sutton et al. 2000), which uses gradient descent to optimize a policy π_θ with parameters $\theta \in \Theta \subseteq \mathbb{R}^{d_\Theta}$; we choose π_θ to be a neural network. This approach requires that we construct a feature map $\phi : S \to \{0,1\}^d$. Then, π_θ takes as input the featurized state $\phi(s)$ and outputs a categorical distribution $\pi_*(a \mid \phi(s))$ over actions $a \in A$. Then, the policy gradient algorithm performs stochastic gradient descent on the objective $J(\pi_\theta)$ and outputs the best policy $\pi_* = \pi_{\theta^*}$. For the kitchen game MDP, we use state features, including whether each subtask of each order is available, the current status of each worker, and the current time step. We take π_θ to be a neural network with 50 hidden units; to optimize $J(\pi_\theta)$, we take 10,000 stochastic gradient steps with a learning rate of 0.001.

Once we have computed π_*, we use our tip inference algorithm to learn an estimate $\hat{Q}$ of the Q-function $Q^{(\pi_*)}$ for π_*. We choose $\hat{Q}$ to be a random forest (Breiman 2001). It operates over the same featurized states as the neural network policy; that is, it has the form $\hat{Q}(\phi(s), a) \approx Q^{(\pi_*)}(s,a)$. Finally, we apply our algorithm to inferring tips on state-action pairs collected from observing human users playing our game. Because our goal is to help human users improve their performance, we restrict the training data set to the bottom 25% performing human users—indeed, our expected improvement is much higher for the bottom 25% (3.6 tips faster for normal, 4.4 ticks faster for disrupted) than for everyone (2.1 ticks faster for normal, 1.8 ticks faster for disrupted), demonstrating that our tip is expected to be most effective for the bottom quartile of participants. In Online Appendix C.4, we show that our algorithm is robust to this choice; that is, it produces the same tips if we instead consider the bottom 50% of participants or all participants.

In addition, we apply two postprocessing steps to the set of candidate tips. First, we eliminate tips that apply in less than 10% of the (featurized) states that occur in the human data set. This step eliminates high-variance tips that may have large benefit but are useful only a small fraction of the time; we omit such tips because our

estimates of their quality tend to have very high variance. Second, we eliminate tips that disagree with the expert policy more than 50% of the time; that is, for a tip (ψ, a), we have $\psi(s) = 1$ and $a \neq \pi^*(s)$ for more than 50% of state-action pairs in the human data set. This step eliminates tips that have large benefits on average but frequently offer incorrect advice that can confuse the human user or cause them to distrust our tips. In Online Appendix C.4, we show that this second elimination step is robust to the choice of threshold.

A.5. Adapting Our Tips to Other Domains

Broadly speaking, a challenge in interpretable machine learning is that the space of interpretable models must be tailored to each new domain to ensure that the model captures insights relevant to that domain in a human-interpretable way. For our virtual kitchen management game, we have tailored our tips to convey useful information by first inferring if-then rules and then aggregating these rules into useful tips. The design decisions include both the postprocessing steps used to prune and aggregate tips as well as the feature map over states used to infer tips. We arrived at this trade-off because we wanted tips that could be easily read and understood by human participants while conveying useful information for improving decision making. The specific choices we made and the postprocessing steps we used were informed by our pilot studies.

When applying our algorithm to a new domain, our approach must be adapted so that it infers tips that are useful for that domain. In general, the goal should be to produce tips that are as informative as possible under the condition that a human worker can understand what the tip is trying to convey in a reasonable amount of time. For tasks where individual decisions must be made quickly, the tip must be very succinct and easy to understand; in these settings, postprocessing strategies such as ours may be necessary to ensure the human understands the tip. Otherwise, more detailed tips, such as the original if-then rules, can be used.

Finally, we briefly comment on when we expect our algorithm's tip to outperform both the human tip and the baseline algorithm's tip. As our results demonstrate, the human tip tends to have higher compliance because it is usually more intuitive, yet it might be suboptimal in settings where the optimal policy is complex/unintuitive. As a consequence, we expect the human tip to be more effective when the optimal strategy is intuitive; alternatively, one can imagine scenarios where the optimal policy is simply too complex for the human to determine (even with our algorithm's tip), making it better to go with a more intuitive but less effective strategy. For the baseline tip, we expect it to only be effective when the sequential structure is relatively unimportant for achieving good performance (e.g., in well-mixed MDPs), and the human can focus on achieving good immediate reward. In this case, a strategy that directly tries to maximize immediate rewards may also be effective.

Appendix B. Additional Details on Experimental Design

We perform separate experiments for each of the two configurations of our game. The high-level structure of our experimental design for each configuration is the same; they differ in terms of when we show tips to the participant and which tips we show. Before starting our game, all participants are shown a set of game instructions and comprehension checks; then, they play a practice scenario twice (with an option to skip the second one). The practice scenario is meant to familiarize participants with the game mechanics and the user interface. In this scenario, they manage three identical chefs to make a single food order (different than the burger order used in the main game). Then, they proceed to play the scenarios for the current configuration. Table B.1 exhibits the number of time steps needed for each of the virtual workers to complete each of the subtasks required to complete a single burger order.

After completing all scenarios, we give each participant a postgame survey regarding their experience with the game. Each participant receives a participation fee of \$0.10 for each round they complete; they also receive a performance-based bonus based on the number of time steps taken to complete each round. The bonus ranges from \$0.15 to \$0.75 per round. Participants provided informed consent, and all study procedures were approved by our institution's institutional review board.

B.1. Phase 1

For each configuration, we recruited 200 participants via Amazon Mechanical Turk (AMT). As part of the postgame survey, we ask the participants to suggest a tip for future players. In particular, we show each participant a comprehensive list of candidate tips and ask them to select the one they believe would most improve the performance of future players. This list of tips is constructed by merging three types of tips: (i) all possible tips in the search space considered by our algorithm (e.g., "Chef shouldn't plate"), (ii) generic tips that arise frequently in our exploratory pilot studies (e.g., "Keep everyone busy at all time"), and (iii) a small number of manually constructed tips obtained by studying the optimal policy (e.g., "Chef should chop as long as there is no cooking task"). Importantly, this list always contains the top tip inferred using our algorithm (Figure B.1).

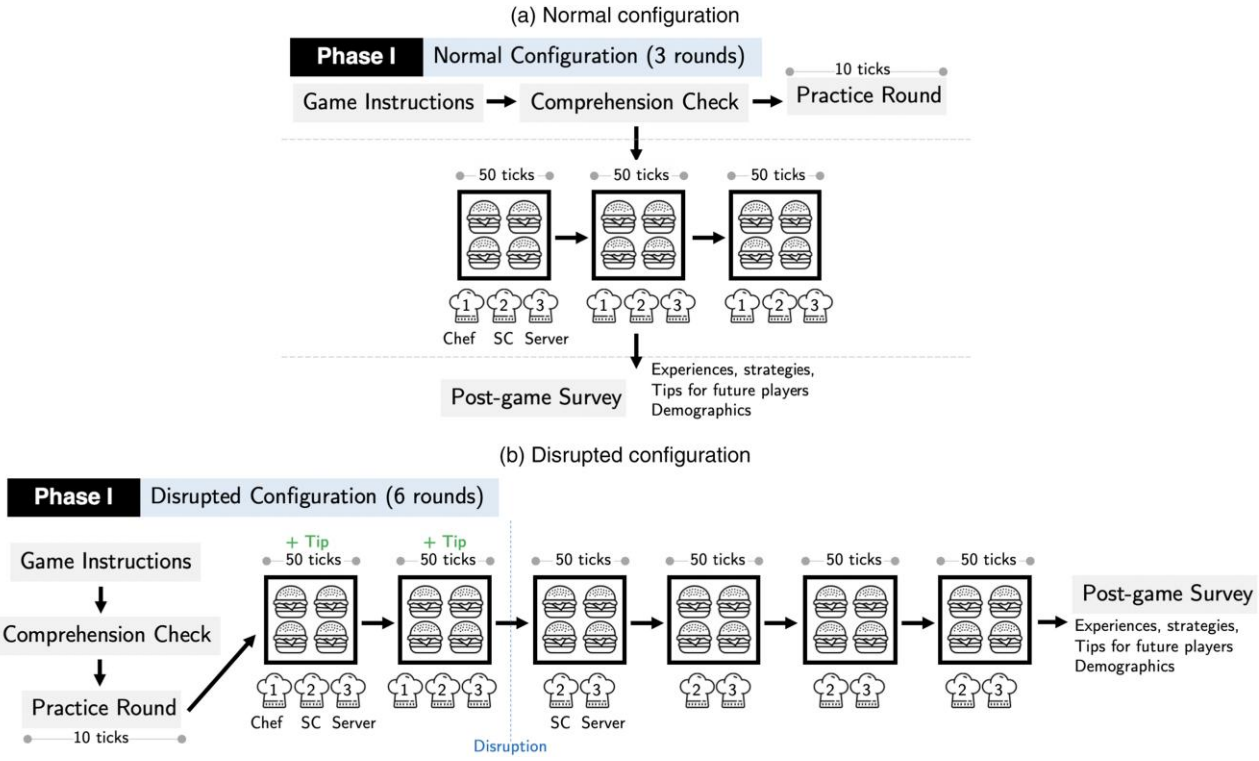
B.2. Inferred Tips

Next, we use participant data from the final round to infer tips in three ways: (i) use our tip inference algorithm in conjunction with the data from phase I, (ii) do the same with the baseline algorithm, and (iii) rank the candidate tips in the postgame survey based on the number of votes by the participants. As shown in Online Appendix C.4, the human tips are robust to the participant subgroup used to construct them; that is, we get the same tips if we restrict only to top performers.

Table B.1. The (Deterministic) Number of Time Steps Each Virtual Worker Requires to Complete a Given Subtask

	Chopping meat	Cooking burger	Plating burger
Chef	1	4	6
Sous-chef	2	8	2
Server	3	12	1

Figure B.1. (Color online) Study Flow for Phase I



For the normal configuration, 183 participants[11] successfully completed the game. The top three tips inferred from each of the sources are reported in Table B.2. For the algorithm tip, "Chef should never plate" is selected as it is expected to be the most effective at shortening completion time (2.43 steps). For the baseline tip, our naïve algorithm selects "Chef should chop once," as it is the most frequently observed state-action pair in the data. Finally, for the human tip, "Strategically leave some workers idle" received the most votes among the participants (28.42%). It is worth noting that all of the tips most voted by past players are in line with the optimal strategy. The first tip captures the key strategy that some virtual workers should be left idle rather than assigned to a time-consuming task. However, it is less specific than other tips. The second and third tips reflect the information participants could learn from assigning different tasks to different workers during the game: the server spends the most time cooking, whereas the chef spends the most time plating.

For the disrupted configuration, 172 participants[12] successfully completed the game. Table B.3 reports the top three tips inferred from each of the sources. The best algorithm tip is "Server should cook twice," with the expected completion time reduction of 2.32 steps. The baseline algorithm chooses "Sous-chef should plate twice" and the human tip "Server should cook once" (equivalently, "Sous-chef should cook three times") got the most votes. Unlike the normal configuration, the top two human tips are not part of the optimal policy. In the optimal policy, sous-chef and server should each cook twice. The third human tip does align with the optimal policy; however, it is much less specific than the other tips. This highlights the increased difficulty for humans to identify the optimal strategy in the disrupted configuration compared to the normal configuration.

B.3. Phase II

Next, we evaluate the effectiveness of these tips. In this phase, participants are randomly assigned to one of four conditions (control, baseline, algorithm, human). We recruited 350 AMT users to play each condition in each configuration, totaling to 2,800 users. The specific tips we show

Table B.2. Top Three Tips Inferred from Different Sources for the Normal Configuration

Normal	Tip 1	Tip 2	Tip 3
Algorithm	**Chef should never plate**	Server plates three times	Server should skip chopping once
Baseline	**Chef should chop once**	Server should plate three times	Sous-chef should plate twice
Human (% voted)	**Strategically leave some workers idle** (28)	Server should never cook (21)	Chef should never plate (13)

Table B.3. Top Three Tips Inferred from Different Sources for the Disrupted Configuration

Disrupted	Tip 1	Tip 2	Tip 3
Algorithm	**Server should cook twice**	Sous-chef should plate once	Server should chop once
Baseline	**Sous-chef should plate twice**	Sous-chef should chop three times	Server should cook twice
Human (% voted)	**Server should cook once** (28)	Server should never cook (24)	Keep everyone busy (17)

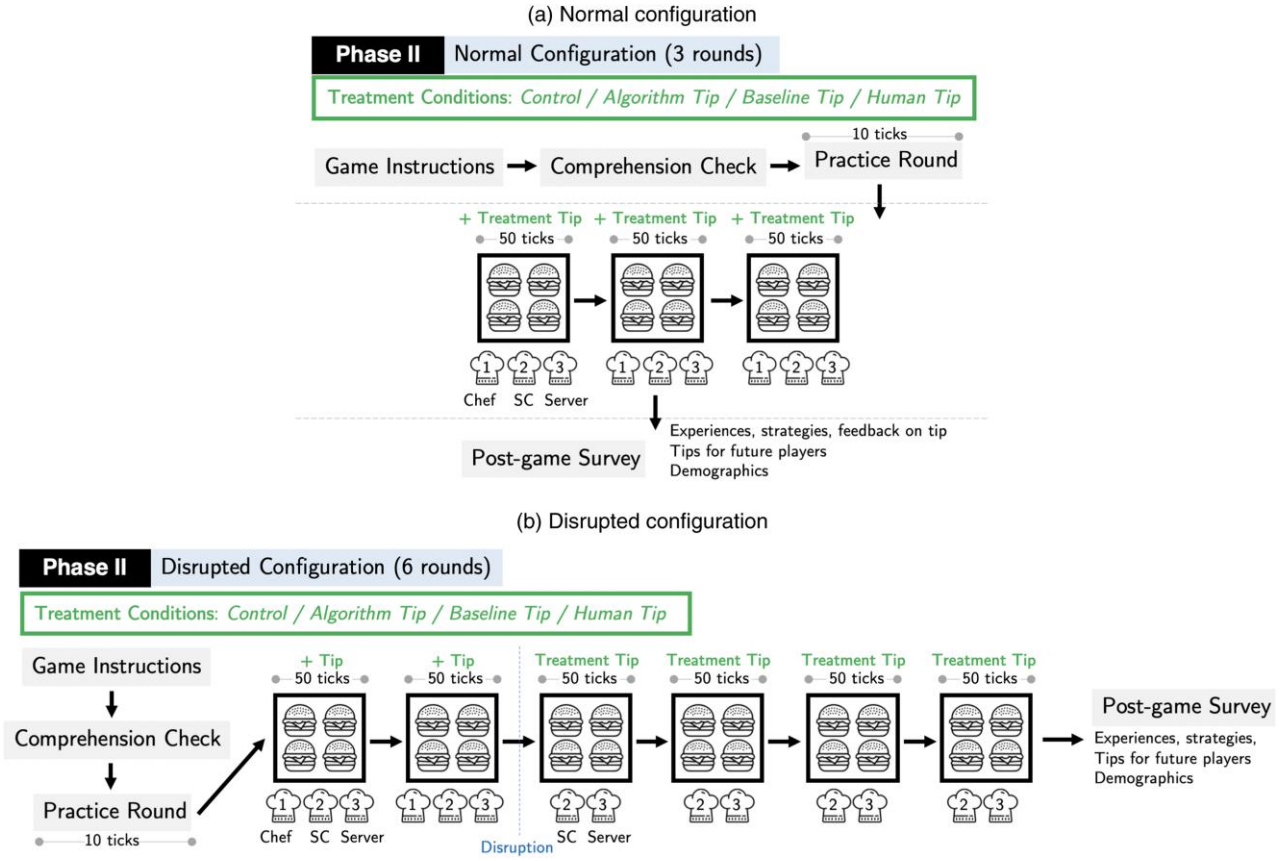
in each round depends not just on the condition but also varies from round to round, depending on the configuration. For the normal configuration, we show the tip for the current condition in all three rounds. However, for the disrupted configuration, the tip for the current condition is specific to the understaffed scenario. Thus, we only show the tip for the current condition in rounds 3–6; in all conditions, for rounds 1 and 2, we show the tip inferred by our algorithm for the fully staffed scenario from the normal configuration. By doing so, we ensure that the tip shown during the fully staffed scenario does not bias our evaluation of the tip for the understaffed scenario.

B.4. Pay Schemes

Normal configuration. In Phase I, participants received \$0.30 as a base pay for their participation. In addition, they could earn a performance-based bonus for each of the three rounds of the game. The optimal (e.g., shortest possible) completion time is 20 time steps, and the maximum time allowed is 50 time steps. The bonus is as follows: \$0.75 if completing the round in exactly 20 time steps, \$0.35 if completing the round in 21 to 22 time steps, \$0.15 if completing the round in 23 to 26 time steps, or no bonus otherwise. The total pay ranges from \$0.30 to \$2.55, with a mean of \$1.00, a median of \$0.95, and a standard deviation of \$0.56. The sum of the total pay is \$182.15 (183 participants). In phase II, which was conducted well into the COVID-19 pandemic, we kept the same base pay but slightly increased the tiered bonus: \$1.25 if completing the round in exactly 20 time steps, \$0.60 if completing the round in 21 to 22 time steps, \$0.25 if completing the round in 23 to 26 time steps, or no bonus otherwise. The total pay ranges from \$0.30 to \$4.05, with a mean of \$1.63, a median of \$1.40, and a standard deviation of \$1.03. The sum of the total pay is \$2,149.70 (1,317 participants) (Figure B.2).

Disrupted configuration. In both phases, participants received \$0.60 as a base pay for their participation. In addition, they could earn a performance-based bonus for each of the six rounds of the game. For the first two rounds, in which they managed a fully staffed kitchen, the bonus

Figure B.2. (Color online) Study Flow for Phase II



scheme is the same as that of phase I of the normal configuration. For the last four rounds in which they managed an understaffed kitchen (optimal completion time is 34 time steps), the bonus is as follows: \$0.75 if completing the round in exactly 34 time steps, \$0.35 if completing the round in 35 to 36 time steps, \$0.15 if completing the round in 37 to 38 time steps, or no bonus otherwise. In phase I, the total pay ranges from \$0.60 to \$3.30, with a mean of \$1.63, a median of \$1.55, and a standard deviation of \$0.60. The sum of the total pay is \$279.55 (172 participants). In phase II, the total pay ranges from \$0.60 to \$4.50, with a mean of \$1.81, a median of \$1.75, and a standard deviation of \$0.68. The sum of the total pay is \$1,829.25 (1,011 participants).

B.5. Hypothetical Disruption

In the postgame survey of both phases of the normal configuration, participants were asked to imagine a hypothetical understaffed scenario where the chef was no longer available in the kitchen and select the best tip that they believed would most help improve performance in such a disruption. Note that these participants did not experience a disruption during their gameplay. The list of tips presented to them is the same as the one offered to the participants in the disruption configuration. Consistently in both phases, the tip that received the most votes is "Server shouldn't cook." Again, this is likely because of the fact that after three rounds of managing the virtual kitchen under the fully staffed scenario, the participants potentially learned the optimal policy that the server should not be assigned to cook any burger. Without the actual experience of managing the disruption, they appeared to be biased toward their strategy learned in the fully staffed scenario, which felt more intuitive to them. This observation highlights one of the key insights of our study that humans' intuition could be far away from the optimal policy, making them less likely to comply with the counterintuitive tip inferred from our algorithm.

Endnotes

[1] The full preregistration document for our study is available at https://aspredicted.org/5kfk-ts2x.pdf.

[2] Results remain highly statistically significant under a Bonferroni correction for multiple-hypothesis testing.

[3] Note that human participants have a slightly different tip search space than our algorithm. However, this discrepancy cannot be the source of the performance difference because in the disrupted configuration, both our algorithm's tip and the human tip are present in both search spaces; participants then chose an incorrect tip.

[4] Whereas our experiments did not reveal the source of the tip, one may be concerned that participants may be able to infer this information in real-world contexts, potentially affecting compliance. To this end, we ran a small pilot study to explore the impact of informing participants of the source of the tip—we found no statistically or economically significant differences in compliance rates as a function of providing source information (see details in Online Appendix C.6).

[5] For the human tip in the normal configuration ("strategically leave some worker(s) idle"), we measured compliance by identifying if the participant ever skipped a potential task assignment when at least one virtual kitchen worker was idle and there was at least one available subtask. Note that we cannot be certain if such "skipping" was strategic, but, given that participants were financially incentivized to complete each round as fast as possible, we expect that participants would not skip an assignment unless they were being strategic.

[6] In the easier normal configuration, participants in all conditions cross-comply with all other tips (which are all part of the optimal policy), but they achieve higher cross-compliance when shown our algorithm's tip; see Online Appendix C.3.

[7] Note that participants in the control and human conditions comply with the human tip at similar rates; that is, the human tip suggests behaviors that are highly likely to be adopted even in the absence of tips.

[8] This follow-up study is preregistered at https://aspredicted.org/d5x3-gr7x.pdf.

[9] A qualitative understanding of the survey responses suggest that providing an intermediate step between human intuition and the optimal action may have confused users and slowed down participants' ability to adapt to the new environment.

[10] We experimented with *combinations* of tips in exploratory pilots and found that AMT workers were unable to operationalize and comply with such complex tips even though they might be part of an optimal strategy.

[11] They are 35 years old on average, 57% are female, and 68% have at least a two-year degree.

[12] They are 36 years old on average, 62% are female, and 78% have at least a two-year degree.

References

Akşin Z, Deo S, Jónasson JO, Ramdas K (2021) Learning from many: Partner exposure and team familiarity in fluid teams. *Management Sci.* 67(2):854–874.

Allon G, Cohen MC, Moon K, Sinchaisri WP (2023) Managing multihoming workers in the gig economy. Preprint, submitted July 16, http://dx.doi.org/10.2139/ssrn.4502968.

Argote L (2012) *Organizational Learning: Creating, Retaining and Transferring Knowledge* (Springer Science & Business Media, New York).

Bastani O, Pu Y, Solar-Lezama A (2018) Verifiable reinforcement learning via policy extraction. *NIPS'18 Proc. 32nd Internat. Conf. Adv. Neural Inform. Processing Systems* (Curran Associates, Inc., Red Hook, NY), 2499–2509.

Bavafa H, Jónasson JO (2021) Recovering from critical incidents: Evidence from paramedic performance. *Manufacturing Service Oper. Management* 23(4):914–932.

Bertsimas D, Dunn J (2017) Optimal classification trees. *Machine Learn.* 106(7):1039–1082.

Brattland H, Høiseth JR, Burkeland O, Inderhaug TS, Binder PE, Iversen VC (2018) Learning from clients: A qualitative investigation of psychotherapists' reactions to negative verbal feedback. *Psychotherapy Res.* 28(4):545–559.

Breiman L (2001) Random forests. *Machine Learn.* 45(1):5–32.

Breiman L, Friedman J, Stone CJ, Olshen RA (1984) *Classification and Regression Trees* (CRC Press, Boca Raton, FL).

Buciluǎ C Caruana R Niculescu-Mizil A(2006 Model compression. *KDD'06 Proc. 12th ACM SIGKDD Internat. Conf. Knowledge Discovery Data Mining* (Association for Computing Machinery, New York), 535–541.

Chan TY, Li J, Pierce L (2014) Learning from peers: Knowledge transfer and sales force productivity growth. *Marketing Sci.* 33(4):463–484.

Chandrasekaran A, Prabhu V, Yadav D, Chattopadhyay P, Parikh D (2018) Do explanations make VQA models more predictable to a human? Preprint, submitted October 29, https://arxiv.org/abs/1810.12366.

Chandrasekaran A, Yadav D, Chattopadhyay P, Prabhu V, Parikh D (2017) It takes two to tango: Towards theory of AI's mind. Preprint, submitted April 3, https://arxiv.org/abs/1704.00717.

Chui M, Manyika J, Bughin J (2012) The social economy: Unlocking value and productivity through social technologies. Technical report, McKinsey Global Institute, New York.

Dietvorst BJ, Simmons JP, Massey C (2015) Algorithm aversion: People erroneously avoid algorithms after seeing them err. *J. Experiment. Psych. General* 144(1):114–126.

Dietvorst BJ, Simmons JP, Massey C (2018) Overcoming algorithm aversion: People will use imperfect algorithms if they can (even slightly) modify them. *Management Sci.* 64(3):1155–1170.

Dorn B, Guzdial M (2010) Learning on the job: Characterizing the programming knowledge and learning strategies of web designers. *CHI'10 Proc. SIGCHI Conf. Human Factors Comput. Systems* (Association for Computing Machinery, New York), 703–712.

Doshi-Velez F, Kim B (2017) Towards a rigorous science of interpretable machine learning. Preprint, submitted February 28, https://arxiv.org/abs/1702.08608.

Eastwood J, Snook B, Luther K (2012) What people want from their professionals: Attitudes toward decision-making strategies. *J. Behav. Decision Making* 25(5):458–468.

Fudenberg D, Liang A (2019) Predicting and understanding initial play. *Amer. Econom. Rev.* 109(12):4112–41.

Fudenberg D, Kleinberg J, Liang A, Mullainathan S (2022) Measuring the completeness of economic models. *J. Political Econom.* 130(4):956–990.

Fügener A, Grahl J, Gupta A, Ketter W (2022) Cognitive challenges in human–Artificial intelligence collaboration: Investigating the path toward productive delegation. *Inform. Systems Res.* 33(2): 678–696.

Gleicher M (2016) A framework for considering comprehensibility in modeling. *Big Data* 4(2):75–88.

Green B, Chen Y (2019) The principles and limits of algorithm-in-the-loop decision making. *Proc. ACM Human-Comput. Interaction*, vol. 3 (Association for Computing Machinery, New York), 1–24.

Gurvich I, O'Leary KJ, Wang L, Van Mieghem JA (2020) Collaboration, interruptions, and changeover times: Workflow model and empirical study of hospitalist charting. *Manufacturing Service Oper. Management* 22(4):754–774.

Herkenhoff K, Lise J, Menzio G, Phillips G (2018) Knowledge diffusion in the workplace. Technical report, University of Minnesota, Minneapolis.

Hinton G, Vinyals O, Dean J (2015) Distilling the knowledge in a neural network. Preprint, submitted March 9, https://arxiv.org/abs/1503.02531.

Huckman RS, Pisano GP (2006) The firm specificity of individual performance: Evidence from cardiac surgery. *Management Sci.* 52(4):473–488.

Ibanez MR, Clark JR, Huckman RS, Staats BR (2018) Discretionary task ordering: Queue management in radiological services. *Management Sci.* 64(9):4389–4407.

Jarosch G, Oberfield E, Rossi-Hansberg E (2021) Learning from coworkers. *Econometrica* 89(2):647–676.

Kagan E, Leider S, Sahin O (2021) Dynamic decision-making in operations management. Johns Hopkins Carey Business School Research Paper No. 21-13, Johns Hopkins Carey Business School, Baltimore.

Kc DS, Staats BR (2012) Accumulating a portfolio of experience: The effect of focal and related experience on surgeon performance. *Manufacturing Service Oper. Management* 14(4):618–633.

Kim SH, Tong J, Peden C (2020) Admission control biases in hospital unit capacity management: How occupancy information hurdles and decision noise impact utilization. *Management Sci.* 66(11):5151–5170.

Kleinberg J, Ludwig J, Mullainathan S, Obermeyer Z (2015) Prediction policy problems. *Amer. Econom. Rev.* 105(5):491–95.

Kneusel RT, Mozer MC (2017) Improving human-machine cooperative visual search with soft highlighting. *ACM Trans. Appl. Perception (TAP)* 15(1):1–21.

Lage I, Ross AS, Kim B, Gershman SJ, Doshi-Velez F (2018) Human-in-the-loop interpretability prior. *NIPS'18 Proc. 32nd Internat. Conf. Adv. Neural Inform. Processing Systems* (Curran Associates, Inc., Red Hook, NY), 10180–10189.

Lai V, Tan C (2019) On human predictions with explanations and predictions of machine learning models: A case study on deception detection. *FAT'19 Proc. Conf. Fairness Accountability Transparency* (Association for Computing Machinery, New York), 29–38.

Letham B, Rudin C, McCormick TH, Madigan D (2015) Interpretable classifiers using rules and Bayesian analysis: Building a better stroke prediction model. *Ann. Appl. Statist.* 9(3):1350–1371.

Logg JM, Minson JA, Moore DA (2019) Algorithm appreciation: People prefer algorithmic to human judgment. *Organ. Behav. Human Decision Processes* 151:90–103.

Lu J, Lee D, Kim TW, Danks D (2019) Good explanation for algorithmic transparency. Preprint, submitted November 11, http://dx.doi.org/10.2139/ssrn.3503603.

Marshall A (2020) Uber changes its rules, and drivers adjust their strategies. *Wired* (February 18), https://www.wired.com/story/uber-changes-rules-drivers-adjust-strategies/.

McIlroy-Young R, Sen S, Kleinberg J, Anderson A (2020) Aligning superhuman AI with human behavior: Chess as a model system. *KDD'20 Proc. 26th ACM SIGKDD Internat. Conf. Knowledge Discovery Data Mining* (Association for Computing Machinery, New York), 1677–1687.

Meyer G, Adomavicius G, Johnson PE, Elidrisi M, Rush WA, Sperl-Hillen JM, O'Connor PJ (2014) A machine learning approach to improving dynamic decision making. *Inform. Systems Res.* 25(2):239–263.

Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, et al. (2015) Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533.

Nonaka I, Takeuchi H (1995) *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation* (Oxford University Press, Oxford, UK).

Pfeffer J, Sutton RI (2000) *The Knowing-Doing Gap: How Smart Companies Turn Knowledge into Action* (Harvard Business School Press, Boston).

Puiutta E, Veith EM (2020) Explainable reinforcement learning: A survey. Holzinger A, Kieseberg P, Tjoa A, Weippl E, eds. *Machine Learning Knowledge Extraction. CD-MAKE 2020*, Lecture Notes in Computer Science, vol. 12279 (Springer, Cham, Switzerland), 77–95.

Ramdas K, Saleh K, Stern S, Liu H (2017) Variety and experience: Learning and forgetting in the use of surgical devices. *Management Sci.* 64(6):2590–2608.

Ribeiro MT, Singh S, Guestrin C (2016) "Why should I trust you?": Explaining the predictions of any classifier. *KDD'16 Proc. 22nd ACM SIGKDD Internat. Conf. Knowledge Discovery Data Mining* (Association for Computing Machinery, New York), 1135–1144.

Ross S, Gordon G, Bagnell D (2011) A reduction of imitation learning and structured prediction to no-regret online learning. *Proc. 14th Internat. Conf. Artificial intelligence Statistics* (JMLR), 627–635.

Rudin C (2019) Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1(5):206–215.

Silver D, Huang A, Maddison CJ, Guez A, Sifre L, Van Den Driessche G, Schrittwieser J, et al. (2016) Mastering the game of go with deep neural networks and tree search. *Nature* 529(7587): 484–489.

Song H, Tucker AL, Murrell KL, Vinson DR (2017) Closing the productivity gap: Improving worker productivity through public relative performance feedback and validation of best practices. *Management Sci.* 64(6):2628–2649.

Spear SJ (2005) Fixing health care from the inside, today. *Harvard Bus. Rev.* 83(9):78–91.

Stites MC, Nyre-Yu M, Moss B, Smutz C, Smith MR (2021) Sage advice? The impacts of explanations for machine learning models on human decision-making in spam detection. Degen H, Ntoa S, eds. *Artificial Intelligence HCI. HCII 2021*, Lecture Notes in Computer Science, vol. 12797 (Springer, Cham, Switzerland), 269–284.

Sull DN, Eisenhardt KM (2015) *Simple Rules: How to Thrive in a Complex World* (Houghton Mifflin Harcourt, Boston).

Sun J, Zhang DJ, Hu H, Van Mieghem JA (2022) Predicting human discretion to adjust algorithmic prescription: A large-scale field experiment in warehouse operations. *Management Sci.* 68(2): 846–865.

Sutton RS, Barto AG (2018) *Reinforcement Learning: An Introduction* (MIT Press, Cambridge, MA).

Sutton RS, McAllester DA, Singh SP, Mansour Y (2000) Policy gradient methods for reinforcement learning with function approximation. *NIPS'99 Proc. 13th Internat. Conf. Adv. Neural Inform. Processing Systems* (MIT Press, Cambridge, MA), 1057–1063.

Szulanski G (1996) Exploring internal stickiness: Impediments to the transfer of best practice within the firm. *Strategic Management J.* 17(S2):27–43.

Tan TF, Netessine S (2019) When you work with a superman, will you also fly? An empirical study of the impact of coworkers on performance. *Management Sci.* 65(8):3495–3517.

Tucker AL, Edmondson AC, Spear S (2002) When problem solving prevents organizational learning. *J. Organ. Change Management* 15(2):122–137.

Verma A, Murali V, Singh R, Kohli P, Chaudhuri S (2018) Programmatically interpretable reinforcement learning. Internat. Conf. Machine Learn. (PMLR), 5045–5054.

Wang F, Rudin C (2015) Falling rule lists. Artificial Intelligence Statist. (PMLR), 1013–1022.

Watkins CJ, Dayan P (1992) Q-learning. *Machine Learn.* 8(3–4):279–292.

Copyright of Management Science is the property of INFORMS: Institute for Operations Research & the Management Sciences and its content may not be copied or emailed to multiple sites without the copyright holder's express written permission. Additionally, content may not be used with any artificial intelligence tools or machine learning technologies. However, users may print, download, or email articles for individual use.

4. 护士配置：Nurse Staffing Under Absenteeism: A Distributionally Robust Optimization Approach (MSOM, 2025)

来源：Manufacturing & Service Operations Management. Vol. 27, No. 2, 2025, pp. 624–639.

https://doi.org/10.1287/msom.2023.0398

作者：Minseok Ryu, Ruiwei Jiang

关键词：护士配置；内生不确定性；分布鲁棒优化；浮动护士池

推荐人：仝宇

推荐理由：本文同时考虑随机护理需求与护士缺勤：前者由患者数量波动引起，后者还会受到配置人数影响。作者建立分布鲁棒护士配置模型，利用可跨病区工作的浮动护士池缓解人员短缺，并为一般护士池结构设计分离算法。对于常见结构，模型可转化为混合整数线性规划。此外，论文在满足目标配置成本的条件下优化护士池设计，以减少交叉培训需求。合作医院数据的案例分析提示，应将缺勤概率较高的病区安排在同一护士池中。

MANUFACTURING & SERVICE OPERATIONS MANAGEMENT
Vol. 27, No. 2, March–April 2025, pp. 624–639
ISSN 1523-4614 (print), ISSN 1526-5498 (online)
https://pubsonline.informs.org/journal/msom

Nurse Staffing Under Absenteeism: A Distributionally Robust Optimization Approach

Minseok Ryu,[a] **Ruiwei Jiang**[b,*]

[a] School of Computing and Augmented Intelligence, Arizona State University, Tempe, Arizona 85281; [b] Department of Industrial and Operations Engineering, University of Michigan, Ann Arbor, Michigan 48109

*Corresponding author

Contact: mryu2@asu.edu, https://orcid.org/0000-0002-1954-7722 (MR); ruiwei@umich.edu, https://orcid.org/0000-0002-3941-5057 (RJ)

Received: July 17, 2023
Revised: February 19, 2024; October 15, 2024
Accepted: December 1, 2024
Published Online in Articles in Advance: January 9, 2025

https://doi.org/10.1287/msom.2023.0398

Copyright: © 2025 INFORMS

Abstract. ***Problem definition*:** We study a nurse staffing problem under random nurse demand and absenteeism. Although the demand uncertainty is exogenous, the absenteeism uncertainty is *decision-dependent*, that is, the number of nurses who show up for work partially depends on the nurse staffing level. For quality of care, hospitals develop float pools of hospital units and train nurses to be able to work in multiple units (termed cross-training) in response to potential nurse shortages. ***Methodology/results*:** We study a distributionally robust nurse staffing (DRNS) model that considers both exogenous and *decision-dependent* uncertainties. We derive a separation algorithm to solve this model under a general structure of float pools. In addition, we identify several pool structures that often arise in practice and recast the corresponding DRNS model as a mixed-integer linear program, which facilitates off-the-shelf commercial solvers. ***Managerial implications*:** Through the numerical case studies, based on the data of a collaborating hospital, we found that modeling decision-dependent absenteeism improves the out-of-sample performance of staffing decisions, and such improvement is positively correlated with the value of flexibility arising from fully utilizing float pools.

Funding: R. Jiang was supported in part by the National Science Foundation [Grant ECCS-1845980].
Supplemental Material: The online appendix is available at https://doi.org/10.1287/msom.2023.0398.

Keywords: nurse staffing • decision-dependent uncertainty • distributionally robust optimization • strong valid inequalities • convex hull

1. Introduction

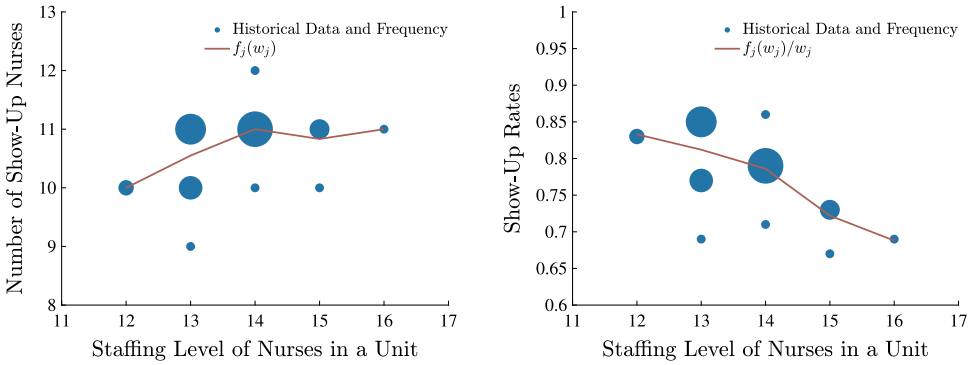
Nurse staffing plays a key role in hospital management. The cost of staffing nurses accounts for over 30% of the overall hospital annual expenditures (see, e.g., Welton 2011). Besides, the nurse staffing level can have significant impact on patient safety, quality of care, and the job satisfaction of nurses (see, e.g., Tevington 2011). In view of that, a number of governing agencies (e.g., California Department of Health 2004 and Victoria Department of Health 2015) have set up minimum nurse-to-patient ratios (NPRs) for various types of hospital units to regulate the staffing decision.

Nurse staffing takes place weeks to months ahead of a specific shift and determines the staffing levels of multiple hospital units based on, for example, the patient census forecast and NPRs. If a unit is found short of nurses because of absence (e.g., family and sick leaves) before the shift, additional workforce such as temporary or off-duty nurses can be recruited. We note that, although nurse staffing does not generate shift schedules and assignments for individual nurses, the output of this study can be used when making those decisions (see Hulshof et al. 2012, Bam et al. 2020).

Nurse staffing is challenging because of the uncertainties of nurse demand and absenteeism. The demand uncertainty has been well documented (Gaynor and Anderson 1995, Boutsioli 2010) and studied in the nurse staffing literature (see, e.g., Davis et al. 2014, Kim and Mehrotra 2015). In contrast, the absenteeism uncertainty has received relatively less attention (see Green et al. 2013, Maass et al. 2017), albeit commonly observed in practice. According to the U.S. Bureau of Labor Statistics (2016), the average absence rate among all nurses in the Veterans Affairs Healthcare System is 6.4%, significantly higher than that among all occupations (2.9%). In response, hospitals develop float pools consisting of multiple hospital units, and a nurse assigned to a pool is cross-trained to be able to work in those units. Hence, a pool nurse can also reinforce when a unit is short of nurses (Inman et al. 2005).

Unlike demand, the nurse absenteeism uncertainty is *decision-dependent*. Specifically, if a unit is staffed with $w \in \mathbb{N}_+$ nurses, then the nurse absenteeism rate depends on w (see Figure 1). Although failing to incorporate such endogeneity may result in understaffing (see Green et al. 2013), unfortunately, modeling endogeneity usually

Figure 1. (Color online) Number of Show-Up Nurses and Show-Up Rates, as Functions of Staffing Levels, Based on the Data from a Collaborating Hospital



Note. Dots represent historical data samples, and the size of dots represents frequency.

makes optimization models computationally prohibitive (see Dupačová 2006). Because of this technical difficulty, existing works often resort to exogenous uncertainty for an approximation or employ certain parametric probability distributions, for example, the absence of each nurse follows an independent Bernoulli distribution with the same probability (Green et al. 2013). A basic challenge to parametric models is that our knowledge of the decision-dependent probability distribution is usually *ambiguous*. Under most circumstances, we only have historical data, for example, past nurse staffing levels and absence records, which can be considered as samples taken from the ambiguous distribution. As a result, the solution obtained from a parametric model can yield unpleasant out-of-sample performance if the chosen model is biased.

In this paper, we propose a *nonparametric* model for both exogenous and decision-dependent uncertainties based on distributionally robust optimization (DRO). Our approach considers an ambiguity set of probability distributions based only on the support and moment information of these uncertainties. To the best of our knowledge, this is the first study of the decision-dependent uncertainty in nurse staffing by using a DRO approach.

1.1. Literature Review

A majority of the nurse staffing literature focuses on deterministic models (see Van den Bergh et al. 2013). For example, Kao and Tung (1981) assessed the need for hiring permanent staffs and temporary helpers and Trivedi (1981) analyzed the trade-offs among hiring full-time, part-time, and overtime nurses. More recently, Yang et al. (2007) compared cross-training and flexible work days and demonstrated that cross-training is far more effective for performance improvement than flexible work days. Similarly, Bam et al. (2020) identified cross-training as a promising extension from their deterministic model. Despite the potential benefit of flexibility brought by float pools and cross-training, Maenhout and Vanhoucke (2013) pointed out that the pool design and staffing are often done manually in a qualitative fashion (also see Stewart et al. 1994). In addition, when the nurse demand and/or absenteeism is random, the deterministic models may underestimate the total staffing cost (see, e.g., Kao and Queyranne 1985).

Existing stochastic nurse staffing models often consider the demand uncertainty only. For example, Campbell (2011) studied a two-stage stochastic programming (SP) model that integrates the staffing and scheduling of cross-trained workers under demand uncertainty and demonstrated the value of cross-training. Likewise, using a two-stage SP model under demand uncertainty, Lu and Lu (2016) studied how mandatory overtime laws can negatively affect the service quality of a nursing home. Different from these works, Green et al. (2013) considered random nurse absenteeism. Based on the data from various hospitals, Green et al. (2013) found that the absence rate depends on the staffing level (also see Figure 1) and ignoring such dependency results in understaffing. They successfully derived a closed-form optimal staffing level for a single unit. In addition, Slaugh et al. (2018) derived an optimal staffing level for a single on-call pool. The problem becomes computationally prohibitive when there are multiple units and/or float pools. For example, Easton (2014) studied a multiunit and one-pool setting and pointed out that the proposed SP model does not scale well, even when staffing levels are fixed. Wang and Gupta (2014) considered a multiunit, no-pool setting and found it computationally demanding to search for an optimal staffing strategy. Consequently, both Easton (2014) and Wang and Gupta (2014) resort to easy-to-use heuristics. SP models may also contrast in the modeling of nurse absenteeism. For example, Green et al. (2013) and Easton (2014) assumed that all nurses are absent independently and with a homogeneous rate, whereas Wang and Gupta (2014) suggested that nurses have heterogeneous absence rates. In addition, the nurse absence

may be correlated because of hidden confounders (e.g., weather and holidays). Consequently, a biased model of the nurse absenteeism may yield suboptimal staffing decisions.

In this paper, we take into account both nurse demand and absenteeism uncertainty in a multiunit and multipool setting. Our DRO model allows heterogeneous absence rates and arbitrary correlations among nurses. Moreover, our model can be solved to global optimum by a separation algorithm and, in several important special cases, by solving a single mixed-integer linear program (MILP).

DRO models have received increasing attention in the recent literature. In particular, DRO has been applied to two-stage stochastic optimization (see, e.g., Bertsimas et al. 2010, 2018; Kong et al. 2013; Hanasusanto and Kuhn 2018), as in this paper. Nevertheless, two-stage DRO is in general computationally prohibitive and proved to be NP-hard by Bertsimas et al. (2010). As compared with this stream of literature, we exploit the nurse staffing problem structure and derive tractable reformulations under practical float pool structures. DRO has also been applied to model decision-dependent uncertainty. Luo and Mehrotra (2020) derived equivalent reformulations of a decision-dependent DRO model under various ambiguity sets. Decision-dependent DRO models have been applied by Noyan et al. (2022) on machine scheduling using an earth mover ambiguity set, by Doan (2022) on retrofitting planning using a Fréchet ambiguity set, and by Basciftci et al. (2021) on facility location using a moment ambiguity set. As compared with the most relevant setting by Basciftci et al. (2021), our staffing decisions impact not only the moments of the random variables but also their support, whose size grows exponentially as the staffing level increases, in contrast to their constant-sized support. We summarize our main contributions as follows:

1. We propose a first decision-dependent DRO approach for nurse staffing, considering both exogenous nurse demand and decision-dependent nurse absenteeism. Our model considers multiple units, multiple float pools, and both long-term and preshift nurse staffing. For general pool structures, we derive a separation algorithm that solves this model to global optimum.

2. For various pool structures arising in practice, we provide a MILP reformulation of the distributionally robust nurse staffing (DRNS) model by deriving an LP representation for a convex maximization problem. In these cases, the computational burden of DRNS is the same as that of the deterministic nurse staffing.

3. We conduct extensive case studies using data from our collaborating hospital. The results demonstrate the value of modeling nurse absenteeism and the computational efficacy of DRNS.

The remainder of the paper is organized as follows. Section 2 describes the DRNS model, Section 3 derives a general solution approach, and Section 4 derives tractable reformulations under special pool structures. We conduct case studies in Section 5, and conclude in Section 6. To ease the exposition, we relegate all proofs to the online appendices.

1.2. Notation

We use $\tilde{\cdot}$ to indicate random variables and $\wedge$ to indicate realizations of the random variables. For example, $\tilde{d}$ represents a random variable and $\hat{d}^1, \ldots, \hat{d}^N$ represent N realizations of $\tilde{d}$. For $a, b \in \mathbb{Z}$, we define $[a] := \{1, 2, \ldots, a\}$ and $[a,b]_{\mathbb{Z}} := \{n \in \mathbb{Z} : a \le n \le b\}$. For $x \in \mathbb{R}$, we define $[x]_+ := \max\{x, 0\}$. For set S, we define its indicator function $\mathbb{1}_S$ such that $\mathbb{1}_S(s) := 1$ if $s \in S$ and $\mathbb{1}_S(s) := 0$ if $s \notin S$, and denote its convex hull by $\mathrm{conv}(S)$. We define $\mathbb{B} := \{0, 1\}$.

2. Distributionally Robust Nurse Staffing

For a specific shift, we consider staffing a group of J hospital units, each facing a random demand of nurses denoted by $\tilde{d}_j$ for all $j \in [J]$. To enhance flexibility, we form I nurse float pools. Each pool $i \in [I]$ is associated with a set P_i of units and each nurse assigned to this pool is capable of working in any unit $j \in P_i$. Because of absenteeism, if we staff unit j with w_j nurses (termed unit nurses), then there will be a random number $\tilde{w}_j$ of nurses showing up for work, where $\tilde{w}_j \in [0, w_j]_{\mathbb{Z}}$. Likewise, $\tilde{y}_i$ nurses show up if we staff pool i with y_i nurses during the shift, where $\tilde{y}_i \in [0, y_i]_{\mathbb{Z}}$. After $\tilde{d}_j$, $\tilde{w}_j$, and $\tilde{y}_i$ are realized, the nurses showing up in pool i can be reassigned to any units in P_i to make up the nurse shortage, if any. At that point, any remaining shortage will be covered by hiring temporary nurses in order to meet the NPR requirement. Formally, for given $\tilde{w} := [\tilde{w}_1, \ldots, \tilde{w}_J]^\top$, $\tilde{y} := [\tilde{y}_1, \ldots, \tilde{y}_I]^\top$, and $\tilde{d} := [\tilde{d}_1, \ldots, \tilde{d}_J]^\top$, the cost of hiring temporary nurses can be obtained from an integer program

$$V(\tilde{w}, \tilde{y}, \tilde{d}) := \min_{z \in \mathbb{Z}_+^{I \times |P_i|}, x \in \mathbb{Z}_+^J, e \in \mathbb{Z}_+^J} \quad \sum_{j=1}^{J} c_j^x x_j \tag{1a}$$

$$\text{s.t.} \quad \sum_{i \in [I]: j \in P_i} z_{ij} + x_j - e_j = \tilde{d}_j - \tilde{w}_j, \quad \forall j \in [J], \tag{1b}$$

$$\sum_{j \in P_i} z_{ij} \le \tilde{y}_i, \ \forall i \in [I], \tag{1c}$$

where variables z_{ij} represent the number of nurses reassigned from pool i to unit j, variables x_j represent the number of temporary nurses[1] hired, variables e_j represent the excessive number of nurses, and parameters c_j^x represent the unit cost of hiring temporary nurses. In the above formulation, constraints (1b) describe three ways of satisfying the demand in unit j, (i) assigning unit nurses, (ii) reassigning pool nurses, and (iii) hiring temporary nurses, and constraints (1c) ensure that the

number of nurses reassigned from pool i does not exceed the number of nurses showing up in that pool.

As mentioned in Section 1, the probability distribution $\mathbb{P}_{\tilde{w},\tilde{y},\tilde{d}}$ of $(\tilde{w},\tilde{y},\tilde{d})$ is ambiguous and depends on the staffing levels (w, y) of units and pools. When historical data of $(\tilde{w},\tilde{y},\tilde{d})$ are available, a natural and interpretable way of modeling such dependence is through its support and moments (e.g., mean values), as a function of (w, y). This motivates us to consider the following decision-dependent ambiguity set $\mathcal{D}(w,y)$:

$$\mathcal{D}(w,y)=\{\mathbb{P}\in\mathcal{P}(\Xi):\mathbb{E}_{\mathbb{P}}[\tilde{d}_j^q]=\mu_{jq},\ \forall j\in[J],\ \forall q\in[Q], \quad (2a)$$

$$\mathbb{E}_{\mathbb{P}}[\tilde{w}_j]=f_j(w_j),\ \forall j\in[J],$$
$$\mathbb{E}_{\mathbb{P}}[\tilde{y}_i]=g_i(y_i),\ \forall i\in[I]\}, \quad (2b)$$

where Ξ represents the support of $(\tilde{w},\tilde{y},\tilde{d})$ and $\mathcal{P}(\Xi)$ represents the set of probability distributions supported on Ξ. We consider a box support $\Xi:=\Xi_{\tilde{w}}\times\Xi_{\tilde{y}}\times\Xi_{\tilde{d}}$, where $\Xi_{\tilde{w}}=\Pi_{j=1}^{J}[0,w_j]_{\mathbb{Z}}$, $\Xi_{\tilde{y}}=\Pi_{i=1}^{I}[0,y_i]_{\mathbb{Z}}$, $\Xi_{\tilde{d}}=\Pi_{j=1}^{J}[d_j^{\mathrm{L}},d_j^{\mathrm{U}}]_{\mathbb{Z}}$, and d_j^{L} and d_j^{U} represent lower and upper bounds of the nurse demand in unit j. In addition, μ_{jq} represents the q-th moment of $\tilde{d}_j$ for all $q\in[Q]$. Furthermore, $f_j:\mathbb{Z}_+\to\mathbb{R}_+$ and $g_i:\mathbb{Z}_+\to\mathbb{R}_+$ are two piecewise linear functions that model the dependence of the mean values $(\mathbb{E}_{\mathbb{P}}[\tilde{w}],\mathbb{E}_{\mathbb{P}}[\tilde{y}])$ on the staffing decisions (w, y).

The ambiguity set $\mathcal{D}(w,y)$ can be calibrated conveniently. Suppose that $\mathbb{P}_{\tilde{w},\tilde{y},\tilde{d}}$ is observed through nurse demand data $\{\hat{d}_j^1,\dots,\hat{d}_j^N\}_{j=1}^{J}$ and attendance records $\{(w_j^1,\hat{w}_j^1),\dots,(w_j^N,\hat{w}_j^N)\}_{j=1}^{J}$ and $\{(y_i^1,\hat{y}_j^1),\dots,(y_i^N,\hat{y}_i^N)\}_{i=1}^{I}$ during the past N days, where, in each pair $(w_j^n,\hat{w}_j^n)$, w_j^n represents the staffing level of unit j in day n and $\hat{w}_j^n$ represents the corresponding number of nurses who actually showed up. Then, μ_{jq} can be obtained from empirical estimates (e.g., $\mu_{j1}=(1/N)\sum_{n=1}^{N}\hat{d}_j^n$, $\mu_{j2}=(1/N)\sum_{n=1}^{N}(\hat{d}_j^n)^2$, and so on). Furthermore, f_j and g_i can be obtained by performing segmented linear regression on the attendance data, using the staffing levels $\{w_j^1,\dots,w_j^N\}$ and $\{y_i^1,\dots,y_i^N\}$ as breakpoints, respectively (see the red solid curves in Figure 1 (left) for an example).

We seek nurse staffing levels that minimize the expected total cost with regard to the worst-case probability distribution in $\mathcal{D}(w,y)$, that is, we consider the following two-stage DRO model:

$$(\mathbf{DRNS}):\ \min_{w,y}\sum_{j=1}^{J}c_j^{\mathrm{w}}w_j+\sum_{i=1}^{I}c_i^{\mathrm{y}}y_i+\max_{\mathbb{P}\in\mathcal{D}(w,y)}\mathbb{E}_{\mathbb{P}}[V(\tilde{w},\tilde{y},\tilde{d})] \quad (3a)$$

$$\text{s.t. } w\in[w^{\mathrm{L}},w^{\mathrm{U}}]_{\mathbb{Z}},\ y\in[y^{\mathrm{L}},y^{\mathrm{U}}]_{\mathbb{Z}}, \quad (3b)$$

$$w,y\in R\cap\mathbb{Z}_+^{I+J}, \quad (3c)$$

where parameters c^{w} and c^{y} represent the unit cost of hiring unit and pool nurses, respectively; constraints (3b) designate lower and upper bounds on staffing levels, and set R represents all remaining restrictions on (w, y), which we assume can be represented via mixed-integer linear inequalities. We remark that (DRNS) decides the staffing levels during a specific shift and does not decide the schedule for individual nurses across multiple shifts. Hence, multistage models do not apply to our problem. Nevertheless, (DRNS) is computationally challenging because (i) $\mathcal{D}(w,y)$ involves exponentially many decision-dependent probability distributions and (ii) it is a two-stage DRO model with integer recourse variables. In the next two sections, we shall derive equivalent reformulations of (DRNS) that facilitate a separation algorithm, and identify practical pool structures that admit more tractable solution approaches.

The ambiguity set $\mathcal{D}(w,y)$ can be extended by incorporating other moment information of $\mathbb{P}_{\tilde{w},\tilde{y},\tilde{d}}$, for example, the correlations of the random demands (see Delage and Ye 2010). But unfortunately, in our case this produces an intractable formulation (see theorem 3.1 in Bertsimas et al. 2010). Nevertheless, in Section 5.5 we shall incorporate higher marginal moments (e.g., variance, skewness, and kurtosis) of $\tilde{w}$ and $\tilde{y}$ into $\mathcal{D}(w,y)$ and derive a tractable and conservative approximation for the ensuing model.

3. Solution Approach: General Pool Structure

In this section, we consider general pool structures, recast (DRNS) as a min-max formulation, and derive a separation algorithm that solves this model to global optimality.

We start by noticing that the integrality restrictions in the second-stage formulation of (DRNS) can be relaxed without loss of generality.

Lemma 1. *For any given $(\tilde{w},\tilde{y},\tilde{d})\in\Xi$, the value of $V(\tilde{w},\tilde{y},\tilde{d})$ remains unchanged if the integrality restriction is relaxed.*

By Lemma 1 (see Online Appendix A for a proof), we rewrite $V(\tilde{w},\tilde{y},\tilde{d})$ as the following dual formulation:

$$V(\tilde{w},\tilde{y},\tilde{d})=\max_{\alpha,\beta}\sum_{j=1}^{J}(\tilde{d}_j-\tilde{w}_j)\alpha_j+\sum_{i=1}^{I}\tilde{y}_i\beta_i \quad (4a)$$

$$\text{s.t. } \beta_i+\alpha_j\le 0,\quad \forall i\in[I],\ \forall j\in P_i, \quad (4b)$$

$$0\le\alpha_j\le c_j^{\mathrm{x}},\quad \forall j\in[J], \quad (4c)$$

where dual variables α_j and β_i are associated with constraints (1b) and (1c), respectively, and constraints (4b) and (4c) are associated with primal variables z_{ij} and (x_j, e_j), respectively. We let Λ denote the dual feasible region, that is, $\Lambda:=\{(\alpha,\beta):(4b)\text{–}(4c)\}$. Strong duality between formulations (1a)–(1c) and (4a)–(4c) holds because (1a)–(1c) has a finite optimal value.

We are now ready to recast (DRNS) as a min-max formulation. To this end, we consider $\mathbb{P}$ as a decision variable and take the dual of the worst-case expectation in (3a). For strong duality, we make the following technical assumption on the ambiguity set $\mathcal{D}(w,y)$.

Assumption 1. *For any staffing levels (w, y) feasible to (DRNS), $\mathcal{D}(w,y)$ is nonempty.*

Assumption 1 is mild. For example, it holds whenever the moments of demands $\{\mu_{jq} : j \in [J], q \in [Q]\}$ are obtained from empirical estimates and the decision-dependent moments $\{g_i(y_i), f_j(w_j) : i \in [I], j \in [J]\}$ lie in the convex hull of their support, that is, $f_j(w_j) \in [0, w_j]$ and $g_i(y_i) \in [0, y_i]$. In Online Appendix B, we present an approach to verify Assumption 1 by solving linear programs. The following proposition presents a min-max reformulation for DRNS (see Online Appendix C for a proof).

Proposition 1. *Under Assumption 1, the (DRNS) model yields the same optimal value and the same set of optimal solutions as the following min-max optimization problem:*

$$\min_{\substack{w,y\\ \gamma,\lambda,\rho}} \max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta) + \sum_{i=1}^{I}(c_i^y y_i + g_i(y_i)\lambda_i) + \sum_{j=1}^{J}\left[c_j^w w_j + \sum_{q=1}^{Q}\mu_{jq}\rho_{jq} + f_j(w_j)\gamma_j\right] \tag{5a}$$

$$\text{s.t. } (3b)-(3c), \tag{5b}$$

where

$$F(\alpha,\beta) := \sum_{j=1}^{J}[(-\alpha_j-\gamma_j)w_j]_+ + \sum_{i=1}^{I}[(\beta_i-\lambda_i)y_i]_+ + \sum_{j=1}^{J}\max_{\tilde{d}_j\in[d_j^L,d_j^U]_{\mathbb{Z}}}\left\{\alpha_j\tilde{d}_j - \sum_{q=1}^{Q}\rho_{jq}\tilde{d}_j^q\right\}. \tag{5c}$$

In the min-max reformulation (5a)–(5b), the additional variables γ, λ, ρ are generated in the process of taking dual. In addition, function $F(\alpha,\beta)$ is jointly convex in (α,β) because, as presented in (5c), $F(\alpha,\beta)$ is the pointwise maximum of functions affine in (α,β). This min-max reformulation is not directly computable because (i) evaluating the objective function (5a) requires us to solve a *convex maximization* problem $\max_{(\alpha,\beta)\in\Lambda}F(\alpha,\beta)$, which is in general NP-hard; and (ii) the formulation includes nonlinear and nonconvex terms $g_i(y_i)\lambda_i$ and $f_j(w_j)\gamma_j$. In what follows, we shall address these two challenges before presenting a separation algorithm for solving (DRNS).

First, we analyze the convex maximization problem and derive optimality conditions in Lemma 2 (see Online Appendix D for a proof). We assume $0 \le c_1^x \le \cdots \le c_J^x$ without loss of generality.

Lemma 2. *There exists an optimal solution $(\overline{\alpha},\overline{\beta})$ to problem $\max_{(\alpha,\beta)\in\Lambda}F(\alpha,\beta)$ such that (a) $\overline{\alpha}_j \in \{0, c_1^x, \ldots, c_j^x\}$ for all $j \in [J]$ and (b) $\overline{\beta}_i = -\max\{\overline{\alpha}_j : j \in P_i\}$ for all $i \in [I]$.*

Remark 1 (Interpretation on $\overline{\alpha}$ and $\overline{\beta}$). $\overline{\alpha}$ are the shadow prices associated with constraints (1b), that is, $\overline{\alpha}_j$ evaluates the increase in staffing cost when the nurse shortage $(\tilde{d}_j - \tilde{w}_j)$ of unit j increases by one. When this happens, a new optimal solution to formulation (1) reassigns a pool nurse from a unit $k \in \{1,\ldots,j\}$ to unit j because of the ascending staffing costs $c_1^x \le \cdots \le c_j^x$. As a result, it is equivalent to transferring the additional shortage from unit j to unit k and accordingly the staffing cost increases by $\overline{\alpha}_j \in \{0, c_1^x, \ldots, c_j^x\}$. In addition, $\overline{\beta}_i$ evaluates the decrease in staffing cost when an additional nurse shows up in pool i. When that happens, a new optimal solution to (1) assigns this nurse to a unit in which the shortage is most costly. As a result, $-\overline{\beta}_i = \max\{\overline{\alpha}_j : j \in P_i\}$.

Lemma 2 enables us to avoid enumerating the infinite number of elements in Λ and focus only on a finite set of $(\overline{\alpha},\overline{\beta})$ values. In addition, we introduce binary variables to encode the special structure identified in the optimality conditions. Specifically, for all $j \in [J]$ and $k \in [j]$, we define binary variable t_{jk} such that $t_{jk} := 1$ if $\overline{\alpha}_j = c_k^x$, and $t_{jk} := 0$ otherwise. For all $i \in [I]$, we denote by $J(i)$ the largest index in P_i, that is, $J(i) = \max\{j : j \in P_i\}$. For all $i \in [I]$ and $k \in [J(i)]$, we define binary variable $s_{ik} := 1$ if $\overline{\beta}_i = -c_k^x$ and $s_{ik} := 0$ otherwise. In other words, $s_{ik} = 1$ if k is the largest index in $[J(i)]$ with $t_{jk} = 1$ for some $j \in P_i$, in line with Condition (b) in Lemma 2. Variables (t, s) need to satisfy the following constraints to make the encoding well defined:

$$\sum_{k=1}^{j} t_{jk} \le 1, \quad \forall j \in [J], \tag{6a}$$

$$\sum_{k=1}^{J(i)} s_{ik} \le 1, \quad \forall i \in [I], \tag{6b}$$

$$s_{ik} \le \sum_{j\in P_i: j\ge k} t_{jk}, \quad \forall i \in [I],\ \forall k \in [J(i)], \tag{6c}$$

$$s_{ik} + t_{j\ell} \le 1, \quad \forall i \in [I],\ \forall k \in [J(i)],\ \forall j \in P_i,\ \forall \ell \in [k+1, j]_{\mathbb{Z}}, \tag{6d}$$

$$t_{jk} \le \sum_{\ell=1}^{J(i)} s_{i\ell}, \quad \forall i \in [I],\ \forall j \in P_i,\ \forall k \in [j], \tag{6e}$$

$$s_{ik} \in \mathbb{B}, \quad \forall i \in [I],\ \forall k \in [J(i)],\ t_{jk} \in \mathbb{B}, \quad \forall j \in [J],\ \forall k \in [j], \tag{6f}$$

where constraints (6a) describe that, for all $j \in [J]$, $t_{jk} = 1$ holds for at most one $k \in [j]$; constraints (6b) describe that $s_{ik} = 1$ holds for at most one $k \in [J(i)]$; constraints (6c) designate that if $s_{ik} = 1$, then there is a $j \in P_i$ with $t_{jk} = 1$; constraints (6d) describe that if $s_{ik} = 1$, then there cannot be any $j \in P_i$ and $\ell \ge k+1$ with $t_{j\ell} = 1$; and

constraints (6e) ensure that all $t_{jk} = 0$ if $s_{i\ell} = 0$ for all $\ell \in [J(i)]$. This recasts $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ as an integer linear program presented in Theorem 1 (see Online Appendix E for a proof). For ease of exposition, we introduce auxiliary variables $r_j := 1 - \sum_{k=1}^{j} t_{jk}$ and $p_i := 1 - \sum_{k=1}^{J(i)} s_{ik}$, whose values depend entirely on variables t and s.

Theorem 1. *The problem* $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ *yields the same optimal value as the following integer linear program:*

$$\max_{t,s,r,p} \sum_{j=1}^{J}\left(c_j^r r_j + \sum_{k=1}^{j} c_{jk}^t t_{jk}\right) + \sum_{i=1}^{I}\left(c_i^p p_i + \sum_{k=1}^{J(i)} c_{ik}^s s_{ik}\right) \tag{7a}$$

$$\text{s.t. } (t,s,r,p) \in \mathcal{H} := \Bigg\{ (6a)-(6f),\ r_j + \sum_{k=1}^{j} t_{jk} = 1, \quad \forall j \in [J],\ \ p_i + \sum_{k=1}^{J(i)} s_{ik} = 1,\ \ \forall i \in [I] \Bigg\}, \tag{7b}$$

where $c_j^r := [-\gamma_j w_j]_+ + \max_{\tilde{d}_j \in [d_j^L, d_j^U]_{\mathbb{Z}}} \{-\sum_{q=1}^{Q} \rho_{jq} \tilde{d}_j^q\}$, $c_{jk}^t := [(-c_k^x - \gamma_j) w_j]_+ + \max_{\tilde{d}_j \in [d_j^L, d_j^U]_{\mathbb{Z}}} \{c_k^x \tilde{d}_j - \sum_{q=1}^{Q} \rho_{jq} \tilde{d}_j^q\}$, $c_i^p := [-\lambda_i y_i]_+$, *and* $c_{ik}^s := [(-c_k^x - \lambda_i) y_i]_+$.

Second, we linearize the terms $f_j(w_j)\gamma_j$ and $g_i(y_i)\lambda_i$. For all $j \in [J]$, although $f_j(w_j)$ can be nonlinear and nonconvex, thanks to the integrality of w_j, we can rewrite $f_j(w_j)$ as an affine function based on a binary expansion of w_j. Specifically, we introduce binary variables $\{u_{j\ell} : \ell \in [w_j^U - w_j^L]\}$ such that $w_j = w_j^L + \sum_{\ell=1}^{w_j^U - w_j^L} u_{j\ell}$, where we interpret $u_{j\ell}$ as whether we assign at least $w_j^L + \ell$ nurses to unit j. That is, $u_{j\ell} = 1$ if $w_j \ge w_j^L + \ell$ and $u_{j\ell} = 0$ otherwise. Then, defining $\Delta_{j\ell} := f_j(w_j^L + \ell) - f_j(w_j^L + \ell - 1)$ for all $\ell \in [w_j^U - w_j^L]$, we have

$$\begin{aligned} f_j(w_j) &= f_j(w_j^L) + \sum_{\ell=1}^{w_j - w_j^L} [f_j(w_j^L + \ell) - f_j(w_j^L + \ell - 1)] \\ &= f_j(w_j^L) + \sum_{\ell=1}^{w_j^U - w_j^L} [f_j(w_j^L + \ell) - f_j(w_j^L + \ell - 1)] \mathbb{1}_{[w_j^L + \ell, w_j^U]}(w_j) \\ &= f_j(w_j^L) + \sum_{\ell=1}^{w_j^U - w_j^L} \Delta_{j\ell} u_{j\ell}. \end{aligned}$$

It follows that $f_j(w_j)\gamma_j = f_j(w_j^L)\gamma_j + \sum_{\ell=1}^{w_j^U - w_j^L} \Delta_{j\ell} u_{j\ell} \gamma_j$. We can linearize the bilinear terms $u_{j\ell}\gamma_j$ by defining continuous variables $\varphi_{j\ell} := u_{j\ell}\gamma_j$ and incorporating the following standard McCormick inequalities (see McCormick 1976):

$$\gamma_j - M(1 - u_{j\ell}) \le \varphi_{j\ell} \le M u_{j\ell}, \ \ \forall j \in [J],\ \forall \ell \in [w_j^U - w_j^L], \tag{8a}$$

$$-M u_{j\ell} \le \varphi_{j\ell} \le \gamma_j + M(1 - u_{j\ell}), \ \ \forall j \in [J],\ \forall \ell \in [w_j^U - w_j^L], \tag{8b}$$

where M represents a sufficiently large positive constant. Likewise, we rewrite $g_i(y_i)\lambda_i$ as $g_i(y_i^L)\lambda_i + \sum_{\ell=1}^{y_i^U - y_i^L} \delta_{i\ell} v_{i\ell} \lambda_i$ by using constants $\delta_{i\ell} := g_i(y_i^L + \ell) - g_i(y_i^L + \ell - 1)$ for all $\ell \in [y_i^U - y_i^L]$ and binary variables $\{v_{i\ell} : \ell \in [y_i^U - y_i^L]\}$, where $v_{i\ell} = 1$ if $y_i \ge y_i^L + \ell$ and $v_{i\ell} = 0$ otherwise. We linearize the bilinear terms $v_{i\ell}\lambda_i$ by continuous variables $\nu_{i\ell} := v_{i\ell}\lambda_i$ and the McCormick inequalities

$$\lambda_i - M(1 - v_{i\ell}) \le \nu_{i\ell} \le M v_{i\ell}, \ \forall i \in [I],\ \forall \ell \in [y_i^U - y_i^L], \tag{8c}$$

$$-M v_{i\ell} \le \nu_{i\ell} \le \lambda_i + M(1 - v_{i\ell}), \ \forall i \in [I],\ \forall \ell \in [y_i^U - y_i^L]. \tag{8d}$$

In computation, a large coefficient M slows down the solution of (DRNS). Theoretically, for the correctness of the linearization (8a)–(8d), M needs to be larger than $|\gamma_j|$ and $|\lambda_i|$ for all $j \in [J]$ and $i \in [I]$, respectively. The following proposition (see Online Appendix F for a proof) derives uniform lower and upper bounds of γ_j and λ_i, leading to a small value of M.

Proposition 2. *There exists an optimal solution* $(\gamma^*, \lambda^*, \rho^*)$ *to formulation* (5a)–(5b) *such that* $\gamma_j^* \in [-c_j^x, 0]$ *for all* $j \in [J]$ *and* $\lambda_i^* \in [-c_{J(i)}^x, 0]$ *for all* $i \in [I]$.

Proposition 2 indicates that (i) we can set $M := c_j^x$ in (8a)–(8d) without loss of optimality and (ii) as all γ_j and λ_i are nonpositive at optimality, we can replace McCormick inequalities (8a) and (8c) as $\gamma_j \le \varphi_{j\ell} \le 0$ and $\lambda_i \le \nu_{i\ell} \le 0$, respectively, both of which are now *big-M-free*. In addition, we incorporate the following constraints to break the symmetry among binary variables:

$$u_{j\ell} \ge u_{j(\ell+1)}, \ \ \forall j \in [J],\ \forall \ell \in [w_j^U - w_j^L - 1], \tag{8e}$$

$$v_{i\ell} \ge v_{i(\ell+1)}, \ \ \forall i \in [I],\ \forall \ell \in [y_i^U - y_i^L - 1]. \tag{8f}$$

The above analysis recasts (DRNS) into a mixed-integer program, which is summarized in the following theorem without proof.

Theorem 2. *Under Assumption 1, the (DRNS) model yields the same optimal value as the following mixed-integer program:*

$$\min_{\substack{u,v,\varphi,\nu \\ \gamma,\lambda,\rho,\theta}} \theta + \sum_{i=1}^{I}(c_i^y y_i^L + g_i(y_i^L)\lambda_i + \sum_{\ell=1}^{y_i^U - y_i^L}(c_i^y v_{i\ell} + \delta_{i\ell}\nu_{i\ell})) + \sum_{j=1}^{J}\Bigg[\sum_{q=1}^{Q}\mu_{jq}\rho_{jq} + c_j^w w_j^L + f_j(w_j^L)\gamma_j + \sum_{\ell=1}^{w_j^U - w_j^L}(c_j^w u_{j\ell} + \Delta_{j\ell}\varphi_{j\ell})\Bigg] \tag{9a}$$

$$\text{s.t. } (8a)-(8f), \tag{9b}$$

$$\theta \ge \sum_{j=1}^{J}\left(c_j^r r_j + \sum_{k=1}^{j} c_{jk}^t t_{jk}\right) + \sum_{i=1}^{I}\left(c_i^p p_i + \sum_{k=1}^{J(i)} c_{ik}^s s_{ik}\right), \quad \forall (t,s,r,p) \in \mathcal{H}, \tag{9c}$$

$$u_{j\ell} \in \mathbb{B}, \ \forall j \in [J],\ \forall \ell \in [w_j^U - w_j^L],\ v_{i\ell} \in \mathbb{B}, \quad \forall i \in [I],\ \forall \ell \in [y_i^U - y_i^L], \tag{9d}$$

where set $\mathcal{H}$ is defined in (7b) and coefficients c_j^r, c_{jk}^t, c_i^p, and c_{ik}^s are represented through

$$c_j^r = \left[-\gamma_j w_j^L - \sum_{\ell=1}^{w_j^U - w_j^L} \varphi_{j\ell}\right]_+ + \max_{\tilde{d}_j \in [d_j^L, d_j^U]_{\mathbb{Z}}} \left\{-\sum_{q=1}^{Q} \rho_{jq} \tilde{d}_j^q\right\}, \quad (9e)$$

$$c_{jk}^t = \left[(-c_k^x - \gamma_j) w_j^L - \sum_{\ell=1}^{w_j^U - w_j^L} (c_k^x u_{j\ell} + \varphi_{j\ell})\right]_+ + \max_{\tilde{d}_j \in [d_j^L, d_j^U]_{\mathbb{Z}}} \left\{c_k^x \tilde{d}_j - \sum_{q=1}^{Q} \rho_{jq} \tilde{d}_j^q\right\}, \quad (9f)$$

$$c_i^p = \left[-\lambda_i y_i^L - \sum_{\ell=1}^{y_i^U - y_i^L} \nu_{i\ell}\right]_+, \quad (9g)$$

$$c_{ik}^s = \left[(-c_k^x - \lambda_i) y_i^L - \sum_{\ell=1}^{y_i^U - y_i^L} (c_k^x v_{i\ell} + \nu_{i\ell})\right]_+. \quad (9h)$$

Algorithm 1 (A Separation Algorithm for Solving the (DRNS) Model (3))

1: Initialization: Set the set of cuts $\mathcal{H}_{\text{sep}} \leftarrow \emptyset$.
2: Solve the master problem

$$\begin{aligned}(\mathbf{MP}): \min_{\substack{u,v,\varphi,\nu \\ \gamma,\lambda,\rho}} \ & \theta + \sum_{i=1}^{I} \left(c_i^y y_i^L + g_i(y_i^L)\lambda_i + \sum_{\ell=1}^{y_i^U - y_i^L} (c_i^y v_{i\ell} + \delta_{i\ell}\nu_{i\ell})\right) \\ & + \sum_{j=1}^{J} \left[\sum_{q=1}^{Q} \mu_{jq}\rho_{jq} + c_j^w w_j^L + f_j(w_j^L)\gamma_j \right. \\ & \left. + \sum_{\ell=1}^{w_j^U - w_j^L} (c_j^w u_{j\ell} + \Delta_{j\ell}\varphi_{j\ell})\right] \\ \text{s.t. } & \text{(9b)}, \text{(9d)}, \\ & \theta \geq \sum_{j=1}^{J} \left(c_j^r r_j + \sum_{k=1}^{j} c_{jk}^t t_{jk}\right) + \sum_{i=1}^{I} \left(c_i^p p_i + \sum_{k=1}^{J(i)} c_{ik}^s s_{ik}\right), \\ & \qquad \forall (t,s,r,p) \in \mathcal{H}_{\text{sep}},\end{aligned}$$

3: Compute $(c_j^{r*}, c_{jk}^{t*}, c_i^{p*}, c_{ik}^{s*})$ by (9e)–(9h) and $(u^*, v^*, \varphi^*, \nu^*, \gamma^*, \lambda^*, \rho^*)$ by (**MP**).
4: Solve (7) using $(c_j^{r*}, c_{jk}^{t*}, c_i^{p*}, c_{ik}^{s*})$ and record an optimal solution (t^*, s^*, r^*, p^*).
5: **if** $\theta^* \geq \sum_{j=1}^{J}(c_j^{r*} r_j^* + \sum_{k=1}^{j} c_{jk}^{t*} t_{jk}^*) + \sum_{i=1}^{I}(c_i^{p*} p_i^* + \sum_{k=1}^{J(i)} c_{ik}^{s*} s_{ik}^*)$ **then**
6: Stop and return (u^*, v^*) as an optimal solution to (DRNS).
7: **else**
8: Add a cut (9c) into (**MP**) by setting $\mathcal{H}_{\text{sep}} \leftarrow \mathcal{H}_{\text{sep}} \cup \{(t^*, s^*, r^*, p^*)\}$. Go to step 2.
9: **end if**

The reformulation (9a)–(9d) facilitates the separation algorithm (see, e.g., Nemhauser and Wolsey 1999), also known as delayed constraint generation. We notice that (9c) involves $(J+1)!$ many constraints, making it computationally prohibitive to solve (9a)–(9d) in one shot. Instead, the separation algorithm incorporates constraints (9c) on the fly. Specifically, this algorithm first solves a relaxation of the reformulation by overlooking constraints (9c). Then, we check if the optimal solution thus obtained violates any of (9c). If yes, then we add one violated constraint back into the relaxation and resolve. We call this added constraint a "cut" and note that each cut describes a convex feasible region. This procedure is repeated until an optimal solution is found to satisfy all of constraints (9c). We summarize this procedure in Algorithm 1.

Theorem 3. *Algorithm 1 finds a globally optimal solution to the (DRNS) model (3) in a finite number of iterations.*

To strengthen the integer program (7), we derive valid inequalities for its feasible region in Lemma 3 (see Online Appendix H for a proof).

Lemma 3. *Any $(t,s,r,p) \in \mathcal{H}$ satisfies the following inequalities:*

$$\sum_{\ell=k}^{j} t_{j\ell} \leq \sum_{\ell=k}^{J(i)} s_{i\ell}, \qquad \forall i \in [I],\ \forall j \in P_i,\ \forall k \in [j], \quad (10)$$

where J(i) is the largest index in P_i. In addition, inequalities (10) imply constraints (6e).

Intuitively, inequalities (10) designate that if $\overline{\alpha}_j \geq c_k^x$, then $\overline{\beta}_i \leq -c_k^x$, echoing the optimality conditions on $\overline{\alpha}_j$ and $\underline{\beta}_i$ in Lemma 2. In the following section, we show that, under special but practical pool structures, inequalities (10) are strong enough to describe the convex hull of $\mathcal{H}$. That is, with inequalities (10), we will be able to solve integer program (7) as a linear program and obtain a binary-valued optimal solution. In Section 5.2, we demonstrate that (10) significantly accelerates the solution of (DRNS) under both general and special pool structures.

4. Tractable Cases: Practical Pool Structures

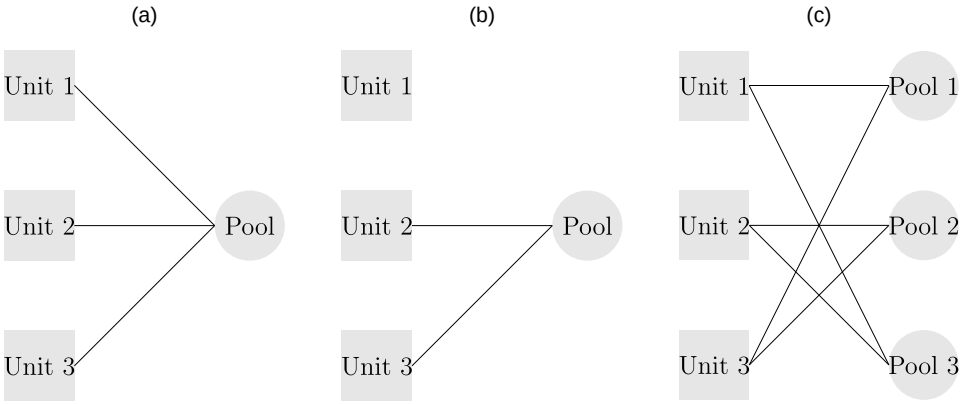
We consider the following three special nurse pool structures.

Structure 1 (One Pool). In this structure, $I = 1$, that is, there is a nurse pool shared among all units (see Figure 2(a) for an example).

Structure D (Disjoint Pools). All nurse pools are disjoint, that is, for all $i_1, i_2 \in [I]$ and $i_1 \neq i_2$, it holds that $P_{i_1} \cap P_{i_2} = \emptyset$ (see Figure 2(b) for an example).

Structure C (Chained Pools). The nurse pools form a long chain, that is, there are $I = J$ pools and the bipartite graph consisting of the units, pools, and pool-unit assignment forms one undirected cycle. Figure 2(c)

Figure 2. Examples of Practical Pool Structures



Notes. (a) Structure 1. (b) Structure D. (c) Structure C.

displays an example with cycle P_1–Unit 1–P_3–Unit 2–P_2–Unit 3–P_1.

Structure 1 can be utilized when all units have similar functionalities and so they can all share one nurse pool. Accordingly, nurses assigned to this pool need to be cross-trained for all units so that they are able to undertake the tasks in them. Structure D is less demanding than one pool, as each pool covers only a subset of units with similar functionality. Accordingly, the amount of cross-training under this structure significantly decreases. Structure C has been applied in production systems to increase flexibility (see, e.g., Jordan and Graves 1995; Chen et al. 2015, 2019; Wang and Zhang 2015). Under this structure, every unit is covered by two nurse pools. Accordingly, a pool nurse needs to be cross-trained for only two units. All three structures have been considered and compared in a nurse staffing context (see, e.g., Inman et al. 2005). Under these practical pool structures, we derive tractable reformulations of the (DRNS) model. Our derivation leads to MILP reformulations that facilitate powerful off-the-shelf software (e.g., GUROBI).

4.1. One Pool

Under Structure 1, that is, $I = 1$, $P_1 = [J]$, and $J(1) = J$, we rewrite the feasible region $\mathcal{H}$ of integer program (7) as follows:

$$\mathcal{H}_1 = \Bigg\{(t,s,r,p) : r_j + \sum_{k=1}^{j} t_{jk} = 1,\ \forall j \in [J],\ p + \sum_{k=1}^{J} s_k = 1, \tag{11a}$$

$$\sum_{k=1}^{j} t_{jk} \le 1,\ \forall j \in [J], \tag{11b}$$

$$\sum_{k=1}^{J} s_k \le 1, \tag{11c}$$

$$s_k \le \sum_{j=k}^{J} t_{jk},\ \forall k \in [J], \tag{11d}$$

$$s_k + t_{j\ell} \le 1,\ \forall k \in [J],\ \forall j \in [J],\ \forall \ell \in [k+1, j]_{\mathbb{Z}}, \tag{11e}$$

$$t_{jk} \le \sum_{\ell=1}^{J} s_\ell,\ \forall j \in [J],\ \forall k \in [j], \tag{11f}$$

$$s_k \in \mathbb{B},\ \forall k \in [J],\ t_{jk} \in \mathbb{B},\ \forall j \in [J],\ \forall k \in [j] \Bigg\}. \tag{11g}$$

We claim that valid inequalities (10), in conjunction with constraints (11a)–(11d), are sufficient to describe the convex hull of $\mathcal{H}_1$ as described in Proposition 3 (see Online Appendix I for a proof).

Proposition 3. *It holds that* $\bar{\mathcal{H}}_1 = \mathrm{conv}(\mathcal{H}_1)$, *where*

$$\bar{\mathcal{H}}_1 := \Bigg\{(t,s,r,p) : (11a) - (11d),\ \sum_{\ell=k}^{j} t_{j\ell} \le \sum_{\ell=k}^{J} s_\ell,\ \forall j \in [J],\ \forall k \in [j], \tag{12a}$$

$$s_k \in \mathbb{R}_+,\ \forall k \in [J],\ t_{jk} \in \mathbb{R}_+,\ \forall j \in [J],\ \forall k \in [j] \Bigg\}. \tag{12b}$$

Better still, this yields a closed-form solution to the convex maximization problem $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$.

Theorem 4. *Under Structure 1, the optimal value of* $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ *equals*

$$\max\Bigg\{c_1^p + \sum_{j=1}^{J} c_j^r,\ \max_{\substack{j^* \in [J],\\ k^* \in [j^*]}} \Bigg\{\sum_{j=1}^{J} c_j^r + c_{1k^*}^s + (c_{j^*k^*}^t - c_{j^*}^r) + \sum_{j \in [J]\setminus\{j^*\}} \Big(\max_{k \in [\min\{j,k^*\}]} \{c_{jk}^t - c_j^r\}\Big)\Bigg\}\Bigg\},$$

where c_j^r, c_{jk}^t, c_1^p, *and* c_{1k}^s *are defined through* (9e)–(9h).

Theorem 4 (see Online Appendix K for a proof) enables us to reduce the $(J+1)!$ many constraints (9c) in the reformulation of (DRNS) to only $\frac{1}{2}J(J+1)+1$ many, thanks to the closed-form solution. This leads to the MILP reformulation of (DRNS) in Proposition 4 (see Online Appendix L for a proof).

Proposition 4. *Under Assumption 1 and Structure 1, the (DRNS) model yields the same optimal objective value as the following MILP:*

$$
\begin{aligned}
Z_1^\star := \min\ & \theta + c_1^y y_1^L + g_1(y_1^L)\lambda_1 + \sum_{\ell=1}^{y_1^U - y_1^L} (c_1^y v_{1\ell} + \delta_{1\ell}\nu_{1\ell}) \\
& + \sum_{j=1}^{J}\left[\sum_{q=1}^{Q}\mu_{jq}\rho_{jq} + c_j^w w_j^L + f_j(w_j^L)\gamma_j \right. \\
& \left. + \sum_{\ell=1}^{w_j^U - w_j^L}(c_j^w u_{j\ell} + \Delta_{j\ell}\varphi_{j\ell})\right] \\
\text{s.t.}\ & (8a)-(8f),\ (9d), \\
& \theta \ge \phi_1^e + \sum_{j=1}^{J}(\zeta_j^e + \eta_j^e), \\
& \theta \ge \sum_{j=1}^{J}(\zeta_j^e + \eta_j^e) + \phi_{1k^*}^x + (\zeta_{j^*k^*}^x + \eta_{j^*k^*}^x) \\
& - (\zeta_{j^*}^e + \eta_{j^*}^e) + \sum_{j\in[J]\setminus\{j^*\}} \chi_{jk^*},\ \forall j^*\in[J],\ \forall k^*\in[j^*], \\
& \chi_{jk^*} \ge (\zeta_{jk}^x + \eta_{jk}^x) - (\zeta_j^e + \eta_j^e),\ \forall j\in[J],\ \forall k^*\in[J], \\
& \qquad \forall k\in[\min\{j,k^*\}], \qquad (13a) \\
& \left.\begin{aligned} & \zeta_j^e \ge -\gamma_j w_j^L - \sum_{\ell=1}^{w_j^U - w_j^L}\varphi_{j\ell},\ \zeta_j^e \ge 0, \\ & \eta_j^e \ge -\sum_{q=1}^{Q}\tilde{d}_j^q\rho_{jq},\ \forall \tilde{d}_j\in[d_j^L, d_j^U]_{\mathbb{Z}}, \end{aligned}\right\} \ \forall j\in[J], \qquad (13b) \\
& \left.\begin{aligned} & \zeta_{jk}^x \ge (-c_k^x - \gamma_j)w_j^L - \sum_{\ell=1}^{w_j^U - w_j^L}(c_k^x u_{j\ell} + \varphi_{j\ell}),\ \zeta_{jk}^x \ge 0, \\ & \eta_{jk}^x \ge c_k^x\tilde{d}_j - \sum_{q=1}^{Q}\tilde{d}_j^q\rho_{jq},\ \forall \tilde{d}_j\in[d_j^L, d_j^U]_{\mathbb{Z}}, \end{aligned}\right\} \\
& \qquad \forall j\in[J],\ \forall k\in[j], \qquad (13c) \\
& \phi_1^e \ge -\lambda_1 y_1^L - \sum_{\ell=1}^{y_1^U - y_1^L}\nu_{1\ell},\ \phi_1^e \ge 0, \\
& \phi_{1k}^x \ge (-c_k^x - \lambda_1)y_1^L - \sum_{\ell=1}^{y_1^U - y_1^L}(c_k^x v_{1\ell} + \nu_{1\ell}),\ \phi_{1k}^x \ge 0, \\
& \qquad \forall k\in[J].
\end{aligned}
$$

A special case of Structure 1 is when there is no nurse float pool, which is equivalent to assigning all units to a pool with no nurses. We call this Structure 0, where $y_1^L = y_1^U = 0$. A MILP reformulation of (DRNS) under Structure 0 follows from Proposition 4:

$$
\begin{aligned}
Z_0^\star := \min\ & \theta + \sum_{j=1}^{J}\left[\sum_{q=1}^{Q}\mu_{jq}\rho_{jq} + c_j^w w_j^L + f_j(w_j^L)\gamma_j \right. \\
& \left. + \sum_{\ell=1}^{w_j^U - w_j^L}(c_j^w u_{j\ell} + \Delta_{j\ell}\varphi_{j\ell})\right] \\
\text{s.t.}\ & (8a),\ (8b),\ (8e),\ (9d),\ (13a)\text{–}(13c), \\
& \theta \ge \sum_{j=1}^{J}(\zeta_j^e + \eta_j^e), \\
& \theta \ge \sum_{j=1}^{J}(\zeta_j^e + \eta_j^e) + (\zeta_{j^*k^*}^x + \eta_{j^*k^*}^x) - (\zeta_{j^*}^e + \eta_{j^*}^e) \\
& + \sum_{j\in[J]\setminus\{j^*\}}\chi_{jk^*},\ \forall j^*\in[J],\ \forall k^*\in[j^*].
\end{aligned}
$$

We notice that, whenever $y_1^L = 0$, any feasible nurse staffing levels under Structure 0 are also feasible to (DRNS) under Structure 1. It follows that $Z_1^\star \le Z_0^\star$. In addition, because Structure 1 provides the most flexibility and Structure 0 has zero flexibility, we interpret the difference $Z_0^\star - Z_1^\star$ as the (maximum) value of flexibility.

4.2. Disjoint Pools

Under Structure D, we denote by $\mathcal{H}_D$ the feasible region $\mathcal{H}$ of the integer program (7). Intuitively, as the nurse pools are disjoint, $\mathcal{H}_D$ becomes separable in index i, that is, separable among the nurse pools and the units pertaining to each pool. Hence, we obtain $\mathrm{conv}(\mathcal{H}_D)$ by convexifying the projection of $\mathcal{H}_D$ in each pool and then taking their Cartesian product. It follows that, once again, the convex maximization problem $\max_{(\alpha,\beta)} F(\alpha,\beta)$ admits a closed-form solution and (DRNS) can be recast as a MILP. In particular, we reduce the exponentially many constraints (9c) to only $0.5J(J+1)+I$ many. We summarize these results in Proposition 5 (see Online Appendix M for a proof).

Proposition 5. *Under Structure D, it holds that* $\mathrm{conv}(\mathcal{H}_D) = \{(t,s,r,p) : (7b),(10)\}$. *In addition, for fixed* $(u,v,\gamma,\lambda,\rho)$, *the optimal value of* $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ *equals*

$$
\sum_{i=1}^{I}\left[\max\left\{c_i^p + \sum_{j\in P_i}c_j^r,\ \max_{\substack{j^*\in P_i,\\ k^*\in[j^*]}}\left\{\sum_{j\in P_i}c_j^r + c_{ik^*}^s + (c_{j^*k^*}^t - c_{j^*}^r) \right.\right.\right.
$$

$$
\left.\left.\left. + \sum_{j\in P_i\setminus\{j^*\}}\left(\max_{k\in[\min\{j,k^*\}]}\{c_{jk}^t - c_j^r\}\right)\right\}\right\}\right],
$$

where c_j^r, c_{jk}^t, c_i^p, *and* c_{ik}^s *are defined through* (9e)–(9h).

Furthermore, under Assumption 1, the (DRNS) model yields the same optimal objective value as the following MILP:

$$Z_D^\star := \min \sum_{i=1}^{I} \left(\theta_i + c_i^y y_i^L + g_i(y_i^L)\lambda_i + \sum_{\ell=1}^{y_i^U - y_i^L} (c_i^y v_{i\ell} + \delta_{i\ell} \nu_{i\ell}) \right) + \sum_{j=1}^{J} \left[\sum_{q=1}^{Q} \mu_{jq} \rho_{jq} + c_j^w w_j^L + f_j(w_j^L)\gamma_j + \sum_{\ell=1}^{w_j^U - w_j^L} (c_j^w u_{j\ell} + \Delta_{j\ell} \varphi_{j\ell}) \right]$$

$$\text{s.t. (8a)–(8f), (9d), (13a)–(13c),}$$

$$\theta_i \geq \phi_i^e + \sum_{j \in P_i} (\zeta_j^e + \eta_j^e), \ \forall i \in [I],$$

$$\theta_i \geq \sum_{j \in P_i} (\zeta_j^e + \eta_j^e) + \phi_{ik^*}^x + (\zeta_{j^*k^*}^x + \eta_{j^*k^*}^x) - (\zeta_{j^*}^e + \eta_{j^*}^e) + \sum_{j \in P_i \setminus \{j^*\}} \chi_{jk^*}, \quad \forall i \in [I], \forall j^* \in P_i, \forall k^* \in [j^*],$$

$$\phi_i^e \geq -\lambda_i y_i^L - \sum_{\ell=1}^{y_i^U - y_i^L} \nu_{i\ell}, \ \phi_i^e \geq 0, \ \forall i \in [I], \tag{14a}$$

$$\phi_{ik}^x \geq (-c_k^x - \lambda_i) y_i^L - \sum_{\ell=1}^{y_i^U - y_i^L} (c_k^x v_{i\ell} + \nu_{i\ell}), \ \phi_{ik}^x \geq 0, \quad \forall i \in [I], \forall k \in [J(i)]. \tag{14b}$$

4.3. Chained Pools

Under Structure C, the inequalities (10) can strengthen the mixed-integer set $\mathcal{H}$ but no longer describe $\text{conv}(\mathcal{H})$. Nevertheless, we adopt an alternative approach to recast the convex maximization problem $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ as a linear program. We start by noticing that, by renumbering the units if needed, the chained pools can be made $P_i = \{i, i+1\}$ for all $i \in [I-1]$ and $P_I = \{I, 1\}$. For ease of exposition, we denote by $\sigma(i)$ the original index of unit i before renumbering. We define an order $\prec$ on $[I]$ such that $i \prec j$ if and only if $\sigma(i) < \sigma(j)$, and we say $i \preceq j$ if $i \prec j$ or $i = j$. In addition, we let $i \curlyvee j$ represent the index of higher order between units i and j, that is, $i \curlyvee j = j$ if $i \preceq j$ and $i \curlyvee j = i$ otherwise. We create a dummy unit $I+1$, which is a duplicate of unit 1, and refer to these two units interchangeably.

Example 1. Consider the chained pools in Figure 3(a) with 3 units. After renumbering (see Figure 3(b)), we have $\sigma(1) = 1$, $\sigma(2) = 3$, and $\sigma(3) = 2$. In addition, $1 \prec 3 \prec 2$ and $2 \curlyvee 3 = 2$.

Next, because $P_i = \{i, i+1\}$ for all $i \in [I]$, variables s_{ik} in the integer program (7) depend on $t_{[i]}$ and $t_{[i+1]}$ only, where $t_{[i]} := \{t_{ik} : k \preceq i\}$. Indeed, the definition of s_{ik} implies

$$s_{ik} = h_k(t_{[i]}, t_{[i+1]}) := \begin{cases} 1 & \text{if } \max\{t_{ik}, t_{(i+1)k}\} = 1, \ t_{i\ell} = 0 \text{ for all } \ell \succ k, \\ & \text{and } t_{(i+1)\ell} = 0 \text{ for all } \ell \succ k, \\ 0 & \text{otherwise.} \end{cases}$$

Plugging these representations into formulation (7) yields a reformulation of $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ based on variables t only:

$$\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta) = \sum_{i=1}^{I} (c_i^r + c_i^p) + \sum_{i=1}^{I} \left[\sum_{k \preceq i} (c_{ik}^t - c_i^r) t_{ik} + \sum_{k \preceq i \curlyvee (i+1)} (c_{ik}^s - c_i^p) h_k(t_{[i]}, t_{[i+1]}) \right]. \tag{15}$$

The reformulation (15) decomposes objective function based on index $i \in [I]$ and enables us to solve $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ by a dynamic program (DP), that is, we sequentially optimize $t_{[1]}, t_{[2]}, \ldots, t_{[I]}$. To this end, we define the state of the DP in stage 1 as $t_{[1]} \equiv t_{11} \in \mathcal{B}_1$ and the states in stage i as $(t_{[1]}, t_{[i]}) \in \mathcal{B}_1 \times \mathcal{B}_i$ for all $i \in [2, I]_{\mathbb{Z}}$, where $\mathcal{B}_i := \{t_{[i]} \in \mathbb{B}^{\sigma(i)} : \sum_{k \preceq i} t_{ik} \leq 1\}$ represents the feasible region of $t_{[i]}$ described by constraints (6a). In addition, we formulate the DP as $\max_{(t_{[1]}, t_{[I]}) \in \mathcal{B}_1 \times \mathcal{B}_I} \{V_I(t_{[1]}, t_{[I]}) + \sum_{k^1 (1 \curlyvee I)} (c_{Ik}^s - c_I^p) h_k(t_{[I]}, t_{[1]})\}$ where the value functions $V_i(\cdot)$ are recursively defined through

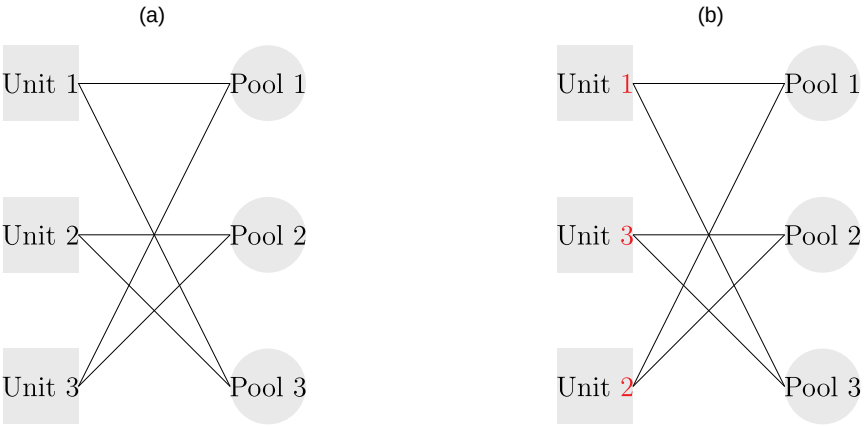
$$V_1(t_{[1]}) := \sum_{i=1}^{I} (c_i^r + c_i^p) + (c_{11}^t - c_1^r) t_{11} \text{ and}$$

$$V_i(t_{[1]}, t_{[i]}) := \max_{t_{[i-1]} \in \mathcal{B}_{i-1}} \left\{ V_{i-1}(t_{[1]}, t_{[i-1]}) + \sum_{k \preceq i} (c_{ik}^t - c_i^r) t_{ik} + \sum_{k \preceq (i-1) \curlyvee i} (c_{(i-1)k}^s - c_{i-1}^p) h_k(t_{[i-1]}, t_{[i]}) \right\}$$

for all $i \in [2, I]_{\mathbb{Z}}$ and $(t_{[1]}, t_{[i]}) \in \mathcal{B}_1 \times \mathcal{B}_i$. For all $i \in [I]$, the value function $V_i(t_{[1]}, t_{[i]})$ represents the "cumulative reward" up to stage i, that is, the terms in (15) that involve $t_{[1]}, \ldots, t_{[i]}$ only. We note that, as $t_{[1]}$ is involved in the final-stage reward, the DP stores the value of $t_{[1]}$ in the state throughout stages $2, \ldots, I$.

We further interpret the DP as a longest-path problem on an acyclic directed network $(\mathcal{N}, \mathcal{A})$. Specifically, the set of nodes $\mathcal{N}$ consists of I layers, denoted by $\{N_i\}_{i=1}^I$. For all $i \in [I]$, layer i consists of the states of the DP in stage i, that is, $\mathcal{N}_1 = \mathcal{B}_1 = \{0, 1\}$ and $\mathcal{N}_i = \mathcal{B}_1 \times \mathcal{B}_i$ for all $i \in [2, I]_{\mathbb{Z}}$. In addition, $\mathcal{A}$ consists of arcs that connect two nodes in neighboring layers, as long as the two nodes share a common $t_{[1]}$ value, that is, $\mathcal{A} = \{[t_{[1]}, (t_{[1]}, t_{[2]})] : t_{[1]} \in \mathcal{B}_1, t_{[2]} \in \mathcal{B}_2\} \cup \{[(t_{[1]}, t_{[i-1]}), (t_{[1]}, t_{[i]})] : t_{[i-1]} \in \mathcal{B}_{i-1}, t_{[i]} \in \mathcal{B}_i, \ \forall i \in [3, I]_{\mathbb{Z}}\}$. Finally, we incorporate into $\mathcal{N}$ a

Figure 3. (Color online) Renumbering Units in Structure C



Notes. (a) Before. (b) After.

starting node $\mathtt{S}$ and a terminal node $\mathtt{T}$, and into $\mathcal{A}$ arcs from $\mathtt{S}$ to all nodes in $\mathcal{N}_1$ and from all nodes in $\mathcal{N}_I$ to $\mathtt{T}$. Then, the DP is equivalent to the longest-path problem from $\mathtt{S}$ to $\mathtt{T}$ on $(\mathcal{N},\mathcal{A})$. We formally state this result in the following theorem. For notation brevity, we say $t_{[i]} = e_{ik}$ if $t_{ik} = 1$ for a $k \preceq i$, while $t_{i\ell} = 0$ for all $\ell \preceq i$ and $\ell \neq k$; and we say $t_{[i]} = e_{i0}$ if $t_{ik} = 0$ for all $k \preceq i$. In Figure 4, we depict $(\mathcal{N},\mathcal{A})$ for Example 1.

Theorem 5. *Define* $\{c_{[m,n]} : [m,n] \in \mathcal{A}\}$, *the length of the arcs in network* $(\mathcal{N},\mathcal{A})$, *such that*

$$c_{[\mathtt{S},t_{[1]}]} = \sum_{i=1}^{I} (c_i^r + c_i^p) + (c_{11}^t - c_1^r)t_{11} \quad \forall t_{[1]} \in \mathcal{B}_1,$$

$$c_{[t_{[1]},(t_{[1]},e_{2\ell})]} = \begin{cases} (c_{11}^s - c_1^p)t_{11} & \text{if } \ell = 0 \\ (c_{2k}^t - c_2^r) + (c_{1k}^s - c_1^p) & \text{if } \ell \neq 0 \end{cases} \quad \forall t_{[1]} \in \mathcal{B}_1,\ \forall \ell \preceq 2,$$

$$c_{[(t_{[1]},e_{(i-1)k}),(t_{[1]},e_{i\ell})]} = \begin{cases} 0 & \text{if } k = 0,\ \ell = 0 \\ c_{(i-1)k}^s - c_{i-1}^p, & \text{if } k \neq 0,\ \ell = 0 \\ (c_{i\ell}^t - c_i^r) + (c_{(i-1)(k\curlyvee\ell)}^s - c_{i-1}^p), & \text{if } \ell \neq 0 \end{cases}$$

$$\forall t_{[1]} \in \mathcal{B}_1,\ \forall i \in [3,I]_{\mathbb{Z}},\ \forall k \preceq i-1,\ \forall \ell \preceq i,$$

$$\text{and } c_{[(t_{[1]},e_{Ik}),\mathtt{T}]} = \begin{cases} (c_{I1}^s - c_I^p)t_{11} & \text{if } k = 0 \\ c_{Ik}^s - c_I^p & \text{if } k \neq 0 \end{cases} \quad \forall t_{[1]} \in \mathcal{B}_1,\ \forall k \preceq I.$$

Then, $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ *equals the length of the longest* $\mathtt{S}$–$\mathtt{T}$ *path on* $(\mathcal{N},\mathcal{A})$, *that is,*

$$\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta) = \max_{x\in[0,1]^{\mathcal{A}}} \sum_{[m,n]\in\mathcal{A}} c_{[m,n]} x_{[m,n]}$$

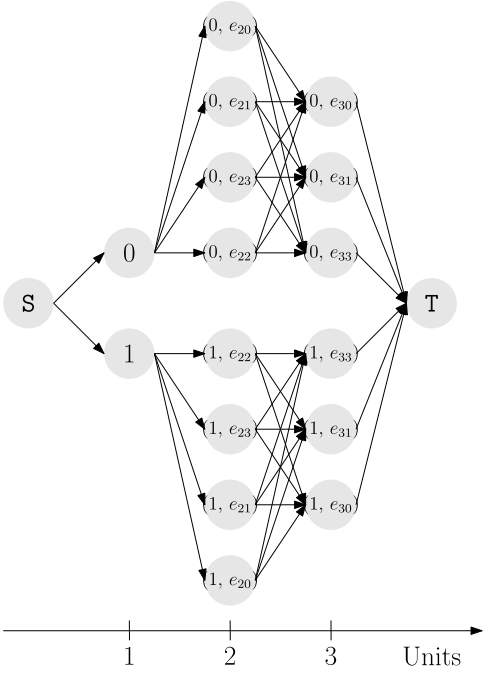
$$\text{s.t.} \sum_{n:[m,n]\in\mathcal{A}} x_{[m,n]} - \sum_{n:[n,m]\in\mathcal{A}}$$

$$x_{[n,m]} = \begin{cases} 1 & \text{if } m = \mathtt{S} \\ 0 & \text{if } m \neq \mathtt{S},\mathtt{T} \\ -1 & \text{if } m = \mathtt{T} \end{cases}, \ \forall m \in \mathcal{N}.$$

We note that $(\mathcal{N},\mathcal{A})$ is acyclic and it consists of $O(I^2)$ nodes and $O(I^3)$ arcs. Hence, $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ can be obtained in time *polynomial of the problem input*. Accordingly, we replace the exponentially many constraints (9c) in the formulation of (DRNS) with only $O(I^3)$ many linear ones. This yields the MILP reformulation in Proposition 6 (see Online Appendix O for a proof).

Proposition 6. *Under Structure C and Assumption 1, the (DRNS) model yields the same optimal objective value as*

Figure 4. Longest-Path Problem on an Acyclic Directed Network



the following MILP:

$$\min\ \theta + \sum_{i=1}^{I}(c_i^y y_i^L + g_i(y_i^L)\lambda_i + \sum_{\ell=1}^{y_i^U - y_i^L}(c_i^y v_{i\ell} + \delta_{i\ell} v_{i\ell})) + \sum_{j=1}^{I}\left[\sum_{q=1}^{Q}\mu_{jq}\rho_{jq} + c_j^w w_j^L + f_j(w_j^L)\gamma_j + \sum_{\ell=1}^{w_j^U - w_j^L}(c_j^w u_{j\ell} + \Delta_{j\ell}\varphi_{j\ell})\right]$$

$$\text{s.t.}\ \ (8a)\text{–}(8f),\ (9d),\ (13b)\text{–}(13c),\ (14a)\text{–}(14b),$$

$$\theta \geq \sum_{i=1}^{I}(c_i^r + c_i^p) + \pi_{\mathrm{S}} - \pi_{\mathrm{T}},$$

$$\pi_{\mathrm{S}} - \pi_{t_{[1]}} \geq ((\zeta_{11}^x + \eta_{11}^x) - (\zeta_1^e + \eta_1^e))t_{11} \quad \forall t_{[1]} \in \mathcal{B}_1,$$

$$\pi_{t_{[1]}} - \pi_{(t_{[1]}, e_{2\ell})} \geq \begin{cases} (\phi_{11}^x - \phi_1^e)t_{11} & \text{if } \ell = 0 \\ (\zeta_{2\ell}^x + \eta_{2\ell}^x) - (\zeta_2^e + \eta_2^e) + (\phi_{1\ell}^x - \phi_1^e) & \text{if } \ell \neq 0 \end{cases} \quad \forall t_{[1]} \in \mathcal{B}_1,\ \forall \ell \preceq 2,$$

$$\pi_{(t_{[1]}, e_{(i-1)k})} - \pi_{(t_{[1]}, e_{i\ell})} \geq \begin{cases} 0, & \text{if } k = 0, \ell = 0 \\ \phi_{(i-1)k}^x - \phi_{i-1}^e, & \text{if } k \neq 0, \ell = 0 \\ (\zeta_{i\ell}^x + \eta_{i\ell}^x) - (\zeta_i^e + \eta_i^e) + (\phi_{(i-1)(k\curlyvee\ell)}^x - \phi_{i-1}^e), & \text{if } \ell \neq 0 \end{cases}$$

$$\forall t_{[1]} \in \mathcal{B}_1,\ \forall i \in [3, I]_{\mathbb{Z}},\ \forall k \preceq i-1,\ \forall \ell \preceq i,$$

$$\pi_{(t_{11}, e_{Ik})} - \pi_{\mathrm{T}} \geq \begin{cases} (\phi_{I1}^x - \phi_I^e)t_{11} & \text{if } k = 0 \\ \phi_{Ik}^x - \phi_I^e & \text{if } k \neq 0 \end{cases} \quad \forall t_{[1]} \in \mathcal{B}_1,\ \forall k \preceq I.$$

5. Numerical Experiments

We report numerical experiments on the (DRNS) model, using data from our collaborating hospital (see Section 5.1). We summarize our main findings as follows:

1. Under the practical nurse pool structures as introduced in Section 4, the MILP reformulations of (DRNS) lead to a significant speed-up over the separation algorithm (see Section 5.2).

2. Allowing the absenteeism rate to vary with respect to the staffing level, as opposed to treating it as a constant, delivers better out-of-sample performance (see Section 5.3).

3. Modeling nurse absenteeism, as opposed to overlooking it, produces significantly better staffing decisions and out-of-sample performance (see Section 5.4).

4. Incorporating higher moments of random show-up nurses leads to an approximate solution which could have better out-of-sample performance compared with (DRNS) (see Section 5.5).

In all experiments, we solve optimization models by GUROBI 9.0.1 via Python 2.7 on a personal laptop with an Intel® Core™ i7-4850HQ CPU@2.3 GHz and 16 GB RAM.

5.1. Data

Our test instances are based on five units, abbreviated as CVC5, 4B, 4C, 7A1, and 7B in our collaborating hospital. We obtain the nurse demand and attendance entries of these five units from July 2010 to June 2014. To calibrate the ambiguity set $\mathcal{D}(w, y)$, we randomly split the data into a training data set (taking up 80% of all data entries) and a testing data set (20%). We set $Q := 2$, that is, we consider mean value μ_{j1} and standard deviation sd_j of the nurse demand, and we set $(d_j^{\mathrm{L}}, d_j^{\mathrm{U}}, \mu_{j1}, \mathrm{sd}_j)$ as the empirical min, max, mean, and standard deviation of the training data, respectively. Likewise, we set $(w_j^{\mathrm{L}}, w_j^{\mathrm{U}}, f_j(w_j))$ as the empirical min, max, and mean of the training data. For pool nurses, we assume a constant absence rate such that, for all $i \in [I]$, $g_i(y_i) := (1 - A_i^{\mathrm{p}})y_i$, where A_i^{p} is randomly extracted from the interval $[0, 0.02]$. Additionally, we set $y_i^{\mathrm{L}} := 0$ and $y_i^{\mathrm{U}} := \sum_{j \in P_i}(d_j^{\mathrm{U}} - w_j^{\mathrm{L}})$. We summarize the parameters of $\mathcal{D}(w, y)$ in Table 1.

5.2. Computational Efficacy

We evaluate the computational efficacy of three approaches on solving (DRNS) under general and special pool structures: the separation algorithm described in Algorithm 1 (denoted by Sep), the separation algorithm with valid inequalities (10) (denoted by $\mathrm{Sep_{VI}}$), and the MILP reformulations derived in Section 4 in case of special pool structures (denoted by MILP). To evaluate the computational efficacy, we duplicate the five-unit system to create 10- and 50-unit systems. For each system, we consider four different pool structures, including general pool and Structures 1, D, and C. We report the computing time (in wall-clock seconds) in Table 2. From this table, we observe that valid inequalities (10) significantly improve the computational efficacy of the separation algorithm. For example, the computing time of $\mathrm{Sep_{VI}}$ is a small fraction of that of Sep

Table 1. Parameters of $\mathcal{D}(w, y)$ in the Five-Unit System

	Demand				Decision-dependent nurse show-up		
Unit j	d_j^{L}	d_j^{U}	μ_{j1}	sd_j	w_j^{L}	w_j^{U}	$\{(w_j, f_j(w_j)) : w_j \in [w_j^{\mathrm{L}}, w_j^{\mathrm{U}}]_{\mathbb{Z}}\}$
1	4	12	10.39	1.75	9	12	{(9, 7), (10, 7.55), (11,7.57), (12,7.85)}
2	6	14	12.91	1.34	10	13	{(10,8.5), (11,8.89), (12,9), (13,9)}
3	8	18	16.60	1.55	14	17	{(14,11.64), (15,11.68), (16,11.86), (17,12)}
4	5	14	12.08	1.48	8	13	{(8,7), (9,7.5), (10,8), (11,8.75), (12,8.82), (13,9)}
5	6	16	14.58	1.42	12	16	{(12,10), (13,10.55), (14,11), (15,10.83), (16,11)}

Table 2. Wall-Clock Seconds Taken to Solve (DRNS) by Separation Algorithm (Sep), Separation Algorithm with Valid Inequalities (Sep_{VI}), and MILP Reformulations in Section 4

	General[a]		Structure 1			Structure D[b]			Structure C		
J	Sep	Sep_{VI}	Sep	Sep_{VI}	MILP	Sep	Sep_{VI}	MILP	Sep	Sep_{VI}	MILP
5	0.41	0.27	0.22	0.20	0.18	0.23	0.22	0.16	0.37	0.26	0.25
10	4.02	0.93	0.57	0.46	0.52	0.71	0.40	0.22	23.15	0.83	0.67
50	285.19	50.93	81.27	15.72	17.20	165.85	44.62	8.22	Limit[c]	3,187.35	1,078.28

[a]$P_1 = \{j \in [J] : j \text{ is even}\}$, $P_2 = \{j \in [J] : j \text{ is odd}\}$, $P_3 = \{j \in [J] : j \le J/2\}$, and $P_4 = \{j \in [J] : j > J/2\}$.
[b]$P_i = \{4i-3, 4i-2, 4i-1, 4i\}$, $\forall i \in [\lfloor J/4 \rfloor]$, and $P_{\lfloor J/4 \rfloor+1} = \{4\lfloor J/4 \rfloor + 1, \ldots, J\}$.
[c]Time limit = two hours.

in most instances, and Sep_{VI} solves the most challenging instance ($J = 50$ under Structure C) to optimum within the time limit, whereas Sep cannot. In addition, MILP prevails under special pool structures. For example, it solves all instances to global optimum in 18 minutes.

5.3. Comparing Constant and Varying Absenteeism Rates

When characterizing the ambiguity set $\mathcal{D}(w, y)$, we can model f_j and g_i simply as linear functions of the staffing levels (w, y). Linear f_j and g_i imply a constant nurse show-up rate (i.e., the expected number of show-up nurses divided by the staffing levels), which is independent of the staffing decisions. This gives rise to a DRO^{exo} model with constant absenteeism (or show-up) rates. That is, the DRO^{exo} approach assumes that the expected number of the nurses showing up is a linear function of the staffing level with a constant slope. This is in contrast to DRNS, wherein the absenteeism rates may vary pertaining to the staffing level (see, e.g., Figure 1 (right)). In this section, we evaluate the out-of-sample performance of DRO^{exo} and DRNS solutions to compare these two modeling choices.

We compute the constant show-up rates in DRO^{exo} by averaging across all nurse attendance records in the data. In addition, we present the formulation of DRO^{exo} in Online Appendix P. We report the out-of-sample average staffing costs of DRNS and DRO^{exo} in Table 3. From this table, we observe that DRNS leads to a lower staffing cost on average and tends to need fewer temporary nurses than DRO^{exo}. Under a general pool structure, as well as under Structures 1 and C, the gap in out-of-sample performance exceeds 25%. This demonstrates the value of modeling varying absenteeism rates. To examine this observation further, we compare the optimal staffing levels (w^*, y^*) of DRNS and DRO^{exo} in Table 4. From this table, we observe that DRNS staffs more pool nurses and assigns fewer unit nurses. This is because, in our instances, the nurses become less likely to show up as the staffing level increases (see Figure 1 (right)). The increasing absenteeism rate discourages DRNS from staffing more unit nurses. In contrast, the DRO^{exo} approach assumes a constant absenteeism rate and overstaffs unit nurses, resulting in out-of-sample disappointment.

5.4. The Value of Modeling Absenteeism

As discussed in Section 1, modeling nurse absenteeism incurs decision-dependent uncertainty and computational challenges. It is hence worth examining what (DRNS) buys us, that is, the value of modeling absenteeism. To this end, we generate 400 instances by tuning parameters of the five-unit system under Structure 1. Specifically, we (i) generate random c_j^{x} from the interval $[450, 1500]$, (ii) perturb μ_{j1} in Table 1 by adding a random noise $\tilde{\alpha}_j$ supported on the interval $[-1, 1]$, and (iii) generate random sd_j from the interval $[1, 2]$. In addition, we consider a variant of (DRNS) that overlooks the nurse absenteeism, in which we assume that all assigned nurses show up. Then, we compare the out-of-sample performance of the optimal nurse staffing decisions produced by (DRNS) and that produced by overlooking absenteeism. Fixing the nurse staffing levels as in a (DRNS) optimal solution (w^*, y^*), we

Table 3. Out-of-Sample Comparison Between DRO^{exo} and DRNS

	Average staffing cost			Average number of temp nurses	
	DRO^{exo}	DRNS	GAP (%)[a]	DRO^{exo}	DRNS
Structure 0	35,030	33,996	3.04	17.91	18.87
Structure 1	31,826	25,268	25.95	12.92	4.93
Structure D	32,597	31,051	4.98	14.08	14.12
Structure C	32,859	25,706	27.83	13.94	4.95
General	31,908	24,912	28.08	12.99	4.15

[a]$\text{GAP} = (Z^{\text{DRO(exo)}} - Z^{\text{DRNS}})/Z^{\text{DRNS}} \times 100\%$, where $Z^{\text{DRO(exo)}}$ and Z^{DRNS} are average staffing costs of DRO^{exo} and (DRNS) in the out-of-sample simulation, respectively.

Table 4. Optimal Staffing Levels Produced by DRO^{exo} and DRNS

	DRO^{exo}		DRNS	
	w^*	y^*	w^*	y^*
Structure 0	10, 13, 17, 12, 16	0	10,11, 14, 11, 14	0
Structure 1	9, 13, 17, 12, 16	5	9, 10, 14, 8, 12	17
Structure D	9, 13, 17, 12, 16	4	9, 11, 14, 11, 14	5
Structure C	9, 13, 17, 12, 16	0, 1, 1, 0, 3	9, 10, 14, 8, 12	3, 4, 3, 5, 3
General	9, 13, 17, 12, 16	4, 1	9, 10, 14, 8, 12	9, 9

generate a large number of scenarios for nurse demand and absenteeism. Exposing (w^*, y^*) under these scenarios produces an out-of-sample estimate of the average staffing cost with absenteeism, which we denote by Z_{abs}. Using the same set of scenarios, we examine the optimal solution produced by overlooking absenteeism and obtain an out-of-sample average cost without absenteeism, denoted by $Z_{w/o}$. Using the same out-of-sample procedures, we compute the average number of temporary nurses hired when considering absenteeism (denoted by x_{abs}) and when overlooking it (denoted by $x_{w/o}$).

We depict the values of $Z_{w/o}$ (x-coordinate) and Z_{abs} (y-coordinate) obtained in 400 replications in Figure 5(a). From this figure, we observe that most dots are below the 45-degree line, indicating that $Z_{w/o} - Z_{abs} > 0$, that is, modeling nurse absenteeism yields nurse staffing levels with better out-of-sample performance. In addition, we group the dots based on the relative value of flexibility $\text{OVG} := (Z_0^\star - Z_1^\star)/Z_0^\star \times 100\%$. From Figure 5(a), we observe that the difference $Z_{w/o} - Z_{abs}$ shows an increasing trend as OVG increases. That is, modeling nurse absenteeism becomes more valuable as the value of flexibility increases. This makes sense because when a unit is short of supply because of nurse absenteeism, making it up with pool nurses is less expensive than doing so with temporary nurses. As a result, setting up nurse pools can effectively mitigate the impacts of nurse absenteeism. In Figure 5(b), we depict the values of $x_{w/o}$ and x_{abs} obtained in the 400 replications and make similar observations.

5.5. Higher-Order Moments of Random Show-Up Nurses

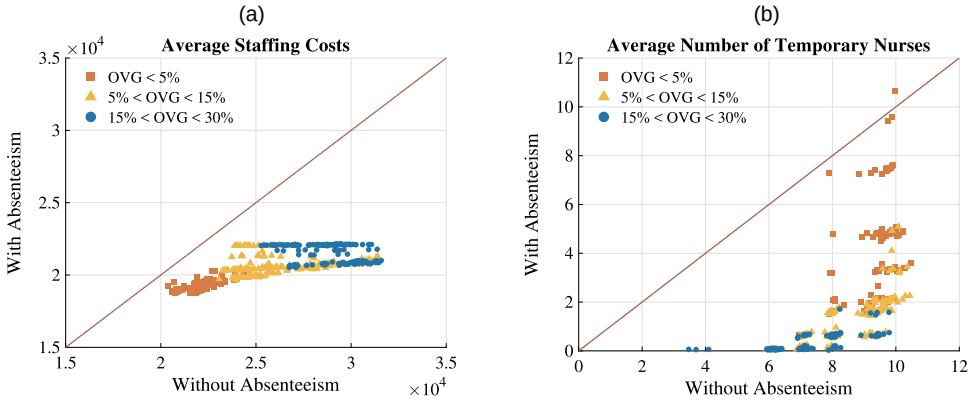
So far, the ambiguity set $\mathcal{D}(w,y)$ only considers the first-order moments (or mean values) for the random show-up nurses $(\tilde{w},\tilde{y})$. In this section, we incorporate higher-order (up to the Q-th order, with $Q \geq 2$) moments of $(\tilde{w},\tilde{y})$ and extend $\mathcal{D}(w,y)$ to be

$$\mathcal{D}(w,y) = \{\mathbb{P} \in \mathcal{P}(\Xi):\ \mathbb{E}_{\mathbb{P}}[\tilde{d}_j^q] = \mu_{jq},\quad \mathbb{E}_{\mathbb{P}}[\tilde{w}_j^q] = f_{jq}(w_j),\quad \forall j \in [J],\ \forall q \in [Q], \tag{16a}$$

$$\mathbb{E}_{\mathbb{P}}[\tilde{y}_i^q] = g_{iq}(y_i),\quad \forall i \in [I],\ \forall q \in [Q]\}. \tag{16b}$$

As noted in Section 2, the extended $\mathcal{D}(w,y)$ can be calibrated through the nurse demand data and attendance records, and we denote as DRNS$^{(Q)}$ the ensuing staffing model using the extended $\mathcal{D}(w,y)$. We recall that the support set Ξ depends on the staffing levels w and y. This makes DRNS$^{(Q)}$ intractable, as its reformulation boils down to a set of integer programs with polynomial objective functions and changing feasible regions pertaining to (w, y). Fortunately, relaxing (w,y) to their

Figure 5. (Color online) An Out-of-Sample Performance Comparison of Considering vs. Overlooking Nurse Absenteeism



Note. Note that $\text{OVG} := (Z_0^\star - Z_1^\star)/Z_0^\star \times 100\%$, where $Z_0^\star$ (respectively, $Z_1^\star$) is the optimal value of (DRNS) with no nurse pools (respectively, under Structure 1).

upper bounds $(w^{\mathrm{U}}, y^{\mathrm{U}})$ renders the feasible region decision-independent. In this case, we can solve these integer programs by enumerating a linear number of staffing levels. This produces a conservative and tractable approximation for DRNS$^{(Q)}$:

$$\min_{\gamma,\lambda,\rho^w,\rho^y} \max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta) + \sum_{i=1}^{I}\left[c_i^{\mathrm{y}} y_i + \sum_{q=1}^{Q} g_{iq}(y_i)\lambda_{iq}\right] + \sum_{j=1}^{J}\left[c_j^{\mathrm{w}} w_j + \sum_{q=1}^{Q}\mu_{jq}\rho_{jq} + \sum_{q=1}^{Q} f_{jq}(w_j)\gamma_{jq}\right] \tag{17a}$$

$$\text{s.t.} \quad (3b)\text{–}(3c), \tag{17b}$$

where $\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta)$ is recast as a MILP:

$$\max_{(\alpha,\beta)\in\Lambda} F(\alpha,\beta) := \max_{(t,s,r,p)\in\mathcal{H}} \sum_{j=1}^{J}\left(c_j^{\mathrm{r}} r_j + \sum_{k=1}^{j} c_{jk}^{\mathrm{t}} t_{jk}\right) + \sum_{i=1}^{I}\left(c_i^{\mathrm{p}} p_i + \sum_{k=1}^{J(i)} c_{ik}^{\mathrm{s}} s_{ik}\right) \tag{17c}$$

with

$$c_j^{\mathrm{r}} = \max_{\tilde{w}_j\in[0,w_j^{\mathrm{U}}]_{\mathbb{Z}}}\left\{-\sum_{q=1}^{Q}\gamma_{jq}\tilde{w}_j^q\right\} + \max_{\tilde{d}_j\in[d_j^{\mathrm{L}},d_j^{\mathrm{U}}]_{\mathbb{Z}}}\left\{-\sum_{q=1}^{Q}\rho_{jq}\tilde{d}_j^q\right\}, \tag{17d}$$

$$c_{jk}^{\mathrm{t}} = \max_{\tilde{w}_j\in[0,w_j^{\mathrm{U}}]_{\mathbb{Z}}}\left\{-c_k^{\mathrm{x}}\tilde{w}_j - \sum_{q=1}^{Q}\gamma_{jq}\tilde{w}_j^q\right\} + \max_{\tilde{d}_j\in[d_j^{\mathrm{L}},d_j^{\mathrm{U}}]_{\mathbb{Z}}}\left\{c_k^{\mathrm{x}}\tilde{d}_j - \sum_{q=1}^{Q}\rho_{jq}\tilde{d}_j^q\right\}, \tag{17e}$$

$$c_i^{\mathrm{p}} = \max_{\tilde{y}_i\in[0,y_i^{\mathrm{U}}]_{\mathbb{Z}}}\left\{-\sum_{q=1}^{Q}\lambda_{iq}\tilde{y}_i^q\right\}, \tag{17f}$$

$$c_{ik}^{\mathrm{s}} = \max_{\tilde{y}_i\in[0,y_i^{\mathrm{U}}]_{\mathbb{Z}}}\left\{-c_k^{\mathrm{x}}\tilde{y}_i - \sum_{q=1}^{Q}\lambda_{iq}\tilde{y}_i^q\right\}. \tag{17g}$$

Then, the approximate DRNS$^{(Q)}$ model can be solved using a similar separation algorithm as Algorithm 1. We provide a detailed derivation for (17) in Online Appendix Q.

We compare DRNS$^{(Q)}$ with the baseline DRNS in out-of-sample simulations using various orders of moments $Q = 1, 2, 3, 4$ and various nurse pool structures, and we report the results in Table 5. From this table, we observe that DRNS$^{(Q)}$ performs better as Q increases. This is expected because the ambiguity set incorporates more distributional information of $(\tilde{w}, \tilde{y})$ as Q increases. In addition, although we obtain an approximate solution to DRNS$^{(Q)}$, the solution eventually outperforms the baseline DRNS as Q increases to three or four. This demonstrates the value of incorporating higher-order moment information, as well as the tightness of the approximation.

Table 5. Comparison of the Out-of-Sample Average Costs Between DRNS and the DRNS$^{(Q)}$ Models

	DRNS	DRNS$^{(1)}$	DRNS$^{(2)}$	DRNS$^{(3)}$	DRNS$^{(4)}$
Structure 0	**33,996**	**33,996**	**33,996**	**33,996**	**33,996**
Structure 1	25,268	32,808	24,956	**24,821**	**24,821**
Structure D	31,051	33,424	32,607	**29,864**	**29,864**
Structure C	25,706	32,833	27,867	24,901	**24,264**
General	24,912	32,822	25,174	**24,574**	**24,574**

Note. The lowest cost for each nurse pool structure is highlighted in bold.

6. Conclusions

We studied a two-stage (DRNS) model for nurse staffing under both exogenous demand uncertainty and decision-dependent absenteeism uncertainty. We derived a min-max reformulation for (DRNS) under general nurse pool structures, leading to a separation algorithm that provably finds a globally optimal solution within a finite number of iterations. Under practical pool structures including one pool, disjoint pools, and chained pools, we derived MILP reformulations for (DRNS) and significantly improved the computational efficacy. Through numerical case studies, we found that modeling absenteeism improves the out-of-sample performance of staffing decisions, and such improvement is positively correlated with the value of flexibility.

For future research, it complements the DRNS model to penalize the nurse shortage on a relative scale, for example, $\sum_{j=1}^{J}(\tilde{d}_j - \tilde{w}_j)^+/\tilde{w}_j$, which penalizes the shortage more heavily when there are fewer nurses showing up. This reflects the increased workload on the remaining nurses when absence occurs.

Endnote

[1] When the NPR requirement is not mandated, we interpret the temporary nurses as a measure of nurse shortage, which may decrease the quality of care and increase the workload of the remaining nurses.

References

Bam M, Zhang Z, Denton BT, Duck M, Van Oyen MP (2020) Planning models for skills-sensitive surgical nurse staffing: A case study at a large academic medical center. *IISE Trans. Healthcare Systems Engrg.* 10(4):277–293.

Basciftci B, Ahmed S, Shen S (2021) Distributionally robust facility location problem under decision-dependent stochastic demand. *Eur. J. Oper. Res.* 292(2):548–561.

Bertsimas D, Sim M, Zhang M (2018) Adaptive distributionally robust optimization. *Management Sci.* 65(2):604–618.

Bertsimas D, Doan XV, Natarajan K, Teo CP (2010) Models for minimax stochastic linear optimization problems with risk aversion. *Math. Oper. Res.* 35(3):580–602.

Boutsioli Z (2010) Demand variability, demand uncertainty and hospital costs: A selective survey of the empirical literature. *Global J. Health Sci.* 2(1):138–149.

California Department of Health (2004) California RN Staffing Ratio Law. https://www.cga.ct.gov/2004/rpt/2004-R-0212.htm.

Campbell GM (2011) A two-stage stochastic program for scheduling and allocating cross-trained workers. *J. Oper. Res. Soc.* 62(6):1038–1047.

Chen X, Zhang J, Zhou Y (2015) Optimal sparse designs for process flexibility via probabilistic expanders. *Oper. Res.* 63(5):1159–1176.

Chen X, Ma T, Zhang J, Zhou Y (2019) Optimal design of process flexibility for general production systems. *Oper. Res.* 67(2):516–531.

Davis A, Mehrotra S, Holl J, Daskin MS (2014) Nurse staffing under demand uncertainty to reduce costs and enhance patient safety. *Asia-Pacific J. Oper. Res.* 31(01):1450005-1–1450005-19.

Delage E, Ye Y (2010) Distributionally robust optimization under moment uncertainty with application to data-driven problems. *Oper. Res.* 58(3):595–612.

Doan XV (2022) Distributionally robust optimization under endogenous uncertainty with an application in retrofitting planning. *Eur. J. Oper. Res.* 300(1):73–84.

Dupačová J (2006) Optimization under exogenous and endogenous uncertainty. University of West Bohemia, Pilsen.

Easton FF (2014) Service completion estimates for cross-trained workforce schedules under uncertain attendance and demand. *Production Oper. Management* 23(4):660–675.

Gaynor M, Anderson GF (1995) Uncertain demand, the structure of hospital costs, and the cost of empty hospital beds. *J. Health Econom.* 14(3):291–317.

Green LV, Savin S, Savva N (2013) "Nursevendor Problem": Personnel staffing in the presence of endogenous absenteeism. *Management Sci.* 59(10):2237–2256.

Hanasusanto GA, Kuhn D (2018) Conic programming reformulations of two-stage distributionally robust linear programs over Wasserstein balls. *Oper. Res.* 66(3):849–869.

Hulshof PJ, Kortbeek N, Boucherie RJ, Hans EW, Bakker PJ (2012) Taxonomic classification of planning decisions in healthcare: A structured review of the state of the art in OR/MS. *Health Systems* 1(2):129–175.

Inman RR, Blumenfeld DE, Ko A (2005) Cross-training hospital nurses to reduce staffing costs. *Health Care Management Rev.* 30(2):116–125.

Jordan WC, Graves SC (1995) Principles on the benefits of manufacturing process flexibility. *Management Sci.* 41(4):577–594.

Kao EP, Queyranne M (1985) Budgeting costs of nursing in a hospital. *Management Sci.* 31(5):608–621.

Kao EP, Tung GG (1981) Aggregate nursing requirement planning in a public healthcare delivery system. *Socio-Econom. Planning Sci.* 15(3):119–127.

Kim K, Mehrotra S (2015) A two-stage stochastic integer programming approach to integrated staffing and scheduling with application to nurse management. *Oper. Res.* 63(6):1431–1451.

Kong Q, Lee CY, Teo CP, Zheng Z (2013) Scheduling arrivals to a stochastic service delivery system using copositive cones. *Oper. Res.* 61(3):711–726.

Lu SF, Lu LX (2016) Do mandatory overtime laws improve quality? Staffing decisions and operational flexibility of nursing homes. *Management Sci.* 63(11):3566–3585.

Luo F, Mehrotra S (2020) Distributionally robust optimization with decision dependent ambiguity sets. *Optim. Lett.* 14(8):2565–2594.

Maass KL, Liu B, Daskin MS, Duck M, Wang Z, Mwenesi R, Schapiro H (2017) Incorporating nurse absenteeism into staffing with demand uncertainty. *Health Care Management Sci.* 20(1):141–155.

Maenhout B, Vanhoucke M (2013) An integrated nurse staffing and scheduling analysis for longer-term nursing staff allocation problems. *Omega* 41(2):485–499.

McCormick GP (1976) Computability of global solutions to factorable nonconvex programs: Part I–Convex underestimating problems. *Math. Programming* 10(1):147–175.

Nemhauser GL, Wolsey LA (1999) *Integer and Combinatorial Optimization* (John Wiley & Sons, Inc., Hoboken, NJ).

Noyan N, Rudolf G, Lejeune M (2022) Distributionally robust optimization under a decision-dependent ambiguity set with applications to machine scheduling and humanitarian logistics. *INFORMS J. Comput.* 34(2):729–751.

Slaugh VW, Scheller-Wolf AA, Tayur SR (2018) Consistent staffing for long-term care through on-call pools. *Production Oper. Management* 27(12):2144–2161.

Stewart BD, Webster DB, Ahmad S, Matson JO (1994) Mathematical models for developing a flexible workforce. *Internat. J. Production Econom.* 36(3):243–254.

Tevington P (2011) Mandatory nurse-patient ratios. *Medsurg Nursing* 20(5):265–268.

Trivedi VM (1981) A mixed-integer goal programming model for nursing service budgeting. *Oper. Res.* 29(5):1019–1034.

U.S. Bureau of Labor Statistics (2016) Absences from work of employed full-time wage and salary workers by occupation and industry. https://www.bls.gov/cps/cpsaat47.htm.

Van den Bergh J, Beliën J, De Bruecker P, Demeulemeester E, De Boeck L (2013) Personnel scheduling: A literature review. *Eur. J. Oper. Res.* 226(3):367–385.

Victoria Department of Health (2015) Safe Patient Care (Nurse to Patient and Midwife to Patient Ratios) Act. https://www2.health.vic.gov.au/health-workforce/nursing-and-midwifery/safe-patient-care-act.

Wang WY, Gupta D (2014) Nurse absenteeism and staffing strategies for hospital inpatient units. *Manufacturing Service Oper. Management* 16(3):439–454.

Wang X, Zhang J (2015) Process flexibility: A distribution-free bound on the performance of k-chain. *Oper. Res.* 63(3):555–571.

Welton JM (2011) Hospital nursing workforce costs, wages, occupational mix, and resource utilization. *J. Nursing Admin.* 41(7/8):309–314.

Yang KK, Webster S, Ruben RA (2007) An evaluation of worker cross training and flexible workdays in job shops. *IIE Trans.* 39(7):735–746.

Copyright of Manufacturing & Service Operations Management (M&SOM) is the property of INFORMS: Institute for Operations Research & the Management Sciences and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.

5. 无人机基站选址：3-D Dynamic UAV Base Station Location Problem (IJOC, 2021)

来源：INFORMS Journal on Computing. Vol. 33, No. 3, 2021, pp. 839–860.
https://doi.org/10.1287/ijoc.2020.1034

作者：Cihan Tugrul Cicek, Zuo-Jun Max Shen, Hakan Gultekin, Bulent Tavli

关键词：无人机基站；动态选址；拉格朗日分解；连续近似；非线性优化

推荐人：付一郎

推荐理由：本文研究单个无人机基站在有限规划期内的三维动态覆盖选址，以满足地面用户随时间变化的数据需求。由于垂直移动和非凸覆盖函数使传统二维选址方法难以适用，作者建立非凸混合整数非线性规划，并提出拉格朗日分解算法。针对大规模实例中求解效果有限的问题，论文进一步构造连续近似模型。数值实验显示，该近似方法兼顾计算时间和解的准确性，还可为精确算法提供高质量初始解。

http://pubsonline.informs.org/journal/ijoc

INFORMS JOURNAL ON COMPUTING
Vol. 33, No. 3, Summer 2021, pp. 839–860
ISSN 1091-9856 (print), ISSN 1526-5528 (online)

3-D Dynamic UAV Base Station Location Problem

Cihan Tugrul Cicek,[a,b] Zuo-Jun Max Shen,[b] Hakan Gultekin,[c,d] Bulent Tavli[e]

[a] Department of Industrial Engineering, Atilim University, 06830, Incek, Ankara, Turkey; [b] Department of Industrial Engineering and Operations Research, University of California, Berkeley, Berkeley, California 94720; [c] Department of Mechanical & Industrial Engineering, Sultan Qaboos University, AL-Khoud 123, Muscat, Oman; [d] Department of Industrial Engineering, TOBB University of Economics and Technology, 06560, Cankaya, Ankara, Turkey; [e] Department of Electrical and Electronics Engineering, TOBB University of Economics and Technology, 06560, Cankaya, Ankara, Turkey

Contact: cihan.cicek@atilim.edu.tr, https://orcid.org/0000-0002-3532-2638 (CTC); maxshen@berkeley.edu, https://orcid.org/0000-0003-4538-8312 (Z-JMS); hgultekin@squ.edu.om, hgultekin@etu.edu.tr, https://orcid.org/0000-0002-5416-3249 (HG); btavli@etu.edu.tr, https://orcid.org/0000-0002-9615-1983 (BT)

Received: October 26, 2019
Revised: May 31, 2020; August 14, 2020; September 17, 2020
Accepted: September 27, 2020
Published Online in Articles in Advance: January 11, 2021

https://doi.org/10.1287/ijoc.2020.1034

Copyright: © 2021 INFORMS

Abstract. We address a dynamic covering location problem of an unmanned aerial vehicle base station (UAV-BS), in which the location sequence of a single UAV-BS in a wireless communication network is determined to satisfy data demand arising from ground users. This problem is especially relevant in the context of smart grid and disaster relief. The vertical movement ability of the UAV-BS and nonconvex covering functions in wireless communication restrict utilizing classical planar covering location approaches. Therefore, we develop new formulations to this emerging problem for a finite time horizon to maximize the total coverage. In particular, we develop a mixed-integer nonlinear programming formulation that is nonconvex in nature and propose a Lagrangean decomposition algorithm (LDA) to solve this formulation. Because of the high complexity of the problem, the LDA is still unable to find good local solutions to large-scale problems. Therefore, we develop a continuum approximation (CA) model and show that CA would be a promising approach in terms of both computational time and solution accuracy. Our numerical study also shows that the CA model can be a remedy to build efficient initial solutions for exact solution algorithms.

Summary of Contribution: This paper addresses a facet of mixed integer nonlinear programming formulations. Dynamic facility location problems (DFLPs) arise in a wide range of applications. However, classical DFLPs typically focus on the two-dimensional spaces. Emerging technologies in wireless communication and some other promising application areas, such as smart grids, have brought new location problems that cannot be solved with classical approaches. For practical reasons, many research attempts to solve this new problem, especially by researchers whose primary research area is not OR, have seemed far from analyzing the characteristics of the formulations. Rather, solution-oriented greedy heuristics have been proposed. This paper has two main objectives: (i) to close the gap between practical and theoretical sides of this new problem with the help of current knowledge that OR possesses to solve facility location problems and (ii) to support the findings with an exhaustive computational study to show how these findings can be applied to practice.

History: Accepted by David Alderson, Area Editor for Network Optimization: Algorithms and Applications.
Funding: C. Tugrul Cicek acknowledges the support of Fulbright Commission Turkey under the Visiting Researcher Scholarship Program [Grant VSR2018].

Keywords: UAV base station • dynamic location • Lagrangean decomposition • continuum approximation • nonlinear optimization

1. Introduction

Unmanned aerial vehicles (UAVs) as aerial base stations (BSs) are a promising technology to provide wireless services to ground users. For example, UAV-BSs can provide emergency communication services to rescue teams after a disaster or can help improving wireless capacity in congested networks (Bor-Yaliniz and Yanikomeroglu 2016). Among all other opportunities, rapid deployment and mobility advantages of UAV-BSs enforce players in the telecommunication sector to involve aerial components (e.g., drones or balloons) in addition to existing terrestrial components (e.g., macrocell base stations) in their operations. In this way, the quality of service and experience can be improved. Quality of service typically includes performance metrics on how service providers monitor the service provision, such as the cost of service or coverage. Quality of experience, on the other hand, includes metrics on how users perceive the services, such as reliability or price (Cao et al. 2018).

A key challenge to involving UAV-BSs in the design of wireless networks is to determine UAV-BS locations because the demand is instantaneous and highly dynamic in a territory over which the UAV-BS is planned to serve. Recent works have studied single and multiple UAV-BS locations for static networks, in which the demand and location from which the demand is originated are fixed and known. However, compared with static networks, the UAV-BS location problem for dynamic networks is more challenging when the demand and locations are random. Although there have been some attempts to solve this challenging dynamic problem, much uncertainty still exists about how to incorporate different behaviors of ground users into the problem, such as changing locations and different service-level requests at different time epochs.

To solve this problem, we introduce a new 3-D maximal covering location problem (3MCLP) to model a wireless communication network in which a single UAV-BS is used to maximize coverage in a finite service area by offering wireless services to users who have dynamic demand within a finite time horizon. Although the coverage functions have typically been assumed to be convex and dependent solely on the distance in the classical covering location problems, the coverage function in wireless networks considers both the distance and the angle between a particular user and the UAV-BS. Therefore, classical covering location approaches cannot help to solve this new problem.

In this study, we deliberately address the 3MCLP of the UAV-BS in a dynamic environment in which both the demand and locations of users change in time. Our objective is to determine the optimal location sequence of the UAV-BS for a limited time horizon to maximize the overall coverage while satisfying user demand. In particular, we first develop a nonconvex mixed integer nonlinear programming (MINLP) formulation for this dynamic problem and propose two algorithms to solve it. First, we develop a Lagrangean decomposition algorithm (LDA), in which we can decompose the original problem into relatively easier-to-solve smaller problems after applying a Lagrangean relaxation (LR) technique with additional convex approximations of some nonconvex functions. Second, we develop a continuum approximation (CA) model to reformulate the problem and propose a CA algorithm to solve this new model. We test the two algorithms through a computational study with synthetic data.

The main contributions of our study can be summarized as follows:

1. A new 3-D covering location problem is introduced, in which the vertical dimension and the temporal change in demand is involved.

2. A nonconvex coverage function is proposed, in which the coverage level does not solely depend on the distance to the UAV-BS, but it is a function of both the distance and the angle between the users and the UAV-BS.

3. A new MINLP formulation is developed, and its characteristics are analyzed.

4. Two new algorithms are developed as a remedy to overcome the computational burden of the discrete formulation, and they have improved the computational efficiency.

The rest of the paper is organized as follows: A comprehensive literature review is given in Section 2. The system model is given in Section 3, and a discrete programming formulation of the problem is developed in Section 4 together with the LDA to solve this problem. Section 5 provides the details of the CA approach. We present computational results of the proposed solution algorithms in Section 6 and conclude the study in Section 7.

2. Literature Review

Promising research including UAV-BSs (such as enhancing network capacity, improving quality of experience, extending coverage) has been attracting significant interest despite the relatively new appearance of this topic. In particular, location optimization problems have attracted significant interest because they have a crucial impact on wireless network performance (Cicek et al. 2019b). In this section, we present a high-level overview of the UAV-BS literature with a special focus on location problems together with its reflection in the operations research literature and present the differences of our study from the existing literature.

UAV-assisted wireless networks are starting to be investigated with the launch of emerging technologies in the design of heterogeneous telecommunication networks in which both terrestrial and aerial components can be jointly utilized (Cao et al. 2018). Among all others, one of the challenges in such networks is to optimally determine the UAV-BS locations not only to improve the communication channel performance between the UAV-BSs and users, but also to maintain a reliable backhaul connection with the satellite or the terrestrial networks (Li et al. 2019). Mozaffari et al. (2016), Merwaday et al. (2016), Sharma et al. (2016), Lyu et al. (2017), Alzenad et al. (2018), and Bor-Yaliniz et al. (2019) study a static, uncapacitated UAV-BS location problem with different objectives, such as coverage; spectral efficiency, that is, the unit data transmission rate per unit bandwidth; or latency, that is, the average delay time observed when responding to a data request from a user.

The literature on UAV-enabled communications is rich and encompasses many aspects of communications (Zeng et al. 2016, 2019a). In Xu et al. (2018), trajectory optimization of a UAV that provides wireless

energy transfer to a set of energy receivers is investigated. In Zhan et al. (2018), the use of UAVs to collect data from a wireless sensor network is investigated, and the trajectory of the UAV and the wake-up schedule of the sensor nodes are optimized. In Zhang et al. (2019a), a cellular enabled UAV communications paradigm is investigated. The objective is to minimize the mission completion of a UAV, which is served by terrestrial BSs, by optimizing its trajectory. In Zhang et al. (2019b), physical layer security aspects of UAV-enabled communications networks are addressed to maximize the average secrecy rates of the UAV-to-ground and ground-to-UAV transmissions by joint optimization of the UAV trajectory and transmit power. In Han et al. (2020), an approach for high reliability and low latency communication of UAV swarms is proposed, and it exploits both cellular communications and device-to-device communications.

In multiple UAV systems, UAVs need to communicate among themselves, and such communications can be in any direction of the 3-D space. As such, many antenna positions and orientations are possible. In fact, it is shown in Badi et al. (2019) that UAVs themselves can act as local scatterers, which results in elevated channel depolarization and decreased cross-polarization discrimination (XPD). Furthermore, experimental analysis reveals that the relative direction of UAVs has a significant impact on XPD values, 3-D link performance is critically affected by the elevation angle, and spectral efficiency can be more than doubled by cross-polarized antenna configurations.

To improve the overall service quality and cover more realistic cases, Kalantari et al. (2016; 2017a, b), Shi et al. (2018), and Cicek et al. (2019a) revisit the problem by incorporating the capacity of the network, for example, the maximum data rate that can be offered to users, into the problem setup and develop new problem structures and solution approaches. However, none of these studies consider the temporal change in the parameters and mainly focus on a specific time epoch or a snapshot of the network.

Although there is extensive literature investigating UAV-BS location models in the static setup, there has been surprisingly little work investigating how a dynamic problem setup in which both UAV-BSs and users are allowed to move within the planning horizon can change the network performance. Fotouhi et al. (2016) solve a single UAV-BS location problem to improve the spectral efficiency for a finite time horizon with three different heuristic algorithms, in which the UAV-BS altitude and velocity are assumed to be fixed. Mozaffari et al. (2017) study the energy aspect of the UAV-BSs. The authors propose three heuristic algorithms to find the locations of multiple UAV-BSs to minimize the total transmit power for serving multiple internet-of-things devices located on the ground. Again, the UAV altitudes are assumed to be fixed.

Propulsion energy dissipation of UAVs as a function of speed has been investigated in detail in the literature. In fact, it is shown that minimum energy dissipation is achieved at a certain nonzero speed (i.e., not hovering) (Phung and Morin 2013, Di Franco and Buttazzo 2015, Karydis and Kumar 2017, Zeng and Zhang 2017, Zeng et al. 2019b, Ebrahimi et al. 2020). In Zeng and Zhang (2017) and Zeng et al. (2019b), UAV trajectory optimization frameworks considering both communications objectives and energy dissipation minimization (by utilizing the aforementioned relationship between propulsion energy dissipation and speed) are proposed, and significant performance improvements are reported.

Liu et al. (2018) study the single multiple UAV-BS location problem to design an energy-efficient wireless network in which the maximum service area of the UAV-BSs are assumed to be known and fixed and propose a deep reinforcement learning algorithm. Recently, Wang et al. (2019a) study an adaptive UAV location problem in which the altitude of the UAV-BS is assumed to be fixed and the objective is to maximize the average throughput provided to users. An adaptive scheme is proposed in which ground users are divided into two or four identical sectors, and a single sector is selected to locate the UAV-BS at each time epoch by maximizing the coverage probability. However, none of these studies consider (i) varying user demand and (ii) moving ability of the UAV-BSs in the vertical dimension.

There exist more studies related to the routing decisions of single or multiple UAVs to optimize different objectives in a dynamic setup, such as data package delivery. However, such studies require a different understanding of the networks with a focus on the hovering ability of the UAVs, such as the velocity and/or the order of which users or ground terminals are served. Therefore, those studies are typically treated as routing problems, which are fundamentally different from covering problems; thus, these studies are out of the scope of our study and are not included in our literature review. A broad review of the technologies used in UAV-assisted networks and various applications of UAV-BSs in telecommunication can be found in the reviews of Hayat et al. (2016), Cao et al. (2018), and Li et al. (2019).

Although the research on UAV-BS location is fairly recent, covering location problems have been studied for decades in the operations research community. After the seminal work of Church and ReVelle (1974), in which the locations of a fixed number of public facilities are determined to cover as much population as possible, MCLP has become an essential component

for a wide range of applications, such as disaster management, fire protection, public services, and telecommunications (Brotcorne et al. 2003, Gentili and Mirchandani 2012, Tu et al. 2016, Chauhan et al. 2019, Cordeau et al. 2019). Readers are referred to Farahani et al. (2012) and Murray (2016) for general MCLP problems and to Berman et al. (2010) for extensions of MCLP. Because the goal of almost all of the existing studies on MCLP are mainly based on determining the locations of buildings, such as warehouses or plants, the utilized models are typically limited to $\mathbb{R}^2$. However, it is envisioned that UAVs play a crucial role in transforming several industries to technology-driven operations (Gupta et al. 2016), and it would not be surprising that widespread adoption of UAVs will occur progressively in many more applications and industries to improve efficiency and operability.

Aligned with this prediction, some studies have considered using UAVs as an emerging tool to improve the system performance in different industries. Humanitarian logistics has been one of the highly popular areas in which UAVs are used as complementary tools to extend and leverage disaster relief operations (Erdelj et al. 2017, Malandrino et al. 2019). Last-mile delivery and emergency response planning have been some other areas in which UAVs are utilized because of their cost advantage and mobility (Murray and Chu 2015, Haidari et al. 2016, Pulver et al. 2016, Kim et al. 2017). A recent survey on different civil applications of UAVs can be found in Otto et al. (2018).

A significant drawback of the current studies related to UAV location is to ignore the flexibility of the UAVs to dynamically change their position in both horizontal and vertical directions. However, integrating this ability into the problem formulation increases the complexity substantially. Therefore, two common approaches are adopted: (i) fixing the UAV altitudes or (ii) discarding the decision variables related to the vertical dimension by some simplifying assumptions, which yield suboptimal decisions.

Apart from simplifying a challenging task by introducing different relaxations or assumptions on the vertical dimension, in most of the studies, mathematical formulations adopt discrete programming techniques. Thus, even moderately sized problems easily become intractable. In this study, we combine the flexibility of positioning the UAV-BSs in the vertical dimension and a more realistic nonconvex coverage function to develop a mathematical programming formulation and propose two solution algorithms to mitigate suboptimal decisions.

3. System Model

We consider a UAV-assisted wireless communication network in which a single UAV-BS is used to provide wireless services to multiple users within a finite horizon. It is assumed that locations and demand of users are known a priori, and the UAV-BS is fed with this information to design its trajectory beforehand. Such an approach is adopted in different studies with different objectives, such as maximizing average information rate (Jiang et al. 2019), minimizing energy consumption (Wang et al. 2019b), and maximizing secrecy (Zhang et al. 2019b).

The UAV-BS is assumed to have an infinite capacity because of the dedicated backhaul links to ground base stations. That is, the demand can be infinitely satisfied without regarding the capacity of the UAV-BS because any capacity requirement can be instantly fulfilled by allocating more bandwidth from the ground base stations (Alzenad et al. 2018, Ghanavi et al. 2018). Because backhaul requires wireless links, interference between UAV-BS–to-user (fronthaul) and backhaul links should be considered. Similar to interference-avoiding methods studied in the literature (Mozaffari et al. 2015, Qiu et al. 2020), we assume that different frequency bands are employed in fronthaul and backhaul links. Moreover, the time-division duplexing is assumed, in which equally allocated time slots are adopted. In such a scheme, the UAV-BS is assumed to receive a signal from the users in the first time slot and forward signals to ground base stations in the next slot.

Relaxing the backhaul capacity limitation and interference decreases the complexity of the problem; however, thoroughly analyzing this simpler setup provides powerful insights for designing more complex structures. Thus, we keep the capacitated network design as future research and exclude ground base stations from the model.

3.1. Proposed Network Structure

Let $S \subseteq \mathbb{R}^2$ and $Q \subseteq \mathbb{R}^3$ be the convex and bounded regions within which demand arises and the UAV-BS flies, respectively. We use $\mathcal{I} = \{1,\dots,n\}$ and $\mathcal{T} = \{1,\dots,T\}$ to denote demand nodes and time intervals, respectively. We assume that n users can move inside S during the entire time horizon, and no additional users are allowed to enter. Similarly, the UAV-BS is assumed to serve inside Q for all intervals.

The UAV-BS is assumed to serve for a limited time horizon because of battery concerns and can move at specific time epochs to respond to the changing environment. Note that the UAV-BS is assumed to be capable of performing successful flying operations independent of the duration and number of intervals. However, we incorporate a penalty factor to our model to prevent aggressive movements of the UAV, which is explained in the sequel.

The time epochs are assumed to be fixed so that the entire time horizon can be divided into fixed intervals.

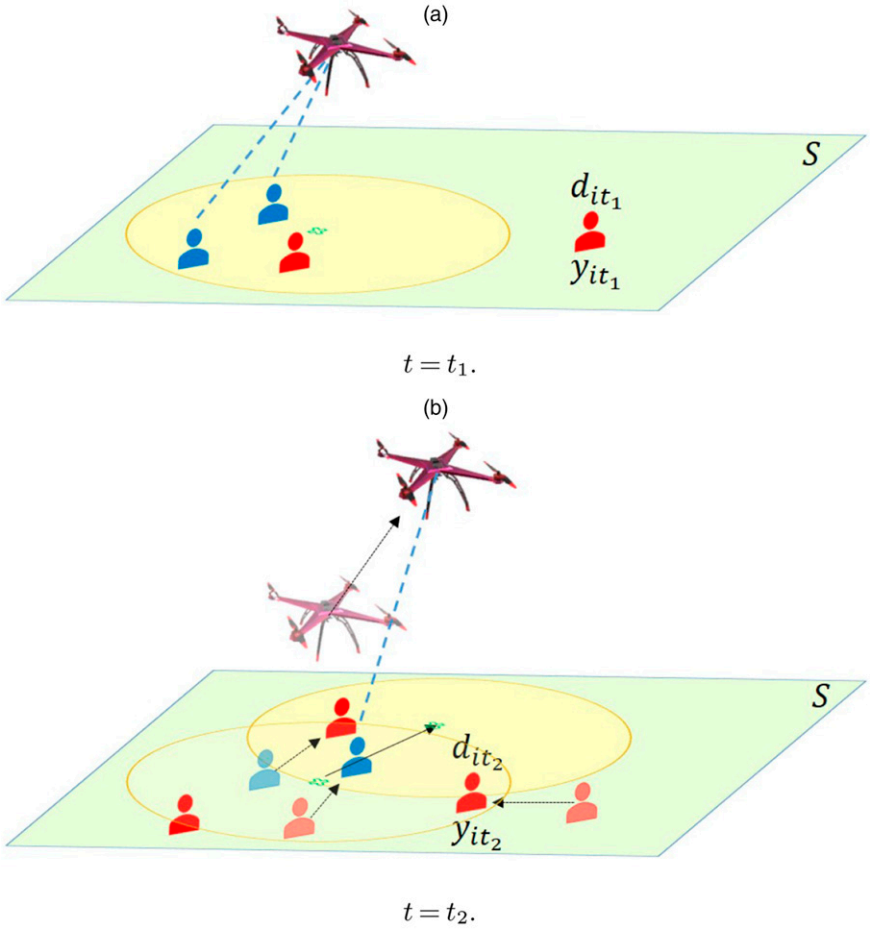
Each user is associated with a nonnegative weight, $w_i \in [0, 1]$, and assumed to be covered in a specific time interval if and only if the interval-specific maximum signal loss threshold (MSLT) of the user, $d_{it} \in \mathbb{R}$, is not exceeded. The details of signal loss are given in the next section. This loss is similar to the spatial distance constraint of the classical MCLP, in which a facility is assumed to cover a customer unless the distance is greater than a maximum threshold value.

We assume that the coverage level of a user depends solely on the signal quality provided by the UAV-BS. This quality may depend on several other factors, such as transmit power at the transmitter and the obstructions on or near the signal path. However, we assume that the signal power at the UAV-BS is fixed, and all other factors can be ignored. In particular, the signal quality can be measured by a loss function, $L : Q \times S \rightarrow \mathbb{R}$, that depends on the locations of the users and UAV-BS (see Section 3.2 for a detailed explanation).

Let $y_{it} \in S$ be the location of user $i \in \mathcal{I}$ in interval $t \in \mathcal{T}$. The UAV-BS is located at $x_t \in Q$ in interval $t \in \mathcal{T}$ and covers a set of users in its covering area $C_t(x_t) \subseteq \mathcal{I}$. A user is fully covered in a particular interval if the loss level in that interval is below a certain value, and the coverage level gradually decreases until the loss level exceeds the MSLT of the corresponding user. Thus, C_t can be defined as $C_t(x_t) := \{i : L(x_t, y_{it}) \leq d_{it}\}$. Throughout this paper, unless otherwise specified, we use boldface capital letters to denote matrices and lowercase letters to denote vectors consisting of scalar parameters or variables denoted by the same letter, for example, $\mathbf{w}$ is the vector whose components are w_i for $i \in \mathcal{I}$, and $\mathbf{Y}$ is the matrix whose components are y_{it} for $i \in \mathcal{I}$ and $t \in \mathcal{T}$.

An illustration of the described wireless network is given in Figure 1. Users may move within S between time intervals. The discs depict the covering area in an interval with cross symbols depicting the projection of the UAV-BS onto the ground. We assume that user MSLTs are independent of each other in each interval. A user is covered in a specific interval if the signal loss is below the corresponding MSLT value of that interval. Hence, users who are covered in an interval may not be covered in the next interval or vice versa. Users connected to the UAV-BS with a dashed

Figure 1. (Color online) Illustration of Network for Different Time Intervals



line depict the users who are covered in an interval, and all other users are not covered. The change in the coverage in different time intervals can be observed in this figure. Noncoverage can arise in two cases: (i) Relocation of a user or the UAV-BS can degrade the signal, and the new signal quality may not satisfy user demand anymore. (ii) User demand (MSLT value) can change, and the loss level may exceed the new value.

We made several simplifying assumptions in our model. However, arguably, the most important communication-theoretic aspect of UAV-BSs is the path loss function itself when compared with many other applications of the communication theory. We adopt the path loss model for UAV-assisted networks proposed by Al-Hourani et al. (2014), which has been heavily cited in the literature. Indeed, avoiding extremely complicated models by focusing on the most important aspects of UAV-BS communications is important for extracting invaluable insights that otherwise would be extremely challenging (if not impossible) to unearth.

3.2. Service Function

UAV-assisted wireless communication networks have several unique characteristics when compared with existing terrestrial communication networks. One of the most significant differences is based on the signal loss, namely path loss (PL), between transmitters and receivers, which are the UAV-BS and users, respectively, in our context. The higher the PL, the higher the decrease of transmitted signal power. It is not possible to successfully decode the transmitted signal if the received signal power is lower than a threshold. It means that the received power level should not be much lower than the noise level. Therefore, higher transmit power should be utilized to compensate for the higher PL levels. In the classical terrestrial networks, base stations are equipped with transmitters that can provide much higher transmission power levels to compensate for high PLs. However, mobile vehicles, such as UAVs, have less space and capacity to handle such high-power transmission equipment. Therefore, managing PL in a UAV-assisted network is more important.

There exist several PL models in the literature to define the loss level in UAV-assisted networks (Khawaja et al. 2019). We adopt the model proposed by Al-Hourani et al. (2014), which has been the most cited model to date. In this model, users are assumed to be divided into two groups: the first group has the probability of having line-of-sight (LoS) connections with low PL, and the second group does not have LoS but still can maintain a relatively poor connection with high PL.

Let $r(x,y) = \|K(x-y)\|$, $H(x,y) = \|V(x-y)\|$, and $\theta(x,y) = (180/\pi)\arctan(H(x,y)/r(x,y))$ be the horizontal and vertical distances and the elevation angle between a UAV-BS located at $x \in Q$ and a user at $y \in S$, respectively. Here, $\|\cdot\|$ is the Euclidean norm, and K and V are linear transformations defined to transform 3-D location vectors to horizontal ($\mathbb{R}^2$) and vertical ($\mathbb{R}$) location vectors, respectively. Then, the signal loss between the UAV-BS at $x \in Q$ and a user at $y \in S$ is defined as the following nonconvex function:

$$L(x,y) = F + 10\eta \log_{10}(\|x-y\|) + \frac{B}{1+\alpha e^{-\beta(\theta(x,y)-\alpha)}}, \quad (1)$$

where $F = 10\eta \log_{10}(\frac{4\pi f}{c}) + \phi_{\text{NLOS}}$ and $B = \phi_{\text{LoS}} - \phi_{\text{NLoS}}$ are constant loss parameters with channel frequency f in Hz, speed of light c in m/s. The parameters η, α, β, ϕ_{LoS}, and ϕ_{NLoS} depend on the environment, which can be a suburban, urban, dense urban, or high-rise urban. The UAV-BSs typically have a finite service area; thus, $\|x-y\|$ and $\theta(x,y)$ terms are also finite although there always exists a constant loss value because of F in this function. Therefore, we can assume that $L(\cdot)$ is bounded (i.e., $L \in [L^-, L^+]$).

Note that we do not include any term to model the effects of small-scale fading in Equation (1); instead, we focus on large-scale path loss. Thus, the path-loss function in Equation (1) can impact our analysis to some extent, especially for non line-of-sight (NLOS) channels because fading is often more severe in NLOS channels. Nevertheless, it is possible to mitigate the effects of small-scale fading in UAV-enabled communications systems (e.g., by utilizing an antenna array, it is possible to harness the diversity of the wireless channels and achieve multiplexing gain) (Khoshkholgh et al. 2019).

Our model uses the maximum signal loss threshold as the demand of a specific user. Such an approach can be easily extended to different demand functions, such as throughput. When a user requests a certain data rate in an interval, this rate can be converted to a maximum signal loss value if the transmit power of the UAV-BS and allocated bandwidth amount are fixed (Cicek et al. 2019a). As we have similar assumptions in our model, we use the signal loss function as the demand function.

$L(.)$ is a monotonically increasing function in r and unimodal in H. In other words, given the altitude of UAV-BS, the signal loss monotonically increases as horizontal distance increases, whereas given the horizontal distance, the loss monotonically increases up to a certain point as the altitude increases and then monotonically decreases (Al-Hourani et al. 2014). We utilize these properties, in particular, to define the service area and develop solution algorithms for the CA model.

4. Discrete Formulation

In this section, we give an MINLP formulation of the described wireless communication network and develop an LDA to solve this formulation. UAVs are fast and relatively practical to use; however, there is an

inherent limitation to their use in terms of battery life. Recall that we assume that the UAV-BS would have sufficient battery installed and be capable of performing any flying operation within the planning horizon. Nevertheless, service time is assumed to remain limited, and we deliberately enforce the UAV-BS to avoid aggressive displacement as much as possible.

The battery of a UAV-BS is typically used for maintaining two operations simultaneously: (i) to carry the UAV and maintain a stable position in the air and (ii) to transmit the signal. Therefore, there exists a trade-off between extending the service time by hovering for longer times and expanding the service area by allocating more power to transmit the signal. A promising way of managing this trade-off is to define a weighting cost parameter, $p \in [0,1]$, that allows the decision maker to observe how different strategies, such as penalizing the movement or relaxing it, affects the coverage. We present a sensitivity analysis for this parameter in Section 6.

Let $\mathbf{X} = (x_1, \ldots, x_T) \in Q^T$ and $g(\mathbf{X}) : Q^T \to \mathbb{R}$ be a function that defines the total relocation measurement of a location sequence of the UAV-BS. A typical definition for $g(\cdot)$, given an initial and terminating location x_0 and x_{T+1}, is the total movement during the entire time horizon, that is,

$$g(\mathbf{X}) := \sum_{t=1}^{T+1} \|x_t - x_{t-1}\|. \tag{2}$$

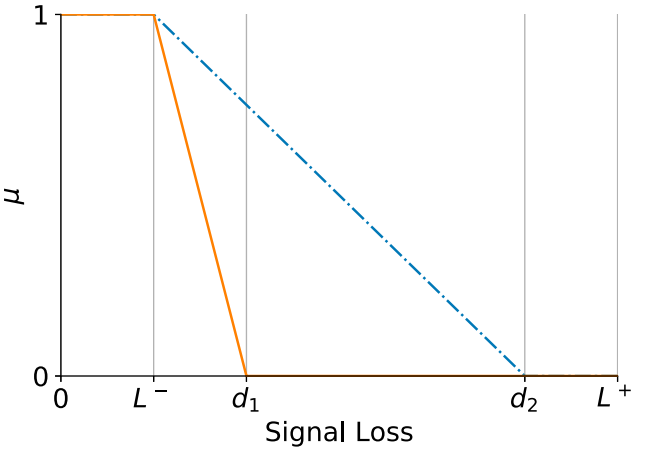
Note that there might be different definitions for $g(\cdot)$, such as the average velocity or average movement. Nevertheless, different measures have no effect on our formulation. Stability of the UAV-BS is regularized by $p \in [0,1]$. Greater p values enforce the UAV-BS to keep its location stable during the entire time horizon, and smaller p values allow the UAV-BS to move around. We assume that the power required during takeoff and landing is negligible or can be managed explicitly; thus, there is no cost at the beginning and end of the service time. Note that the problem can be solved independently for each interval when $p = 0$.

We introduce $\mu : Q \times S \times \mathbb{R} \to [0,1]$ to define the coverage level between a UAV-BS located at $x \in Q$ and a user at $y \in S$ with MSLT $d > L^-$ as

$$\mu(x,y,d) := \max\left\{0, \frac{d - L(x,y)}{d - L^-}\right\}. \tag{3}$$

Observe that μ takes positive values only if the signal loss is below the given MSLT value, d, and gradually increases to one with decreasing loss values. Figure 2 depicts the change in μ with respect to two arbitrary MSLT values, d_1 and d_2.

Figure 2. (Color online) Change of Coverage Level with Respect to Signal Loss



Given the sets of user locations $\mathbf{Y_t} = (y_{1t}, \ldots, y_{nt}) \subset S^n$ and related MSTL values per time intervals $\mathbf{D_t} = (d_{1t}, \ldots, d_{nt}) \subset \mathbb{R}^n$, the discrete 3MCLP, (3MCLP$_\mathrm{D}$), can be formulated as follows:

$$(3\mathrm{MCLP_D}) : \max_{\mathbf{X} \in Q^T} \Omega_\mathrm{D}(\mathbf{X}) = -pg(\mathbf{X}) + \sum_{t \in \mathcal{T}} \sum_{i \in C_t(x_t)} w_i \mu(x_t, y_{it}, d_{it}). \tag{4}$$

Note that the set of covered users, $C(x_t)$, can also be stated in a different way by introducing new binary variables, z_{it}, that denote whether user i is covered in interval t. Having added these new binary variables and placing (3) into (4), we have

$$\begin{aligned}(3\mathrm{MCLP_D}) : \max_{\mathbf{X} \in Q^T, \mathbf{Z} \in \{0,1\}^{n \times T}} \Omega_D(\mathbf{X}, \mathbf{Z}) &= -pg(\mathbf{X}) + \sum_{i=1}^{n} w_i \sum_{t=1}^{T} \frac{d_{it} - L(x_t, y_{it})}{d_{it} - L^-} z_{it}, \\ &= -pg(\mathbf{X}) + \sum_{t=1}^{T} \sum_{i=1}^{n} \left(v_{it} z_{it} - \kappa_{it} z_{it} L(x_t, y_{it})\right),\end{aligned}$$

s.t.

$$L(x_t, y_{it}) - d_{it} \le M(1 - z_{it}), \quad i \in \mathcal{I}, t \in \mathcal{T}, \tag{5}$$

$$z_{it} \in \{0,1\}, \quad i \in \mathcal{I}, t \in \mathcal{T}, \tag{6}$$

where $v_{it} = w_i d_{it}/(d_{it} - L^-)$, $\kappa_{it} = 1/(d_{it} - L^-)$ are auxiliary parameters and M is a sufficiently large nonnegative number (e.g., $M = L^+ - L^-$). Constraints (5) enforce z variables to be zero when the MSLT of a user is exceeded, that is, $d_{it} - L(x_t, y_{it}) < 0$. When $d_{it} - L(x_t, y_{it}) \ge 0$, z variables are not restricted and set to one because the objective is maximization. Constraints (6) are binary restrictions for z variables.

Note that the objective function in ($3MCLP_D$) includes multiplication of decision variables z and L. Therefore, we introduce new auxiliary variables, s_{it}, to overcome this nonlinearity. As a result, the ($3MCLP_D$) can be stated as

$$(3MCLP_D): \max_{\mathbf{X}\in Q^T, \mathbf{Z}\in\{0,1\}^{n\times T}, \mathbf{S}\in\mathbb{R}^{n\times T}} \Omega_D(\mathbf{X},\mathbf{Z},\mathbf{S}) = -pg(\mathbf{X}) + \sum_{t=1}^{T}\sum_{i=1}^{n}(\nu_{it}z_{it} - \kappa_{it}s_{it}),$$

s.t.

Constraints (5)-(6),

$$s_{it} \le Mz_{it}, i \in \mathcal{I}, t \in \mathcal{T}, \quad (7)$$

$$s_{it} \le L(x_t, y_{it}), i \in \mathcal{I}, t \in \mathcal{T}, \quad (8)$$

$$L(x_t, y_{it}) - M(1 - z_{it}) \le s_{it}, i \in \mathcal{I}, t \in \mathcal{T}, \quad (9)$$

$$s_{it} \ge 0, i \in \mathcal{I}, t \in \mathcal{T}. \quad (10)$$

In this formulation, when a z variable is set to zero, constraints (7) and (10) become tight, and the corresponding s variable becomes zero, which yields no coverage in the objective function value. On the other hand, when a z variable is set to one, constraints (8) and (9) become tight, and the corresponding s variable becomes equal to L, which increases the objective function value.

4.1. Lagrangian Decomposition Algorithm

($3MCLP_D$) can be solved by using commercial nonlinear programming solvers, such as BARON, but generally such an approach takes an excessively long time even for moderately sized problems. This fact motivates the development of an LDA, which is a Lagrange heuristic based on Lagrangian relaxation of nonlinear constraints and on simplifying nonconvex functions by convex approximations. LDA has been widely used within the scope of different MINLP applications, such as energy storage, portfolio management, and water network designs. A detailed source of decomposition approaches in MINLP can be found in Nowak (2005) and the references therein.

The LDA is shown to be efficient when a nonconvex MINLP has a block-seperable structure (Nowak 2005), which is the case in our problem after applying a Lagrangian relaxation to $3MCLP_D$. Relaxing all nonconvex constraints (5), (8), and (9) with nonnegative multipliers λ, ϑ, and δ yields the following relaxed formulation:

$$\begin{aligned}(3MCLP_{LR}): \max_{\mathbf{X},\mathbf{Z},\mathbf{S},\lambda,\vartheta,\delta} \Omega_{LR}(\mathbf{X},\mathbf{Z},\mathbf{S},\lambda,\vartheta,\delta) &= -pg(\mathbf{X}) \\ &+ \sum_{t=1}^{T}\sum_{i=1}^{n}\Big(\nu_{it}z_{it} - \kappa_{it}s_{it} \\ &- \lambda_{it}\big[s_{it} - L(x_t, y_{it})\big] \\ &- \vartheta_{it}\big[L(x_t, y_{it}) - M(1 - z_{it}) - s_{it}\big] \\ &- \delta_{it}\big[L(x_t, y_{it}) - d_{it} - M(1 - z_{it})\big]\Big) \\ &= -pg(\mathbf{X}) + \sum_{t=1}^{T}\sum_{i=1}^{n}(\omega_{1,it}z_{it} + \omega_{2,it}s_{it} \\ &+ \omega_{3,it}L(x_t, y_{it}) + \omega_{4,it})\end{aligned}$$

s.t.

Constraints (6), (7), (10)

$$\lambda_{it}, \vartheta_{it}, \delta_{it} \ge 0, i \in \mathcal{I}, t \in \mathcal{T}, \quad (11)$$

where $\mathbf{X} \in Q^T$, $\mathbf{Z} \in \{0,1\}^{n\times T}$, $\mathbf{S} \in \mathbb{R}^{n\times T}$, $\omega_{1,it} = (\nu_{it} - M(\vartheta_{it} + \delta_{it}))$, $\omega_{2,it} = (-\kappa_{it} - \lambda_{it} + \vartheta_{it})$, $\omega_{3,it} = (\lambda_{it} - \vartheta_{it} - \delta_{it})$, and $\omega_{4,it} = (M(\vartheta_{it} + \delta_{it}) + \delta_{it}d_{it})$. $3MCLP_{LR}$ has $(5n+3)T$ variables and $6nT$ constraints.

For given λ, ϑ, and δ, $W = \sum_{t=1}^{T}\sum_{i=1}^{n}\omega_{4,it}$ is a constant and can be discarded when solving the problem. Moreover, ($3MCLP_{LR}$) can be decomposed into two subproblems. The first subproblem is solved for determining $\mathbf{Z}$ and $\mathbf{S}$ variables, and the second subproblem is solved for determining $\mathbf{X}$ variables because there is no remaining interdependency among these variables after relaxing the corresponding constraints, that is,

$$(P_1): \max_{\substack{\mathbf{Z}\in\{0,1\}^{n\times T}\\ \mathbf{S}\in\mathbb{R}^{n\times T}}} \Omega_{P_1}(\mathbf{Z},\mathbf{S}) = \sum_{t=1}^{T}\sum_{i=1}^{n}(\omega_{1,it}z_{it} + \omega_{2,it}s_{it})$$

s.t.

Constraints (6), (7), (10), (11)

$$(P_2): \max_{\mathbf{X}\in Q^T} \Omega_{P_2}(\mathbf{X}) = -pg(\mathbf{X}) + \sum_{t=1}^{T}\sum_{i=1}^{n}\omega_{3,it}L(x_t, y_{it})$$

(P_1) can easily be solved by inspection, in which $\mathbf{Z}$ and $\mathbf{S}$ variables are determined by checking $\omega_{1,it}$ and $\omega_{2,it}$ values. Because z_{it} and s_{it} variables are independent for each $i \in \mathcal{I}$ and $t \in \mathcal{T}$, we can determine the optimal solution by checking the multipliers of each user i in each interval t. Recall that the maximum value of z and s variables are one and M, and the minimum values are zero, respectively. Moreover, s variables can be positive if the associated z variable is one. Because the objective is maximization, we set s and z values to zero if both $\omega_{1,it}$ and $\omega_{2,it}$ are negative. On the other hand, z is set to one whenever $\omega_{1,it}$ is nonnegative without checking the s value. In this case, the s value is set to zero if $\omega_{2,it}$ is negative and to M if $\omega_{2,it}$ is positive. When $\omega_{1,it}$ is negative and $\omega_{2,it}$ is positive, there exist two cases. If the $\omega_{1,it} + M\omega_{2,it}$ value is nonnegative, we set z to one and s to M so that the objective can be improved; otherwise, we set both variables to zero. Table 1 summarizes the optimal solution to (P_1).

(P_2) is relatively difficult to solve because we still have a nonconvex constrained optimization problem. Therefore, we further relax the exponential term in L and assume that each user has a LoS connection, that

Table 1. Optimal Solution of (P_1)

$\omega_{1,it}$	$\omega_{2,it}$	z_{it}	s_{it}	Condition
≥ 0	≥ 0	1	M	—
≥ 0	< 0	1	0	—
< 0	≥ 0	1	M	$\omega_{1,it} + M\omega_{2,it} \geq 0$
		0	0	$\omega_{1,it} + M\omega_{2,it} < 0$
< 0	< 0	0	0	—

is, $e^{-\beta(\theta(x,y)-\alpha)} = 0, \forall x \in Q, y \in S$. Then, relaxed ($P_2$) can be stated as

$$\begin{aligned}(P_2') \ : \max_{\mathbf{X}\in Q^T} \ \Omega'_{P_2}(\mathbf{X}) &= -pg(\mathbf{X}) \\ &+ \sum_{t=1}^{T}\sum_{i=1}^{n} \omega_{3,it}\left(F + 10\eta \log_{10}\left(\|x_t - y_{it}\|\right) + B\right) \\ &= O - pg(\mathbf{X}) + \sum_{t=1}^{T}\sum_{i=1}^{n} \omega_{5,it} \log_{10}\left(\|x_t - y_{it}\|\right),\end{aligned} \tag{12}$$

where $O = \sum_{t=1}^{T}\sum_{i=1}^{n} \omega_{3,it}(F + B)$ and $\omega_{5,it} = 10\eta\omega_{3,it}$. O is a constant term and can be discarded when solving (P_2').

Note that (P_2') seems similar to the original problem in (4) in terms of number of variables. However, the original problem cannot be explicitly solved because C_t sets include an *if* statement, which needs to be transformed to additional new constraints by introduction of new auxiliary variables, $\mathbf{Z} \in \{0,1\}^{n\times T}$. The original problem has $T(2n+3)$ variables and $6nT$ constraints. (P_2'), on the other hand, does not have these auxiliary variables and has only $3T$ variables denoted by $\mathbf{X} \in Q^T$ with no constraints. Although this subproblem seems smaller in size than the original problem, it is still difficult to solve the subproblem because of the nonconvexity in the objective function.

One promising approach to solve (P_2') would be relaxing the relocation penalty term in the objective function, dividing the subproblem into smaller subproblems, and independently determining $x_t \in Q$ for each interval $t \in T$. In fact, such an approach would have been used for the original formulation with reducing the problem into multiple single-interval problems by relaxing the relocation penalty and solving each subproblem independently, for example, using a metaheuristic, such as genetic or particle-swarm algorithms. However, these approaches would be optimal only when all parameters are the same for all intervals, which is defined as the homogeneous case in the sequel. Because such a case would not require relocation of the UAV during the planning horizon, dividing the problem with respect to time would be effective. However, in cases in which parameters, w and d, change in different intervals, which is defined as the heterogeneous case in the sequel, division approaches would suffer from relocation cost and cause suboptimal solutions.

In (P_2'), the relocation penalty term in the objective function is concave because it involves the negative of the sum of convex functions. However, the relaxed coverage term involves the multiplications of a concave logarithmic function with a coefficient. This coefficient is the combination of the Lagrange multipliers, and it is unrestricted in sign. As we have assumed that the Lagrange multipliers are fixed, we can decompose this summation into two parts with positive and negative coefficients. Then, the summation with negative multipliers becomes a convex function, and the summation with positive multipliers becomes a concave function. As a result, we have the following relaxed subproblem:

$$\begin{aligned}(P_2') \ : \max_{\mathbf{X}\in Q^T} \ \Omega'_{P_2}(\mathbf{X}) &= -pg(\mathbf{X}) \\ &+ \sum_{t\in\mathcal{T}}\sum_{i\in\mathcal{I}_t^+} \omega_{5,it} \log_{10}\left(\|x_t - y_{it}\|\right) \\ &+ \sum_{t\in\mathcal{T}}\sum_{i\in\mathcal{I}_t^-} \omega_{5,it} \log_{10}\left(\|x_t - y_{it}\|\right),\end{aligned} \tag{13}$$

where $\mathcal{I}_t^+ = \{i : \omega_{5,it} \geq 0\}$ and $\mathcal{I}_t^- = \{i : \omega_{5,it} < 0\}$. Note that ($P_2'$) can be solved by so-called difference of convex (DC) programming optimization techniques (Horst and Thoai 1999), which is a heuristic method and described later in the text.

The summary of the LDA to solve ($3MCLP_{LDA}$) is given in Algorithm 1. We initialize all Lagrange multipliers and run the algorithm for at most K iterations. At each iteration $k \geq 0$, $\mathbf{Z}_k$, $\mathbf{S}_k$, and $\mathbf{X}_k$ variables are determined. The objective function values of the LR problem and the original problem are calculated with these values. Whenever this gap is closed, the algorithm is terminated. Otherwise, the Lagrange multipliers are updated according to the subgradient technique described by Fisher (2004).

In this technique, each multiplier is updated by adding some values to its current value. This value is calculated as a multiplication of two other values. The first one is the subgradient of the corresponding multiplier according to the objective function of the relaxed problem that is normalized with respect to all three subgradient values of Lagrangean multipliers. The second variable is a nonnegative real number, the step size, which should diminish throughout the algorithm. We use a common procedure from the literature to update the step size, in which the step size is initialized as a real number between zero and two and then halved whenever the objective function value of the relaxed problem is not improved after a fixed number of iterations. After updating the multipliers, we recursively apply the same steps to find new $\mathbf{Z}_k$, $\mathbf{S}_k$, and $\mathbf{X}_k$ values.

As we solve a relaxed version of the second subproblem in each iteration, the algorithm may not converge;

thus, we set a maximum iteration limit K. The algorithm terminates after K iterations with an upper bound, Ω_{UB}, which is the minimum objective function value among all objective function values attained after solving LR problems, to the original problem unless the optimal solution is found.

Algorithm 1 (LDA)

Input: Q, $\mathbf{Y}$, $\mathbf{D}$, $\mathbf{w}$, K.
Initialization: Arbitrarily initialize $\lambda_{it}, \vartheta_{it}, \delta_{it}, \forall i \in \mathcal{I}$, $\forall t \in \mathcal{T}$, $k \leftarrow 0$, $\Omega_{UB} \leftarrow +\infty$.
1: **While** $k < K$, **do**
2: Find $\mathbf{Z}_k$ and $\mathbf{S}_k$ variables according to Table 1 with given $\lambda_{it}, \vartheta_{it}, \delta_{it}$.
3: Set $\mathbf{X}_k \leftarrow \arg\max_{\mathbf{X} \in Q^T} \Omega'_{P_2}(\mathbf{X})$ using given $\lambda_{it}, \vartheta_{it}, \delta_{it}$.
4: **If** $\Omega_{LR}(\mathbf{X}_k, \mathbf{Z}_k, \mathbf{S}_k, \{\lambda_{it}\}, \{\vartheta_{it}\}, \{\delta_{it}\}) < \Omega_{UB}$, **then**
5: $\Omega_{UB} \leftarrow \Omega_{LR}(\mathbf{X}_k, \mathbf{Z}_k, \mathbf{S}_k, \{\lambda_{it}\}, \{\vartheta_{it}\}, \{\delta_{it}\})$
6: **End if**
7: **If** $\Omega_{UB} = \Omega(\mathbf{X}_k)$, **then**
8: **break**
9: **Else**
10: Update $\lambda_{it}, \vartheta_{it}, \delta_{it}$ by using the subgradient technique.
11: **End if**
12: $k \leftarrow k + 1$.
13: **End while**
14: **Return** Ω_{UB}.

The complexity of the LDA depends on two main operations at each iteration. The first operation includes simple algebraic operations to find $\mathbf{Z}$ and $\mathbf{S}$ variables, which take $O(nT)$ time. The second operation includes finding $\mathbf{X}$ variables by applying the DC programming algorithm (DCA) defined by Horst and Thoai (1999). In the DCA, the objective is to create two sequences of variables so that the first sequence of variables converges to the local optimum of the primal problem and the second sequence of variables converges to the local optimum of the dual problem. The key point is that the symmetry between the primal and dual problems would follow a variation on the classical subgradient technique used in convex optimization.

The DCA starts with an arbitrary feasible solution. At each iteration of the algorithm, first, a solution is found within the subgradient domain of the first concave function. Then, by using this solution, a new solution is found within the subgradient domain of the second concave function. It is shown that recursively applying these two steps converges to a local maximum after a finite number of iterations. Indeed, the DCA has two gradient calculation steps at each iteration; thus, the complexity of a single iteration is $O(T)$. Assuming that the maximum number of iterations of DCA is set to $\mathcal{K}$, the worst-case complexity of DCA is $O(\mathcal{K}T)$. Eventually, the worst-case complexity of the LDA is $O(KnT + \mathcal{K}T)$.

Note that any $\mathbf{X}_k$ found during the LDA can be used to find a lower bound for the original problem by inputting $\mathbf{X}_k$ into Equation (4). As a side benefit of the LDA, we can track the optimality gap of a problem while running the algorithm. In particular, we store a list that keeps all $\Omega_D(\mathbf{X}_k)$ values and report the maximum of these values as the lower bound to the original problem, Ω^*_{LB}. In fact, the performance of LDA depends significantly on the gap between the best upper and lower bounds attained after each iteration. We illustrate how this gap changes with parameters in the computational study in Section 6.

Although our formulation and the LDA are significant improvements for a nonconvex constrained optimization problem, the worst-case complexity is still exponential because of the NP-hardness of the underlying problem. Therefore, in the next section, we propose a CA approach to overcome these challenges.

5. Continuum Approximation

The computational burden of discrete formulation increases sharply as the numbers of users and time intervals increase. CA could be a remedy to mitigate this challenge because of fewer data requirements and closed- or near-closed-form solutions. The CA approach was first proposed by Newell (1971) to find optimal dispatching times of public buses and refined by Dasci and Verter (2001), Ouyang and Daganzo (2006), Cui et al. (2010), Daganzo (2010), and Wang and Ouyang (2013) for several static facility location problems. Recently, Wang et al. (2017) applied the CA approach to a dynamic facility location problem, in which facilities are not allowed to be closed after opening and demand is assumed to grow throughout a finite horizon. However, our approach has no assumption on user demand. Also the service area of the facilities is in 3-D space in our case. A recent survey on CA can be found in Ansari et al. (2018).

To the best of our knowledge, none of the CA studies incorporate the vertical dimension into the dynamic problem formulation. Moreover, the problems studied to date are related to facility location problems, in which the objective is typically the minimization of the total cost of the facilities to serve all customers in a finite region. Our approach differs from the existing studies in terms of three pillars. First, we relax the static assumption on facilities and develop a CA model for a problem in which a facility is allowed to move in spatiotemporal continuum without opening/closing decisions. Second, the demand (MSLT in our context) is not necessarily growing in time but randomly determined by each user. Third, we develop a discretization procedure in the 3-D space to find the exact location sequence of a single facility instead of determining a discrete 2-D facility location plan to cover all demand.

In a CA model, all parameters are assumed to be continuous over the service area. For our problem, user weights, $\mathbf{w} \in [0,1]^n$, and demands, $\mathbf{D} \in \mathbb{R}^{n\times T}$, are approximated with continuous functions in spatio-temporal continuum, that is, w_{it} and d_{it} are approximated by $w(y,t) : S \times \mathbb{R} \to \mathbb{R}$ and $d(y,t) : S \times \mathbb{R} \to \mathbb{R}$. All these spatial attributes are assumed to vary continuously and slowly in y and t. To derive the solutions to the CA model, we first analyze the homogeneous case in which all parameters are assumed to be constant over an infinite service area, $\mathbb{R}^2$, and infinite horizon, $\mathbb{R}$, and then use these results as building blocks to extend the derivations for more general cases.

5.1. Homogeneous Case

In a homogeneous scenario, we assume that $\mathbf{w}$ and $\mathbf{D}$ values are constant for all $y \in \mathbb{R}^2$ and $t \in \mathbb{R}$, that is, $w(y,t) = w$, $d(y,t) = d$. Clearly, there is no need to move the UAV-BS in this case as nothing changes for the entire time horizon. Recall that the signal loss of a user depend on two distances, horizontal and vertical. Fixing the vertical distance yields the following useful result:

Proposition 1. *In an infinite homogeneous plane, the optimal covering area should form a regular disc when the UAV-BS altitude is fixed.*

Proof. Suppose that the UAV-BS altitude is fixed to h. Then, the coverage level at any point in S can only change with respect to the horizontal distance, r. Starting from an arbitrary point in S, increasing r values causes an increase in the overall distance between the user and the UAV-BS and a decrease in the angle. Both of these have negative impacts on signal loss. Therefore, signal loss monotonically increases with r. As d is constant everywhere, the coverage eventually drops down to zero after a particular r value independent of the direction because the signal loss eventually exceeds d. Therefore, the optimal service area should be a disc with radius r within which signal loss is less than or equal to d and the UAV-BS is located at the center. □

With Proposition 1, we can argue that the maximum coverage area around any location $y \in S$ can be found where the average signal loss is equal to d in this area. Let $A(y,h)$ denote the size of this covering area around location $y \in S$ when the altitude of the UAV-BS is h. $A(y,h)$ can be considered as another approximation used to define the service area of the UAV-BS in CA, which is defined as a subset of users in the discrete formulation by $C_t \subseteq I$. Because all parameters are constant everywhere for the homogeneous case, the coverage area should be equal at every location, that is, $A(y,h) = A(h)$, $\forall y \in S$.

One key point in the CA technique is to find an approximation for the cost of service with respect to the service area so that any parameter can be expressed as a local property of point $y \in S$. For instance, Daganzo (2010) proposes a total cost formula for a network in which a terminal is used to transfer products from a depot to end users. In their proposed model, outbound and inbound costs are aggregated to determine the total cost. The outbound cost is approximated with respect to estimated unit demand, and the inbound cost is approximated with respect to the average delivery distance within the service area.

In our context, instead of the total cost, we need to approximate the average signal loss in the covering area. As the exact loss value is not known in this area, we approximate it by assuming the loss value to increase at the same rate as the average horizontal distance increases within the covering area. Ouyang and Daganzo (2006) show that the average distance in a disc with area A can be approximated as $2/(3\sqrt{\pi}) \times \sqrt{A}$. We use the same rate to approximate the signal loss increment within the covering area with size $A(h)$.

Around any location $y \in S$, the signal loss is assumed to be initialized with the loss value determined as if the UAV-BS is located just atop the point. Let $\overline{L}(h)$ denote the initial signal loss value when the UAV-BS altitude is h over the point $y \in S$. When enlarging the covering area around this point, the average signal loss within this area is approximated as $\overline{L}(h)(2/3)\sqrt{A(h)/\pi}$. Note that this approximation can be considered as a substitute of the L function in the discrete formulation. Instead of using exact user location, we use the service area to approximate average signal loss.

To approximate the total weight in the service area, on the other hand, we use the same approach as Daganzo (2010) has used. That is, the total weight in the covering area is assumed to linearly increase with the size of the area, that is, $w \cdot A(h)$. As a result, the CA model can be stated as follows:

$$\text{3MCLP}_{\text{C}} : \max_{h, A(h)\geq 0} \ \Omega_{\text{C}}(h, A(h)) = wA(h)\left(\frac{d - \overline{L}(h)(2/3)\sqrt{A(h)/\pi}}{d - L^{-}}\right), \quad (14)$$

where $\overline{L}(h) = F + 10\eta \log_{10}(h) + B$. Note that, for any 3MCLP problem in $\mathbb{R}^3$, this approach can be extended with different $\overline{L}(h)$ functions.

We have no discrete variable in the CA model; hence, this model is expected to be solved more easily than the discrete formulation. The following is a comparison of discrete and continuous problems. In the homogeneous case, the discrete formulation can be reduced to a single-period problem without considering the relocation penalty, and subscript t can be removed. In such a case, each user can be tracked with

$z_i \in \{0,1\}$ and $s_i \in \mathbb{R}$ variables instead of z_{it} and s_{it}. z_i is equal to one if the demand of the corresponding user is satisfied and zero otherwise, and s_i is equal to the signal loss value when $z_i = 1$ and zero otherwise. Because we remove the t subscripts, the UAV-BS location can also be determined with a single vector, $x \in Q$. As a result, the discrete formulation would have $2n + 3$ variables and $6n$ constraints. On the other hand, the CA model has only two variables, h and $A(h)$. Therefore, the computational performance of the CA model would be promising.

In (14), the total weight in the service area is approximated as $wA(h)$, and the average signal loss is approximated as $\overline{L}(h)(2/3)\sqrt{A(h)/\pi}$ because the average distance within a hypothetical circular service area of size A can be approximated as $(2/3)\sqrt{A/\pi}$ (Ouyang and Daganzo 2006). For fixed h, this problem can be solved by the first-order condition of Ω_C, that is, $\nabla\Omega_C(h,A) = 0$, because Ω_C is concave as shown in the following lemma.

Lemma 1. *Ω_C is concave.*

Proof. To show the concavity of Ω_C, we use the Hessian matrix of the function. After a number of algebraic operations, the Hessian of Ω_C can be found as

$$\nabla''\Omega_C(h,A) = \begin{bmatrix} -\frac{20\eta w A^{3/2}}{3\log(10)\sqrt{\pi}(d-L^-)h^2} & -\frac{10\eta w A^{1/2}}{\log(10)\sqrt{\pi}(d-L^-)h} \\ -\frac{10\eta w A^{1/2}}{\log(10)\sqrt{\pi}(d-L^-)h} & -\frac{(F+B)wA^{-1/2}}{2\sqrt{\pi}(d-L^-)} - \frac{5\eta w A^{-1/2}\log_{10}(h)}{\sqrt{\pi}(d-L^-)} \end{bmatrix}. \tag{15}$$

Note that we assume that $d \geq L^-$ and $w, \eta \geq 0$. Therefore, the first order principal minor of the preceding matrix is obviously negative. The second order principal minor is positive when $h \geq 10^{3/\log^2(10)} \approx 3.68$. As the minimum altitude allowed to hover UAVs typically starts from around 50 meters, this value is sufficiently small to assume that this principal minor is positive. Therefore, we have a negative semidefinite Hessian matrix, which yields that Ω_C is concave. □

As a result, the optimal service area at altitude h denoted by $A^*(h)$ and the corresponding total covered weight denoted by $\Omega_C(h, A^*(h))$ can be found as follows:

$$A^*(h) = \pi\left(\frac{d}{\overline{L}(h)}\right)^2, \tag{16}$$

$$\Omega_C\big(h, A^*(h)\big) = \frac{\pi w d^3}{3(d-L^-)\overline{L}^2(h)}. \tag{17}$$

Note that (17) can be maximized when $\overline{L}(h)$ takes its minimum value. Because $\overline{L}(h)$ is a monotonically increasing function in h, for the homogeneous case, the optimal service area can be achieved when h takes its minimum value in Q. As a result, the optimal coverage for the homogeneous case can be achieved at any location at this minimum altitude with the optimal service area found by Equation (16).

5.2. Heterogeneous Case

To solve the heterogeneous case, we first consider the case in which the UAV-BS movement is ignored. Then, we propose a regularization algorithm to improve the relaxed solution by iteratively shifting the UAV-BS locations at different time intervals with repulsive forces.

When the UAV-BS position is assumed to be fixed, the problem can be considered independently for each interval. Recall that w and d are allowed to change in this case. Wang et al. (2017) show that the parameters can be approximated with a level-based approach in the temporal continuum for dynamic problem setup without sacrificing solution accuracy. In particular, the service area within a specific time interval is approximated by using the values at the median of the interval, and then, the solution found according to these values is assumed to be valid for the entire interval.

We use a similar approach and solve all subproblems in which w and d values are assumed to be equal to their values at the median of an interval. Because, in our context, all intervals are assumed to be identical in duration, we can determine the median of each interval beforehand. Let τ_t denote the median of the interval and t and γ denote the length of each interval. Then, we can use $w(y,\tau_t)$ and $d(y,\tau_t)$ as approximations of parameters at each location $y \in S$ in interval t.

Because w and d are assumed to be slow varying functions over S, the service area $A(y,h)$ should also vary slowly. Let $\varphi_t(y, A(y,h))$ denote the covered weight around $y \in S$ when the service area is $A(y,h)$ at a fixed altitude h in interval t. Then, the following problem can be solved for each $y \in S$ for each interval t.

$$\max_{h, A(y,h)\geq 0} \ \varphi_t\big(y, A(y,h)\big) = w(y,\tau_t)A(y,h)\frac{d(y,\tau_t) - \overline{L}(h)(2/3)\sqrt{A(y,h)/\pi}}{d(y,\tau_t) - L^-}. \tag{18}$$

A similar analysis to compare this model with the discrete model can be provided as we present in the homogeneous case. In a heterogeneous case, the discrete model has $T(2n+3)$ variables and $6nT$ constraints. Because all parameters are assumed to be slow varying in S in the CA model, it can be solved point by point without a significant loss in the objective function. Suppose that S is represented by $\bar{s}$ different points, which are evenly distributed with

small distances from each other. Then, for each point, the optimal solution can be found by (16) and (17). As a result, the optimal solution for fixed h can be found with $2\overline{s}\log\overline{s}$ algebraic operations, which can computationally outperform solving an MINLP as in the discrete model.

Equation (18) can be solved with a similar approach introduced in the homogeneous case for any $y \in S$. The only difference is replacing constant parameters with the local parameters for each location y. Let $\varphi_t^*(y, A^*(y,h))$ denote the optimal objective function value associated with each y for fixed h. Then, the location y that yields the maximum $\varphi_t^*(y, A^*(y,h))$ can be considered as the footprint of the UAV-BS on S for interval t if the UAV-BS altitude is h. Let $y_t^*(h) \in S$ denote this footprint in interval t; then, an interval bracket search algorithm (e.g., bisection search) can be used to find the best h that maximizes $\varphi_t^*(y_t^*(h), A^*(y_t^*(h),h))$ over the domain of h in Q. We use h_t^* to denote this altitude for interval t. Then, the UAV-BS location and the total covered weight in interval t are determined as $x_t^* = (y_t^*, h_t^*)$ and $\Omega_t^* := \gamma\varphi_t(y_t^*, A^*(y_t^*, h_t^*))$, respectively. As a result, the total coverage of CA denoted by $\overline{\Omega}_C$ can be found as $\overline{\Omega}_C = \sum_{t\in T}\Omega_t^*$.

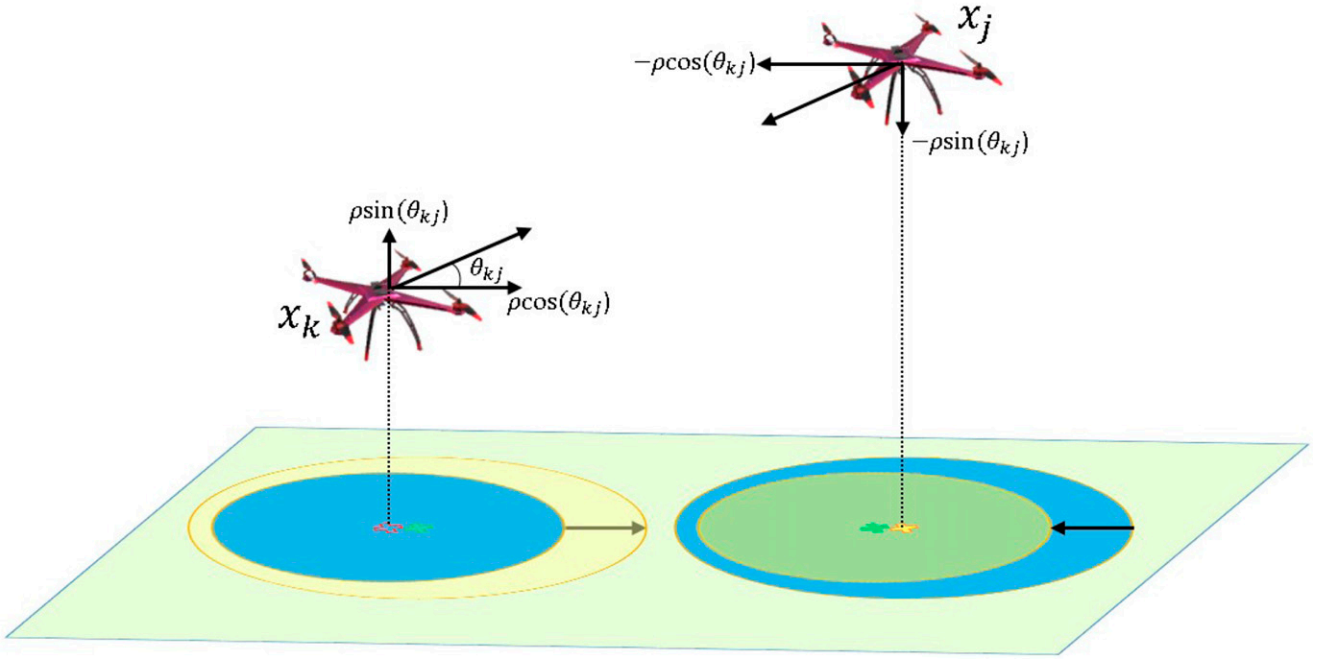
As we relax the UAV-BS movement, this procedure may result in a dispersed location sequence in Q. To alleviate such a result, we propose a regularization algorithm, in which we search the neighborhood of the solution that this procedure yields. Let $\{x_t^*\}$ and $\Omega^* = \Omega_D(\mathbf{X}^*)$ denote the location sequence attained from the CA procedure and the true objective function value of this sequence, respectively. In the regularization algorithm, given a location sequence, at each iteration, we either select the two consecutive intervals that have the highest relocation distance with probability $\mathcal{P}$ or randomly select two intervals with probability $1-\mathcal{P}$. $\mathcal{P}$ can be interpreted as the probability of exploration in searching the neighborhood.

After selecting the intervals, the UAV-BS location in these intervals is moved closer with a step size ρ, and this movement is accepted if the objective function value is improved. Note that we use the true objective function, $\Omega_D(\cdot)$, defined in the original problem. If the objective function value improves because of this movement, then the incumbent objective function value and the location sequence is updated.

The regularization for two arbitrary intervals, k and j, is illustrated in Figure 3. The discs depict the change in these areas in each interval. Empty and filled cross symbols depict the footprint of the UAV-BS before and after regularization, respectively. The angle between selected locations, θ_{kj}, is used to determine horizontal and vertical repulsive forces to move the UAV-BS.

Note that lifting the altitude results in a larger service area, whereas lowering the altitude results in a smaller service area. We use such regularization based on the properties of $L(\cdot)$ to find a balance between improved angle versus worsened distance. Lifting the altitude increases the average distance to the UAV-BS. Thus, the service area is widened to boost the positive impact of increased average angle and vice versa. The rate of shrinkage or widening is applied as the rate that the altitude changes. For instance, lifting the UAV-BS from 200 m to 205 m yields

Figure 3. (Color online) Illustration of Regularization



widening the service area from A to $(1 + 5/200)A$, and moving down from 200 m to 195 m yields shrinking the service area from A to $(1 - 5/200)A$. Moreover, a second force with the same step size, ρ, is applied whenever the service area exceeds S in the opposite direction of the violation.

Note that overlapping is allowed in our problem, which is not the case in the classic 2-D CA algorithms, because our goal is to determine the locations for a single facility at different time intervals. Our algorithm terminates after K consecutive iterations without improvement in the objective function value. The step size, ρ, is updated after each l iterations to a smaller step size by multiplying with a scalar, $\varrho \in [0, 1]$, to avoid aggressive movements in later stages of the algorithm. The CA algorithm is summarized in Algorithm 2.

Algorithm 2 (CA Algorithm)

Input: K, l, $\mathcal{P}$, ρ, ϱ.
Initialization: Determine $\{x_t\}$ by solving (18), and set $\{x_t^*\}$ to these locations. Set Ω^* as the objective function value with respect to $\{x_t^*\}$.
$k \leftarrow 0$, $i \leftarrow 0$, $\rho_k \leftarrow \rho$.
While $k < K$, **do**
  If $i == l$, **then**
    $\rho_k \leftarrow \rho_k \times \varrho$, $i \leftarrow 0$
  End if
  Select two intervals t_1 and t_2 and move x_{t_1} and x_{t_2} by ρ_k closer to each other. Set Ω to the objective function value of this change.
  If $\Omega > \Omega^*$, **then**
    Set $\Omega^* \leftarrow \Omega$ and $k \leftarrow 0$. Update $\{x_t^*\}$ by replacing x_{t_1} and x_{t_2} with their new locations.
  Else
    $k \leftarrow k + 1$
  End if
  $i \leftarrow i + 1$
End while
Return Ω^*, $\{x_t^*\}$.

6. Computational Results

In this section, we provide an extensive computational study to compare the LDA and CA algorithm performances with the optimal solutions and provide insights on how heterogeneity in parameters affects the CA performance. Becasue there is no publicly available data set for 3MCLP, we first generate a synthetic data set and then present the comparison. All the codes are available online at https://github.com/cihantugrulcicek/3DMCLP.git.

6.1. Data Generation

The performance of the proposed algorithms is tested through a series of 3MCLP instances with synthetic data. All instances are generated within $Q = [(0,1500) \times (0,1500) \times (50,500)]$ and $S = [(0,1500) \times (0,1500)]$. The parameters related to the signal loss function are adopted from Al-Hourani et al. (2014) for a suburban environment that has $\eta = 2$, $\alpha = 4.88$, $\beta = 0.43$, $\phi_{\text{LoS}} = 0.1$, $\phi_{\text{NLoS}} = 21$, and $f = 2e9$.

For the CA model, the weight and the MSLT density functions are defined as $w(y,t) = \overline{w}[1 + \Delta_w \cos(\pi\|y\|)]v(t)$ and $d(y,t) = \overline{d}[1 + \Delta_d \cos(\pi\|y\|)]v(t)$, respectively. Here, $\overline{w}$ and $\overline{d}$ control the average values of weight and MSLT density, respectively, and Δ_w and Δ_d control the spatial variability. $v(t)$ is a function of $t \in [0, T]$ that controls the temporal variability. We consider three different functions to see how the different behavior of users in time affect the performance. The first function assumes that the parameter values increase in time, that is, $v(t) = 1 + \log(1 + \Delta_t t)$, and the second function assumes the values decrease in time, that is, $v = \exp(-\Delta_t t)$, where Δ_t controls the degree of trend in time. The third function assumes that the parameters follow a random behavior with a step function, $v(t) = 1 + \Delta_t \psi_k t$ for $t \in [\tau_{k-1}, \tau_k)$, where ψ_k is a uniform random variable taking values in $[-1, 1]$, which is regenerated for each interval k. Note that the system becomes homogeneous when $\Delta_w = \Delta_d = \Delta_t = 0$. The relocation penalty value is determined with respect to average weight per area per time in an instance, that is, $p = (1 + \Delta_p)/T|S| \int_T \int_S w(y,t)dydt$, where $|S|$ denotes the overall area and Δ_p controls the variation from average weight.

For the discrete problems, we adopt a similar approach proposed in Wang et al. (2017). S is divided into s spatial grid cells, and a uniform random point in each cell is used to represent a user location. $\mathcal{T}$ is divided into 10 equal intervals. The weight of each discrete user location is aggregated within each cell for each interval at this random location, and the MSLT value is determined as the average MSLT value inside the cell. Because the original problem is nonconvex, we use a nonconvex commercial solver, BARON, available in the NEOS Server (Czyzyk et al. 1998) to solve discrete problems. The maximum CPU time is set to four hours, and all other solver parameters remain as default. We also set the same time limit for the LDA because the DC programming step of the algorithm can require an excessive solution time for some instances.

Both the LDA and CA algorithms are coded in Python, and a preliminary test has been applied for fine-tuning the parameters used in the CA algorithm. Nine different instances have been generated in which $\overline{w} = 0.5$, $\overline{d} = 105$, and $\Delta_w = \Delta_d = \Delta_p = \Delta_t = 0.2$ for all different trends of the weight and demand functions. We draw l, $\mathcal{P}$, ρ, and ϱ values from $l \in \{50, 100, 150\}$, $\mathcal{P} \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, $\rho \in \{10, 20, 30\}$, and $\varrho \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$ and solve all instances by all possible parameter combinations (405 replications in total).

We observe that l, ρ, and ϱ are dominant parameters in terms of CPU times as these parameters are dominant in determining the step size to relocate the UAV-BS. The replications in which l and ρ take the minimum and ϱ takes the maximum among their alternative values have the longest CPU times and vice versa. However, the improvement in the objective function value is not as precise as it is in CPU time. The average improvement in the objective function value is 0.91%, and the CPU time is almost four times longer than the average CPU time of all replications. This result is based on the fact that, although decreasing the step size with smaller rates may prevent aggressive relocation of the UAV-BS, the algorithm eventually converges to similar solutions. On the other hand, higher $\mathcal{P}$ values yield a 0.84% improvement on average in the objective function values with no additional CPU time. Therefore, we opt to set l, ρ, and ϱ parameters to their average values and $\mathcal{P}$ to the highest value, that is $l = 100$, $\rho = 20$, $\varrho = 0.5$, $\mathcal{P} = 0.9$, for reasonable CPU time and for the sake of better precision.

6.2. Simulation Results

The LDA and CA algorithms are run with an Intel i-5@ 3.20 GHz processor and 8 GB RAM under a Windows 10 operating system. Both the LDA and CA algorithms are set to run for either 1,000 iterations or four hours at most. Instances are generated according to different temporal behavior of users, and only a single function differs for each parameter at a time. As a result, we analyze nine different scenarios in which each parameter takes increasing, decreasing, and random values in time. Ten replications are generated for each scenario with $\overline{w} = 0.5$, $\overline{d} = 105$, and $\Delta_w = \Delta_d = \Delta_p = \Delta_t = 0.2$. The optimality gap and CPU times for BARON, LDA, and CA algorithms are presented in Table 2 for different s values.

Because the CA algorithm includes random parameters, all scenarios are replicated 10 times, and the average and maximum results obtained from all replications are presented. We use the maximum of the best objective function values found by BARON and the maximum of lower bounds found throughout the LDA as the final lower bound and the minimum of upper bound values found by BARON and LDA as the final upper bound while calculating the optimality gaps of the instances.

Table 2 shows that BARON and LDA performances are in close proximity to each other when the number of users is fewer, and both of them suffer from an increase in the number of users and fail to find optimal solutions. BARON finds the optimal solutions for eight instances with 20 users, whereas the LDA finds the optimal solutions to three of the same eight instances. The average optimality gap of the LDA for the remaining five instances is 0.90%. The average optimality gap of BARON for the remaining 19 instances in which the optimal is not found is 4.98%, and the LDA results in an average gap of 4.71% for the same instances. We can conclude that the performance of the LDA increases with increasing grid

Table 2. Simulation Results

		d											
		Increase				Decrease				Random			
		Gap, %		CPU, s		Gap, %		CPU, s		Gap, %		CPU, s	
w	s	BRN	LDA	BRN	LDA	BRN	LDA	BRN	LDA	BRN	LDA	BRN	LDA
Increase													
	20	—	—	4335	7637	—	—	5386	7695	—	1.34	5698	TL
	50	5.75	3.59	TL	TL	2.33	3.25	TL	TL	4.82	3.65	TL	TL
	100	4.38	7.17	TL	TL	4.55	4.76	TL	TL	5.17	4.29	TL	TL
	CA	(7.18, 8.09)		(66, 78)		(5.10, 6.72)		(41, 65)		(8.27, 8.89)		(58, 74)	
Decrease													
	20	—	—	6181	7473	—	1.61	4913	TL	—	1.98	4483	TL
	50	5.90	2.18	TL	TL	5.06	4.11	TL	TL	4.97	3.98	TL	TL
	100	7.99	5.12	TL	TL	7.07	6.95	TL	TL	4.52	7.04	TL	TL
	CA	(8.20, 9.27)		(61, 68)		(7.61, 8.12)		(29, 67)		(9.85, 10.58)		(81, 91)	
Random													
	20	—	1.26	4207	TL	—	1.00	5566	TL	1.63	1.72	TL	TL
	50	3.14	2.27	TL	TL	5.69	5.43	TL	TL	4.21	4.94	TL	TL
	100	4.86	4.89	TL	TL	7.75	7.14	TL	TL	4.89	7.12	TL	TL
	CA	(8.87, 9.56)		(35, 61)		(9.18, 10.30)		(51, 61)		(11.15, 12.71)		(60, 82)	

Notes. The gap and CPU values in CA rows represent the average and maximum values within 10 replications. BRN = BARON, TL = time limit.

Table 3. New Optimality Gap Values of BARON Solutions After Providing CA Output as Initial Solution

		20		*s* 50		100	
w	*d*	GAP	Improvement	GAP	Improvement	GAP	Improvement
Increase	Increase	—	—	4.23	26.35	1.89	56.88
	Decrease	—	—	2.97	49.66	5.47	31.53
	Random	—	—	2.24	28.75	3.78	22.12
Decrease	Increase	—	—	2.10	9.83	3.05	32.93
	Decrease	—	—	3.88	23.34	5.68	19.69
	Random	—	—	3.67	35.63	5.17	33.32
Random	Increase	—	—	2.70	44.00	2.20	57.44
	Decrease	—	—	2.36	52.52	2.30	49.08
	Random	1.21	25.78	2.95	29.87	1.98	59.52

Note. All values are given in percentage.

numbers when compared with BARON. The CA algorithm outperforms both BARON and LDA in terms of CPU times by solving almost all instances in less than a minute. The average of maximum optimality gaps of 10 replications is 9.36%, and the average optimality gap is 8.38% for the CA algorithm.

Figure 4. (Color online) Change in the Gap Values During LDA Iterations

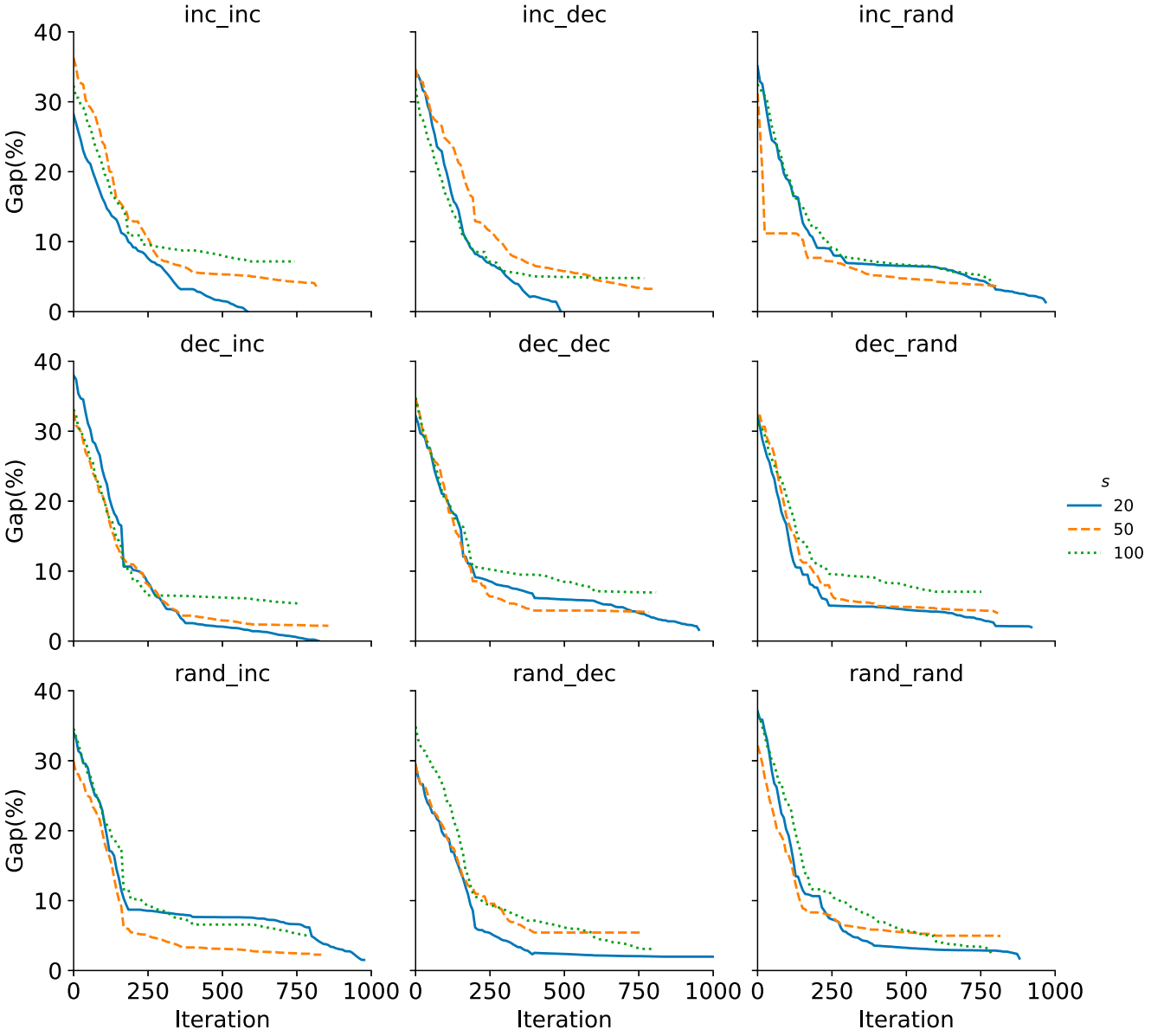
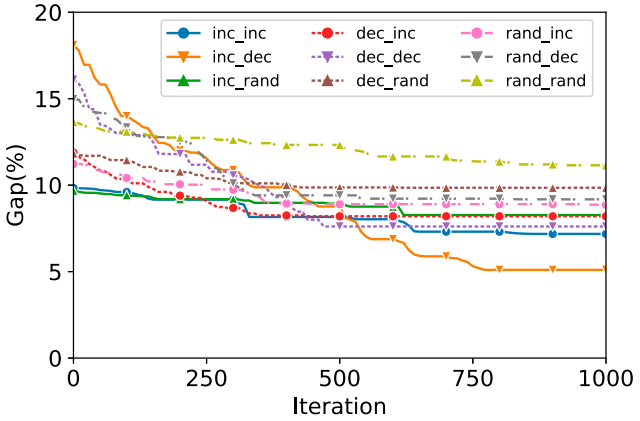


Figure 5. (Color online) Change in the Gap Values During CA Iterations



What is surprising in Table 2 is that the CA algorithm performance worsens for instances in which the weight and the MSLT densities follow a random behavior in time. The optimality gap values of instances in which at least one parameter is randomly determined in time is 2.44% worse than the instances in which both parameters have monotone behavior. This outcome can be explained as follows: in instances with monotone trends in parameters, the UAV-BS is expected to move less compared with random instances. For example, if the MSLT at a location at the beginning is relatively smaller, it follows the same trend until the end of the horizon when the temporal variability is monotone in time. Therefore, optimal service areas are expected to be around the same locations in different time intervals. On the other hand, for instances with random behavior of parameters, there is no obvious trend such as we have under the monotone scenarios. Therefore, the optimal locations are likely to vary, and the total movement is expected to be higher. As we relax the relocation penalty in the CA model, the CA algorithm is likely to suffer from this relaxation and results in suboptimal solutions.

The CPU performance of the CA algorithm motivates us to use this algorithm as an initial solution generator for the exact solution procedures. To see how it may improve the solution accuracy, we resolve each instance with BARON, in which the best solution found by the CA algorithm is provided as an initial solution. Table 3 shows the improvement in the objective function values after running BARON for four hours. Although none of the instances are solved to optimality, the CA solutions enable BARON to improve its performance by 25.78% on average. This finding shows that the CA algorithm can be an efficient preprocessing tool for large-scale problems.

As mentioned earlier, the LDA performance highly depends on the duality gap observed through the iterations. To show how this gap improves in different instances, the gap values are depicted in Figure 4. The instance names are denoted above each plot by concatenating abbreviated trends of weight and demand parameters, respectively; for example, inc_inc denotes the instance in which both w and d increase over time.

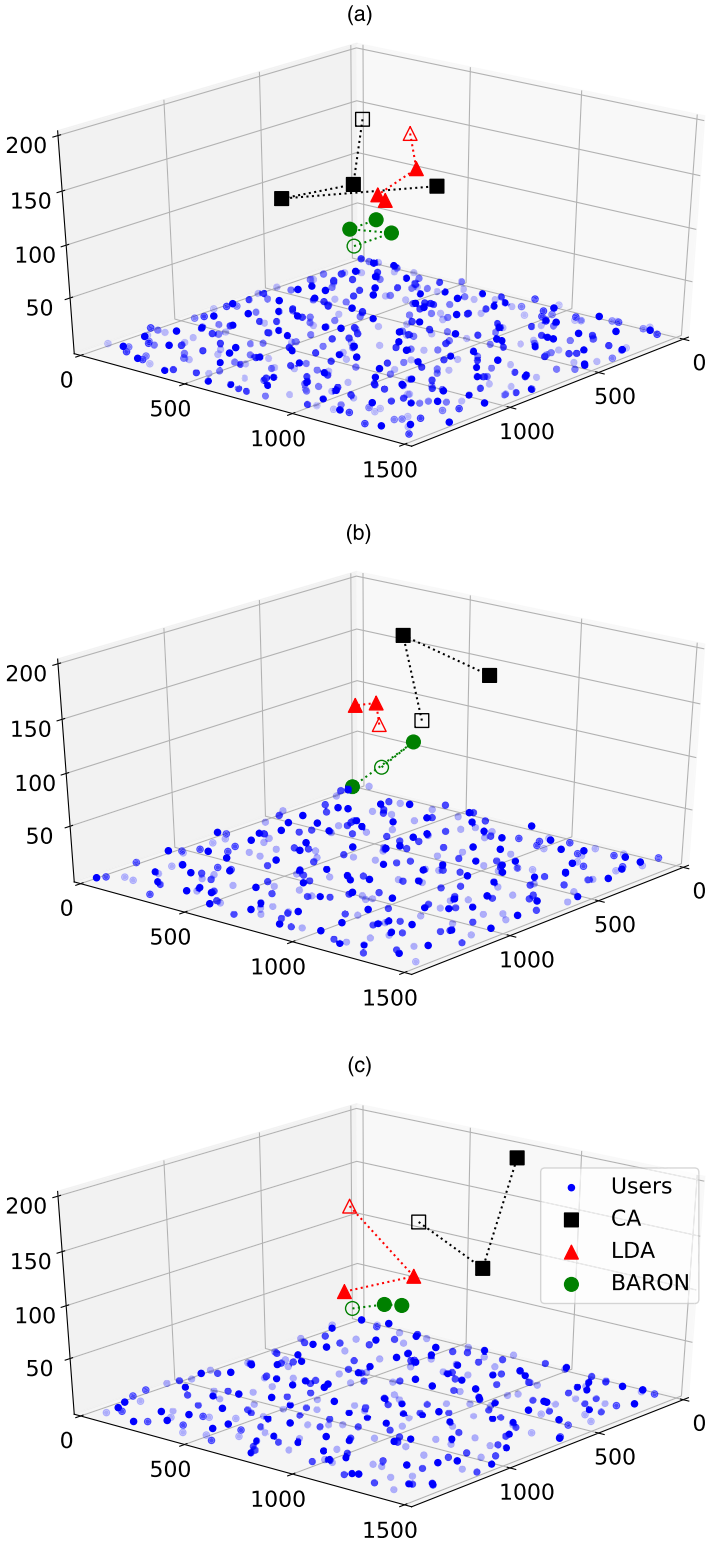
Figure 4 demonstrates how smooth the gap values change through the LDA. The gap values improve after almost each iteration rather than step-wise improvements with long step sizes. As expected, the improvement rate is higher during the first iterations and then decreases toward the termination of the algorithm. The average initial gap value of all instances is 33.18% and decreases to 9.34%, 5.58%, 4.54%, and 3.65% after 25, 50, 75, and 100 iterations, respectively. On average, the LDA terminates after 106, 101, and 97 iterations with the average gap values of 0.99%, 3.71%, and 5.09% for the instances with 20, 50, and 100 grids, respectively. The average gap improvement rates, which are found as the rates of difference between initial and final gaps over the initial gap and averaged over all instances of corresponding grid numbers, are 95.05%, 88.53%, and 84.72% for the instances with 20, 50, and 100 grids, respectively.

Figure 5 shows how the average optimality gap improves during the iterations of the CA algorithm. Each line in this figure illustrates the average gap values and corresponds to a specific instance as explained previously. A significant outcome from this figure is that the CA algorithm keeps improving the gap values until the 755th iteration on average. This confirms that, although the convergence of CA does not seem as smooth as the LDA convergence with a higher number of step-wise decreases, smoothly decreasing step size, ρ, prevents the algorithm from getting stuck in suboptimal location strategies. As a result of such improvements, the CA algorithm performs a 32.44% improvement between the initial and final gap values on average.

Figure 6 illustrates example UAV-BS trajectories found by different solution techniques. An instance with 100 grids in which both w and d decrease is drawn as an example. The trajectories found by BARON, LDA, and CA algorithms are depicted with circle, triangle, and square markers, respectively. Figure 6, (a)–(c), corresponds to the time intervals $1-4$, $5-7$, and $8-10$, respectively. Each trajectory is shown with dashed lines connecting the UAV-BS locations in succeeding intervals in which empty markers denote the starting period of each subfigure. User locations are shown with gradient colors, and the darkest markers correspond to the latest interval of the corresponding subfigure.

Although Figure 6 highlights a single instance, UAV-BS locations follow similar patterns in most of

Figure 6. (Color online) UAV-BS Trajectories for the Instance with 100 Grids and Decreasing Weight and Demand Between the Time Intervals (a) 1 – 4, (b) 5 – 7, and (c) 8 – 10



the instances. BARON and LDA results seem closer, and the CA results slightly differ. Because the CA algorithm attempts to improve a solution in which the relocation is relaxed with a greedy approach, aggressive relocation patterns are likely to be observed. On the other hand, both BARON and LDA prevent aggressive movements although the exact UAV-BS locations are different.

To elaborate on the advantage of allowing the UAV-BS to move vertically, we conduct another numerical analysis. Table 4 presents the summary of UAV-BS movement of each solution method for different behavior of w and d. The first two columns of this table summarize the solutions in terms of total relocated distance, and the last two columns present the number of changes in altitude per interval, in which the numbers in parentheses denote the amount of change. Numbers in the CA rows represent the average and maximum values within 10 replications.

It can be observed that the UAV-BS adjusts its altitude regarding the change in parameters in all methods. The average numbers of such adjustments are 8.3, 8.8, and 9, and the average change in altitude is 39.7, 41.6, and 68.1 meters for BARON, LDA, and CA, respectively. Regarding the total relocation amounts, BARON and LDA obviously have less movement compared with the CA algorithm. The average relocated distances are 1,489, 1,629, and 2,944 for BARON, LDA, and CA, respectively. Moreover, the results confirm that the relocation amounts significantly decrease with an increasing number of grids. Such a result indeed depends on the fact that the distance between points selected in each grid is likely to become smaller with increasing grid numbers. Hence, it is highly likely to have points whose parameter values do not differ significantly from each other. Therefore, the UAV-BS tends to be stable and follows more contiguous trajectories to avoid aggressive movements.

Table 4. Summary of UAV-BS Movement in Terms of Total Distance and Altitude Changes with Respect to Different Solution Approaches

		Relocation, m		Altitude changes	
(w, d)	s	BARON	LDA	BARON	LDA
inc_inc	20	2,105	2,134	8.1(41.9)	9.0(51.6)
	50	1,578	1,323	8.1(42.3)	8.9(38.3)
	100	846	1,255	8.1(28.5)	8.9(34.5)
	CA	(2,674-3,000)		(9.0-9.0)(58.2-68.1)	
inc_dec	20	2,696	2,780	8.1(34.5)	8.8(42.7)
	50	1,438	902	8.2(47.1)	8.7(42.9)
	100	849	762	8.1(49.5)	8.7(47.6)
	CA	(2,770-3,000)		(9.0-9.0)(60.3-69.7)	
inc_rand	20	2,232	2,601	8.1(29.9)	8.8(32.6)
	50	1,396	1,171	8.4(48.1)	8.8(43.4)
	100	819	984	8.4(41.2)	9.0(45.6)
	CA	(2,728-3,208)		(9.0-9.0)(78.8-90.3)	
dec_inc	20	2,168	2,171	8.1(45.5)	8.6(47.9)
	50	1,557	1,841	8.1(30.1)	8.6(33.9)
	100	908	965	8.0(31.8)	8.6(33.4)
	CA	(2,996-4,079)		(9.0-9.0)(61.4-72.2)	
dec_dec	20	2,442	2,784	8.2(33.5)	8.6(37.6)
	50	1,256	959	8.2(41.9)	8.8(37.5)
	100	995	701	8.3(48.2)	8.7(47.4)
	CA	(2,244-3,470)		(9.0-9.0)(58.8-71.0)	
dec_rand	20	2,424	2,861	8.1(25.5)	8.7(28.3)
	50	1,228	1,134	8.3(45.9)	8.8(45.1)
	100	852	772	8.4(42.1)	8.7(40.1)
	CA	(2,786-3,588)		(9.0-9.0)(66.7-83.2)	
rand_inc	20	2,019	2,829	8.2(48.7)	9.0(54.9)
	50	1,137	1,112	8.4(36.1)	9.0(36.6)
	100	802	746	8.5(38.5)	9.0(39.8)
	CA	(2,816-3,083)		(9.0-9.0)(75.5-87.1)	
rand_dec	20	2,235	2,489	8.4(35.5)	9.0(38.1)
	50	1376	1,205	8.4(53.9)	9.0(51.8)
	100	823	1,946	8.4(28.8)	9.0(39.3)
	CA	(2,407-2,473)		(9.0-9.0)(72.4-79.6)	
rand_rand	20	2,271	2,891	8.8(44.6)	9.0(46.3)
	50	1,283	1,301	8.6(41.5)	9.0(42.6)
	100	871	1,353	8.6(35.9)	9.0(43.8)
	CA	(2,540-3,268)		(9.0-9.0)(80.1-93.5)	

In addition to the findings in Table 4 and observations based on the UAV-BS trajectories found for each instance, we evaluate how important it is to let a UAV-BS move in the vertical axes instead of only flying at a single altitude level. For this purpose, we extend the original problem to a new variant in which we enforce the altitude level to remain the same during the entire planning horizon. We keep everything else in the model the same. Hence, a single constraint is added to the original formulation as follows: $h_1 = h_2 = \ldots = h_T$, where h_t denotes the altitude in the interval $t \in \mathcal{T}$. In this way, we impose our model to determine the best single altitude for the same settings with no additional updates on our assumptions.

Because adding this new constraint prevents applying the DC programming in the LDA, we could compare our results with only BARON. We solve the same instances with this new formulation by using BARON with a four-hour time limit and find that allowing the UAV-BS to move in the vertical axis would yield 10.21%, 9.87%, and 9.77% average improvement in the objective function value for 20, 50, and 100 grids, respectively. This result confirms that altitude adjustment of UAV-BSs in such networks would be promising.

To see how our algorithms perform in larger instances, we generate another data set with 200 and 500 grids. As BARON is unable to provide feasible solutions to instances with more than 100 grids within a four-hour time limit, only the performances of the LDA and CA algorithms are reported. Table 5 shows the change in the optimality gap in these larger instances. The table presents the initial and final optimality gap values after four hours reported by the LDA. The first two columns under each grid size demonstrate the gap values obtained by running the natural LDA, and the last two columns demonstrate the gap values of the LDA with the CA solution input as an initial solution. Observe that the average gap values in instances with 200 and 500 grids drop down from 6.92% to 5.72% and from 8.45% to 7.43% within the same CPU time when the CA solution is used, respectively.

Table 5. LDA Performance for Large Instances with CA Output Given as Initial Solution

		s							
		200				500			
		Without CA		With CA		Without CA		With CA	
w	d	Initial	Final	Initial	Final	Initial	Final	Initial	Final
Increase	Increase	33.47	5.98	11.19	5.51	33.40	8.32	11.21	7.57
	Decrease	34.02	6.81	11.42	5.83	35.45	8.52	8.43	7.41
	Random	35.36	7.14	10.59	5.78	34.44	9.09	10.49	7.55
Decrease	Increase	34.10	5.85	9.62	5.37	35.82	8.60	11.11	6.64
	Decrease	31.46	7.61	8.87	6.49	32.67	8.23	11.93	7.68
	Random	27.90	8.02	8.95	5.67	31.90	8.66	11.86	7.71
Random	Increase	26.25	5.65	9.19	5.44	36.78	7.88	10.09	7.10
	Decrease	33.09	6.98	10.96	5.67	37.16	7.92	11.30	7.63
	Random	28.88	8.22	9.32	5.74	32.13	8.85	9.93	7.62

Note. All values are given in percentage.

These results confirm that the CA algorithm also improves the LDA performance as it does in BARON results in Table 3.

To see how heterogeneity affects the CA algorithm's performance, we generate additional series of instances, in which Δ_w, Δ_d, Δ_t, and Δ_p are drawn from $\{0, 0.2, 0.4, 0.6, 0.8\}$. We keep the same $\overline{w}$ and $\overline{d}$ values and change only one of the heterogeneity parameters at a time while fixing all other parameters to zero. Normalized average objective function values with respect to the objective function value of the homogeneous case in which all heterogeneity parameters are set to zero are illustrated in Figure 7.

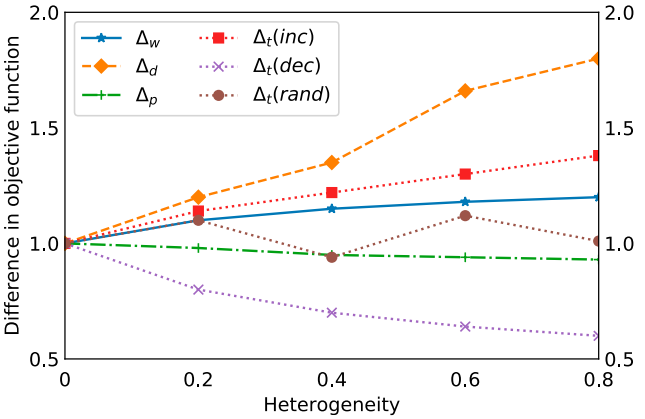
Figure 7 shows that Δ_d and Δ_p have the highest and lowest impacts, respectively. Because the parameters are assumed to vary slowly in the CA model, frequent UAV-BS movement is not an expected policy, thus, the impact of Δ_p on objective function seems negligible. On the other hand, MSLT density is an important factor to determine the service area as shown in Equation (16), thus, high heterogeneity in d has a significant impact on the location decision.

7. Conclusions and Future Work

UAVs have been increasingly utilized in the last decade for various purposes in different industries. In the near future, their adoption and usage are expected to expand significantly, and new problem versions will, most likely, need to be explored. In this study, we defined a 3-D maximal covering location problem to use UAVs as base stations in next-generation wireless communication networks. In particular, the dynamic location of a single UAV-BS that is assumed to have infinite backhaul capacity to serve multiple ground users who are assumed to move inside a finite region within a finite time horizon is determined. We incorporate the vertical dimension into the classic 2-D maximal covering location problems and relax the binary coverage assumption to extend our model to more realistic cases. The problem formulation appeared to be a nonconvex MINLP. Two distinct solution methods are developed to solve this problem. The custom-designed LDA efficiently determines the almost-optimal solutions for medium-sized instances and finds tight bounds for larger instances. The CA algorithm, on the other hand, has the potential to be a valuable tool because of its computational performance by finding efficient solutions in reasonable times. It can also be used as an efficient heuristic to find good initial solutions for exact solution algorithms.

This study can be further extended in several directions. First, it would be interesting to address the set-covering equivalent of this problem, in which the objective is to determine the optimal location policies for multiple UAV-BSs given that users are covered with a minimum service quality threshold. Another possible research direction is to consider a scenario under which the service is required to satisfy certain

Figure 7. (Color online) Impact of Heterogeneity in Parameters



connectivity conditions with no interruption during a specific period. This scenario brings several new constraints that tie consecutive intervals; thus, it is expected to increase complexity substantially. Another important open area is to consider disruption scenarios in which the serving time of a UAV is affected because of several factors, such as weather conditions. These scenarios require a robust approach to design the network; thus, stochastic measures should be considered. A promising area would be studying learning techniques, such as reinforcement learning, to capture the dynamic structure of user locations and demand to decrease the prediction errors.

Acknowledgments

The authors thank the editors and three anonymous referees whose comments and suggestions helped improve and clarify the manuscript significantly.

References

Al-Hourani A, Kandeepan S, Lardner S (2014) Optimal LAP altitude for maximum coverage. *IEEE Wireless Comm. Lett.* 3(6):569–572.

Alzenad M, El-Keyi A, Yanikomeroglu H (2018) 3D placement of an unmanned aerial vehicle base station for maximum coverage of users with different QoS requirements. *IEEE Wireless Comm. Lett.* 7(1):38–41.

Ansari S, Başdere M, Li X, Ouyang Y, Smilowitz K (2018) Advancements in continuous approximation models for logistics and transportation systems: 1996-2016. *Transportation Res. Part B Methodological* 107:229–252.

Badi M, Wensowitch J, Rajan D, Camp JD (2019) Experimental evaluation of antenna polarization and elevation effects on drone communications. *Proc. 22nd Internat. ACM Conf. Model. Anal. Simulation Wireless Mobile Systems* (Association for Computing Machinery, New York), 211–220.

Berman O, Drezner Z, Krass D (2010) Generalized coverage: New developments in covering location models. *Comput. Oper. Res.* 37(10):1675–1687.

Bor-Yaliniz I, Yanikomeroglu H (2016) The new frontier in ran heterogeneity: Multi-tier drone-cells. *IEEE Comm. Magazine* 54(11): 48–55.

Bor-Yaliniz I, El-Keyi A, Yanikomeroglu H (2019) Spatial configuration of agile wireless networks with drone-BSs and user-in-the-loop. *IEEE Trans. Wireless Comm.* 18(2):753–768.

Brotcorne L, Laporte G, Semet F (2003) Ambulance location and relocation models. *Eur. J. Oper. Res.* 147(3):451–463.

Cao X, Yang P, Alzenad M, Xi X, Wu D, Yanikomeroglu H (2018) Airborne communication networks: A survey. *IEEE J. Selected Areas Comm.* 36(9):1907–1926.

Chauhan D, Unnikrishnan A, Figliozzi M (2019) Maximum coverage capacitated facility location problem with range constrained drones. *Transportation Res. Part C Emerging. Tech.* 99:1–18.

Church R, ReVelle C (1974) The maximal covering location problem. *Papers Regional Sci. Assoc.* 32(1):101–118.

Cicek CT, Gultekin H, Tavli B (2019a) The location-allocation problem of drone base stations. *Comput. Oper. Res.* 111:155–176.

Cicek CT, Gultekin H, Tavli B, Yanikomeroglu H (2019b) UAV base station location optimization for next generation wireless networks: Overview and future research directions. *2019 First Internat. Conf. Unmanned Vehicle Systems-Oman* (Institute of Electrical and Electronics Engineers, Washington, DC), 1–6.

Cordeau JF, Furini F, Ljubiċ I (2019) Benders decomposition for very large scale partial set covering and maximal covering location problems. *Eur. J. Oper. Res.* 275(3):882–896.

Cui T, Ouyang Y, Shen ZJM (2010) Reliable facility location design under the risk of disruptions. *Oper. Res.* 58(4):998–1011.

Czyzyk J, Mesnier MP, Moré JJ (1998) The NEOS server. *IEEE J. Comput. Sci. Engrg.* 5(3):68–75.

Daganzo CF (2010) *Logistics Systems Analysis* (Springer, Berlin).

Dasci A, Verter V (2001) A continuous model for production distribution system design. *Eur. J. Oper. Res.* 129(2):287–298.

Di Franco C, Buttazzo G (2015) Energy-aware coverage path planning of UAVs. *Proc. IEEE Internat. Conf. Autonomous Robot Systems Competitions* (Institute of Electrical and Electronics Engineers, Washington, DC), 111–117.

Ebrahimi D, Sharafeddine S, Ho P, Assi C (2020) Autonomous UAV trajectory for localizing ground objects: A reinforcement learning approach. *IEEE Trans. Mobile Comput.* Forthcoming.

Erdelj M, Natalizio E, Chowdhury KR, Akyildiz IF (2017) Help from the sky: Leveraging UAVs for disaster management. *IEEE Pervasive Comput.* 16(1):24–32.

Farahani RZ, Asgari N, Heidari N, Hosseininia M, Goh M (2012) Covering problems in facility location: A review. *Comput. Indust. Engrg.* 62(1):368–407.

Fisher ML (2004) The Lagrangian relaxation method for solving integer programming problems. *Management Sci.* 50(12):1861–1871.

Fotouhi A, Ding M, Hassan M (2016) Dynamic base station repositioning to improve performance of drone small cells. IEEE Globecom Workshops (Institute of Electrical and Electronics Engineers, Washington, DC), 1–6.

Gentili M, Mirchandani P (2012) Locating sensors on traffic networks: Models, challenges and research opportunities. *Transportation Res. Part C Emerging Tech.* 24:227–255.

Ghanavi R, Kalantari E, Sabbaghian M, Yanikomeroglu H, Yongacoglu A (2018) Efficient 3D aerial base station placement considering users mobility by reinforcement learning. *2018 IEEE Wireless Comm. Networking Conf.* (Institute of Electrical and Electronics Engineers, Washington, DC), 1–6.

Gupta L, Jain R, Vaszkun G (2016) Survey of important issues in UAV communication networks. *IEEE Comm. Surveys Tutorials* 18(2): 1123–1152.

Haidari LA, Brown ST, Ferguson M, Bancroft E, Spiker M, Wilcox A, Ambikapathi R, Sampath V, Connor DL, Lee BY (2016) The economic and operational value of using drones to transport vaccines. *Vaccine* 34(34):4062–4067.

Han Y, Liu L, Duan L, Zhang R (2020) Toward reliable UAV swarm communication in D2D-enhanced cellular network Preprint, submitted February 12, https://arxiv.org/abs/2002.04897.

Hayat S, Yanmaz E, Muzaffar R (2016) Survey on unmanned aerial vehicle networks for civil applications: A communications viewpoint. *IEEE Comm. Surveys Tutorials* 18(4):2624–2661.

Horst R, Thoai NV (1999) DC programming: Overview. *J. Optim. Theory Appl.* 103(1):1–43.

Jiang X, Wu Z, Yin Z, Yang W, Yang Z (2019) Trajectory and communication design for UAV-relayed wireless networks. *IEEE Wireless Comm. Lett.* 8(6):1600–1603.

Kalantari E, Yanikomeroglu H, Yongacoglu A (2016) On the number and 3D placement of drone base stations in wireless cellular networks. *Proc. IEEE 84th Vehicular Tech. Conf.* (Institute of Electrical and Electronics Engineers, Washington, DC), 1–6.

Kalantari E, Bor-Yaliniz I, Yongacoglu A, Yanikomeroglu H (2017a) User association and bandwidth allocation for terrestrial and aerial base stations with backhaul considerations. *Proc. IEEE 28th Annual Internat. Sympos. Personal Indoor Mobile Radio Comm.* (Institute of Electrical and Electronics Engineers, Washington, DC), 1–6.

Kalantari E, Shakir MZ, Yanikomeroglu H, Yongacoglu A (2017b) Backhaul-aware robust 3D drone placement in 5G+ wireless

networks. *Proc. IEEE Internat. Conf. Comm. Workshops* (Institute of Electrical and Electronics Engineers, Washington, DC), 109–114.

Karydis K, Kumar V (2017) Energetics in robotic flight at small scales. *Interface Focus* 7(1):20160088:1–20160088:12.

Khawaja W, Guvenc I, Matolak D, Fiebig U, Schneckenberger N (2019) A survey of air-to-ground propagation channel modeling for unmanned aerial vehicles. *IEEE Comm. Surveys Tutorials* 21(3):2361–2391.

Khoshkholgh MG, Navaie K, Yanikomeroglu H, Leung VCM, Shin KG (2019) How do non-ideal UAV antennas affect air-to-ground communications? *Proc. IEEE Internat. Conf. Comm.*, 1–7.

Kim SJ, Lim GJ, Cho J, Côté MJ (2017) Drone-aided healthcare services for patients with chronic diseases in rural areas. *J. Intelligent Robotic Systems* 88(1):163–180.

Li B, Fei Z, Zhang Y (2019) UAV communications for 5G and beyond: Recent advances and future trends. *IEEE Internet Things J.* 6(2):2241–2263.

Liu CH, Chen Z, Tang J, Xu J, Piao C (2018) Energy-efficient UAV control for effective and fair communication coverage: A deep reinforcement learning approach. *IEEE J. Selected Areas Comm.* 36(9):2059–2070.

Lyu J, Zeng Y, Zhang R, Lim TJ (2017) Placement optimization of UAV-mounted mobile base stations. *IEEE Comm. Lett.* 21(3):604–607.

Malandrino F, Chiasserini CF, Casetti C, Chiaraviglio L, Senacheribbe A (2019) Planning UAV activities for efficient user coverage in disaster areas. *Ad Hoc Networks* 89:177–185.

Merwaday A, Tuncer A, Kumbhar A, Guvenc I (2016) Improved throughput coverage in natural disasters: Unmanned aerial base stations for public-safety communications. *IEEE Vehicular Tech. Magazine* 11(4):53–60.

Mozaffari M, Saad W, Bennis M, Debbah M (2015) Drone small cells in the clouds: Design, deployment and performance analysis. *2015 IEEE Global Comm. Conf.* (Institute of Electrical and Electronics Engineers, Washington, D.C.), 1–6.

Mozaffari M, Saad W, Bennis M, Debbah M (2016) Efficient deployment of multiple unmanned aerial vehicles for optimal wireless coverage. *IEEE Comm. Lett.* 20(8):1647–1650.

Mozaffari M, Saad W, Bennis M, Debbah M (2017) Mobile unmanned aerial vehicles (UAVs) for energy-efficient internet of things communications. *IEEE Trans. Wireless Comm.* 16(11):7574–7589.

Murray AT (2016) Maximal coverage location problem: Impacts, significance, and evolution. *Internat. Regional Sci. Rev.* 39(1):5–27.

Murray CC, Chu AG (2015) The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Res. Part C Emerging Tech.* 54:86–109.

Newell GF (1971) Dispatching policies for a transportation route. *Transportation Sci.* 5(1):91–105.

Nowak I (2005) *Relaxation and Decomposition Methods for Mixed Integer Nonlinear Programming* (Birkhäuser, Basal).

Otto A, Agatz N, Campbell J, Golden B, Pesch E (2018) Optimization approaches for civil applications of unmanned aerial vehicles (UAVs) or aerial drones: A survey. *Networks* 72(4):411–458.

Ouyang Y, Daganzo CF (2006) Discretization and validation of the continuum approximation scheme for terminal system design. *Transportation Sci.* 40(1):89–98.

Phung DK, Morin P (2013) Modeling and energy evaluation of small convertible UAVs. *IFAC Proc. Volumes* 46(30):212–219.

Pulver A, Wei R, Mann C (2016) Locating AED enabled medical drones to enhance cardiac arrest response times. *Prehospital Emergency Care* 20(3):378–389.

Qiu C, Wei Z, Feng Z, Zhang P (2020) Backhaul-aware trajectory optimization of fixed-wing UAV-mounted base station for continuous available wireless service. *IEEE Access* 8:60940–60950.

Sharma V, Bennis M, Kumar R (2016) UAV-assisted heterogeneous networks for capacity enhancement. *IEEE Comm. Lett.* 20(6): 1207–1210.

Shi W, Li J, Xu W, Zhou H, Zhang N, Zhang S, Shen X (2018) Multiple drone-cell deployment analyses and optimization in drone assisted radio access networks. *IEEE Access* 6:12518–12529.

Tu W, Li Q, Fang Z, Shaw SL, Zhou B, Chang X (2016) Optimizing the locations of electric taxi charging stations: A spatial–temporal demand coverage approach. *Transportation Res. Part C Emerging Tech.* 65:172–189.

Wang X, Ouyang Y (2013) A continuum approximation approach to competitive facility location design under facility disruption risks. *Transportation Res. Part B Methodological* 50:90–103.

Wang Z, Duan L, Zhang R (2019a) Adaptive deployment for UAV-aided communication networks. *IEEE Trans. Wireless Comm.* 18(9):4531–4543.

Wang X, Lim MK, Ouyang Y (2017) A continuum approximation approach to the dynamic facility location problem in a growing market. *Transportation Sci.* 51(1):343–357.

Wang Z, Xu W, Yang D, Lin J (2019b) Joint trajectory optimization and user scheduling for rotary-wing UAV-enabled wireless powered communication networks. *IEEE Access* 7:181369–181380.

Xu J, Zeng Y, Zhang R (2018) UAV-enabled wireless power transfer: Trajectory design and energy optimization. *IEEE Trans. Wireless Comm.* 17(8):5092–5106.

Zeng Y, Zhang R (2017) Energy-efficient UAV communication with trajectory optimization. *IEEE Trans. Wireless Comm.* 16(6): 3747–3760.

Zeng Y, Wu Q, Zhang R (2019a) Accessing from the sky: A tutorial on UAV communications for 5G and beyond. *Proc. IEEE.* 107(12): 2327–2375.

Zeng Y, Xu J, Zhang R (2019b) Energy minimization for wireless communication with rotary-wing UAV. *IEEE Trans. Wireless Comm.* 18(4):2329–2345.

Zeng Y, Zhang R, Lim TJ (2016) Wireless communications with unmanned aerial vehicles: Opportunities and challenges. *IEEE Comm. Magazine* 54(5):36–42.

Zhan C, Zeng Y, Zhang R (2018) Energy-efficient data collection in UAV enabled wireless sensor network. *IEEE Wireless Comm. Lett.* 7(3):328–331.

Zhang S, Zeng Y, Zhang R (2019a) Cellular-enabled UAV communication: A connectivity-constrained trajectory optimization perspective. *IEEE Trans. Comm.* 67(3):2580–2604.

Zhang G, Wu Q, Cui M, Zhang R (2019b) Securing UAV communications via joint trajectory and power control. *IEEE Trans. Wireless Comm.* 18(2):1376–1389.

Copyright 2021, by INFORMS, all rights reserved. Copyright of Journal on Computing is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.